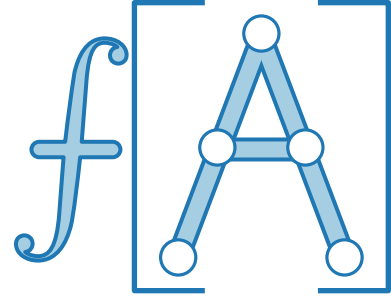
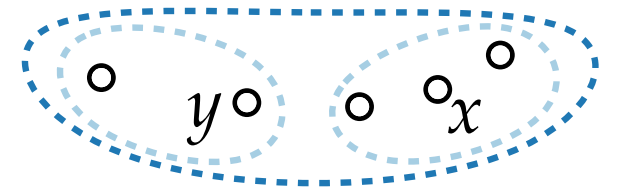
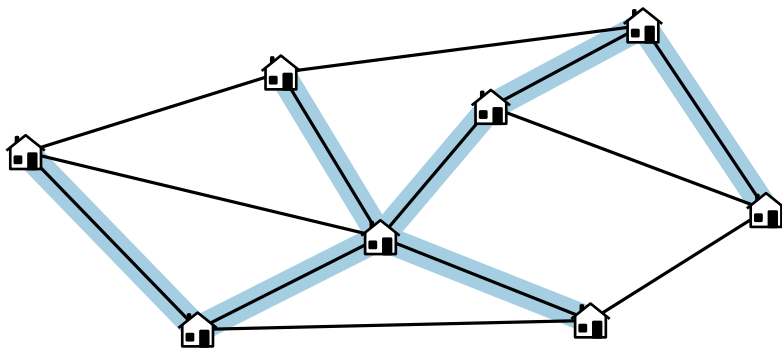




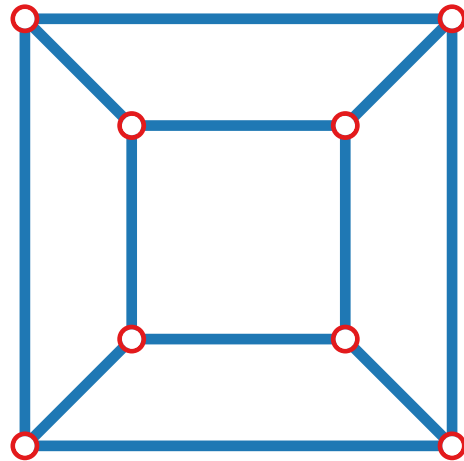
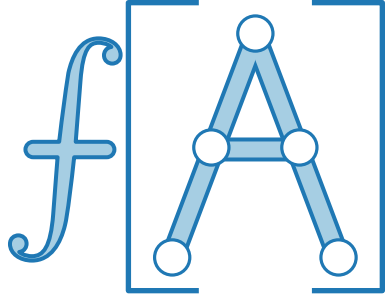
Fortgeschrittene Algorithmen

7. Vorlesung Graphalgorithmen I: Minimale Spannbäume

Teil I: Graphen

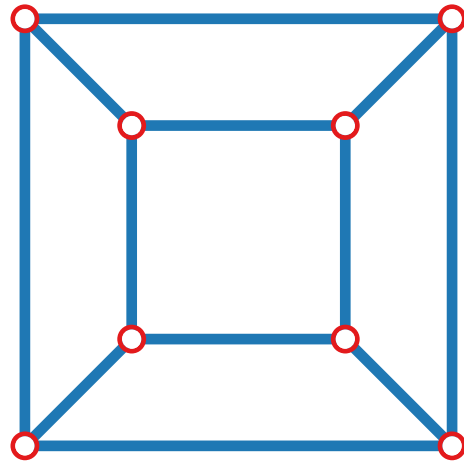
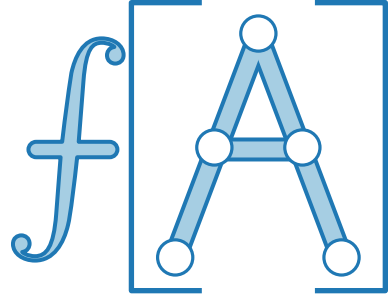


Was ist ein Graph?



Was ist ein Graph?

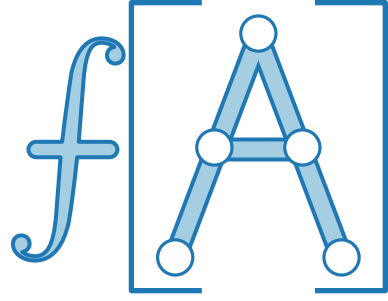
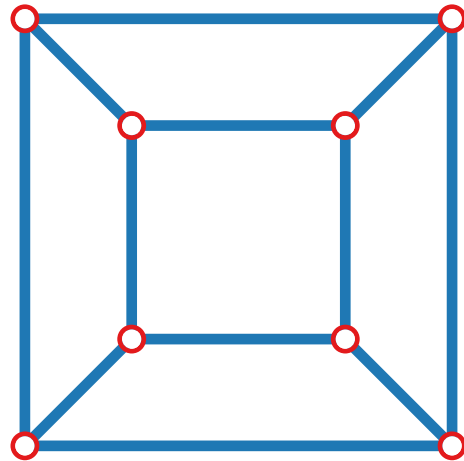
Ein (ungerichteter) Graph ist ein Paar (V, E)



Was ist ein Graph?

Ein (ungerichteter) Graph ist ein Paar (V, E) , wobei

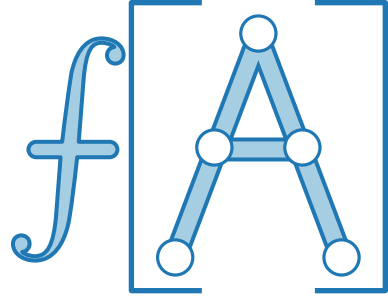
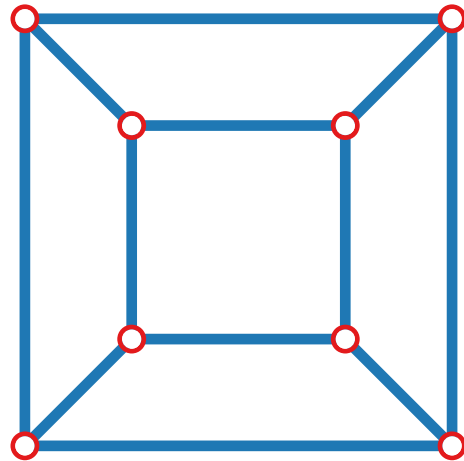
- V **Knotenmenge**



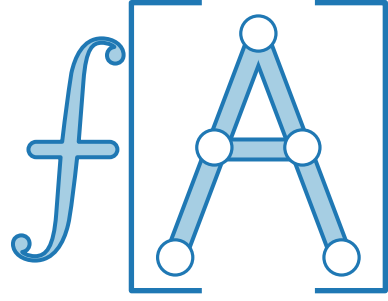
Was ist ein Graph?

Ein (ungerichteter) Graph ist ein Paar (V, E) , wobei

- V **Knotenmenge** und
- $E \subseteq \binom{V}{2} = \{\{u, v\} \subseteq V \mid u \neq v\}$ **Kantenmenge**.

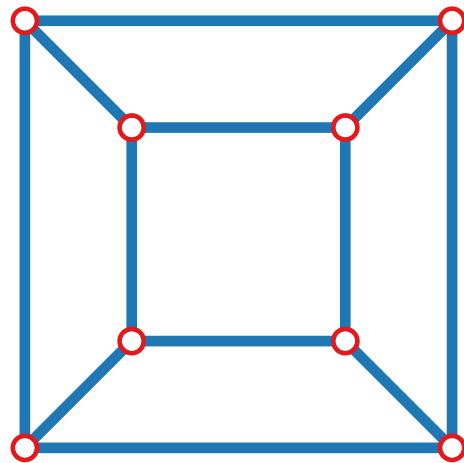


Was ist ein Graph?

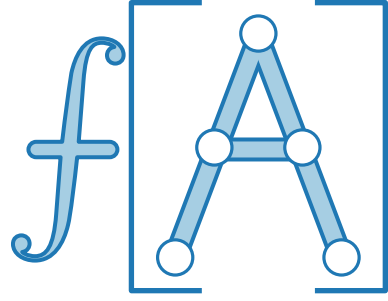


Ein (ungerichteter) Graph ist ein Paar (V, E) , wobei

- V **Knotenmenge** und
- $E \subseteq \binom{V}{2} = \{\{u, v\} \subseteq V \mid u \neq v\}$ **Kantenmenge**.

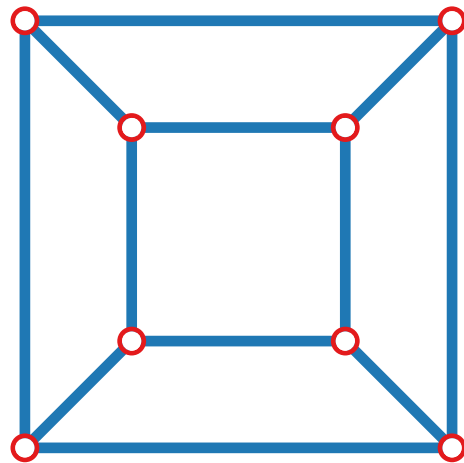


Was ist ein Graph?



Ein (ungerichteter) Graph ist ein Paar (V, E) , wobei

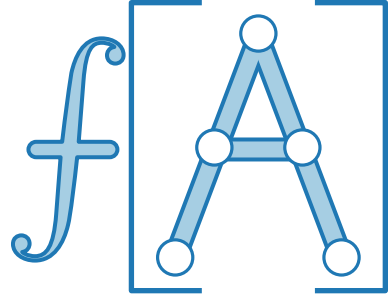
- V **Knotenmenge** und
- $E \subseteq \binom{V}{2} = \{\{u, v\} \subseteq V \mid u \neq v\}$ **Kantenmenge**.



Ein **gerichteter** Graph ist ein Paar (V, E) , wobei

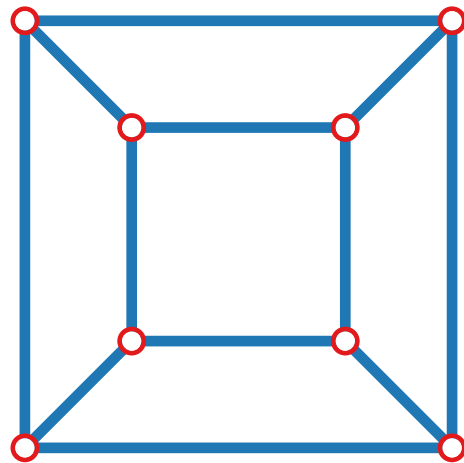
- V **Knotenmenge** und
- $E \subseteq V \times V = \{(u, v) \mid u, v \in V\}$ **Kantenmenge**.

Was ist ein Graph?



Ein (ungerichteter) Graph ist ein Paar (V, E) , wobei

- V **Knotenmenge** und
- $E \subseteq \binom{V}{2} = \{\{u, v\} \subseteq V \mid u \neq v\}$ **Kantenmenge**.



Ein **gerichteter** Graph ist ein Paar (V, E) , wobei

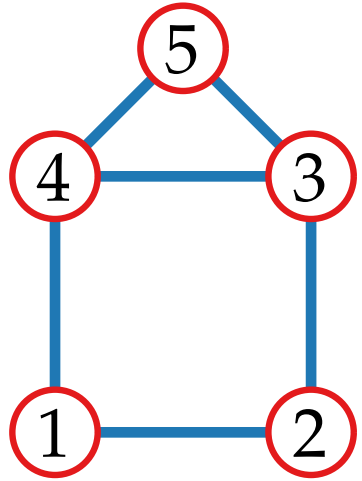
- V **Knotenmenge** und
- $E \subseteq V \times V = \{(u, v) \mid u, v \in V\}$ **Kantenmenge**.

Üblicherweise:

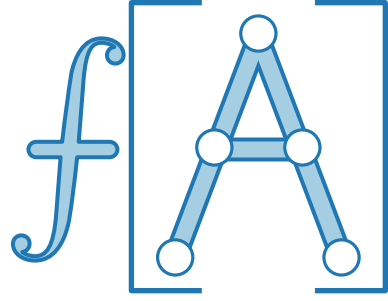
$$n = |V|$$

$$m = |E|$$

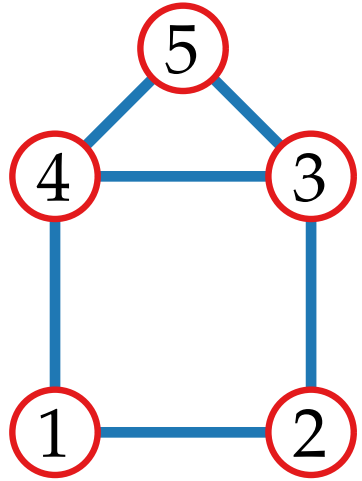
Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

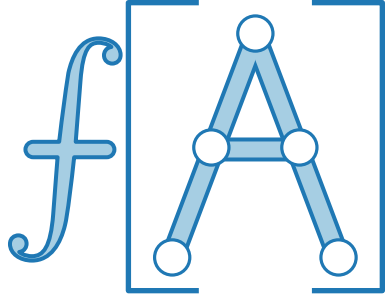


Wie repräsentiere ich einen Graphen?

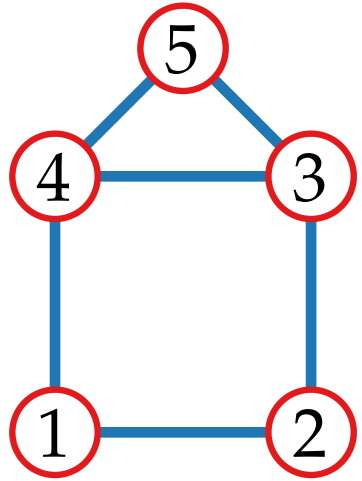


ungerichteter
Graph

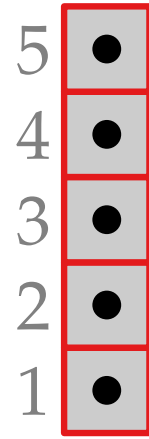
Adjazenzlisten



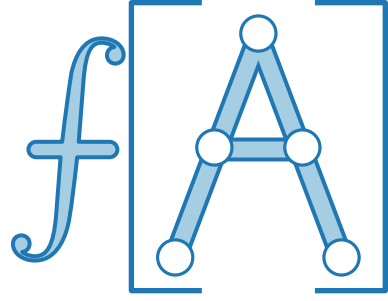
Wie repräsentiere ich einen Graphen?



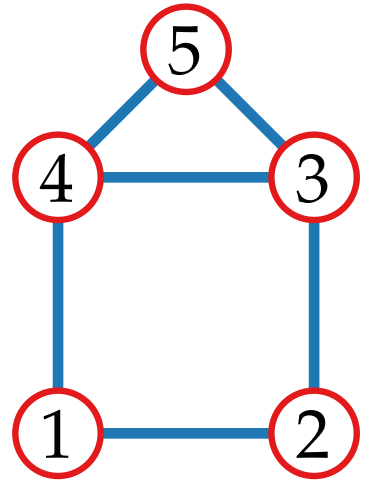
ungerichteter
Graph



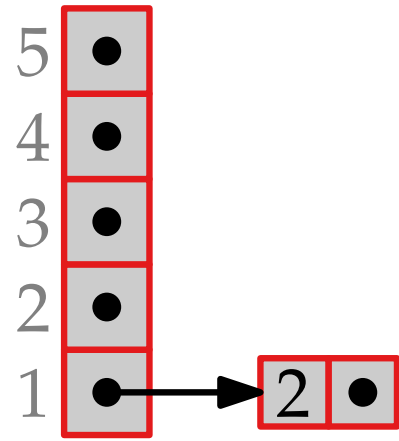
Adjazenzlisten



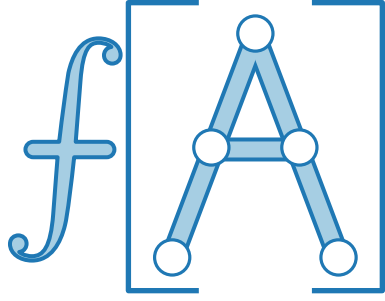
Wie repräsentiere ich einen Graphen?



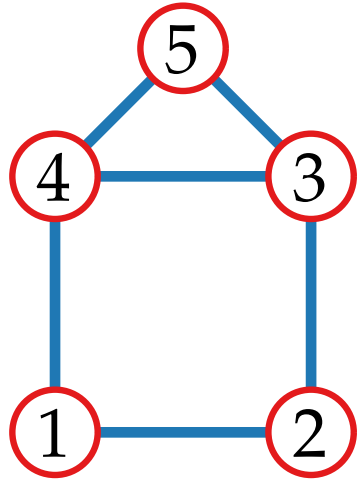
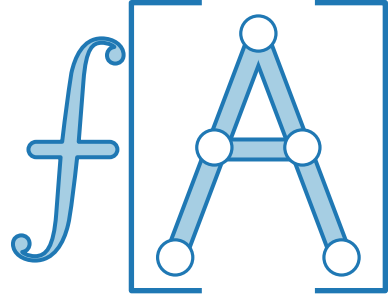
ungerichteter
Graph



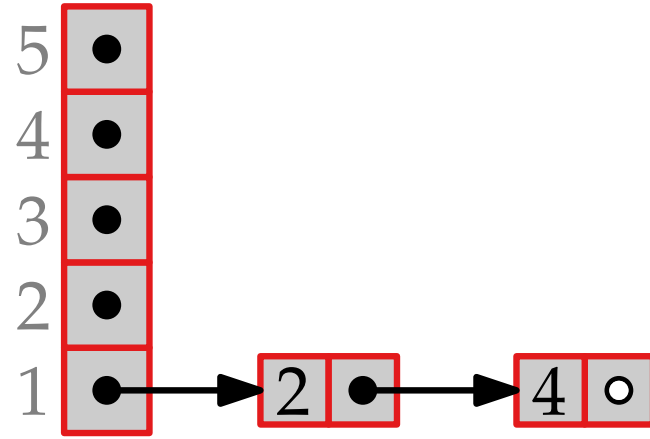
Adjazenzlisten



Wie repräsentiere ich einen Graphen?

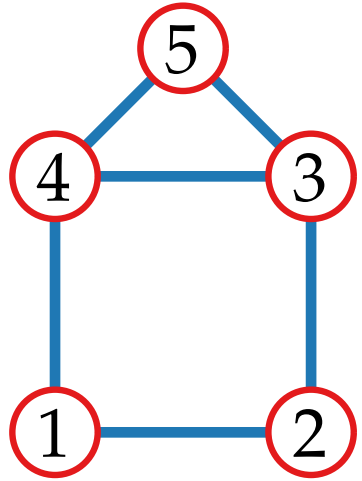
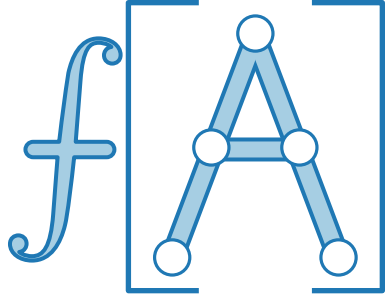


ungerichteter
Graph

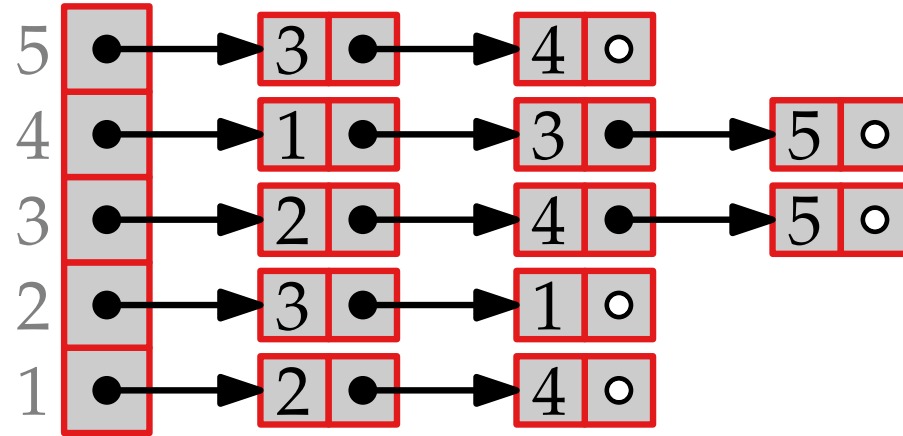


Adjazenzlisten

Wie repräsentiere ich einen Graphen?

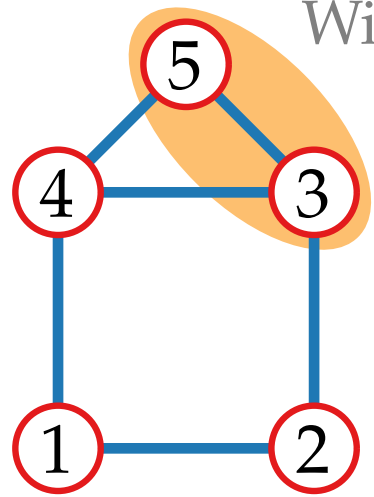
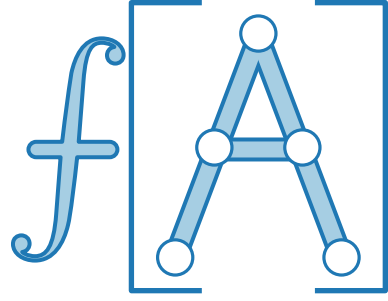


ungerichteter
Graph



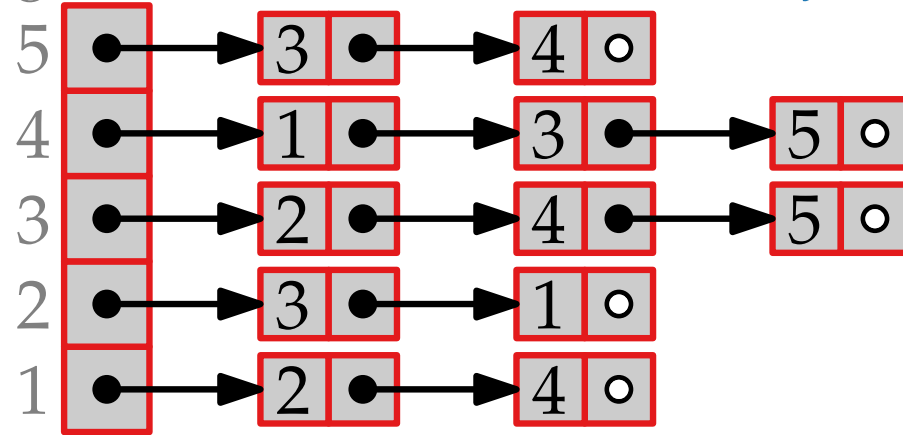
Adjazenzlisten

Wie repräsentiere ich einen Graphen?



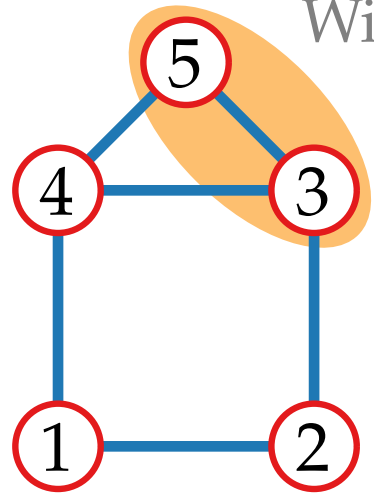
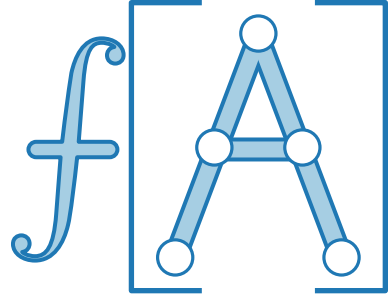
ungerichteter
Graph

Wir sagen: Knoten 3 und 5 sind **adjazent**.



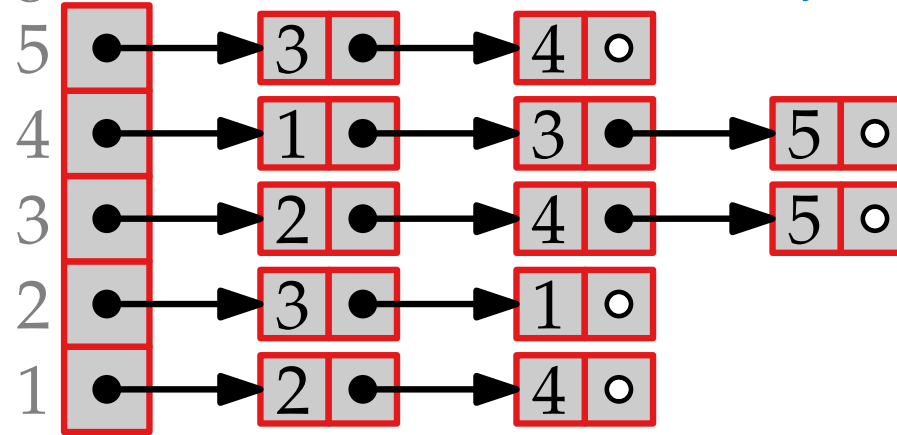
Adjazenzlisten

Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

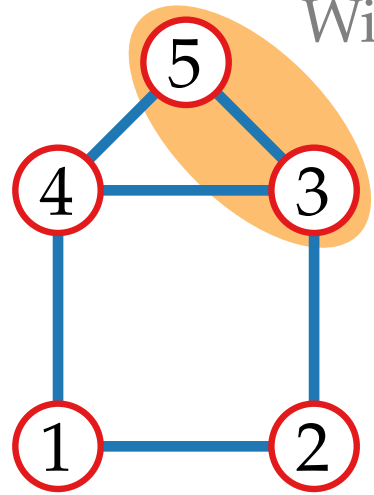
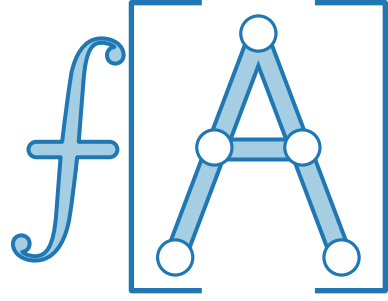
Wir sagen: Knoten 3 und 5 sind **adjazent**.



Adjazenzlisten

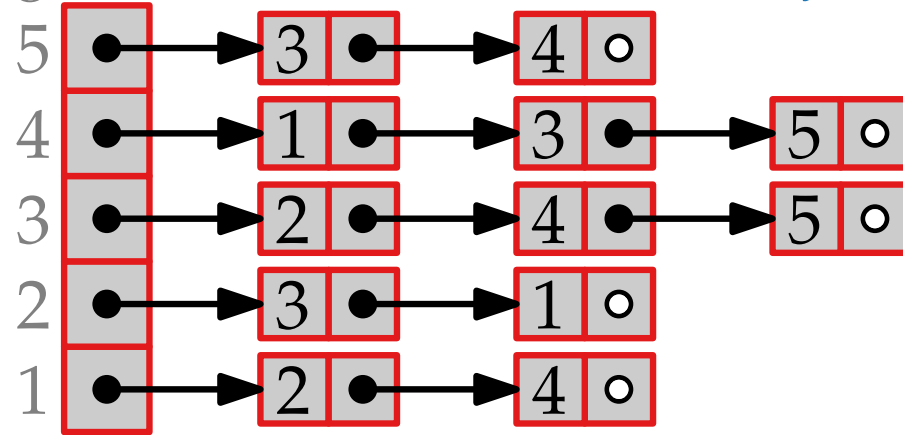
Adjazenzmatrix

Wie repräsentiere ich einen Graphen?

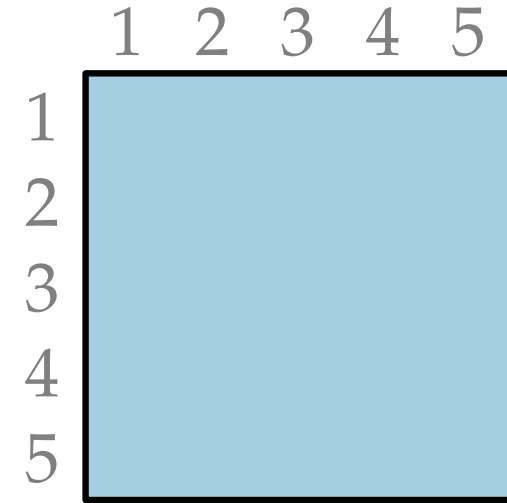


ungerichteter
Graph

Wir sagen: Knoten 3 und 5 sind **adjazent**.

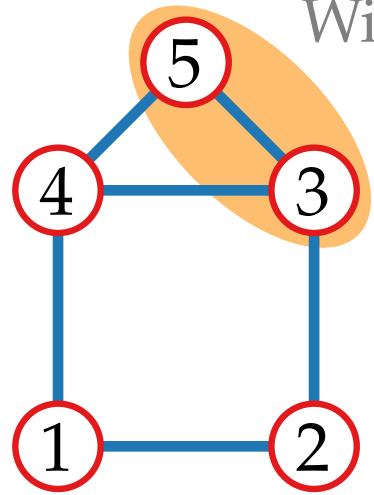
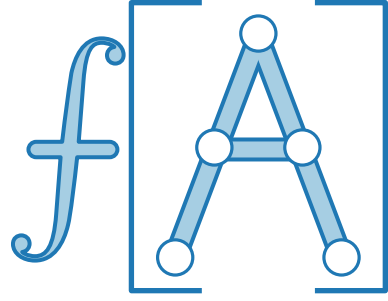


Adjazenzlisten



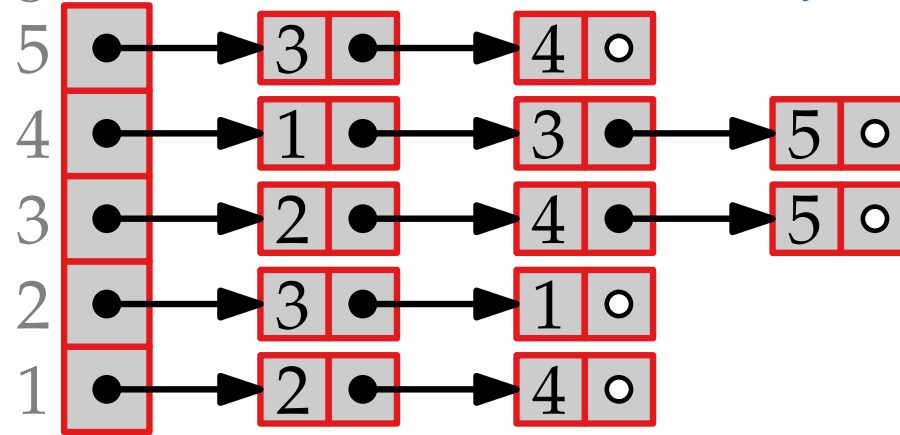
Adjazenzmatrix

Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

Wir sagen: Knoten 3 und 5 sind **adjazent**.

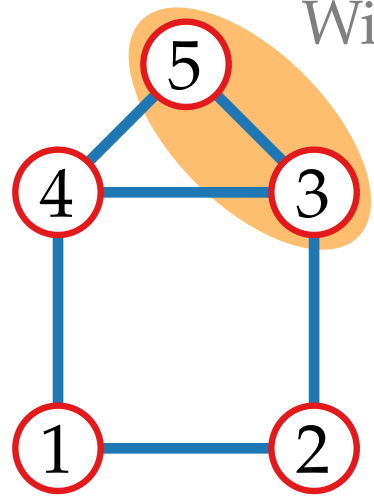
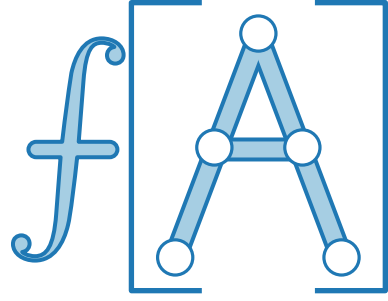


Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

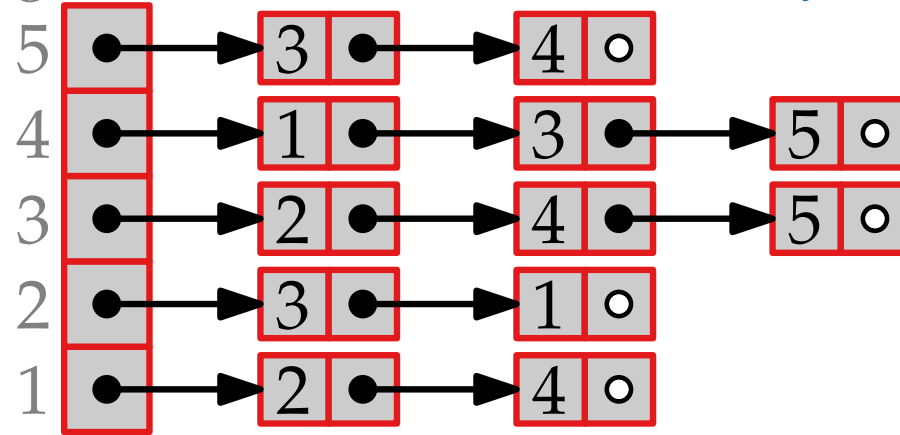
Adjazenzmatrix

Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

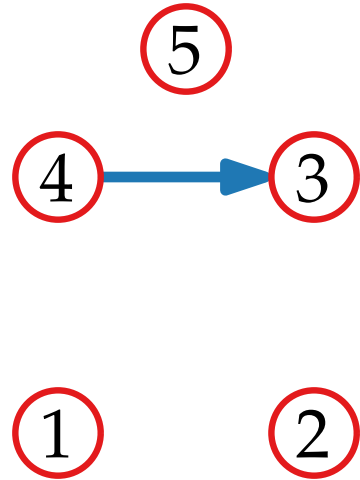
Wir sagen: Knoten 3 und 5 sind **adjazent**.



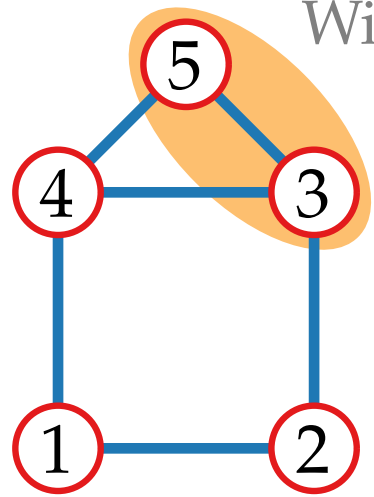
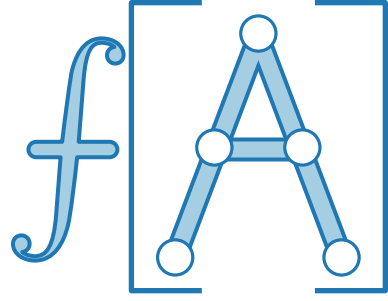
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

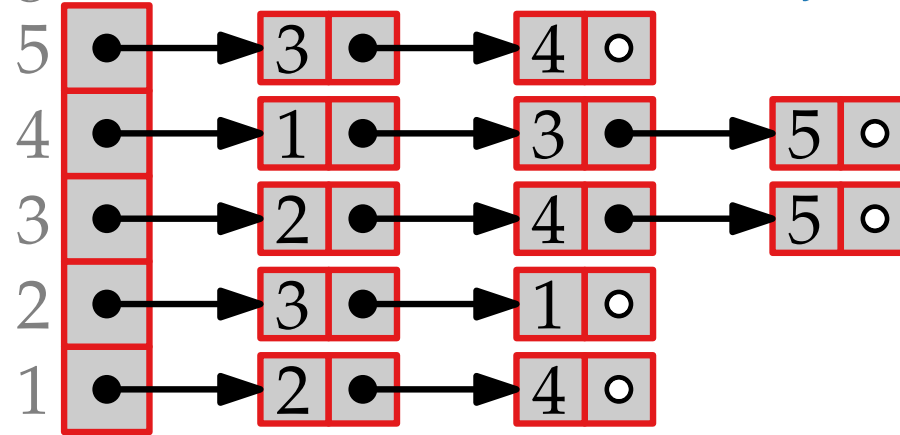


Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

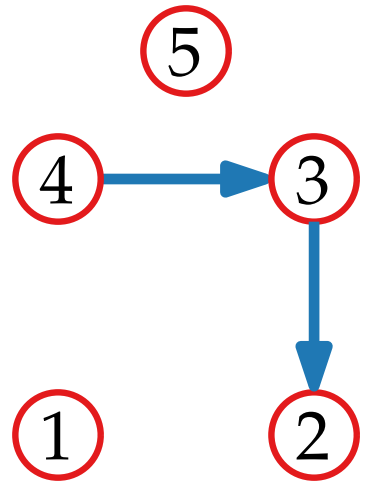
Wir sagen: Knoten 3 und 5 sind **adjazent**.



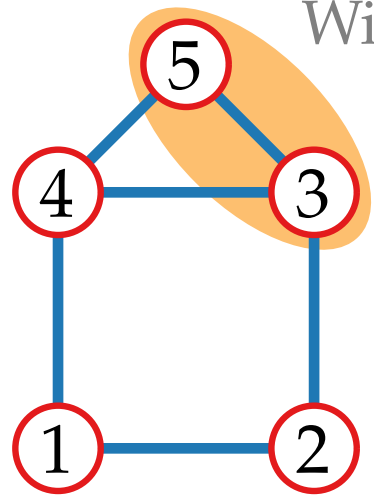
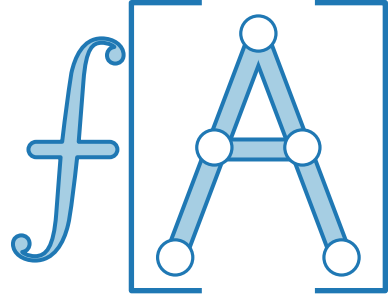
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

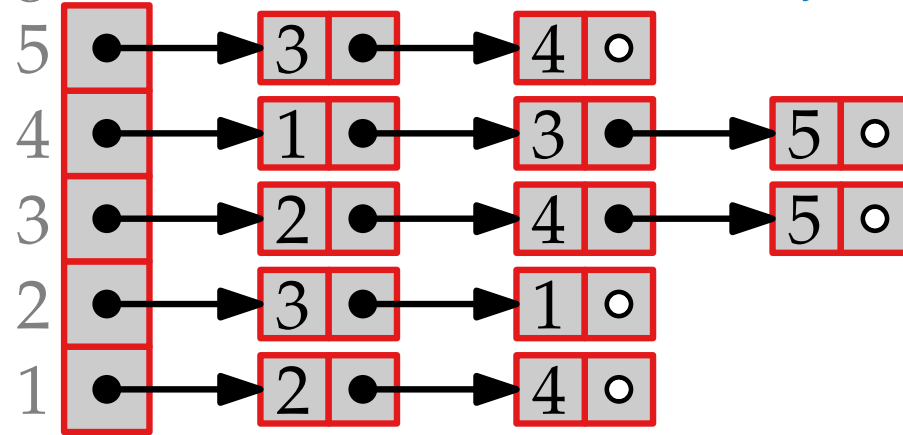


Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

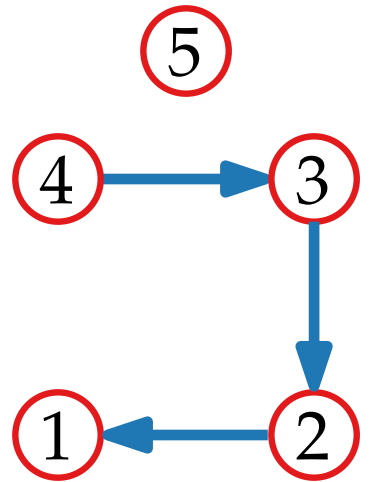
Wir sagen: Knoten 3 und 5 sind **adjazent**.



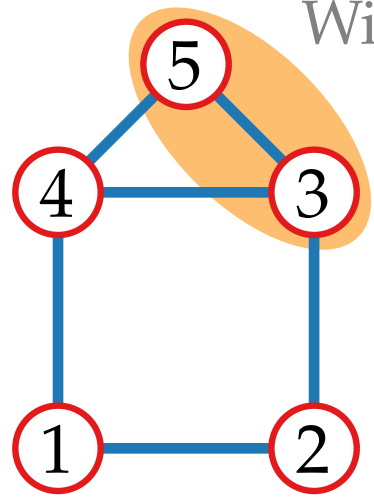
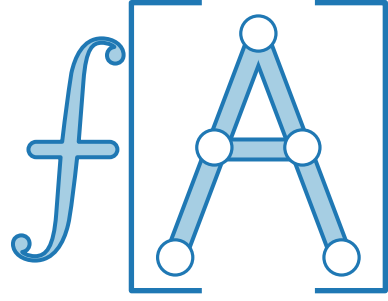
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

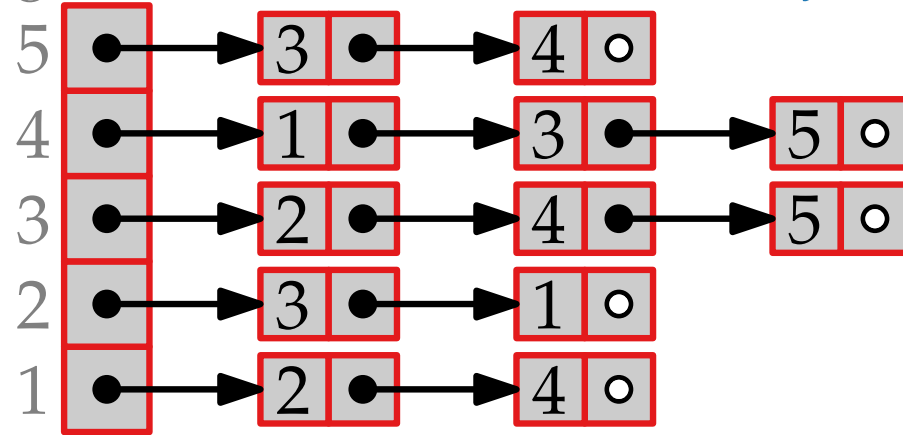


Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

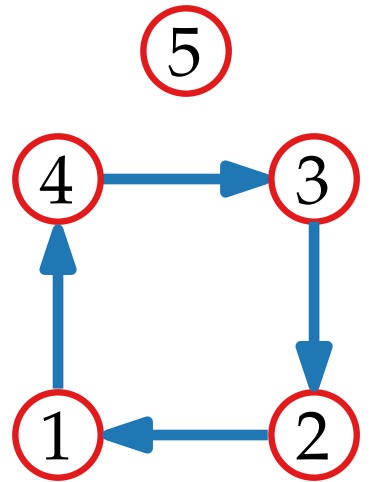
Wir sagen: Knoten 3 und 5 sind **adjazent**.



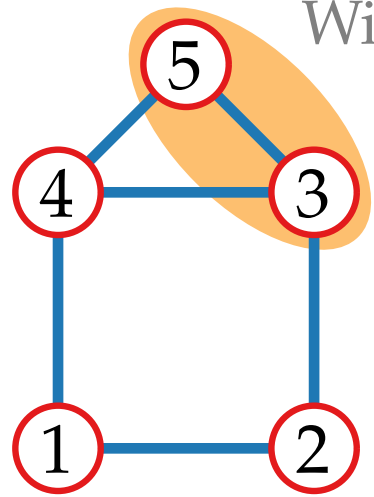
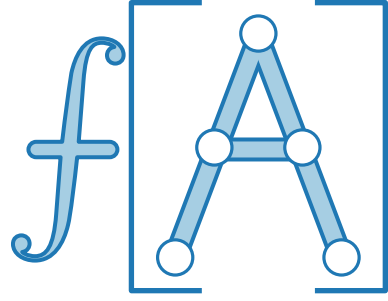
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

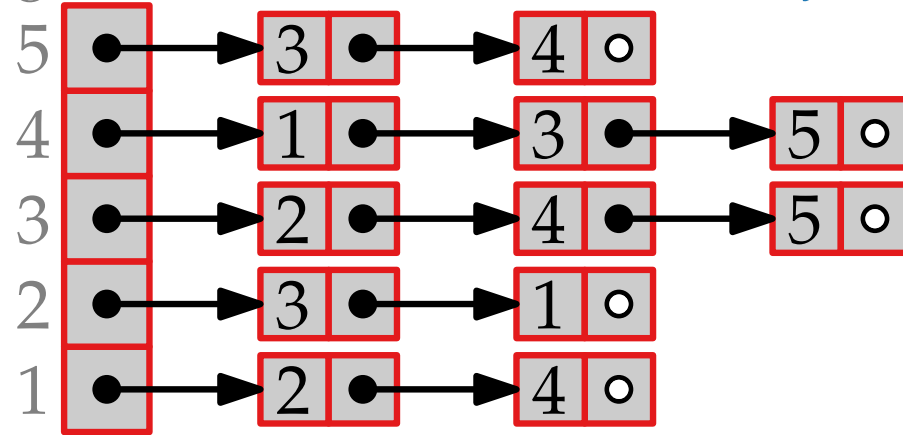


Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

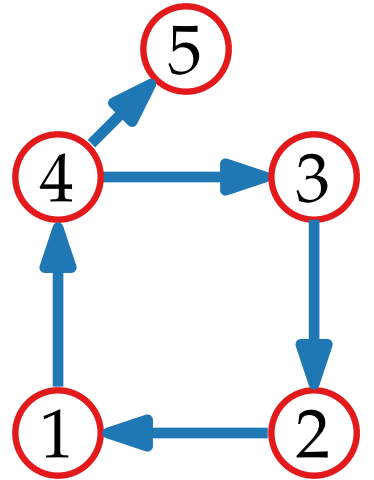
Wir sagen: Knoten 3 und 5 sind **adjazent**.



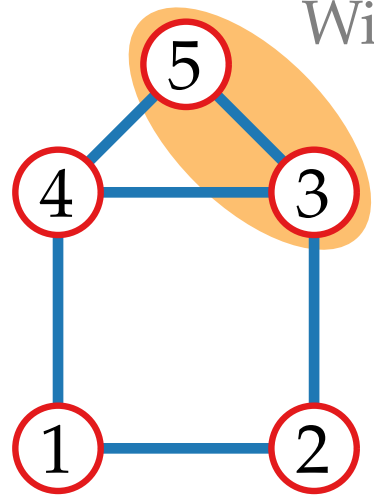
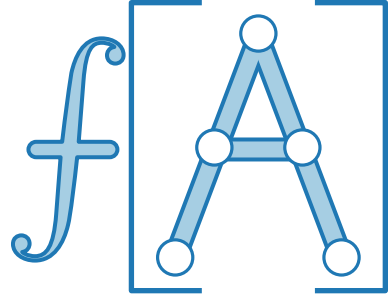
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

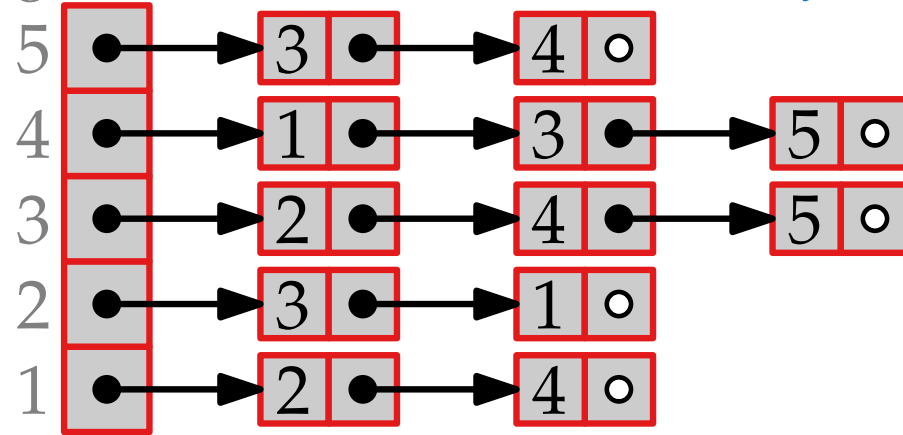


Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

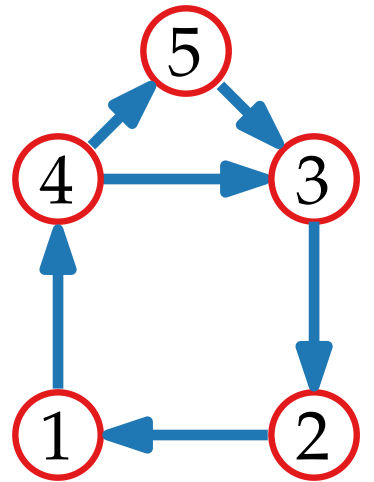
Wir sagen: Knoten 3 und 5 sind **adjazent**.



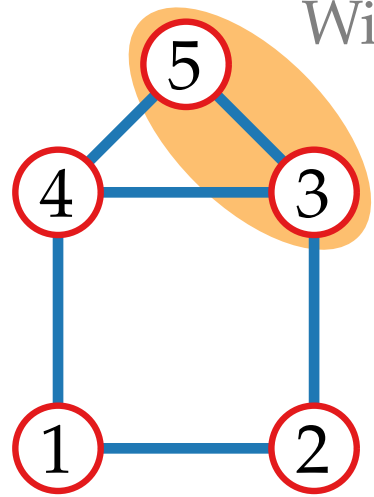
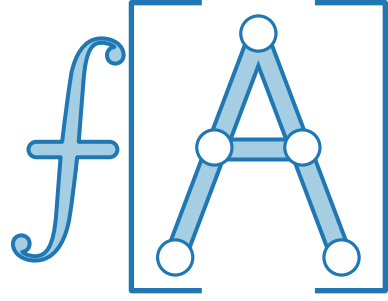
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

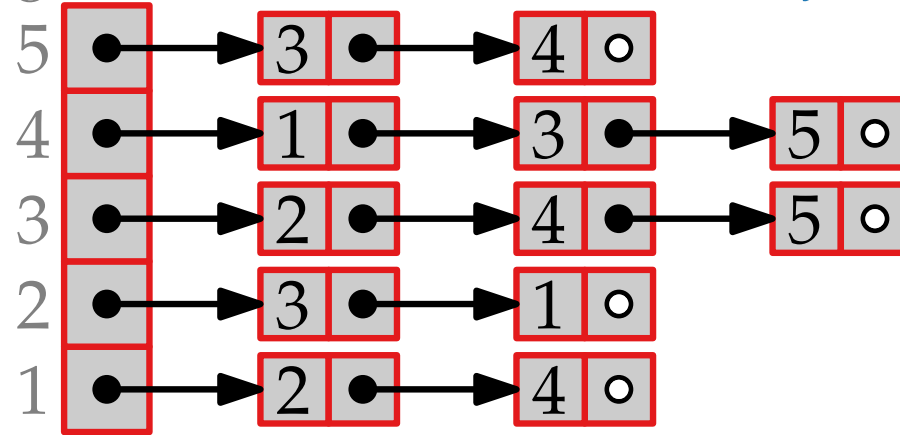


Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

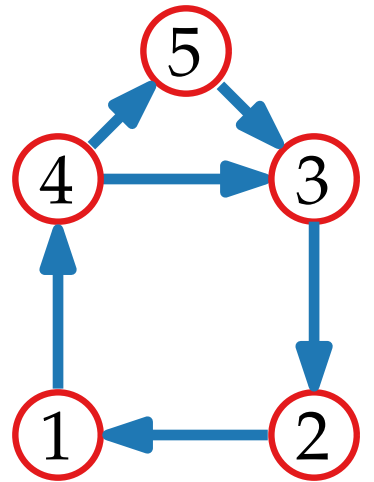
Wir sagen: Knoten 3 und 5 sind **adjazent**.



Adjazenzlisten

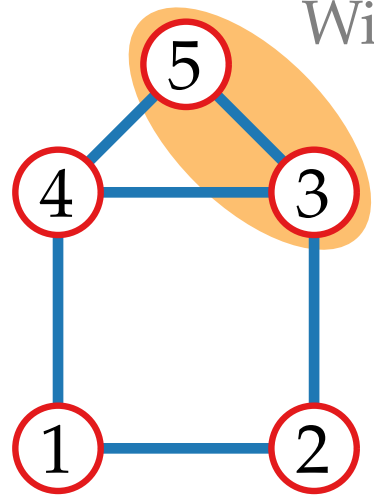
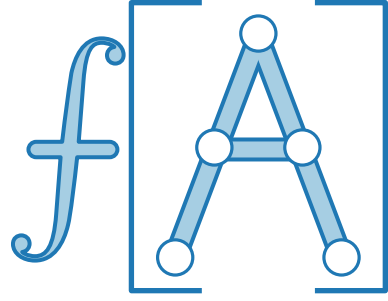
	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix



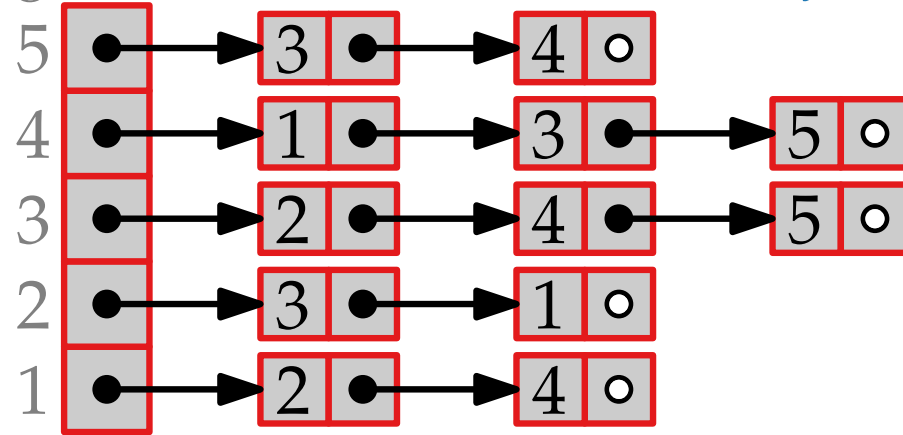
gerichteter
Graph

Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

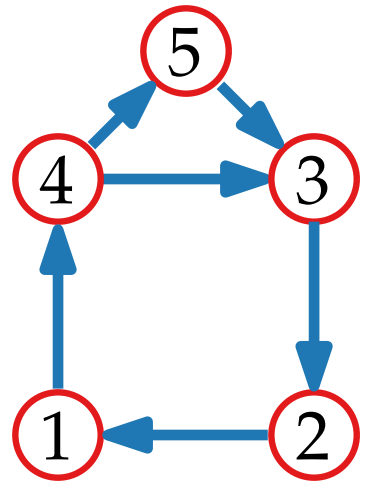
Wir sagen: Knoten 3 und 5 sind **adjazent**.



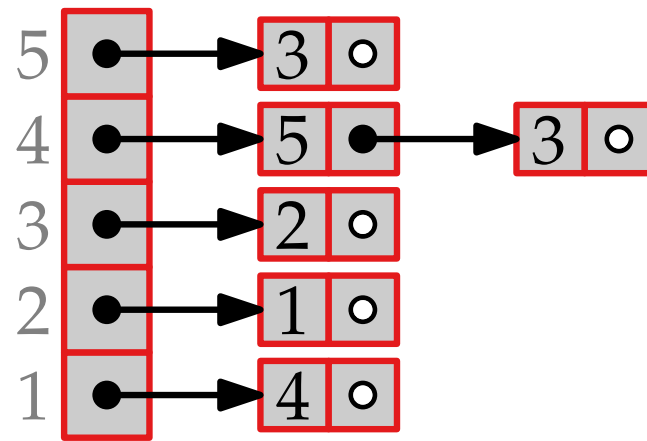
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

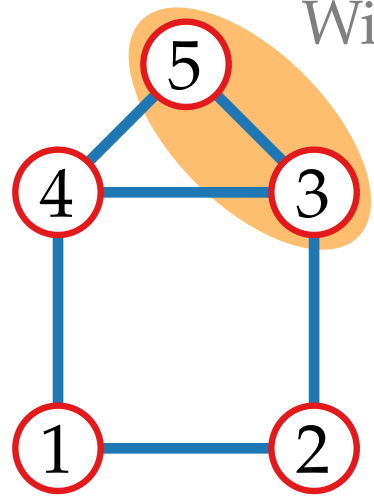
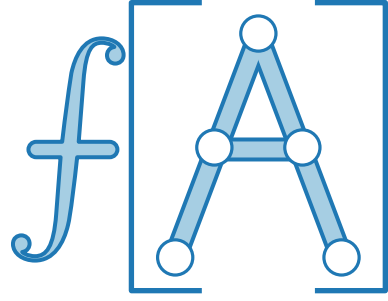


gerichteter
Graph



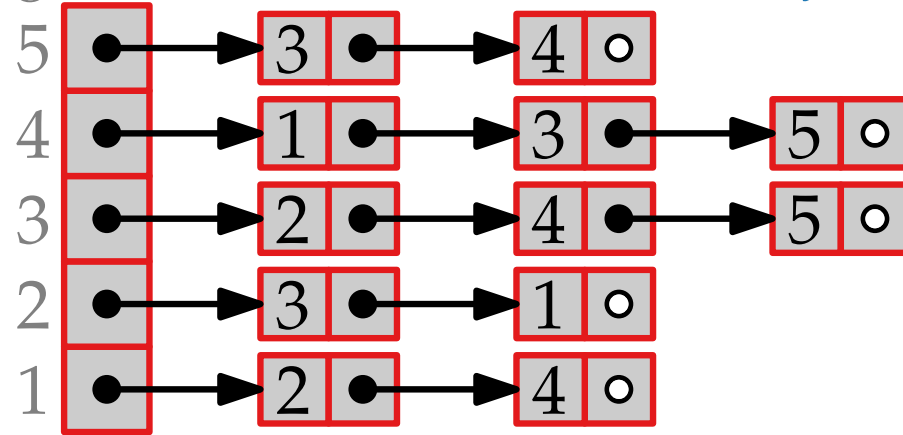
$$\text{Adj}[i] = \{j \in V \mid (i, j) \in E\}$$

Wie repräsentiere ich einen Graphen?



ungerichteter
Graph

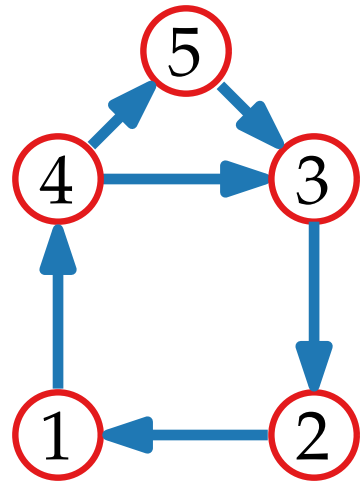
Wir sagen: Knoten 3 und 5 sind **adjazent**.



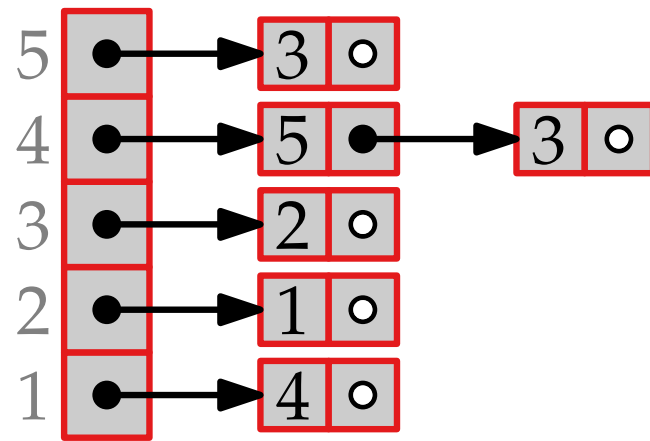
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix



gerichteter
Graph



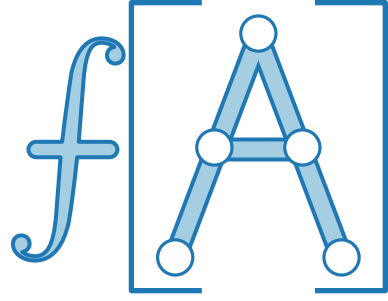
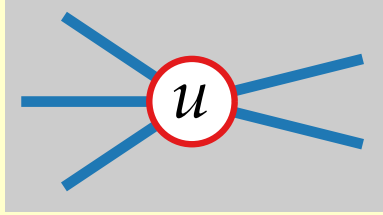
$$\text{Adj}[i] = \{j \in V \mid (i, j) \in E\}$$

	1	2	3	4	5
1	0	0	0	1	0
2	1	0	0	0	0
3	0	1	0	0	0
4	0	0	1	0	1
5	0	0	1	0	0

$$a_{ij} = 1 \Leftrightarrow (i, j) \in E$$

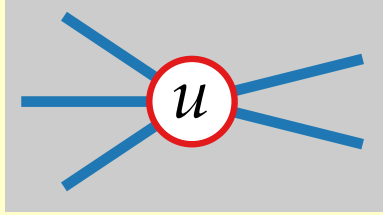
Grad eines Knotens

Def.

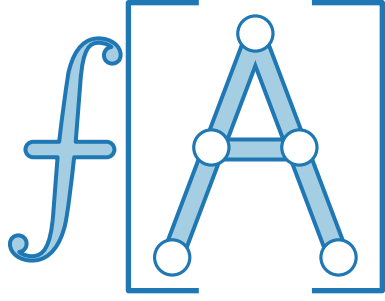


Grad eines Knotens

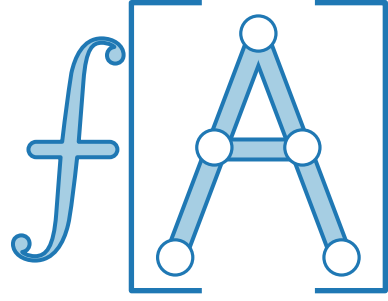
Def.



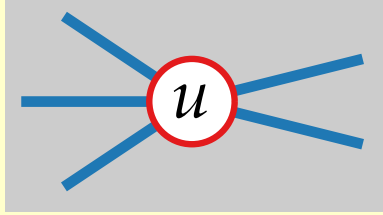
$$\deg(u) = |\text{Adj}[u]|$$



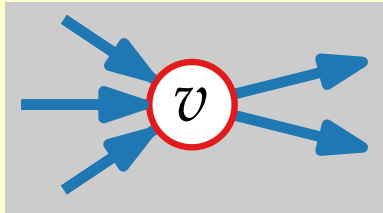
Grad eines Knotens



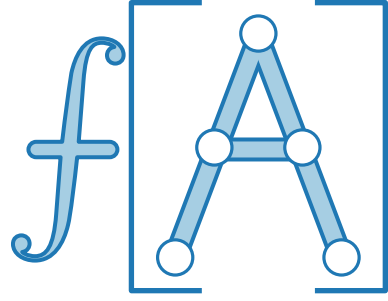
Def.



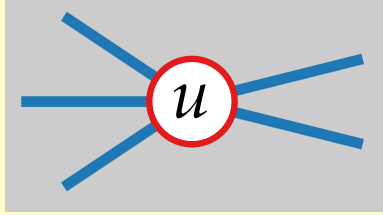
$$\deg(u) = |\text{Adj}[u]|$$



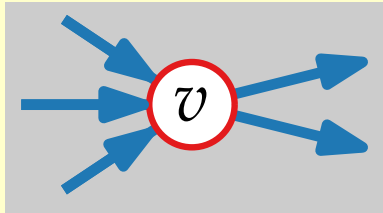
Grad eines Knotens



Def.

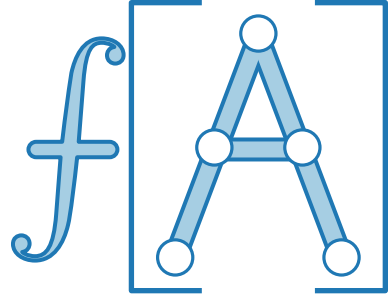


$$\deg(u) = |\text{Adj}[u]|$$

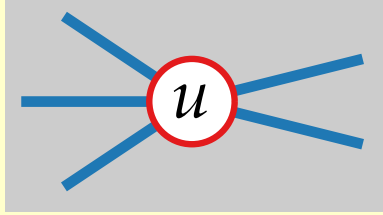


$$\text{outdeg}(v) = |\text{Adj}[v]|$$

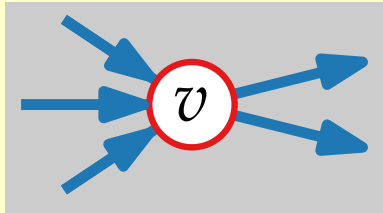
Grad eines Knotens



Def.



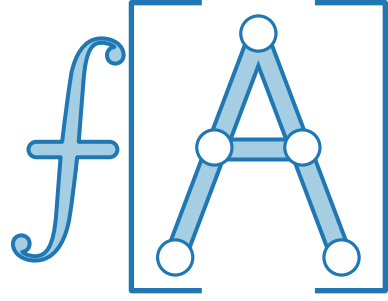
$$\deg(u) = |\text{Adj}[u]|$$



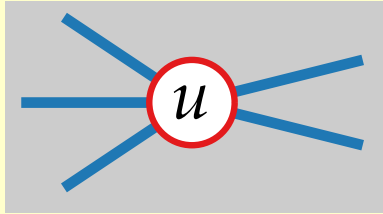
$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in \textcolor{red}{V} : (u, v) \in \textcolor{blue}{E}\}|$$

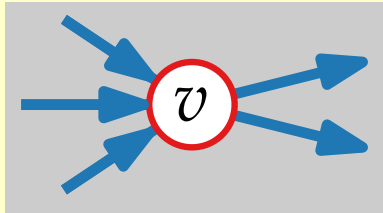
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

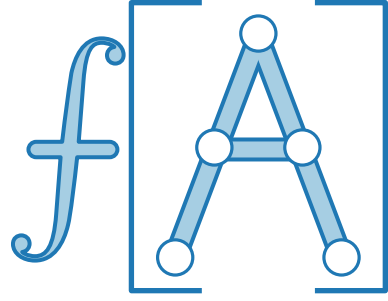
$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

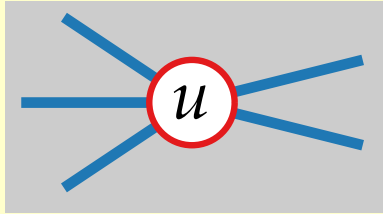
Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade = 

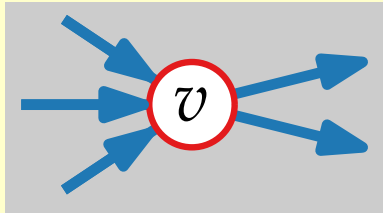
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

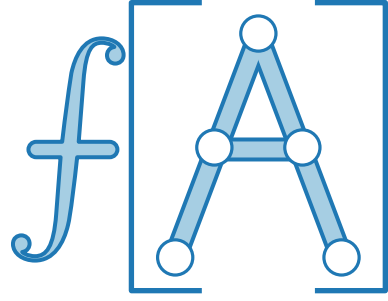
$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

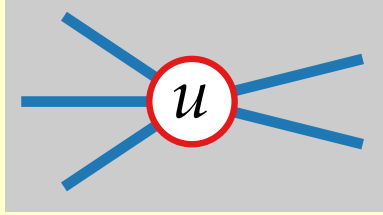
Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

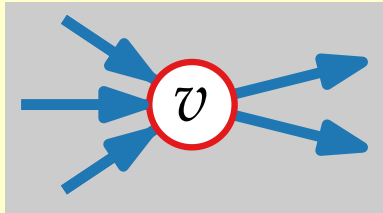
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

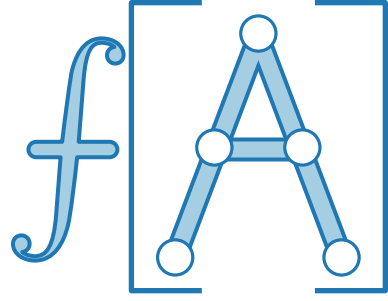
Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

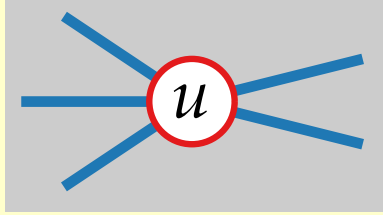
Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

Beweis. Technik des **zweifachen Abzählens**:

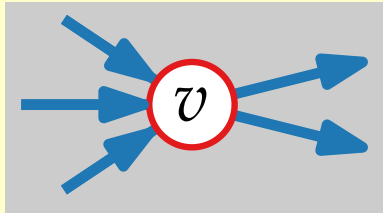
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in \textcolor{red}{V} : (u, v) \in \textcolor{blue}{E}\}|$$

Beob.

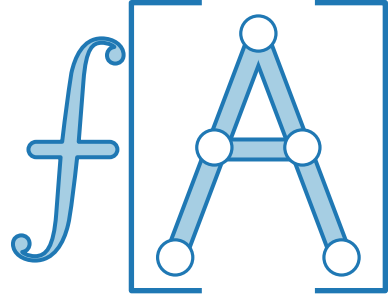
Sei $G = (\textcolor{red}{V}, \textcolor{blue}{E})$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |\textcolor{blue}{E}|$.

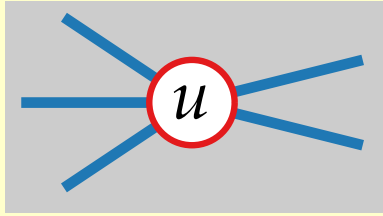
Beweis. Technik des **zweifachen Abzählens**:

Zähle alle Knoten-Kanten-Inzidenzen.

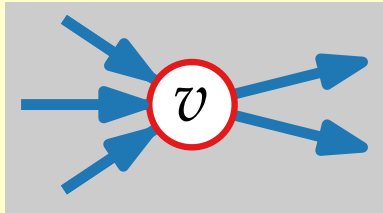
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

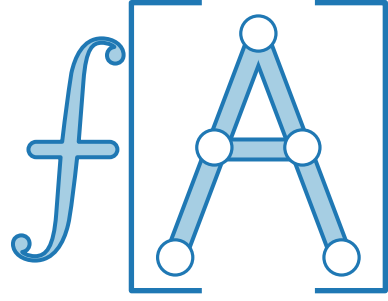
Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

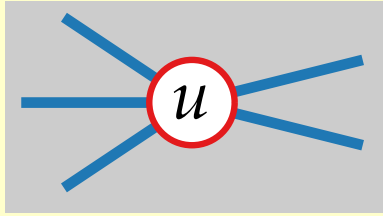
Beweis. Technik des **zweifachen Abzählens**:

Zähle alle Knoten-Kanten-Inzidenzen.

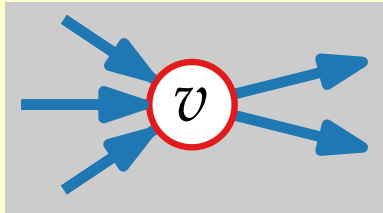
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

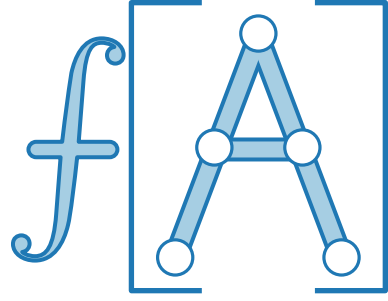
Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

Beweis. Technik des **zweifachen Abzählens**:

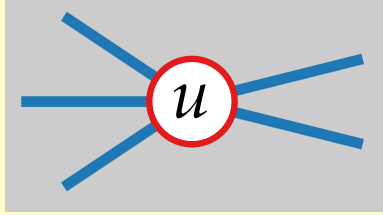
Zähle alle Knoten-Kanten-Inzidenzen.

Eine Kante ist **inzident** zu ihren Endknoten.

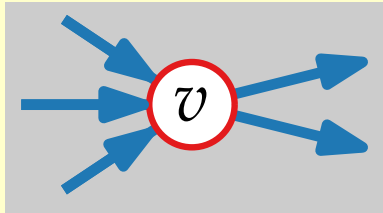
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

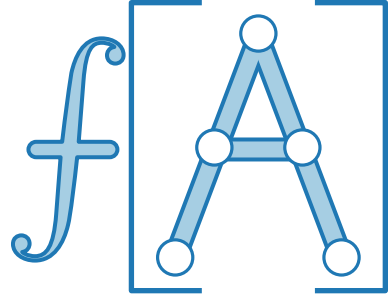
Beweis. Technik des **zweifachen Abzählens**:

Zähle alle Knoten-Kanten-Inzidenzen.

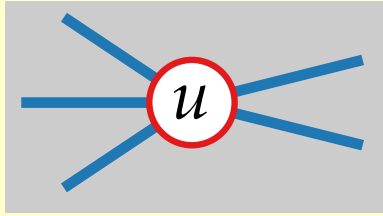
Eine Kante ist **inzident** zu ihren Endknoten.

Ein Knoten ist **inzident** zu allen Kanten, deren Endknoten er ist.

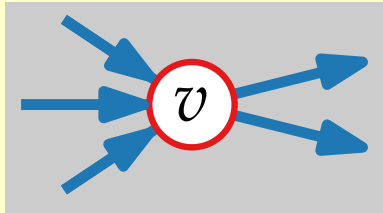
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

Beweis. Technik des **zweifachen Abzählens**:

Zähle alle Knoten-Kanten-Inzidenzen.

Aus Sicht der Knoten:



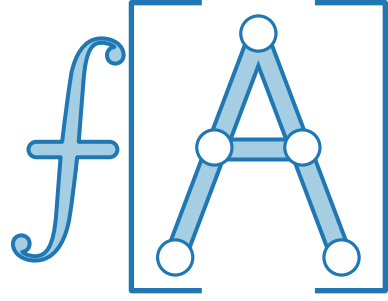
Aus Sicht der Kanten:



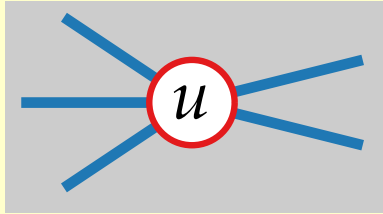
Eine Kante ist **inzident** zu ihren Endknoten.

Ein Knoten ist **inzident** zu allen Kanten, deren Endknoten er ist.

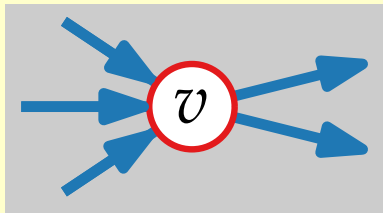
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

Beweis. Technik des **zweifachen Abzählens**:

Zähle alle Knoten-Kanten-Inzidenzen.

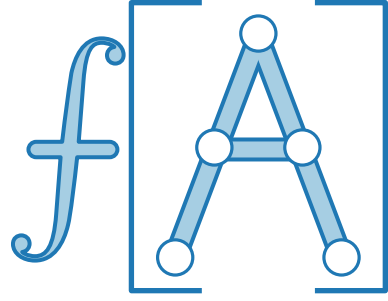
Aus Sicht der Knoten: $\sum_{v \in V} \deg v$

Aus Sicht der Kanten:

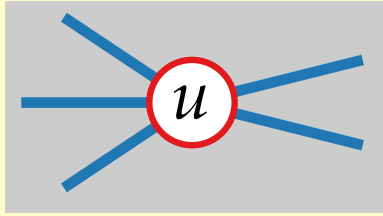
Eine Kante ist **inzident** zu ihren Endknoten.

Ein Knoten ist **inzident** zu allen Kanten, deren Endknoten er ist.

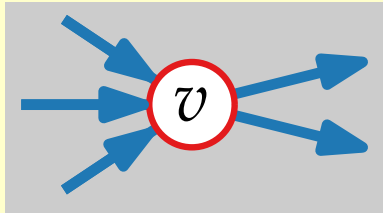
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

Beweis. Technik des **zweifachen Abzählens**:

Zähle alle Knoten-Kanten-Inzidenzen.

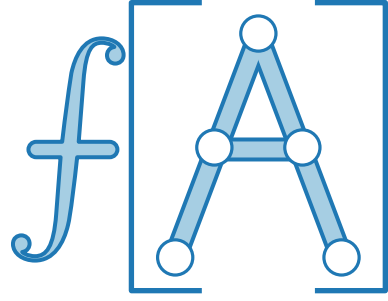
Eine Kante ist **inzident** zu ihren Endknoten.

Aus Sicht der Knoten: $\sum_{v \in V} \deg v$

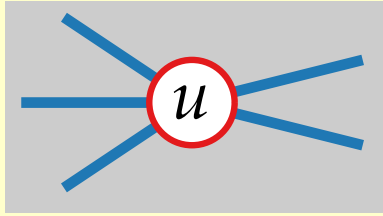
Ein Knoten ist **inzident** zu allen Kanten, deren Endknoten er ist.

Aus Sicht der Kanten: $2 \cdot |E|$

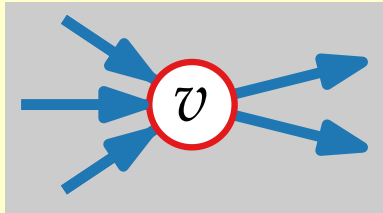
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.

Beweis. Technik des **zweifachen Abzählens**:

Zähle alle Knoten-Kanten-Inzidenzen.

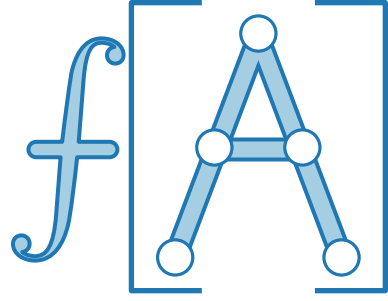
Aus Sicht der Knoten: $\sum_{v \in V} \deg v$

Aus Sicht der Kanten: $2 \cdot |E|$ also gleich!

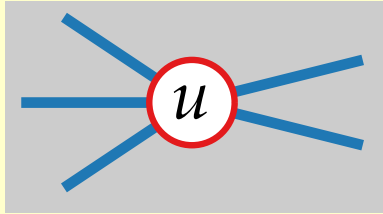
Eine Kante ist **inzident** zu ihren Endknoten.

Ein Knoten ist **inzident** zu allen Kanten, deren Endknoten er ist.

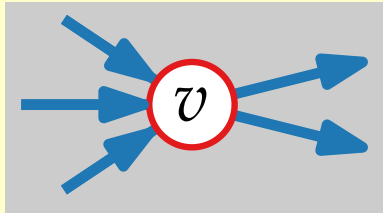
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in \textcolor{red}{V} : (u, v) \in \textcolor{blue}{E}\}|$$

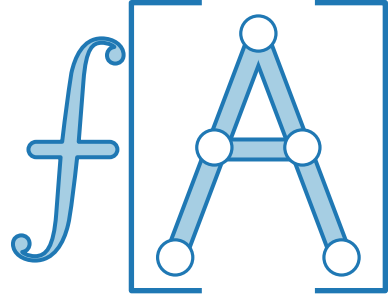
Beob.

Sei $G = (\textcolor{red}{V}, \textcolor{blue}{E})$ ein ungerichteter Graph.

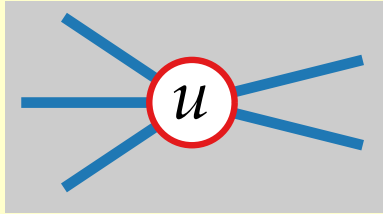
Dann ist die Summe aller Knotengrade $= 2 \cdot |\textcolor{blue}{E}|$.



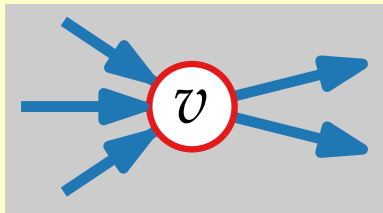
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in \textcolor{red}{V} : (u, v) \in \textcolor{blue}{E}\}|$$

Beob.

Sei $G = (\textcolor{red}{V}, \textcolor{blue}{E})$ ein ungerichteter Graph.

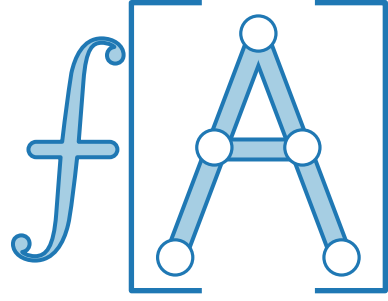
Dann ist die Summe aller Knotengrade $= 2 \cdot |\textcolor{blue}{E}|$.



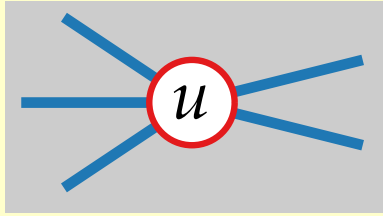
Sätze.

Die Anzahl der Knoten ungeraden Grades ist gerade.

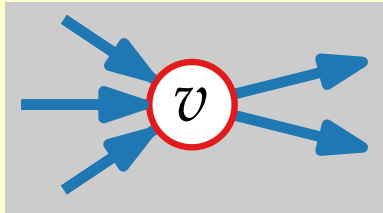
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob. Sei $G = (V, E)$ ein ungerichteter Graph.

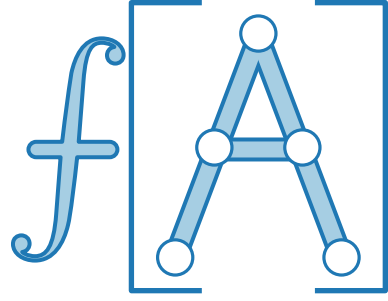
Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



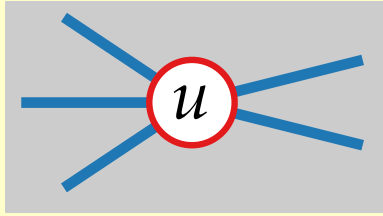
Sätze. Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis. $2 \cdot |E| = \sum_{v \in V} \deg v$

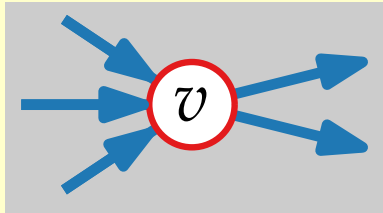
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



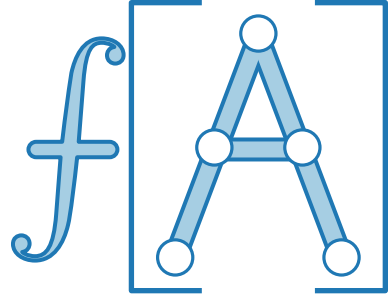
Sätze.

Die Anzahl der Knoten ungeraden Grades ist gerade.

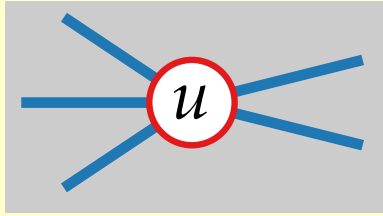
Beweis.

$$2 \cdot |E| = \sum_{v \in V} \deg v = \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$$

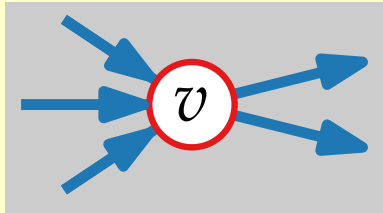
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



Sätze.

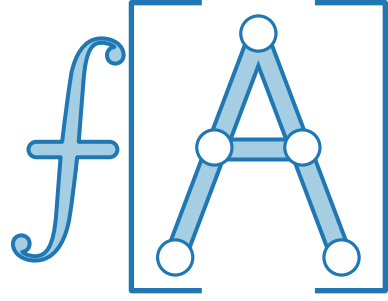
Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis.

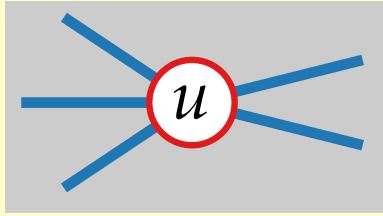
$$2 \cdot |E| = \sum_{v \in V} \deg v = \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$$

gerade!

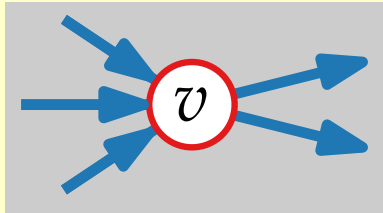
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob. Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



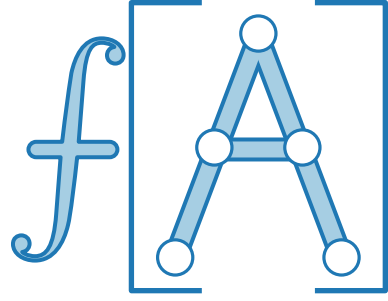
Sätze. Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis. $2 \cdot |E| = \sum_{v \in V} \deg v = \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$

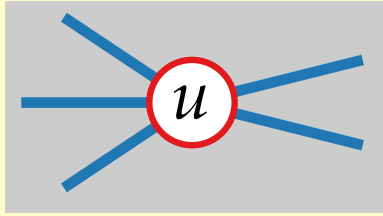
gerade!

gerade!

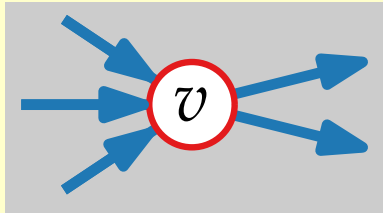
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



Sätze.

Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis.

$$2 \cdot |E| = \sum_{v \in V} \deg v$$

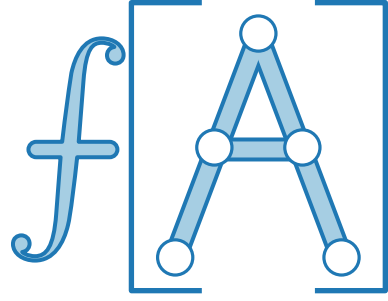
gerade!

gerade!

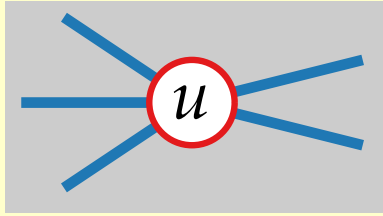
$$= \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$$

gerade!

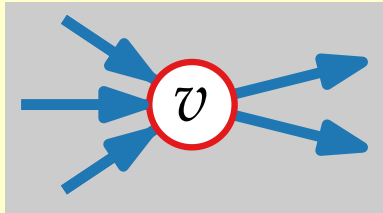
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



Sätze.

Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis.

$$2 \cdot |E| = \sum_{v \in V} \deg v$$

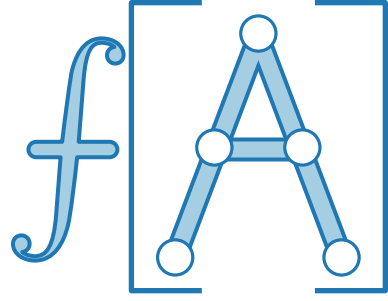
gerade!

gerade!

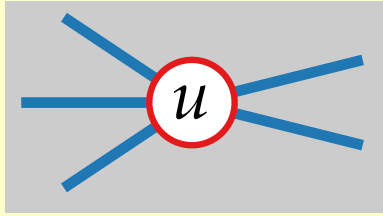
$$= \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$$

gerade!

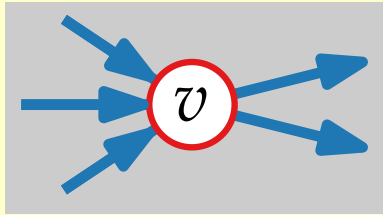
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob. Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



Sätze. Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis. $2 \cdot |E| = \sum_{v \in V} \deg v = \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$

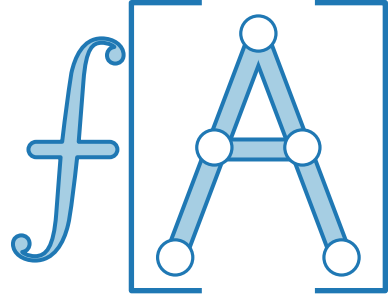
gerade!

gerade!

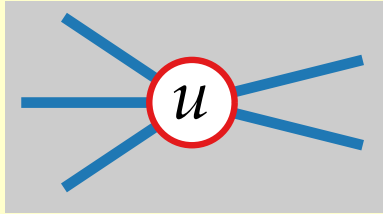
gerade!

$\Rightarrow$ gerade!

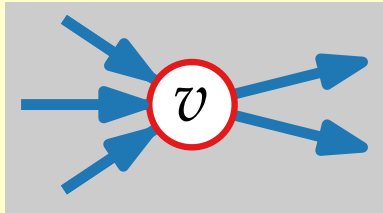
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob.

Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



Sätze.

Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis.

$$2 \cdot |E| = \sum_{v \in V} \deg v = \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$$

gerade!

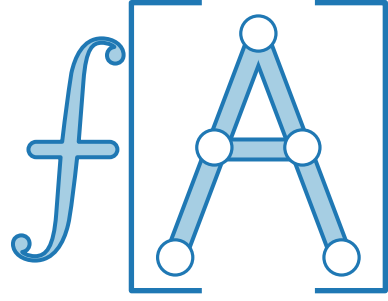
gerade!

gerade!

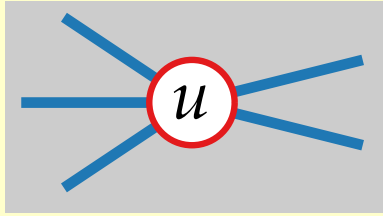
$\Rightarrow$ gerade!

$$\sum_{v \in V_{\text{ung}}} \deg v \text{ gerade} \Rightarrow$$

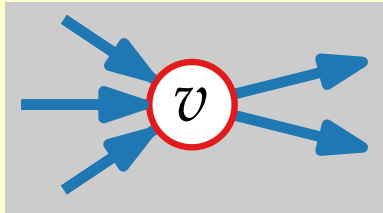
Grad eines Knotens



Def.



$$\deg(u) = |\text{Adj}[u]|$$



$$\text{outdeg}(v) = |\text{Adj}[v]|$$

$$\text{indeg}(v) = |\{u \in V : (u, v) \in E\}|$$

Beob. Sei $G = (V, E)$ ein ungerichteter Graph.

Dann ist die Summe aller Knotengrade $= 2 \cdot |E|$.



Sätze. Die Anzahl der Knoten ungeraden Grades ist gerade.

Beweis. $2 \cdot |E| = \sum_{v \in V} \deg v = \sum_{v \in V_{\text{ger}}} \deg v + \sum_{v \in V_{\text{ung}}} \deg v$

gerade!

gerade!

gerade!

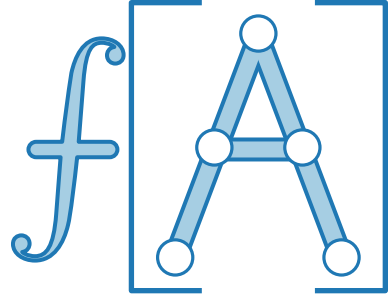
$\Rightarrow$ gerade!

$$\sum_{v \in V_{\text{ung}}} \deg v \text{ gerade} \Rightarrow |V_{\text{ung}}| \text{ ist gerade!}$$



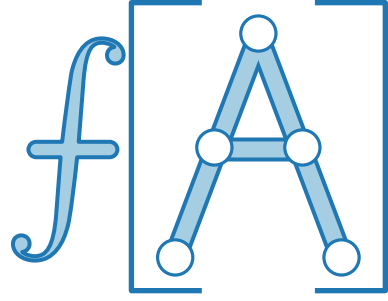
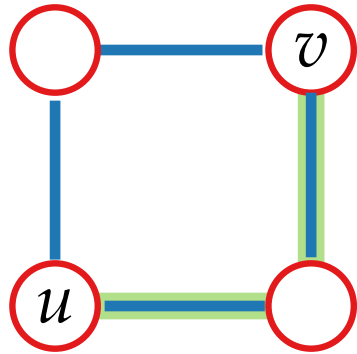
Zusammenhang

Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.



Zusammenhang

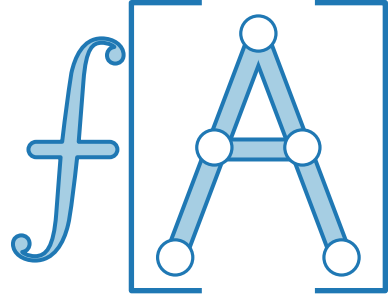
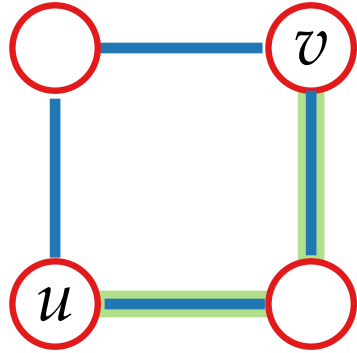
Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.



Zusammenhang

Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

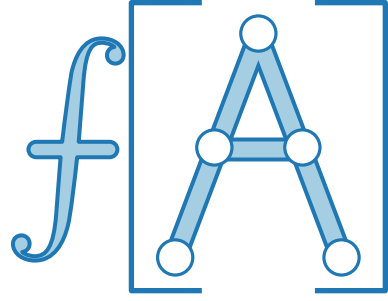
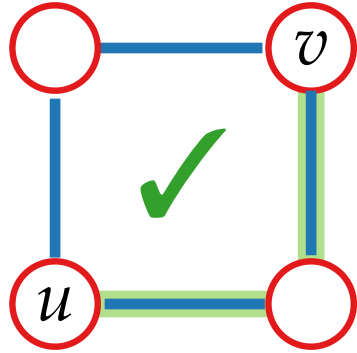
Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.



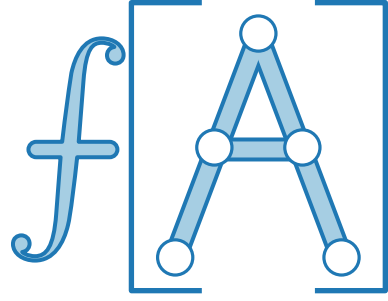
Zusammenhang

Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.

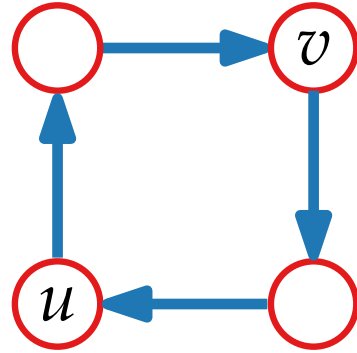
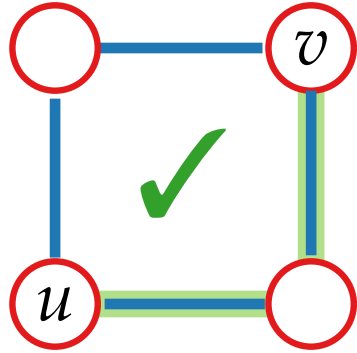


Zusammenhang

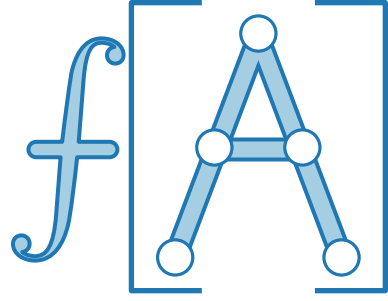


Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.

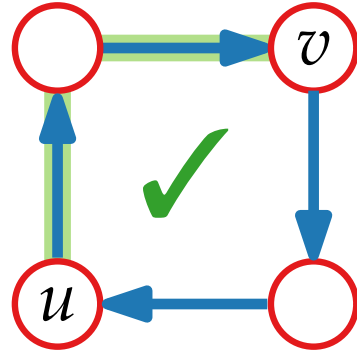
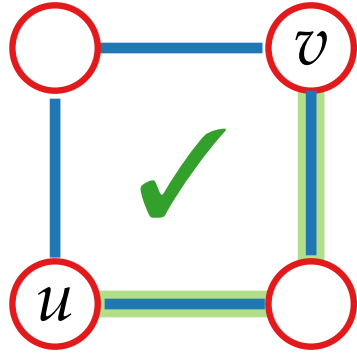


Zusammenhang

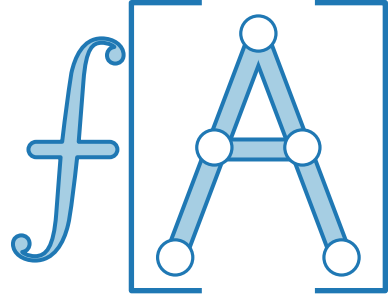


Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.

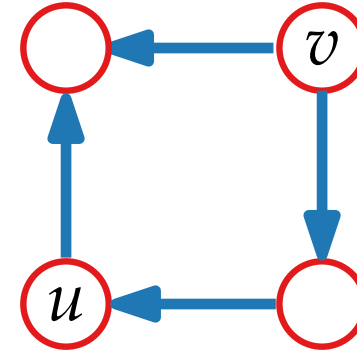
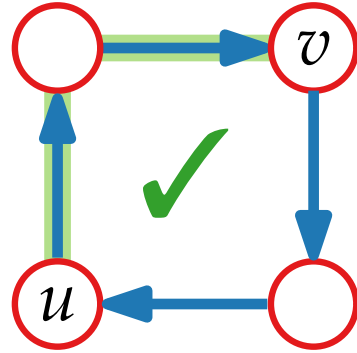
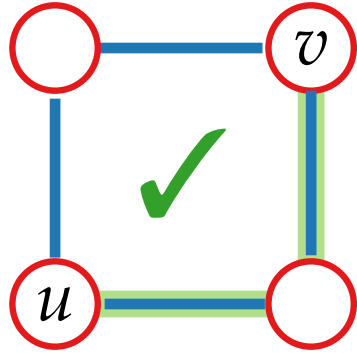


Zusammenhang

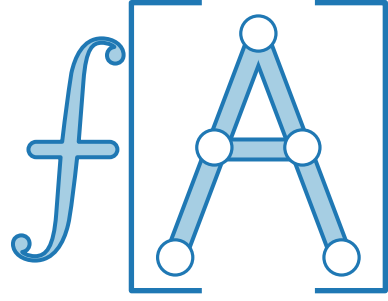


Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.

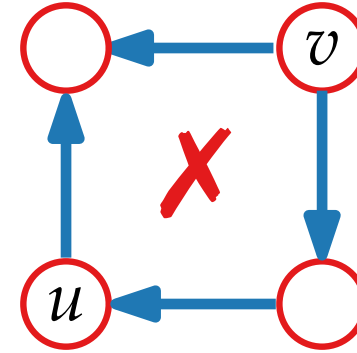
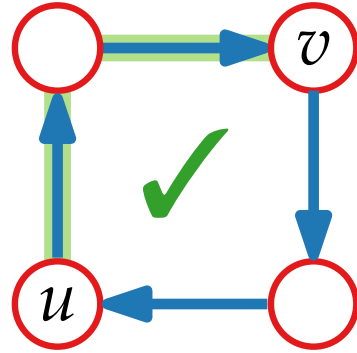
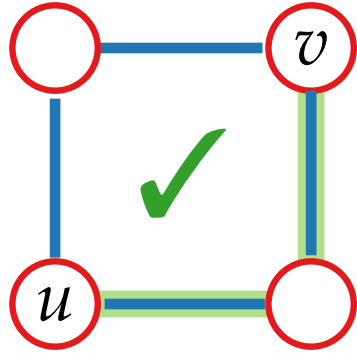


Zusammenhang

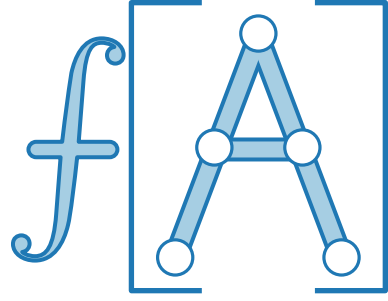


Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.

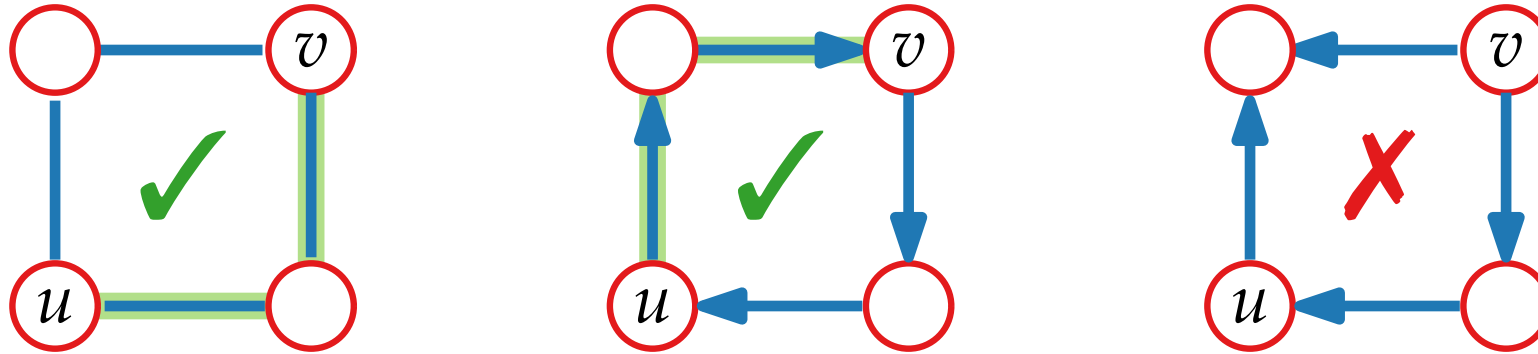


Zusammenhang



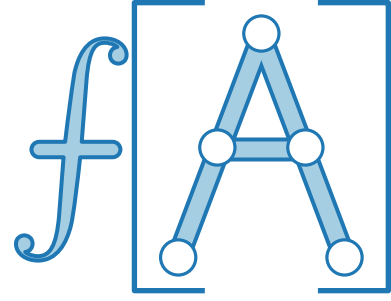
Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.



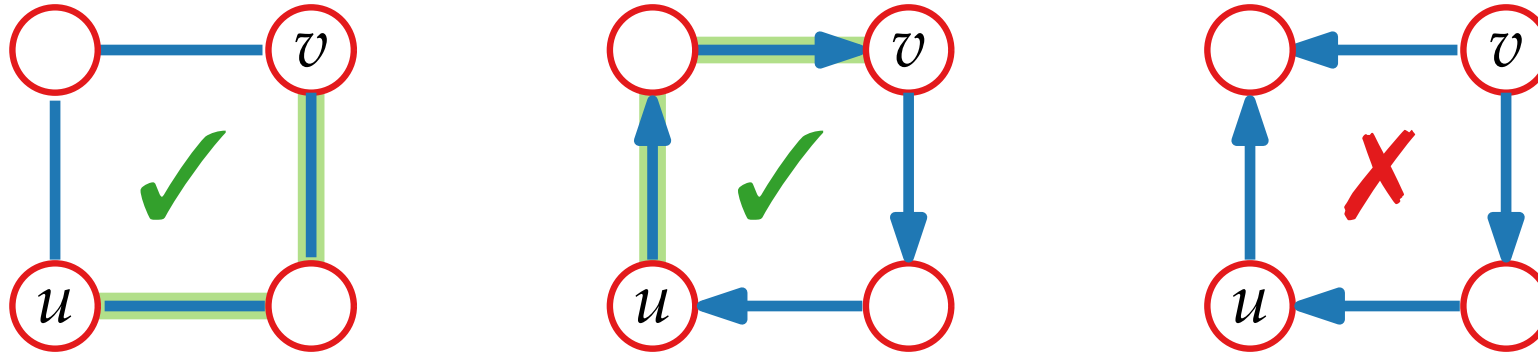
Def. Ein ungerichteter Graph heißt **zusammenhängend**, wenn jedes Knotenpaar voneinander aus erreichbar ist.

Zusammenhang



Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

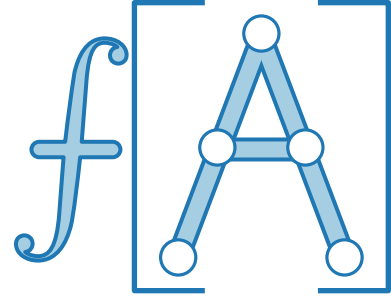
Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.



Def. Ein ungerichteter Graph heißt **zusammenhängend**, wenn jedes Knotenpaar voneinander aus erreichbar ist.

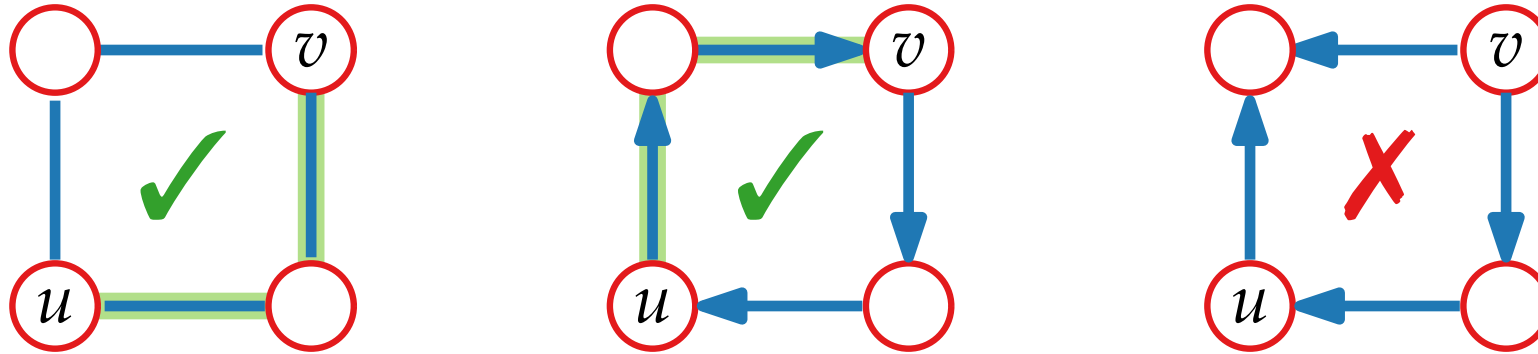
Def. Ein gerichteter Graph heißt **zusammenhängend**, wenn der zugehörige ungerichtete Graph zusammenhängend ist.

Zusammenhang



Def. Ein (un)gerichteter **Pfad** von einem Knoten u zu einem Knoten v ist eine Folge von Kanten, die in u beginnt und in v endet.

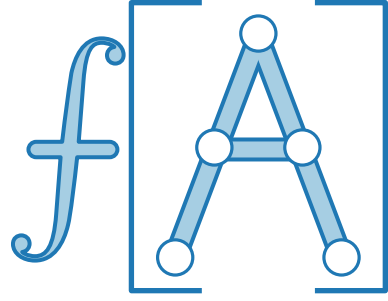
Def. Ein Knoten v ist von einem Knoten u aus **erreichbar**, wenn es einen (un)gerichteten Pfad von u nach v gibt.



Def. Ein ungerichteter Graph heißt **zusammenhängend**, wenn jedes Knotenpaar voneinander aus erreichbar ist.

Def. Ein gerichteter Graph heißt **zusammenhängend**, wenn der zugehörige ungerichtete Graph zusammenhängend ist.

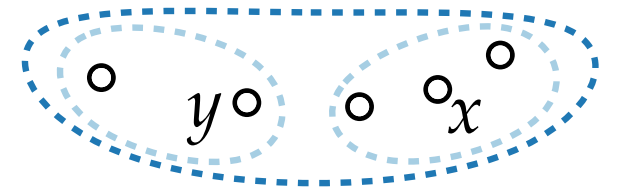
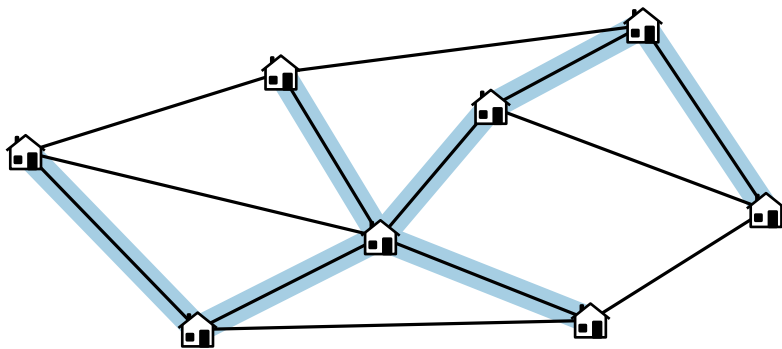
Def. Einen maximalen zusammenhängenden Teilgraph nennt man eine **Zusammenhangskomponente**.



Fortgeschrittene Algorithmen

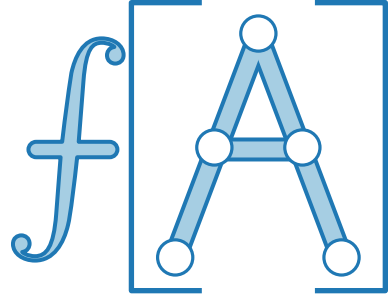
7. Vorlesung Graphalgorithmen I: Minimale Spannbäume

Teil II: Problemstellung



Motivation

Gegeben. Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

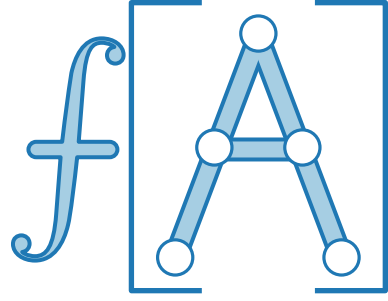


Motivation

ungerichteter, gewichteter Graph

Gegeben.

Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet



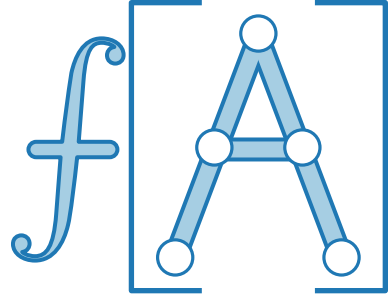
Motivation

ungerichteter, gewichteter Graph

Gegeben.

Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte



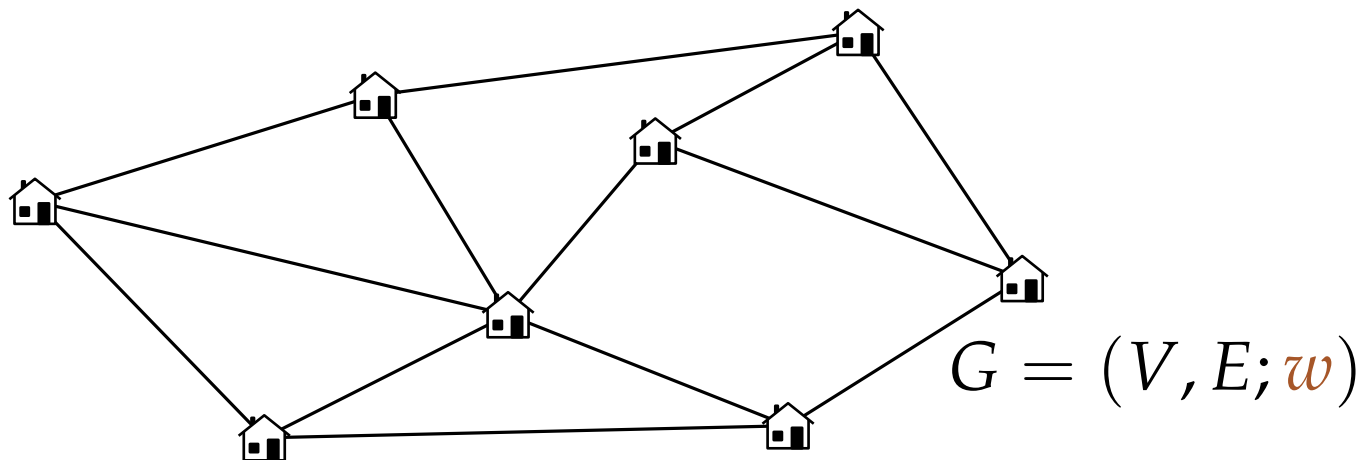
Motivation

ungerichteter, gewichteter Graph

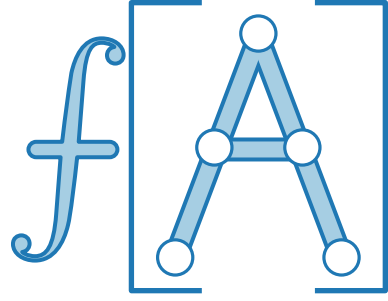
Gegeben.

Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

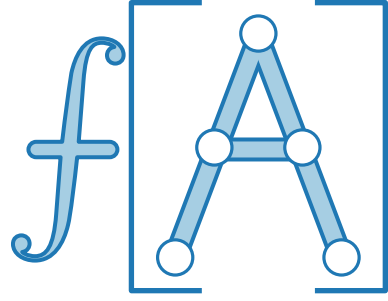
$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte



z.B. mit $w \equiv$ euklid. Abstände



Motivation



ungerichteter, gewichteter Graph

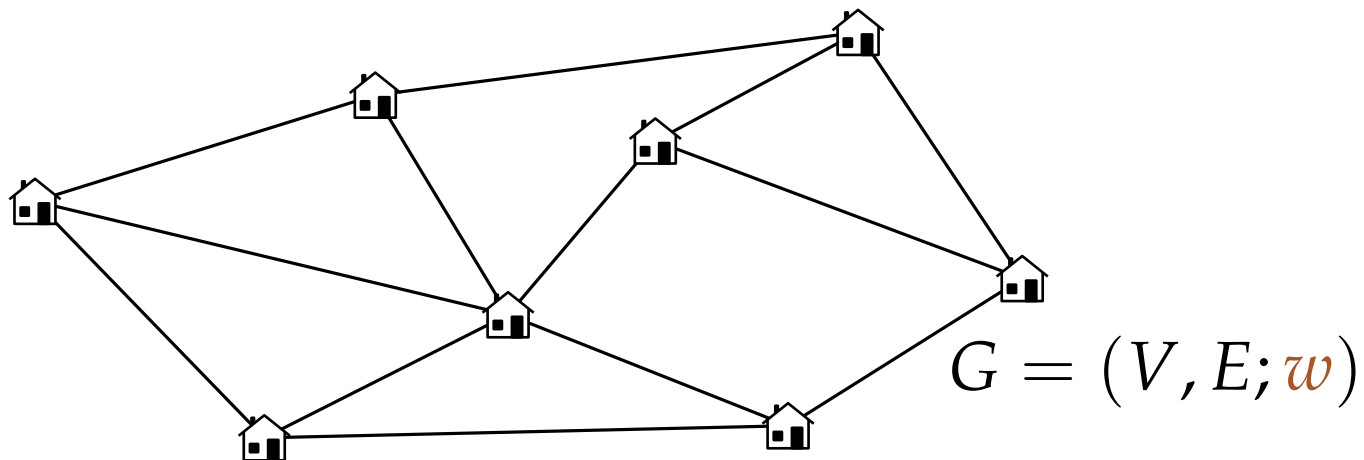
Gegeben.

Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte

Gesucht.

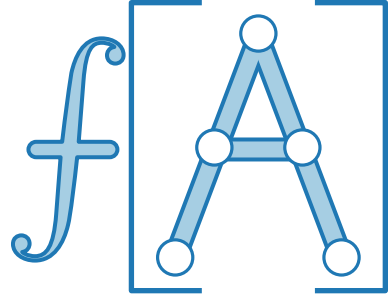
Teilnetz $T = (V, E')$ mit $E' \subseteq E$, so dass



z.B. mit $w \equiv$ euklid. Abstände



Motivation



ungerichteter, gewichteter Graph

Gegeben.

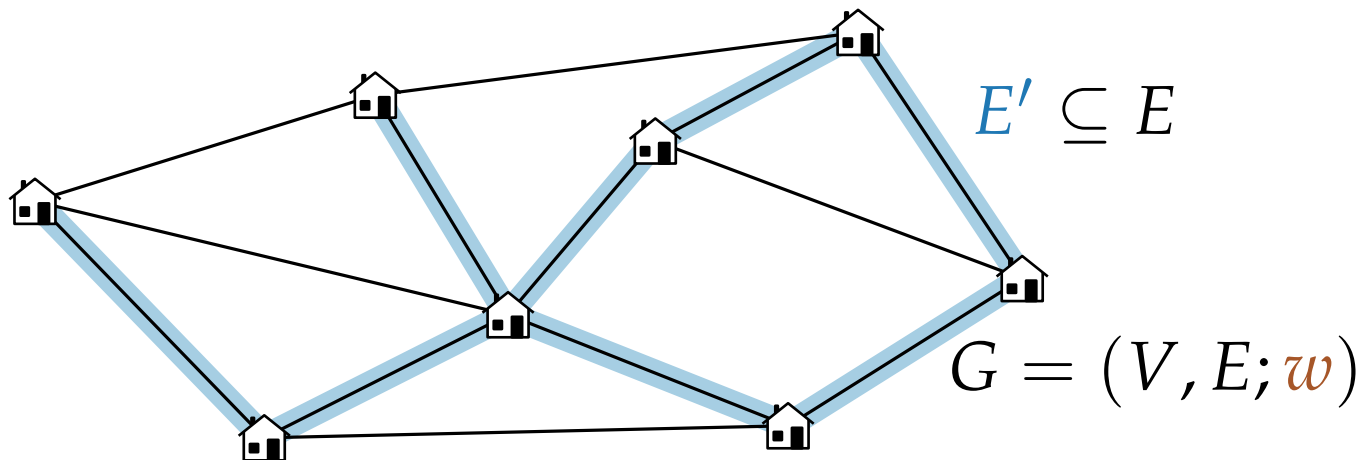
Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte

Gesucht.

Teilnetz $T = (V, E')$ mit $E' \subseteq E$, so dass

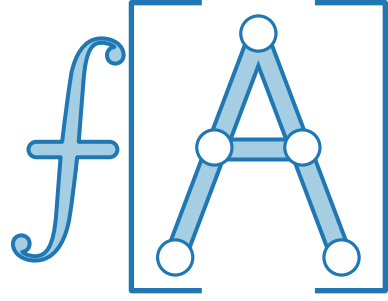
- alle Städte in T erreichbar sind



z.B. mit $w \equiv$ euklid. Abstände



Motivation



ungerichteter, gewichteter Graph

Gegeben.

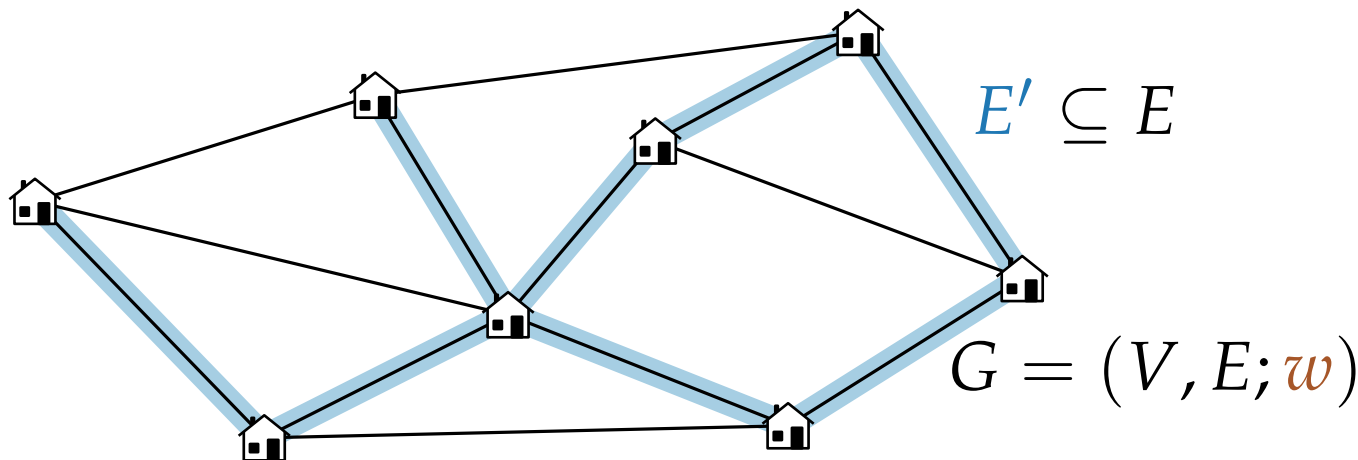
Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte

Gesucht.

Teilnetz $T = (V, E')$ mit $E' \subseteq E$, so dass

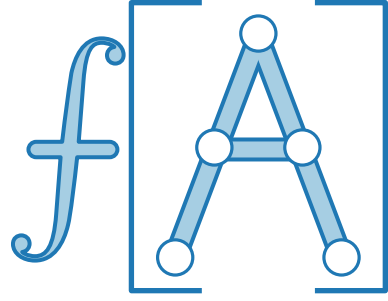
- alle Städte in T erreichbar sind (**T spannt G auf**)



z.B. mit $w \equiv$ euklid. Abstände



Motivation



ungerichteter, gewichteter Graph

Gegeben.

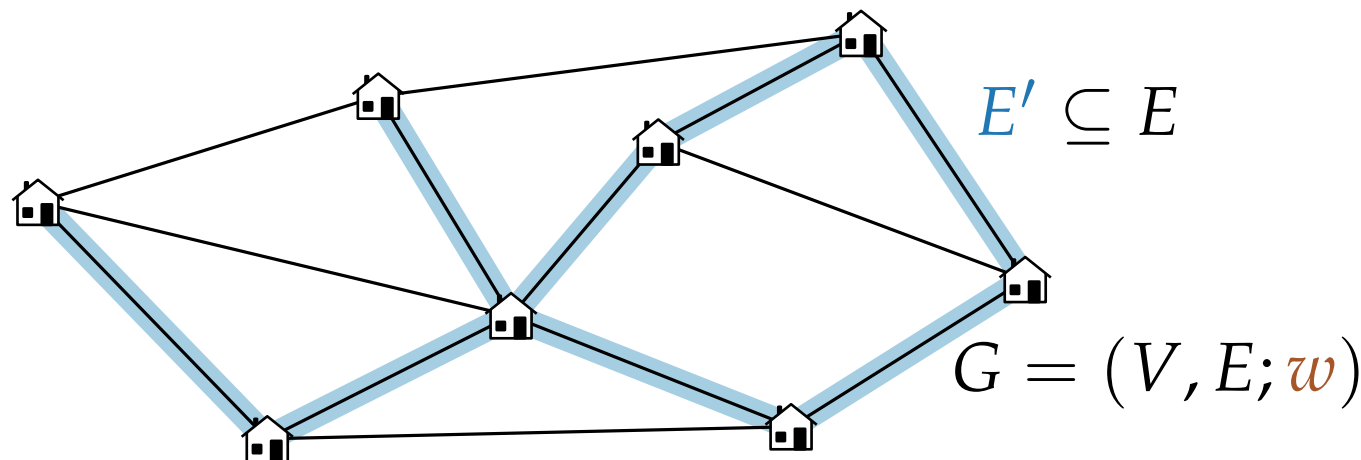
Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte

Gesucht.

Teilnetz $T = (V, E')$ mit $E' \subseteq E$, so dass

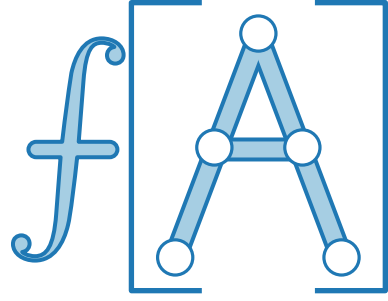
- alle Städte in T erreichbar sind (**T spannt G auf**)
- die „Schneeräumkosten“ $w(E')$ minimal
sind unter allen Teilnetzen, die G aufspannen.



z.B. mit $w \equiv$ euklid. Abstände



Motivation



ungerichteter, gewichteter Graph

Gegeben.

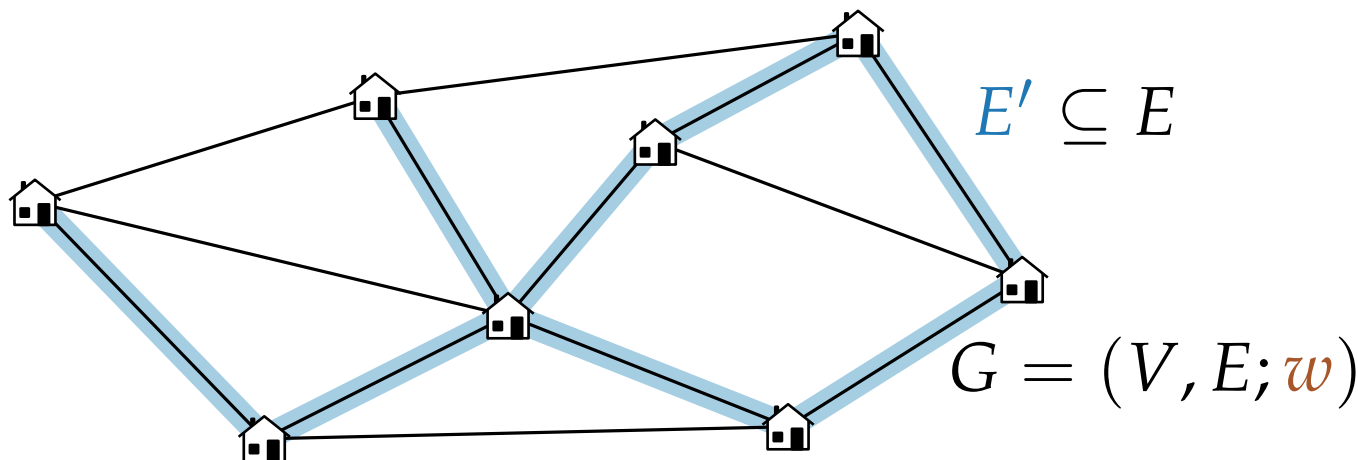
Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte
 $w(E') := \sum_{e \in E'} w(e)$

Gesucht.

Teilnetz $T = (V, E')$ mit $E' \subseteq E$, so dass

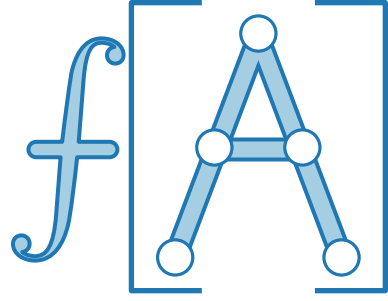
- alle Städte in T erreichbar sind (**T spannt G auf**)
- die „Schneeräumkosten“ $w(E')$ minimal sind unter allen Teilnetzen, die G aufspannen.



z.B. mit $w \equiv$ euklid. Abstände



Motivation



ungerichteter, gewichteter Graph

Gegeben.

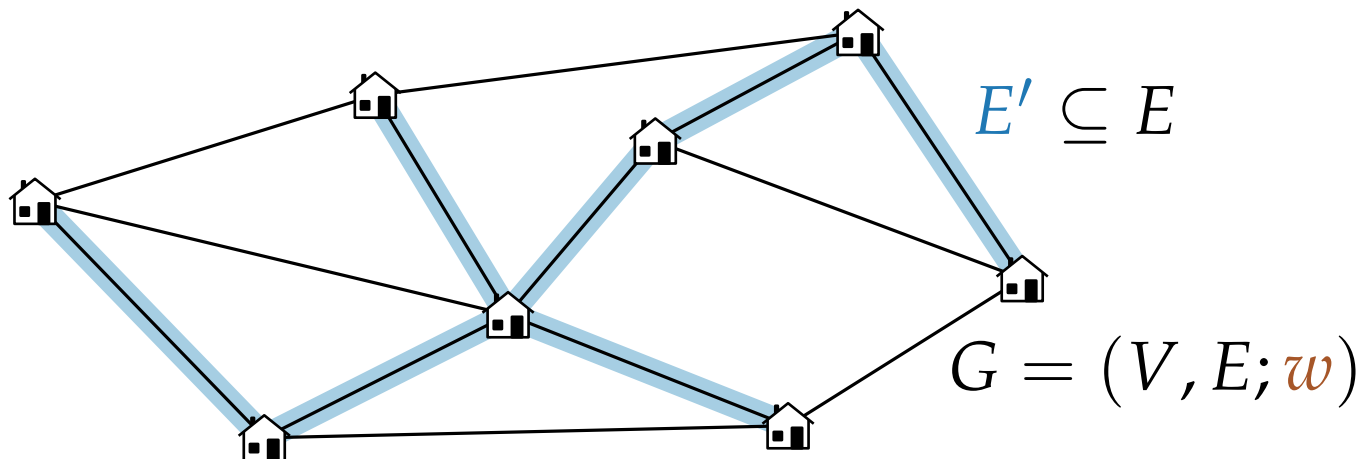
Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte
 $w(E') := \sum_{e \in E'} w(e)$

Gesucht.

Teilnetz $T = (V, E')$ mit $E' \subseteq E$, so dass

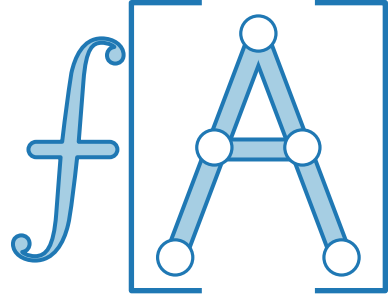
- alle Städte in T erreichbar sind (**T spannt G auf**)
- die „Schneeräumkosten“ $w(E')$ minimal sind unter allen Teilnetzen, die G aufspannen.



z.B. mit $w \equiv$ euklid. Abstände



Motivation



ungerichteter, gewichteter Graph

Gegeben.

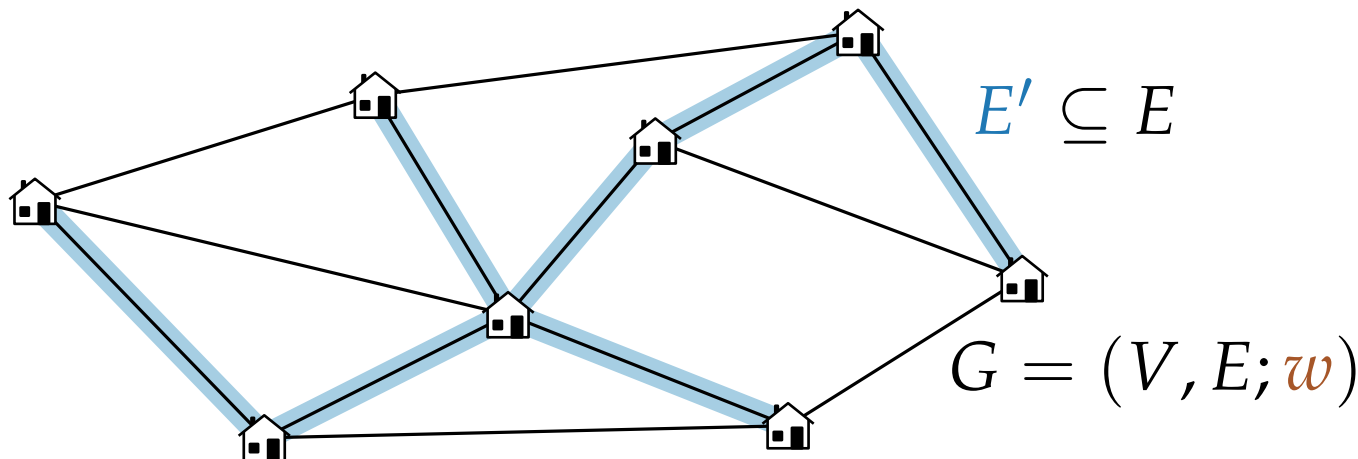
Zusammenhängendes Straßennetz $G = (V, E; w)$,
das eine Menge V von n Städten verbindet

$w: E \rightarrow \mathbb{R}_{>0}$ Kantengewichte
 $w(E') := \sum_{e \in E'} w(e)$

Gesucht.

Teilnetz $T = (V, E')$ mit $E' \subseteq E$, so dass

- alle Städte in T erreichbar sind (**T spannt G auf**)
- die „Schneeräumkosten“ $w(E')$ minimal sind unter allen Teilnetzen, die G aufspannen.

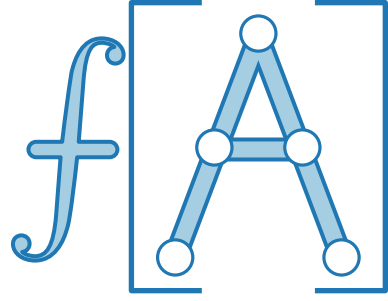
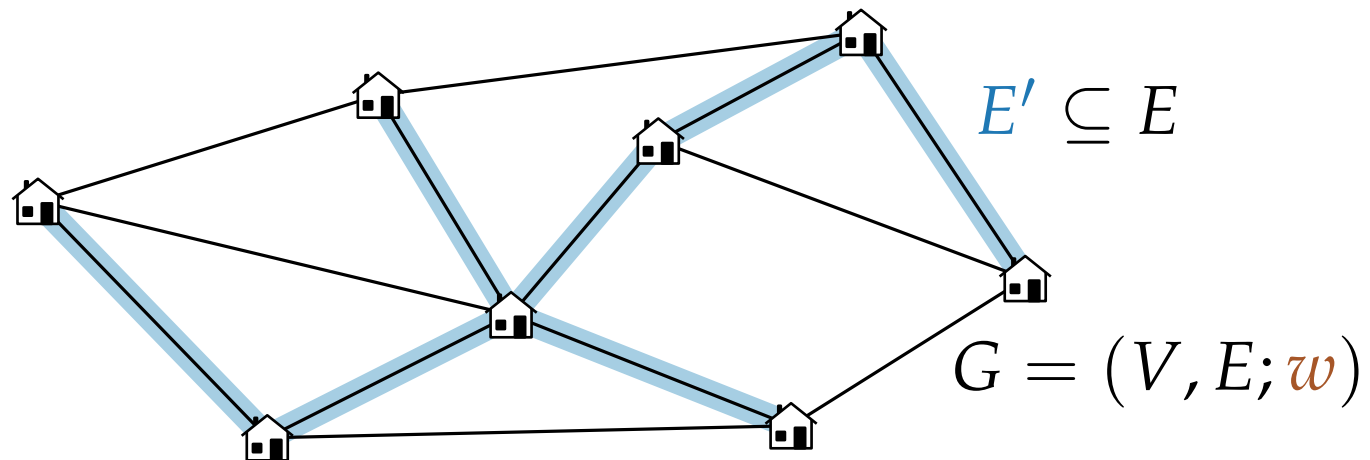


z.B. mit $w \equiv$ euklid. Abstände



Minimaler Spannbaum

Wegen der Minimalität von $w(E')$ gilt:

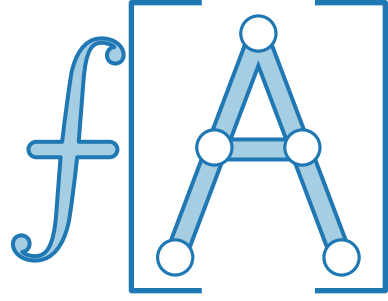
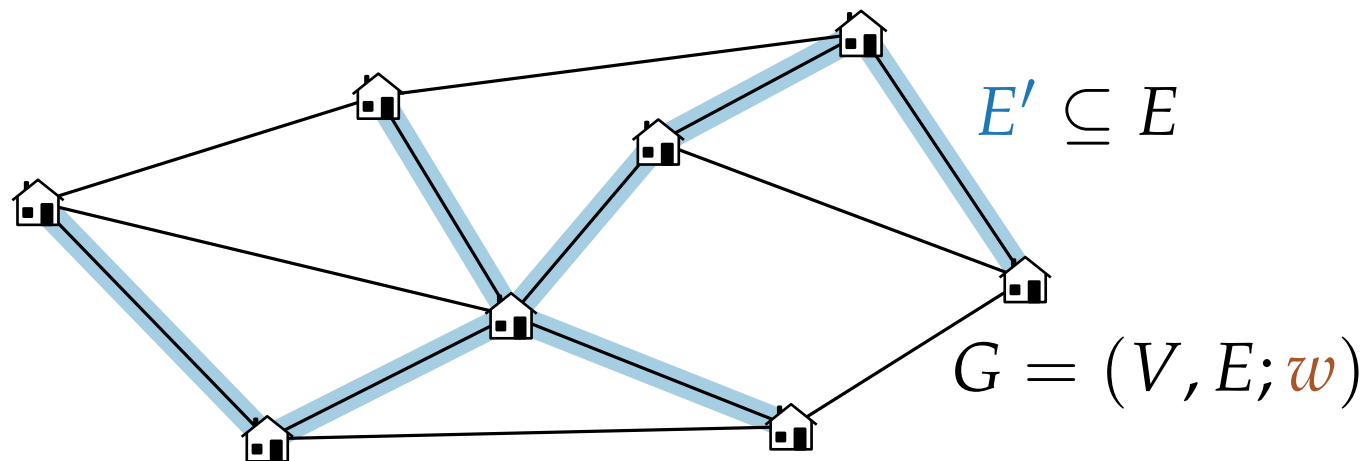


Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brunn



Minimaler Spannbaum

Wegen der Minimalität von $w(E')$ gilt:
 T hat keine Kreise



Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brunn

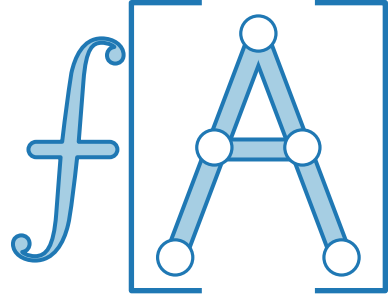


Minimaler Spannbaum

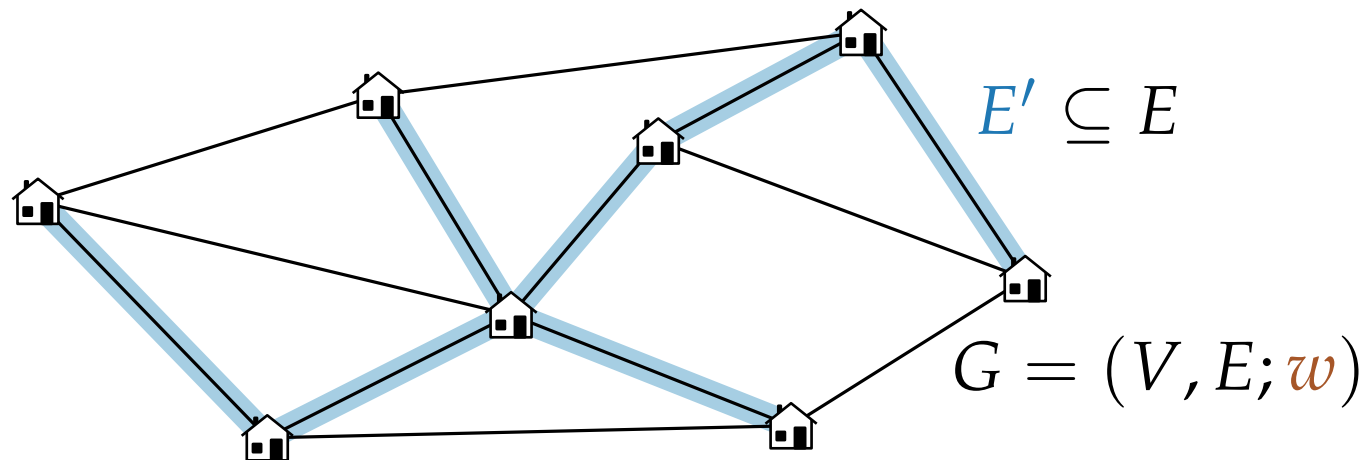
Wegen der Minimalität von $w(E')$ gilt:

T hat keine Kreise

$\Rightarrow T$ ist ein Wald



Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brunn



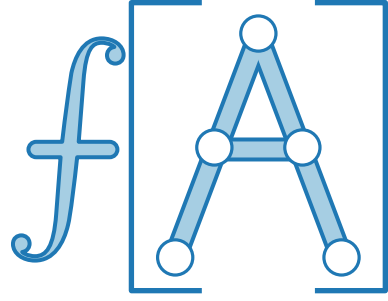
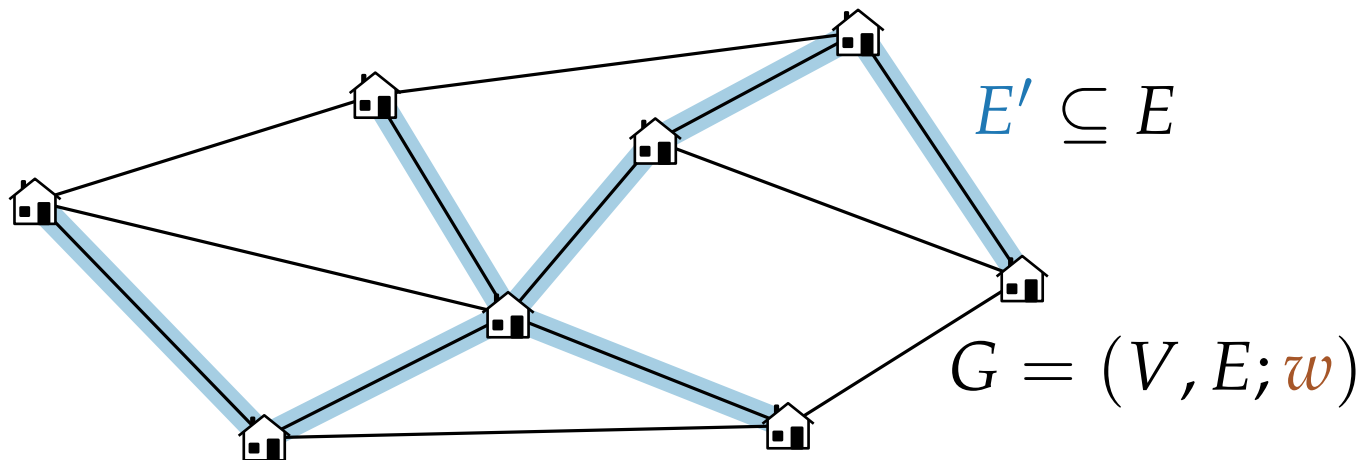
Minimaler Spannbaum

Wegen der Minimalität von $w(E')$ gilt:

T hat keine Kreise

$\Rightarrow T$ ist ein Wald

T „erbt“ Zusammenhang von G



Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brunn

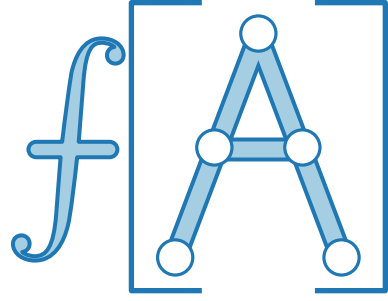


Minimaler Spannbaum

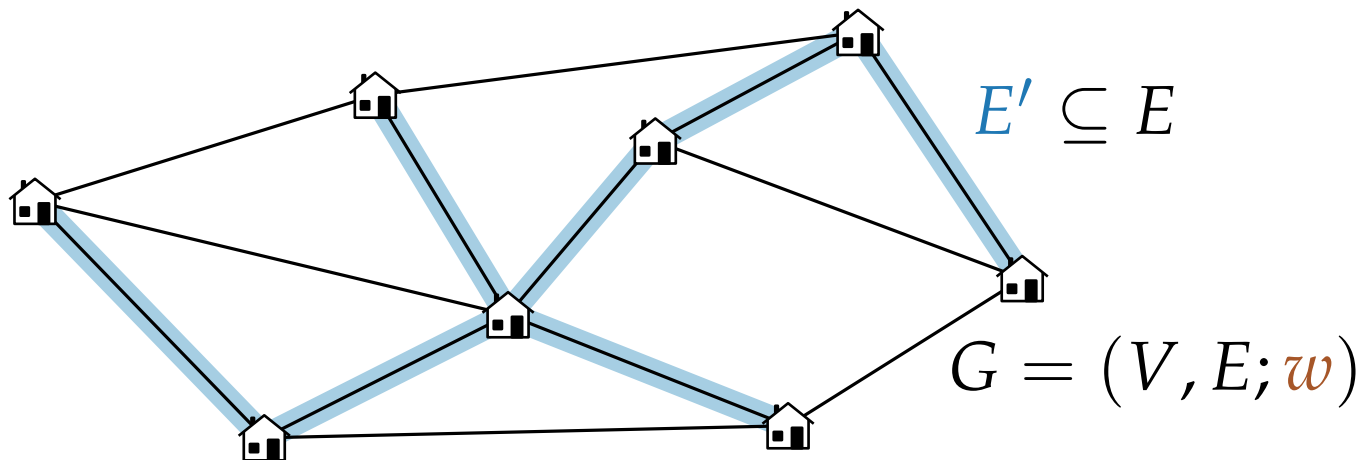
Wegen der Minimalität von $w(E')$ gilt:

T hat keine Kreise $\Rightarrow T$ ist ein Wald

T „erbt“ Zusammenhang von $G \Rightarrow T$ ist ein Baum



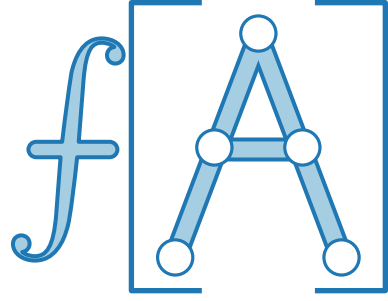
Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brunn



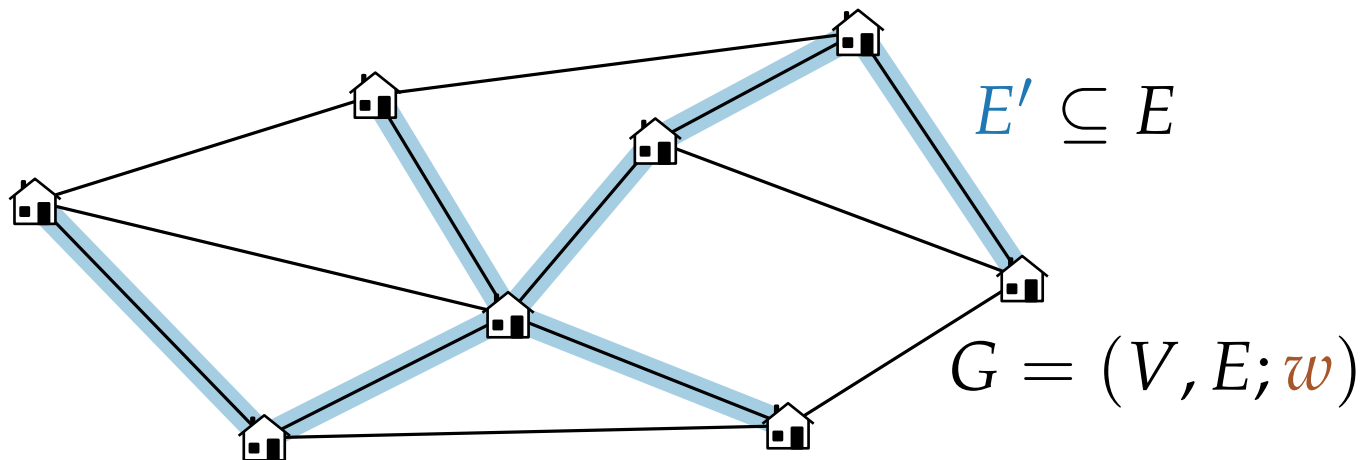
Minimaler Spannbaum

Wegen der Minimalität von $w(E')$ gilt:

- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von $G \Rightarrow T$ ist ein Baum
- T spannt G auf



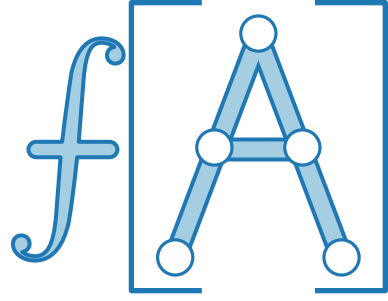
Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn



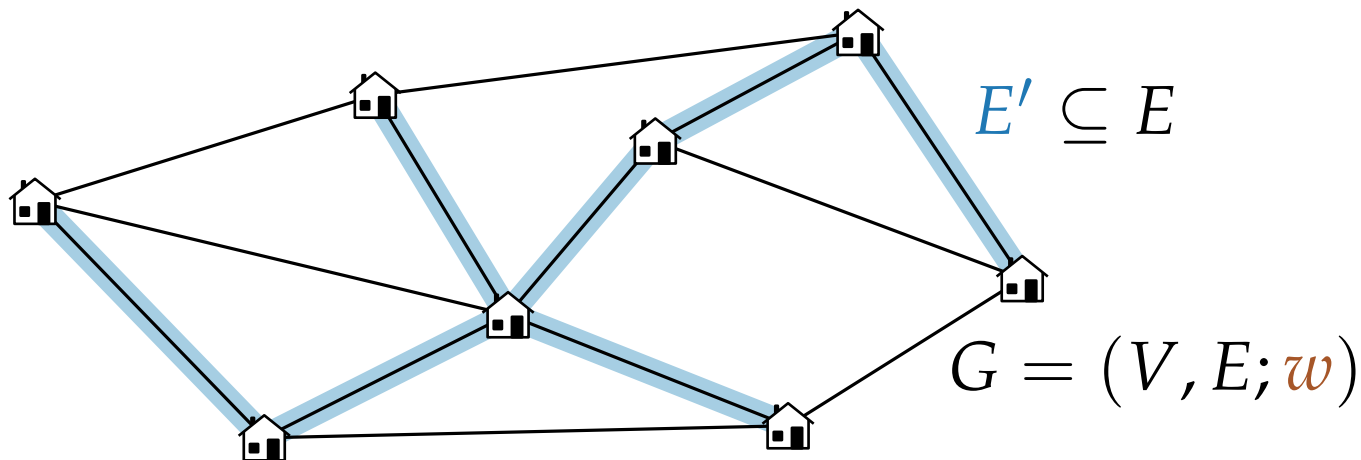
Minimaler Spannbaum

Wegen der Minimalität von $w(E')$ gilt:

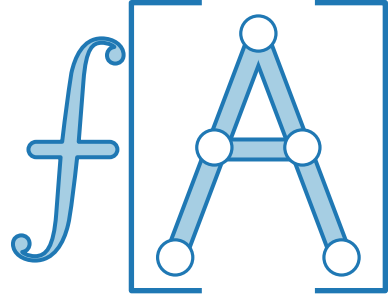
- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von G $\Rightarrow T$ ist ein Baum
- T spannt G auf $\Rightarrow T$ ist Spannbaum von G



Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn



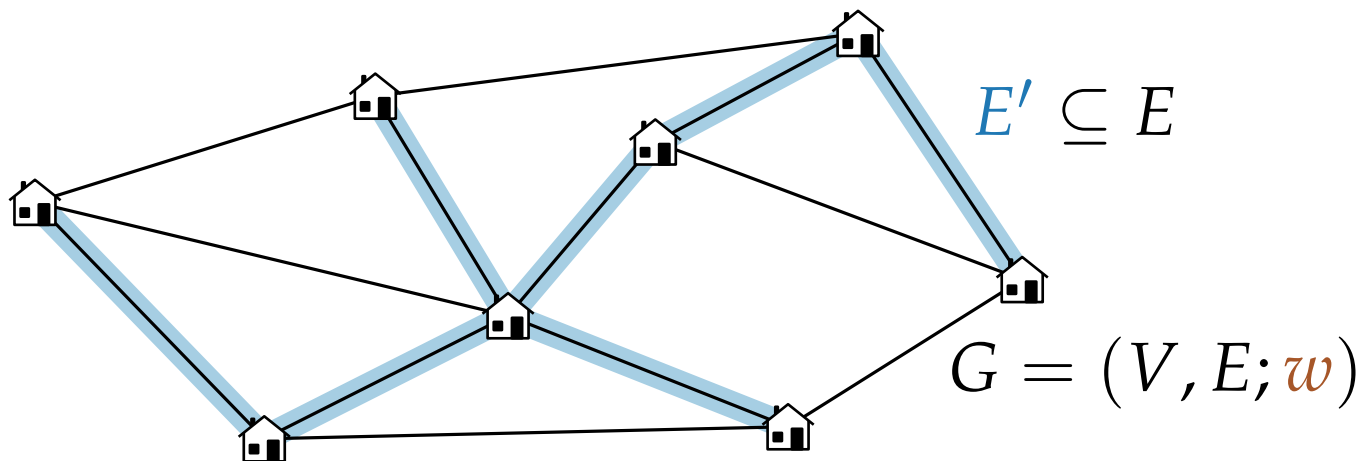
Minimaler Spannbaum



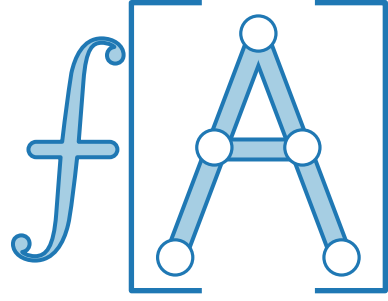
Wegen der Minimalität von $w(E')$ gilt:

- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von G $\Rightarrow T$ ist ein Baum
- T spannt G auf $\Rightarrow T$ ist Spannbaum von G
- T hat minimales Gewicht unter *allen* Spannbäumen von G .

Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn



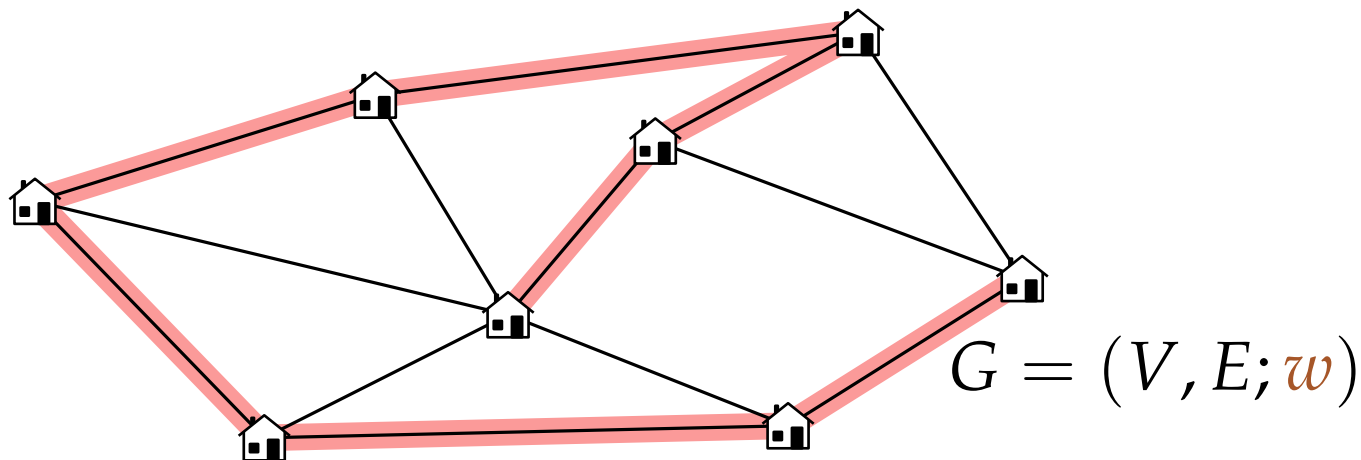
Minimaler Spannbaum



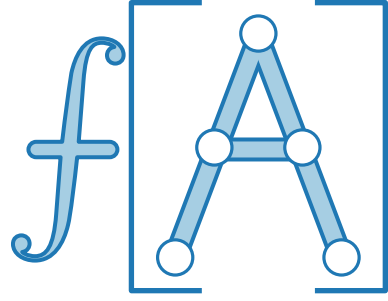
Wegen der Minimalität von $w(E')$ gilt:

- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von G $\Rightarrow T$ ist ein Baum
- T spannt G auf $\Rightarrow T$ ist Spannbaum von G
- T hat minimales Gewicht unter *allen* Spannbäumen von G .

Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn



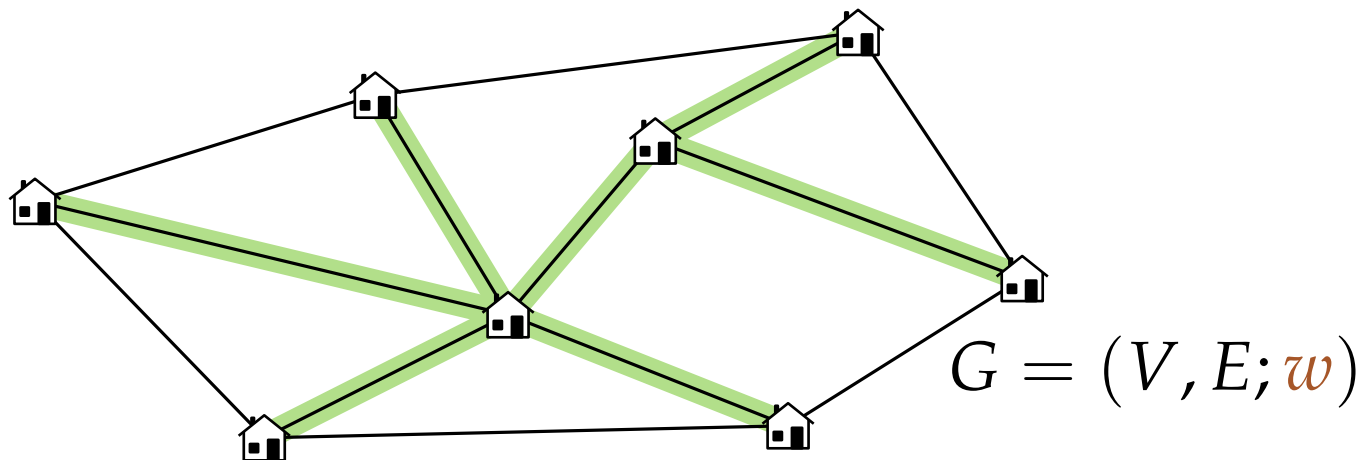
Minimaler Spannbaum



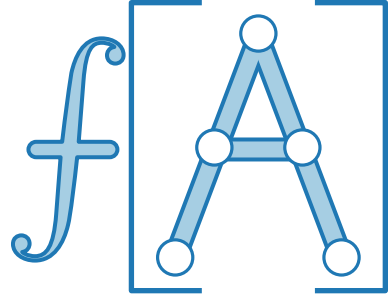
Wegen der Minimalität von $w(E')$ gilt:

- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von G $\Rightarrow T$ ist ein Baum
- T spannt G auf $\Rightarrow T$ ist Spannbaum von G
- T hat minimales Gewicht unter *allen* Spannbäumen von G .

Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn



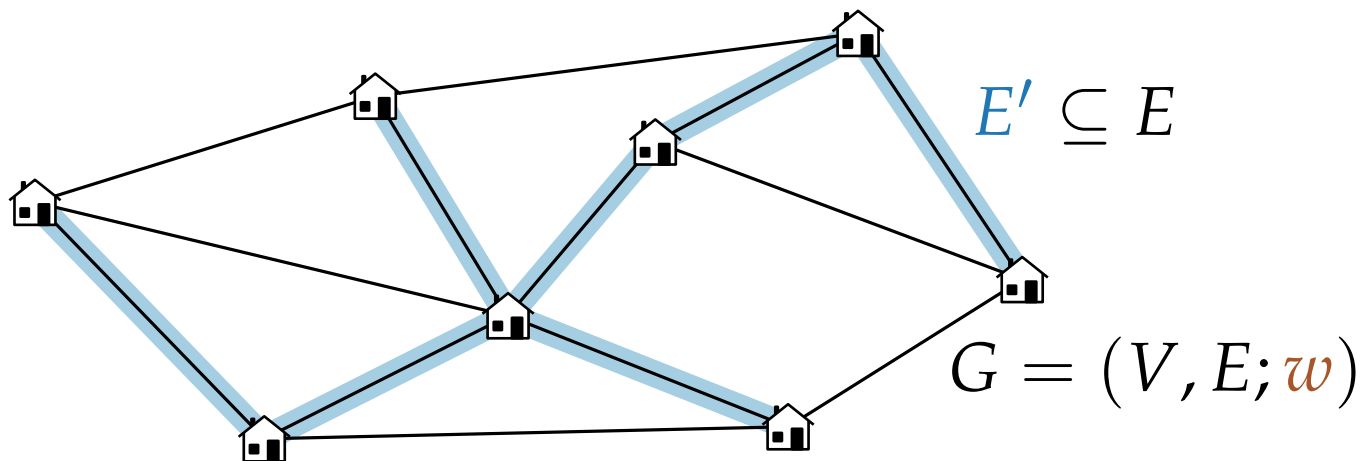
Minimaler Spannbaum



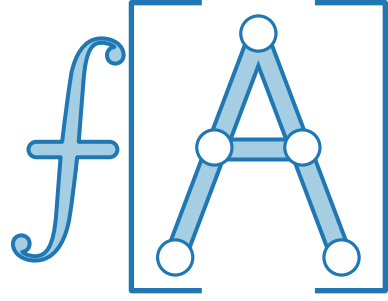
Wegen der Minimalität von $w(E')$ gilt:

- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von G $\Rightarrow T$ ist ein Baum
- T spannt G auf $\Rightarrow T$ ist Spannbaum von G
- T hat minimales Gewicht unter *allen* Spannbäumen von G .
- Wir nennen T kurz **minimalen Spannbaum (MSB)** von G .

Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn



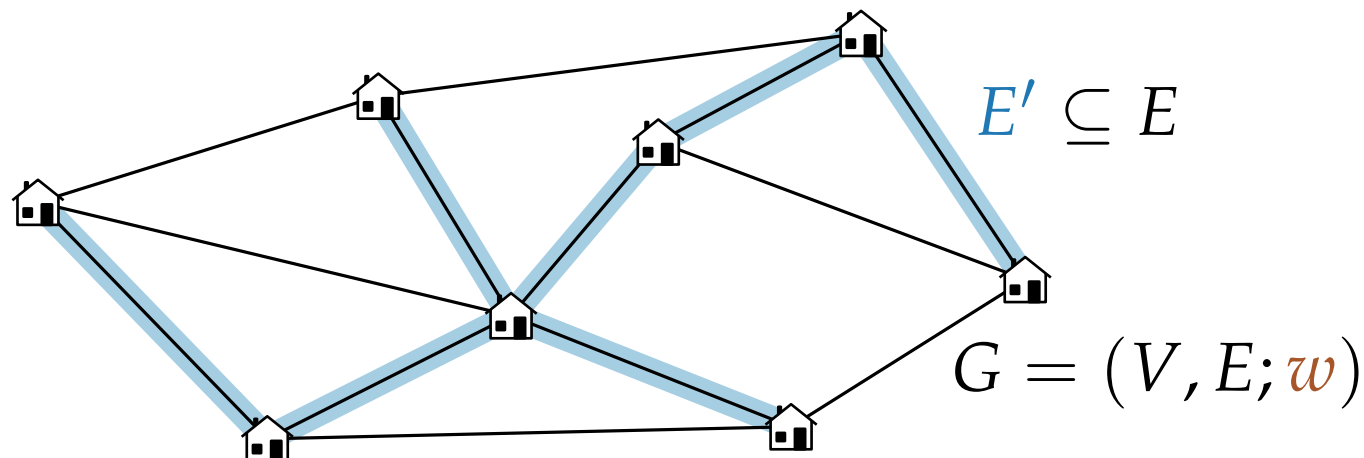
Minimaler Spannbaum



Wegen der Minimalität von $w(E')$ gilt:

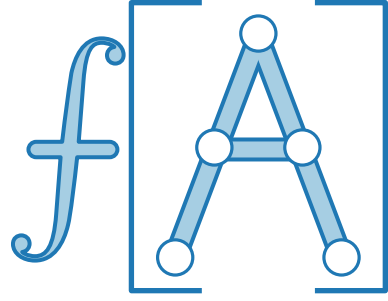
- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von G $\Rightarrow T$ ist ein Baum
- T spannt G auf $\Rightarrow T$ ist Spannbaum von G
- T hat minimales Gewicht unter *allen* Spannbäumen von G .
- Wir nennen T kurz **minimalen Spannbaum (MSB)** von G .

Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn



Beob. $|E'| = ?$

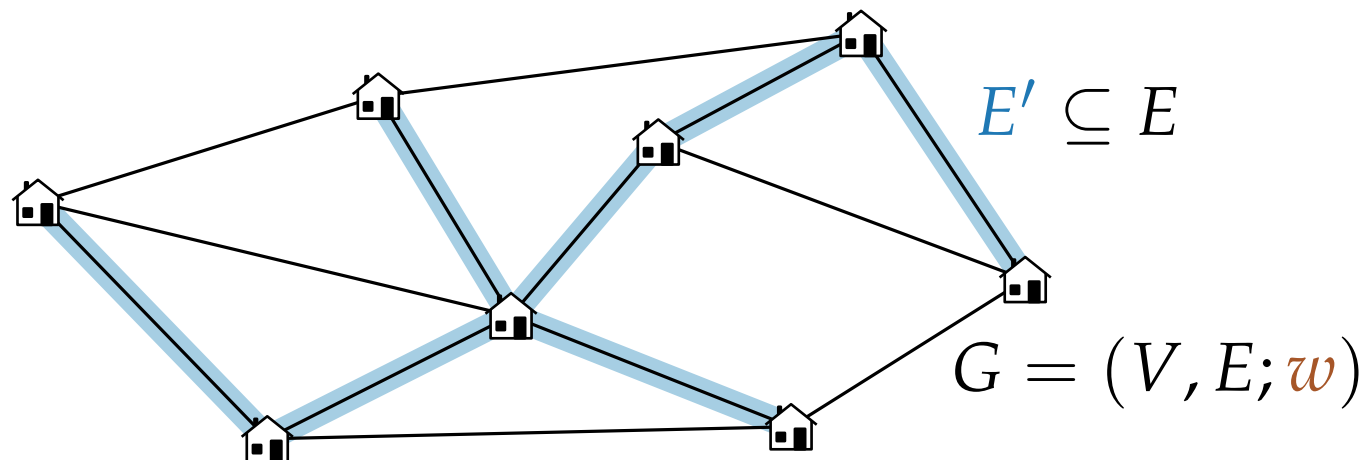
Minimaler Spannbaum



Wegen der Minimalität von $w(E')$ gilt:

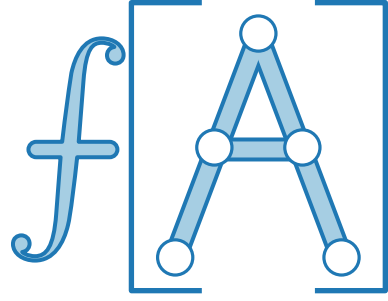
- T hat keine Kreise $\Rightarrow T$ ist ein Wald
- T „erbt“ Zusammenhang von G $\Rightarrow T$ ist ein Baum
- T spannt G auf $\Rightarrow T$ ist Spannbaum von G
- T hat minimales Gewicht unter *allen* Spannbäumen von G .
- Wir nennen T kurz **minimalen Spannbaum (MSB)** von G .

Otakar Borůvka
*1899 Ostroh, Mähren
† 1995 Brünn

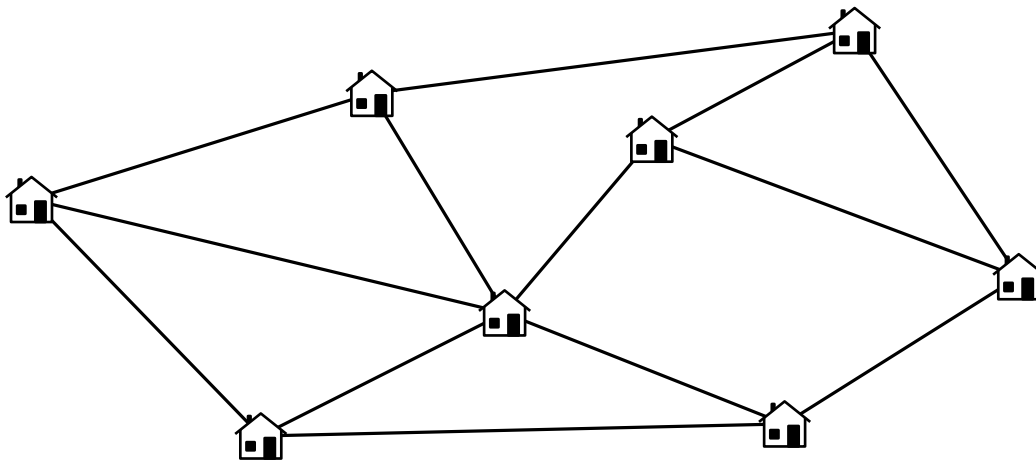


Beob. $|E'| = |V| - 1$

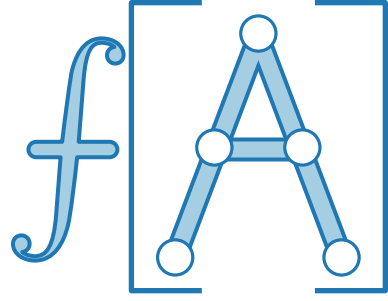
Schnitte



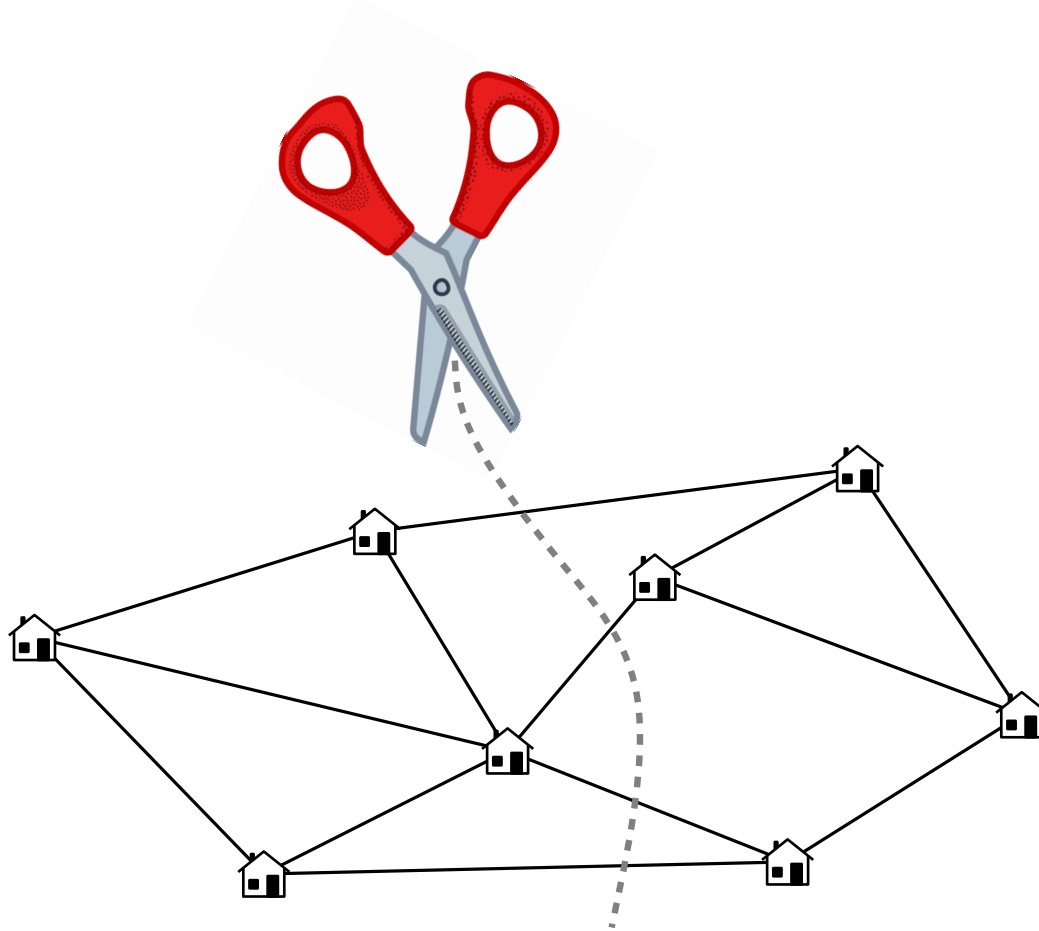
Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.



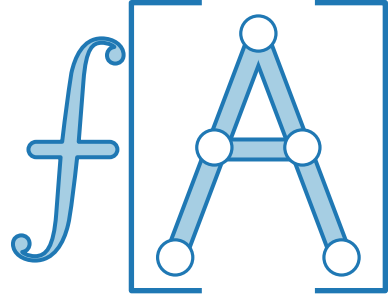
Schnitte



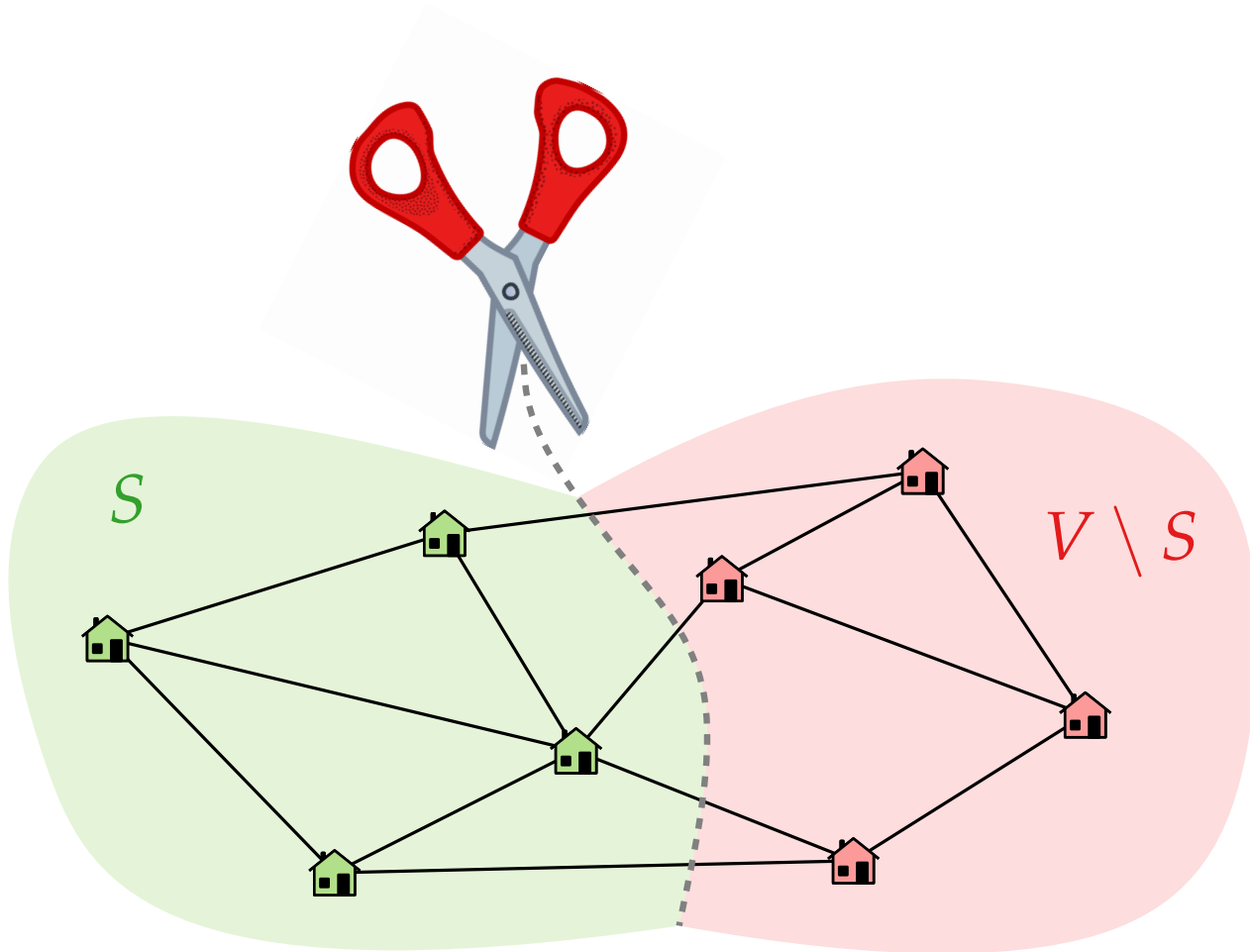
Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.



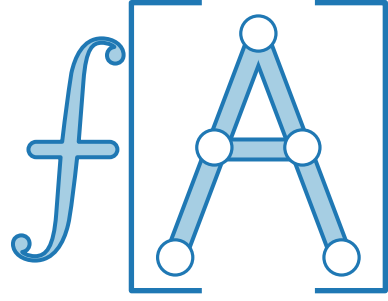
Schnitte



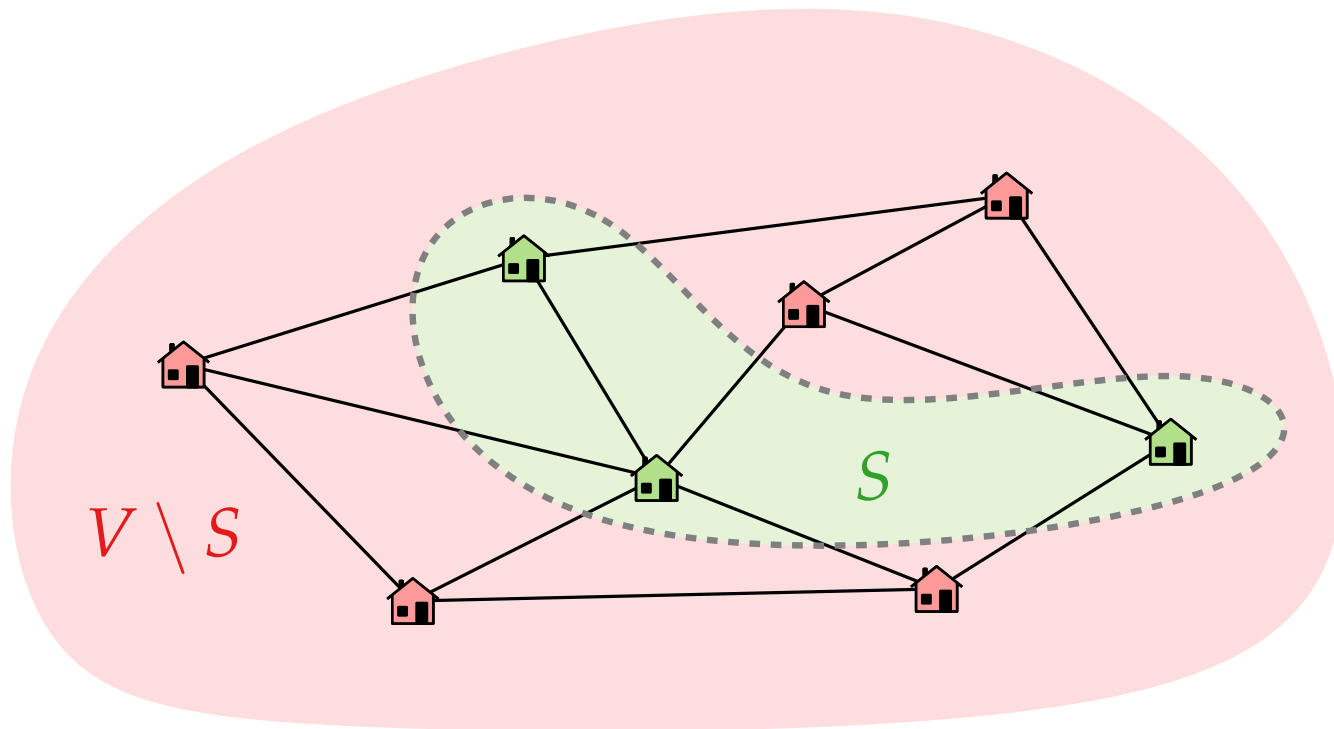
Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.



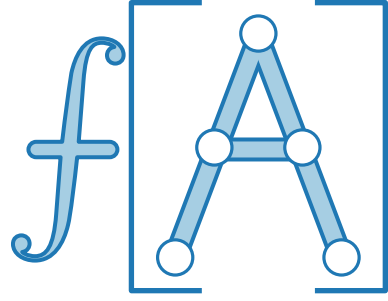
Schnitte



Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.

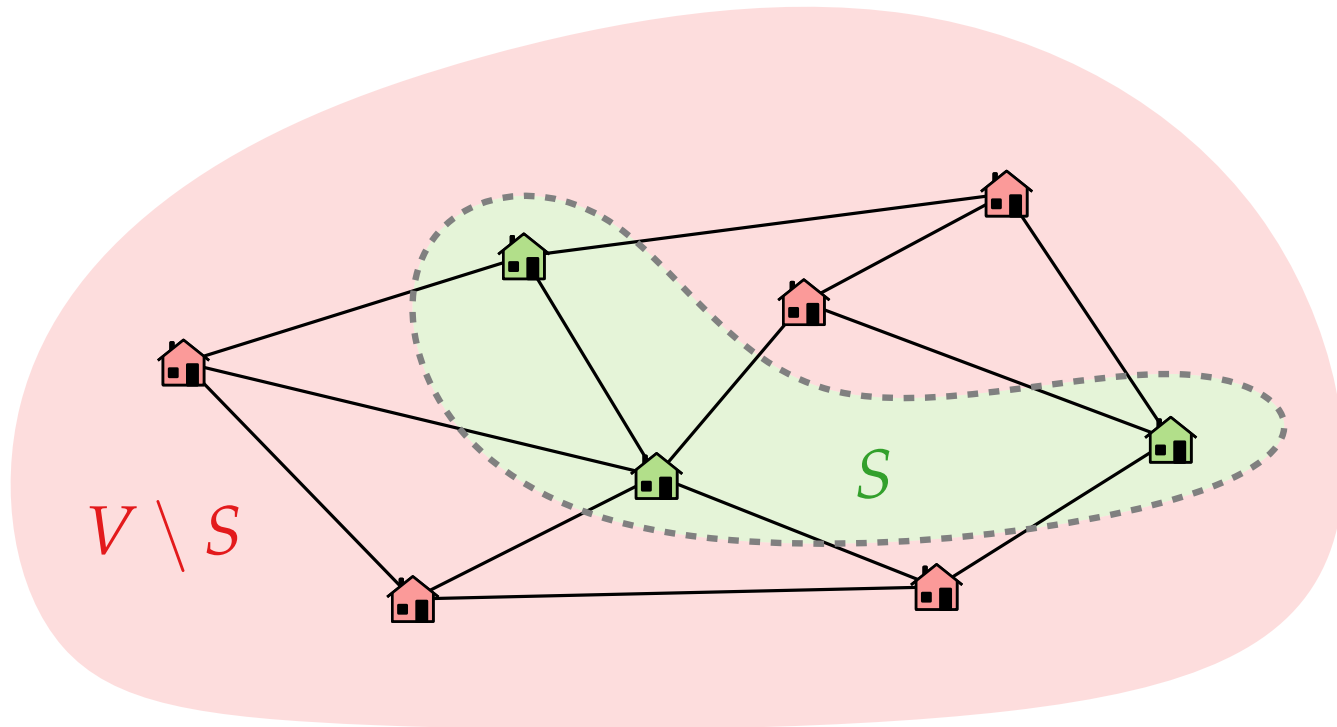


Schnitte

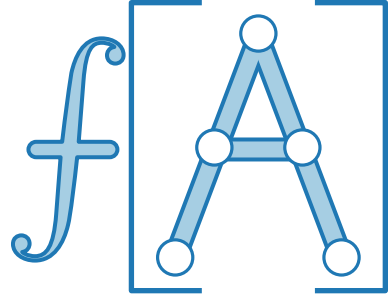


Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.

Eine Kante uv **kreuzt** $(S, V \setminus S)$, wenn $u \in S$ und $v \in V \setminus S$ (oder andersherum).

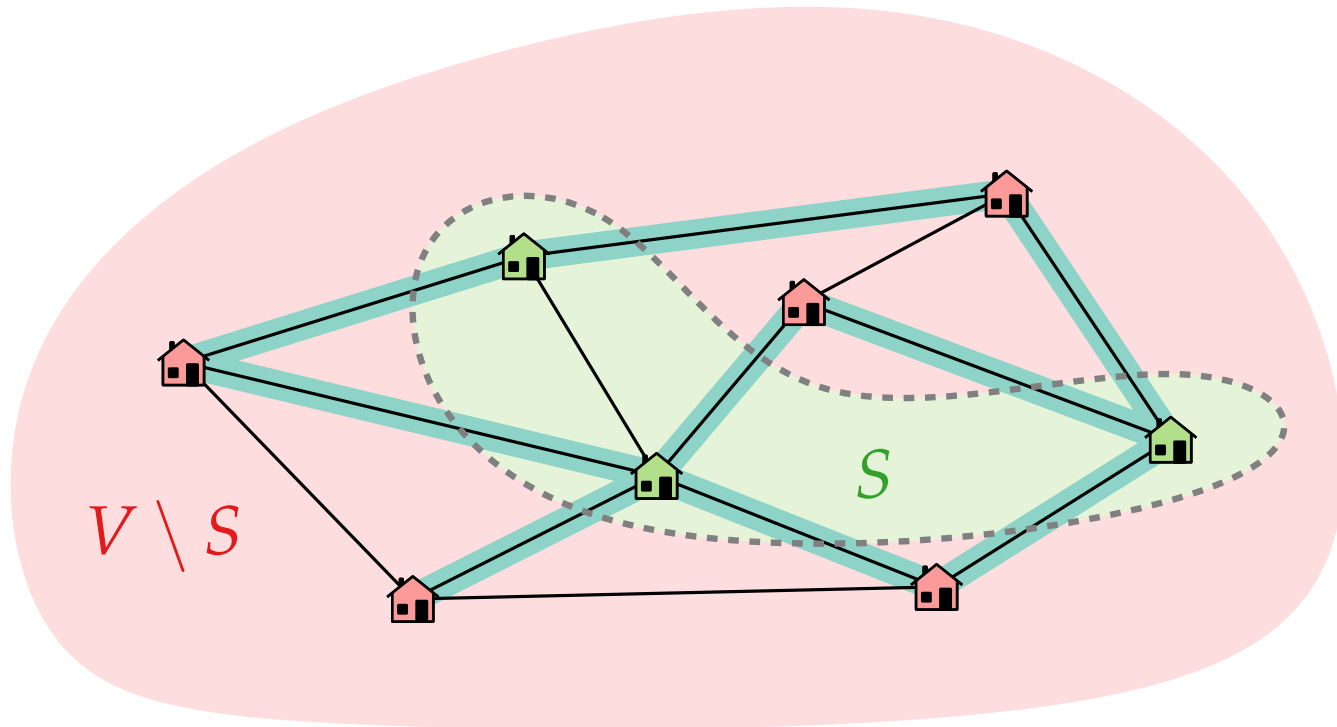


Schnitte

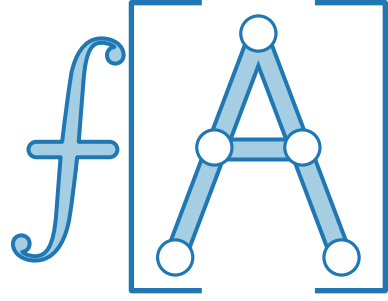


Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.

Eine Kante uv **kreuzt** $(S, V \setminus S)$, wenn $u \in S$ und $v \in V \setminus S$ (oder andersherum).



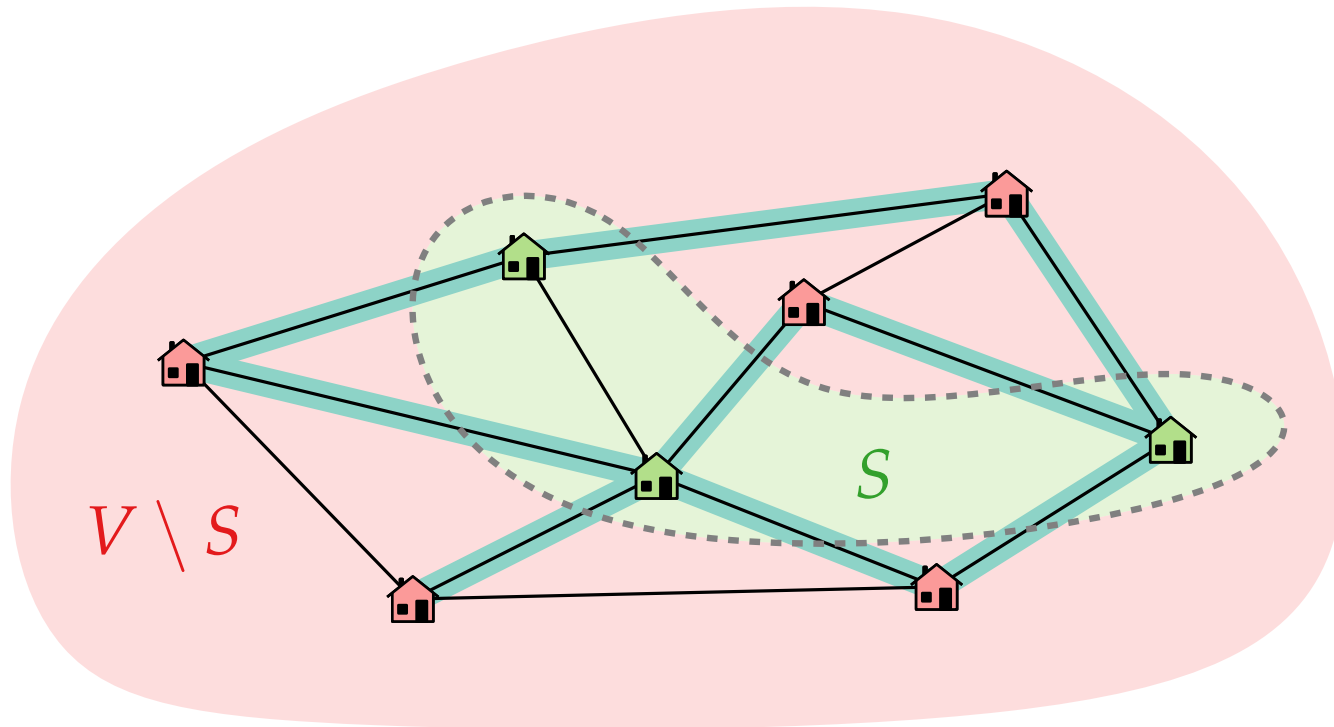
Schnitte



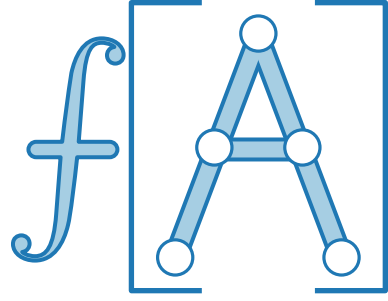
Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.

Eine Kante uv **kreuzt** $(S, V \setminus S)$, wenn $u \in S$ und $v \in V \setminus S$ (oder andersherum).

Eine Kante uv , die einen Schnitt kreuzt, ist **leicht**, wenn alle Kanten, die den Schnitt kreuzen, mindestens $w(uv)$ wiegen.



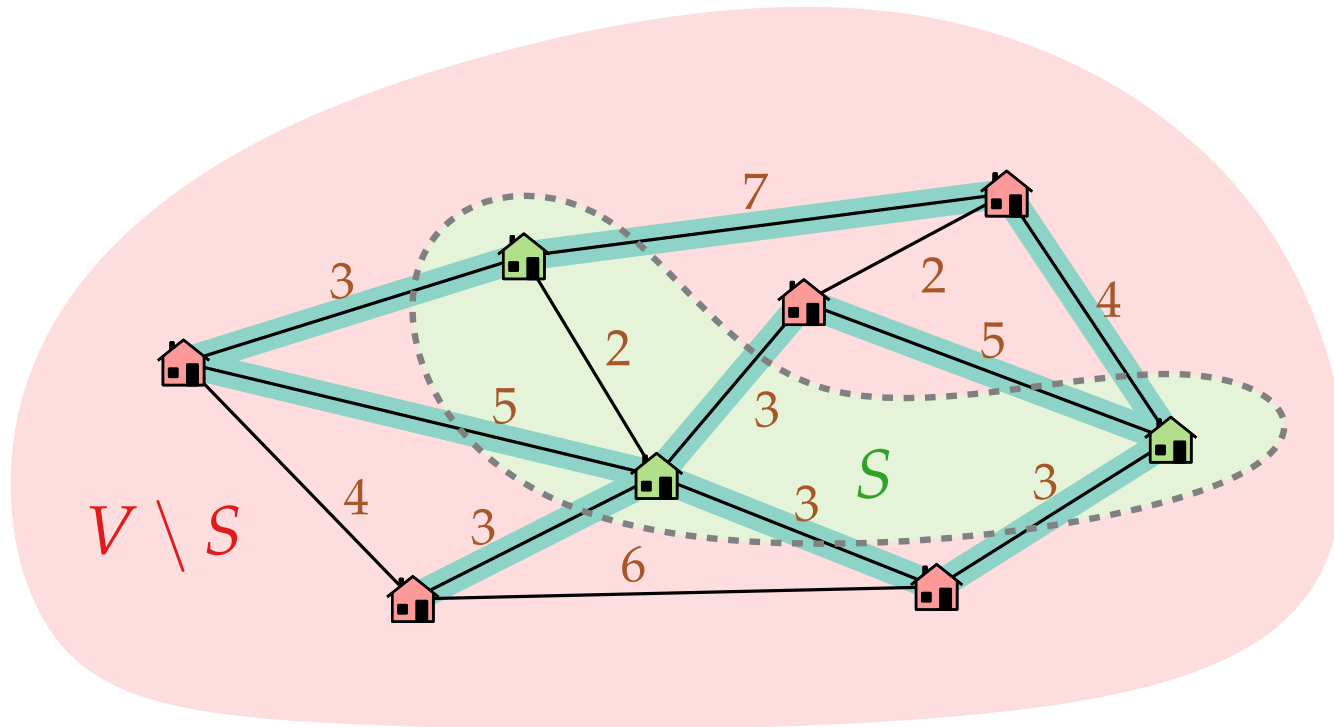
Schnitte



Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.

Eine Kante uv **kreuzt** $(S, V \setminus S)$, wenn $u \in S$ und $v \in V \setminus S$ (oder andersherum).

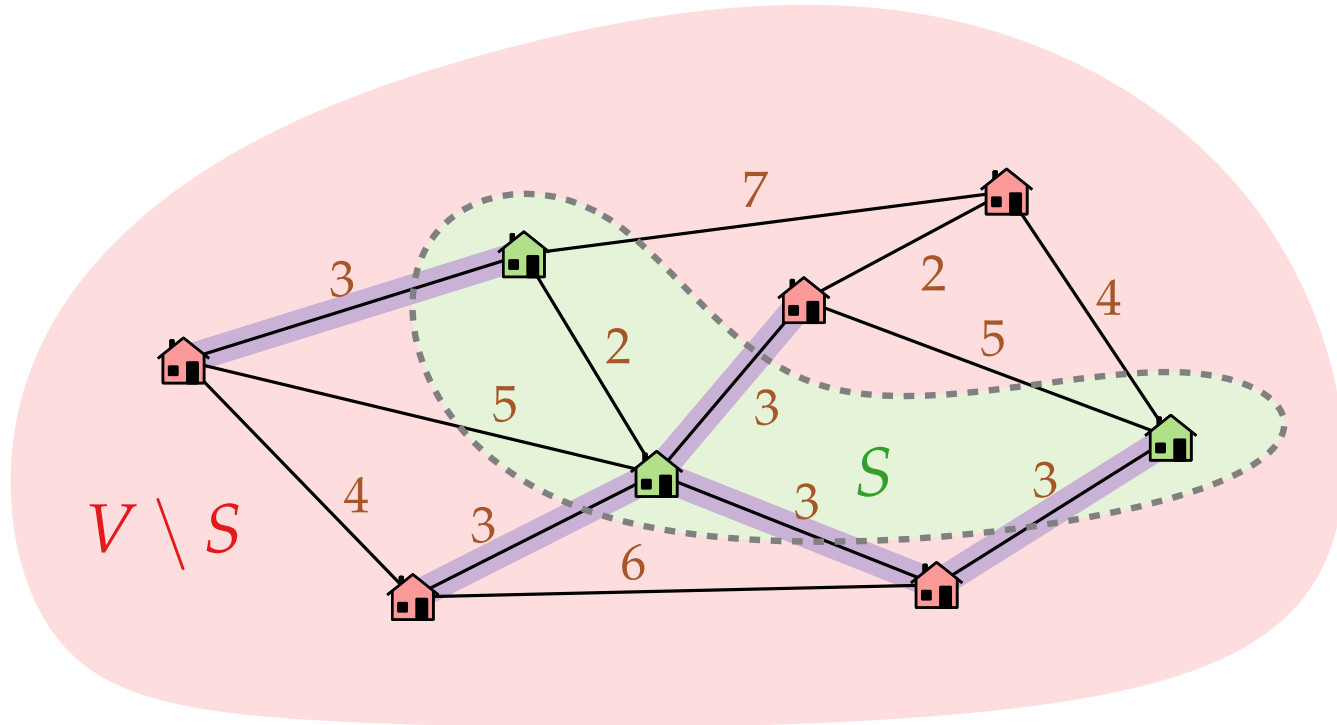
Eine Kante uv , die einen Schnitt kreuzt, ist **leicht**, wenn alle Kanten, die den Schnitt kreuzen, mindestens $w(uv)$ wiegen.

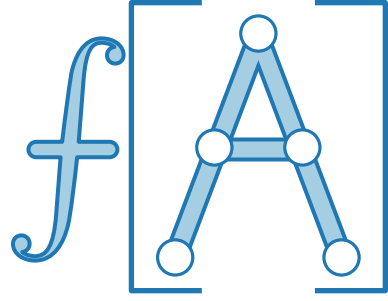


Def. Ein **Schnitt** $(S, V \setminus S)$ eines Graphen $G = (V, E)$ ist eine Zerlegung von V in 2 Teilmengen.

Eine Kante uv **kreuzt** $(S, V \setminus S)$, wenn $u \in S$ und $v \in V \setminus S$ (oder andersherum).

Eine Kante uv , die einen Schnitt kreuzt, ist **leicht**, wenn alle Kanten, die den Schnitt kreuzen, mindestens $w(uv)$ wiegen.

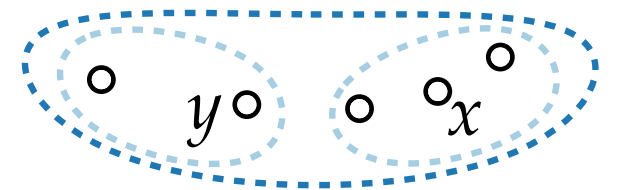
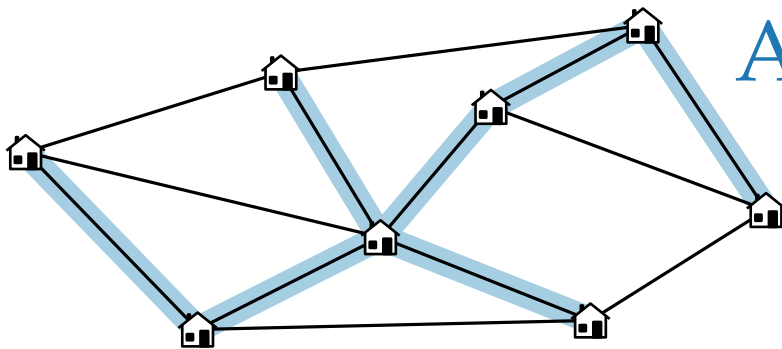




Fortgeschrittene Algorithmen

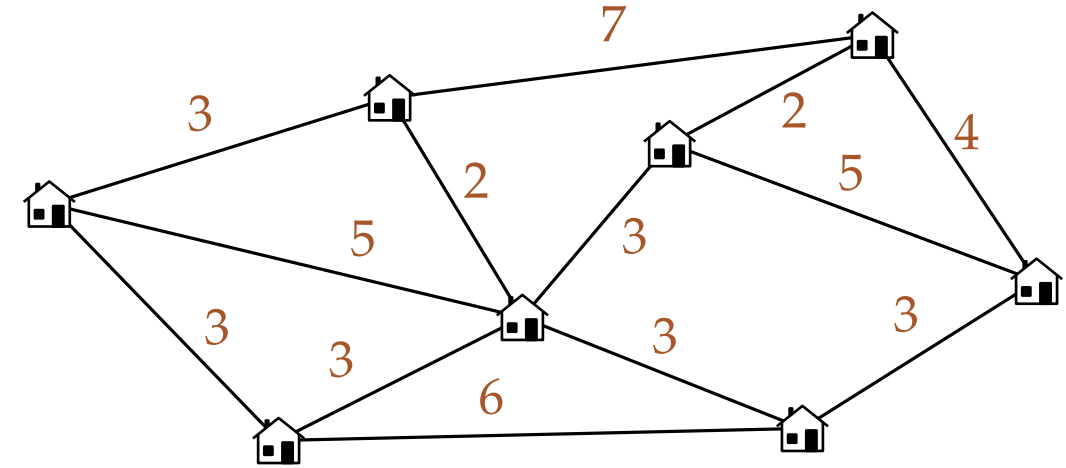
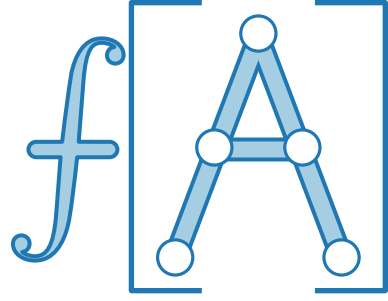
7. Vorlesung Graphalgorithmen I: Minimale Spannbäume

Teil III: Allgemeiner Greedy Algorithmus



Allgemeiner Greedy Algorithmus

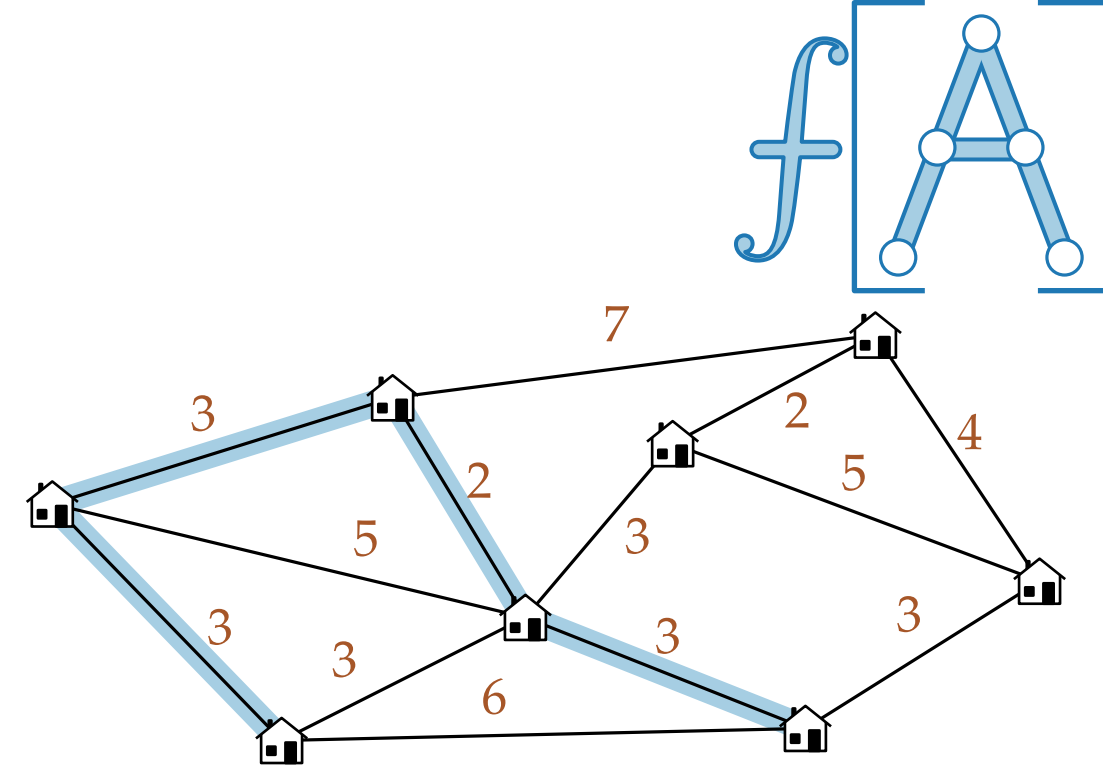
Färbe alle Kanten des Graphen:



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

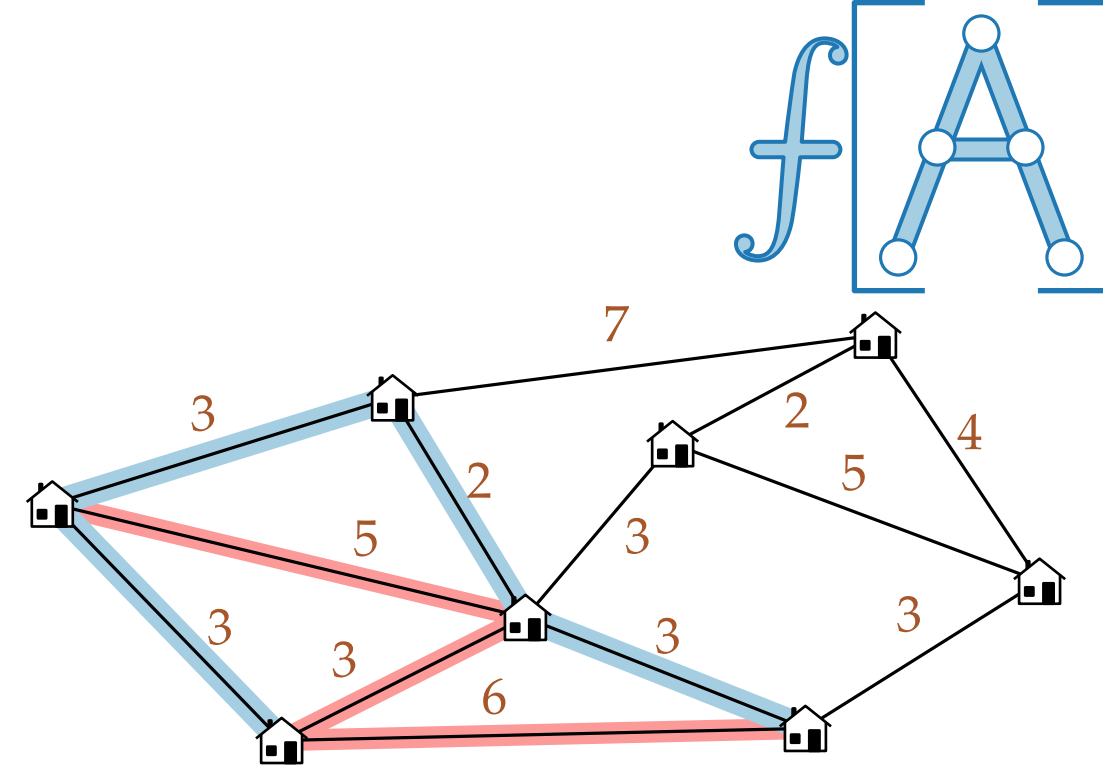
■ blau: Kante aus MSB



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

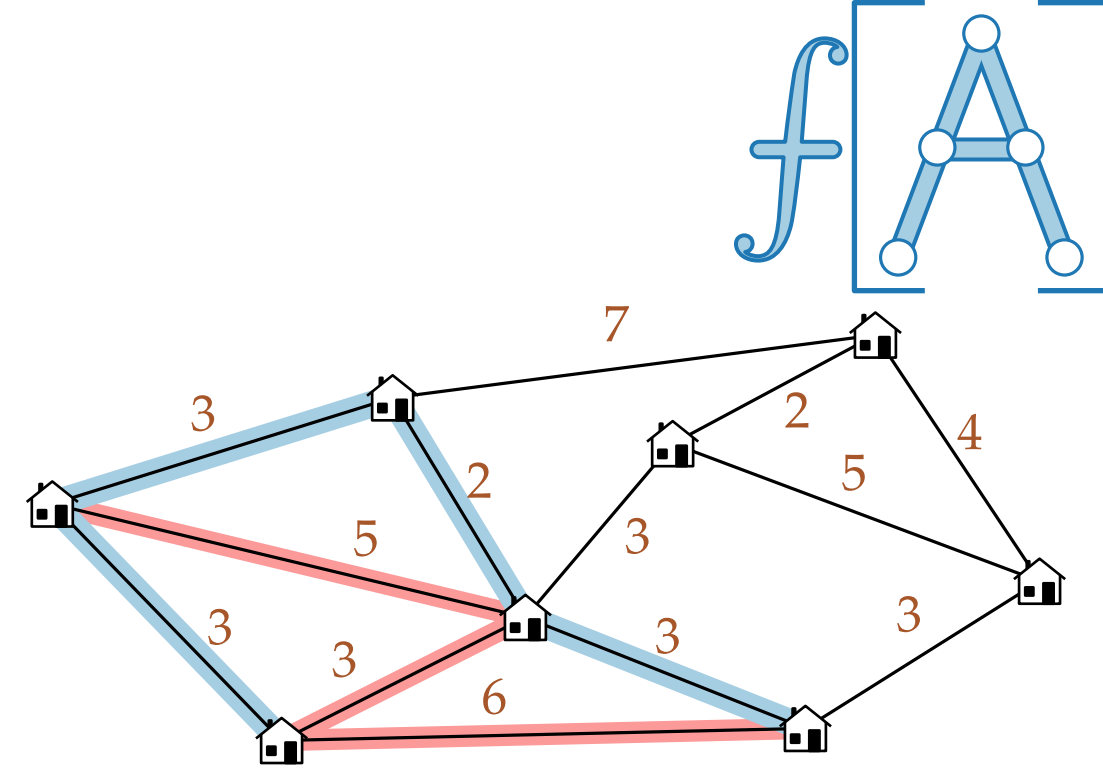
- blau: Kante aus MSB
- rot: Kante nicht aus MSB



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- blau: Kante aus MSB
- rot: Kante nicht aus MSB
- ungefärbt: Noch nicht entschieden

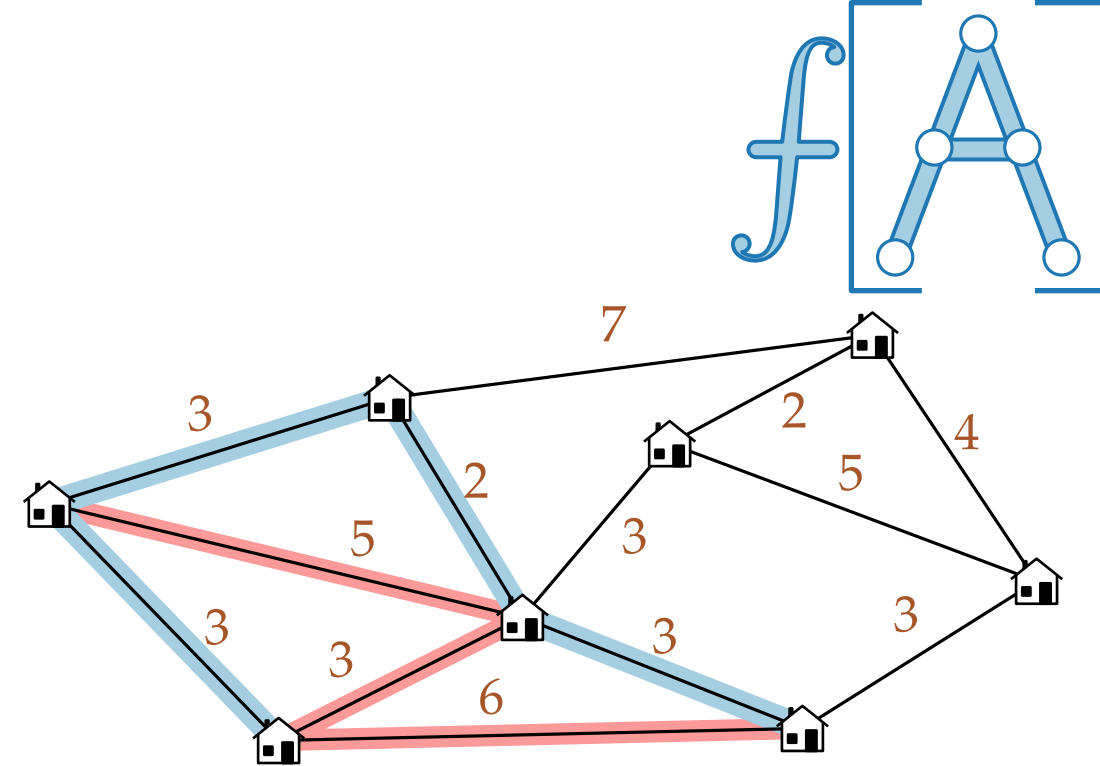


Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- **blau:** Kante aus MSB
- **rot:** Kante nicht aus MSB
- **ungefärbt:** Noch nicht entschieden

Verwende 2 Regeln:



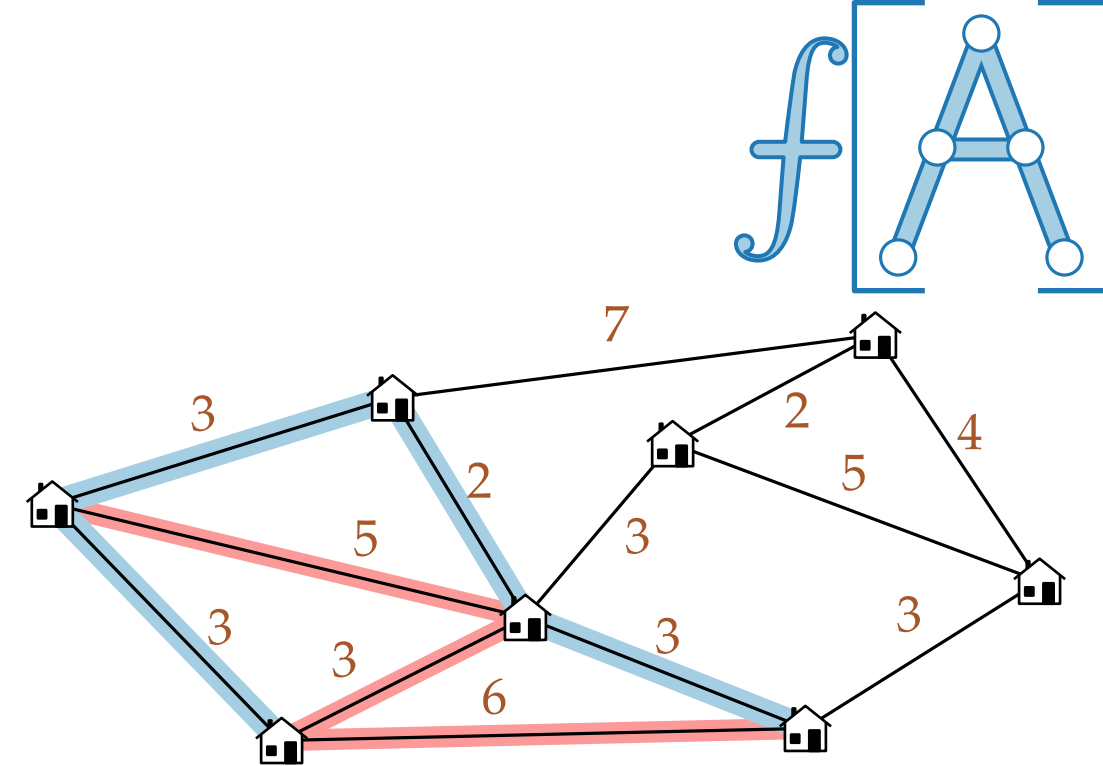
Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- **blau:** Kante aus MSB
- **rot:** Kante nicht aus MSB
- **ungefärbt:** Noch nicht entschieden

Verwende 2 Regeln:

Blaue Regel:



Allgemeiner Greedy Algorithmus

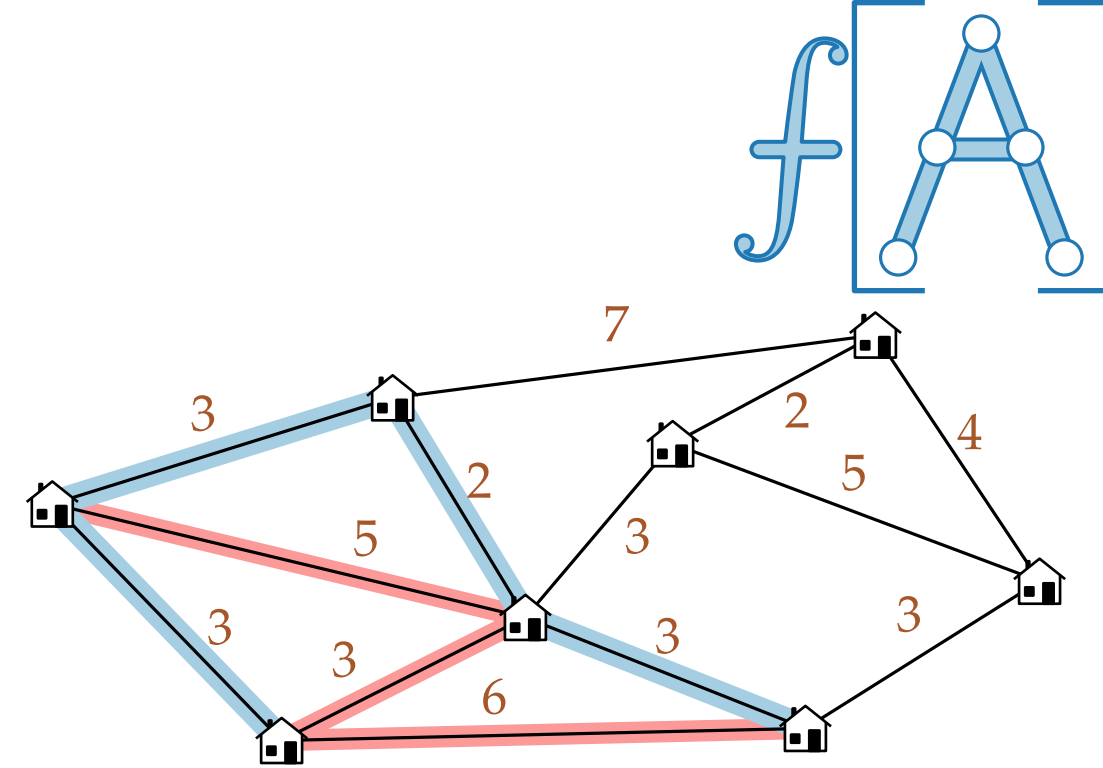
Färbe alle Kanten des Graphen:

- **blau:** Kante aus MSB
- **rot:** Kante nicht aus MSB
- **ungefärbt:** Noch nicht entschieden

Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt



Allgemeiner Greedy Algorithmus

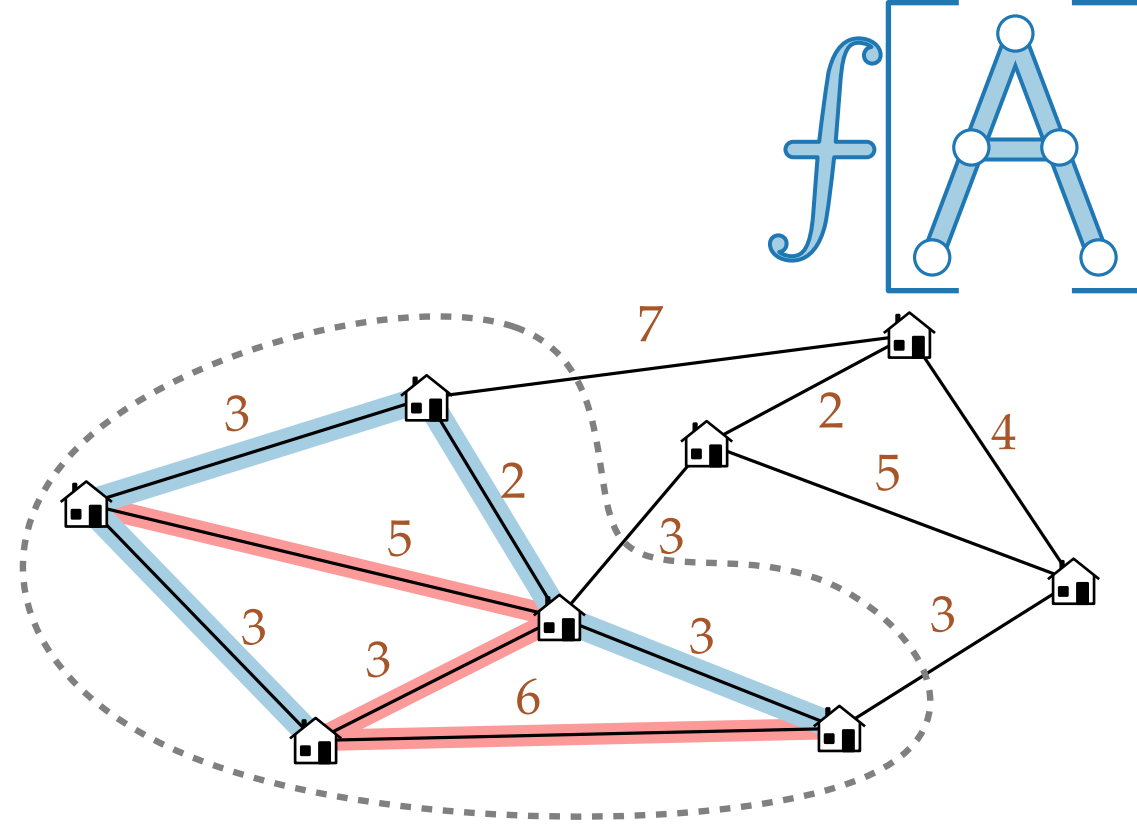
Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt



Allgemeiner Greedy Algorithmus

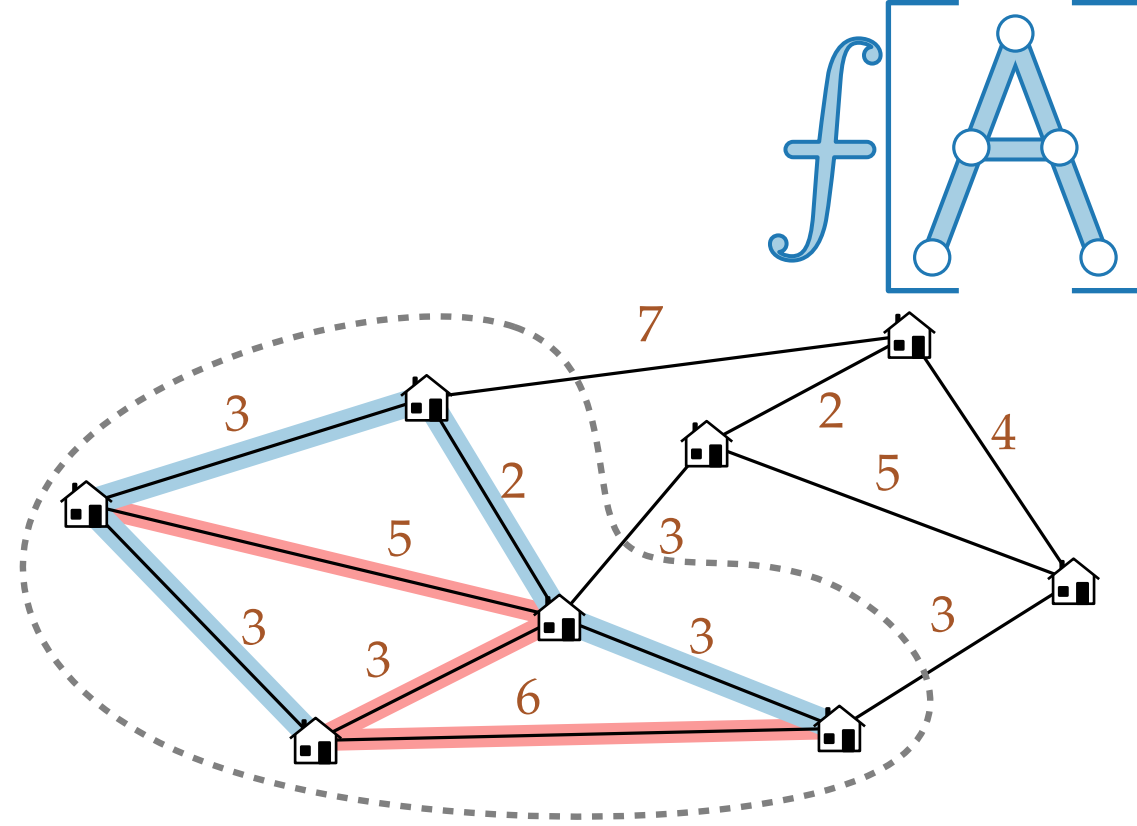
Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**



Allgemeiner Greedy Algorithmus

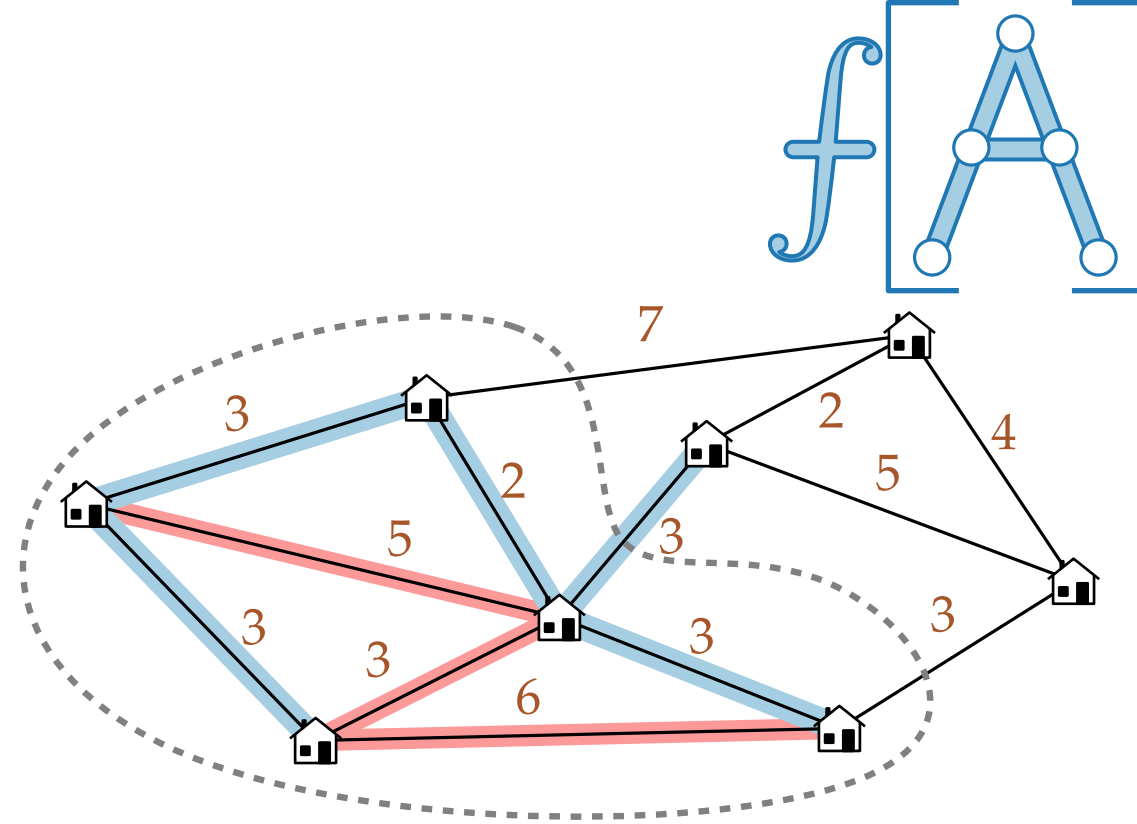
Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

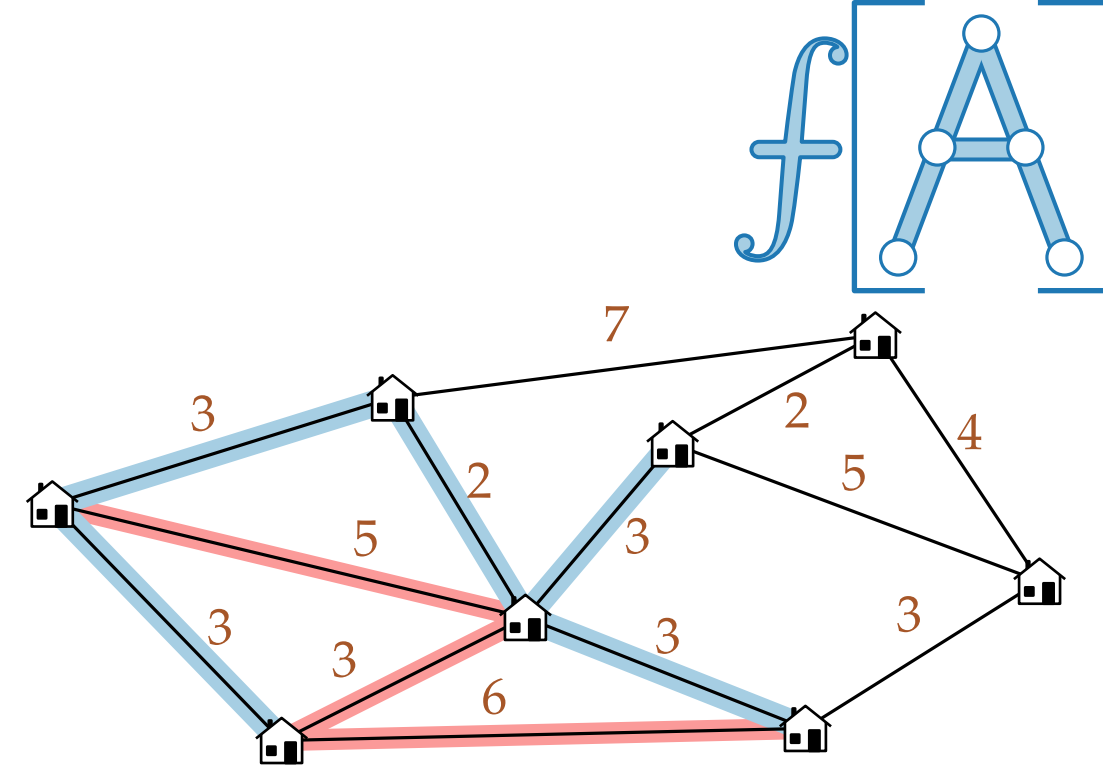
- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Rote Regel:



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

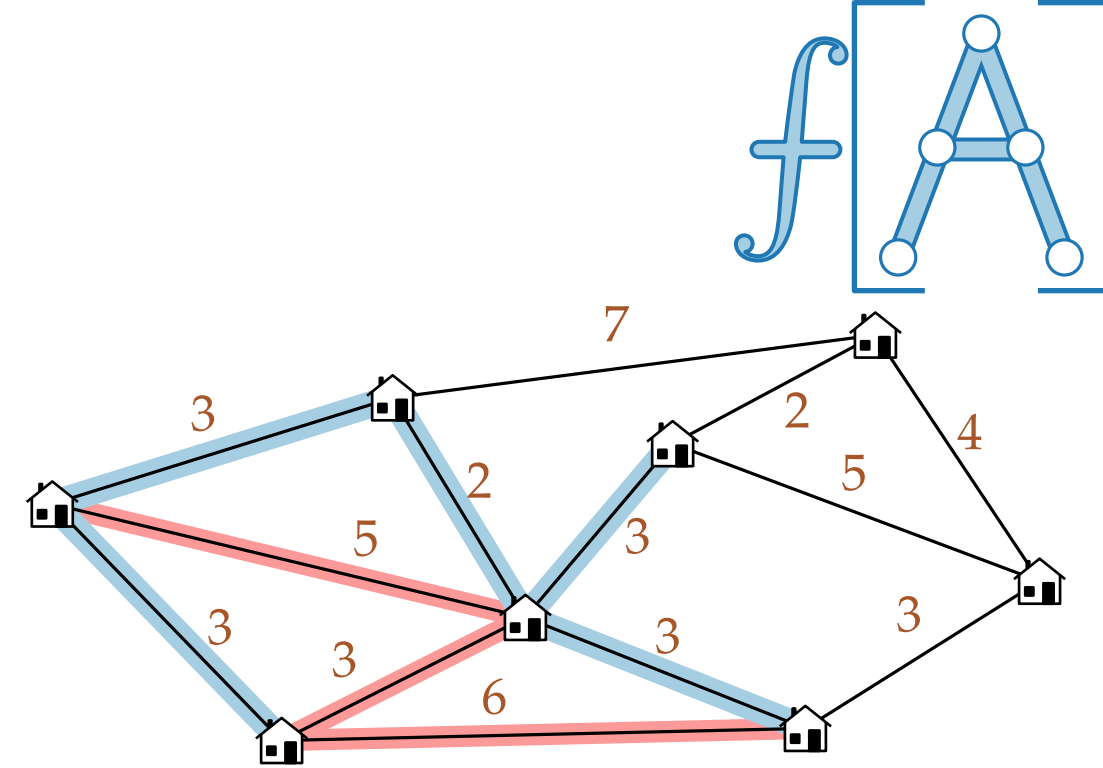
Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Rote Regel:

Wähle Kreis ohne **rote** Kante



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

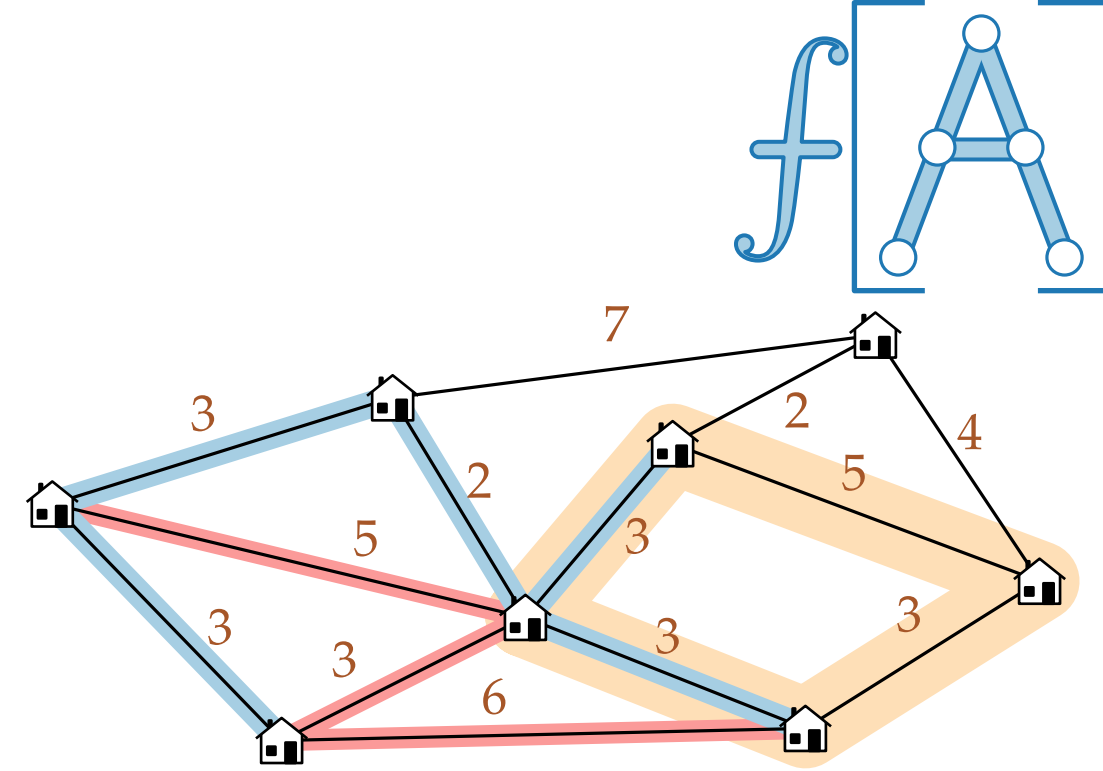
Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Rote Regel:

Wähle Kreis ohne **rote** Kante



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

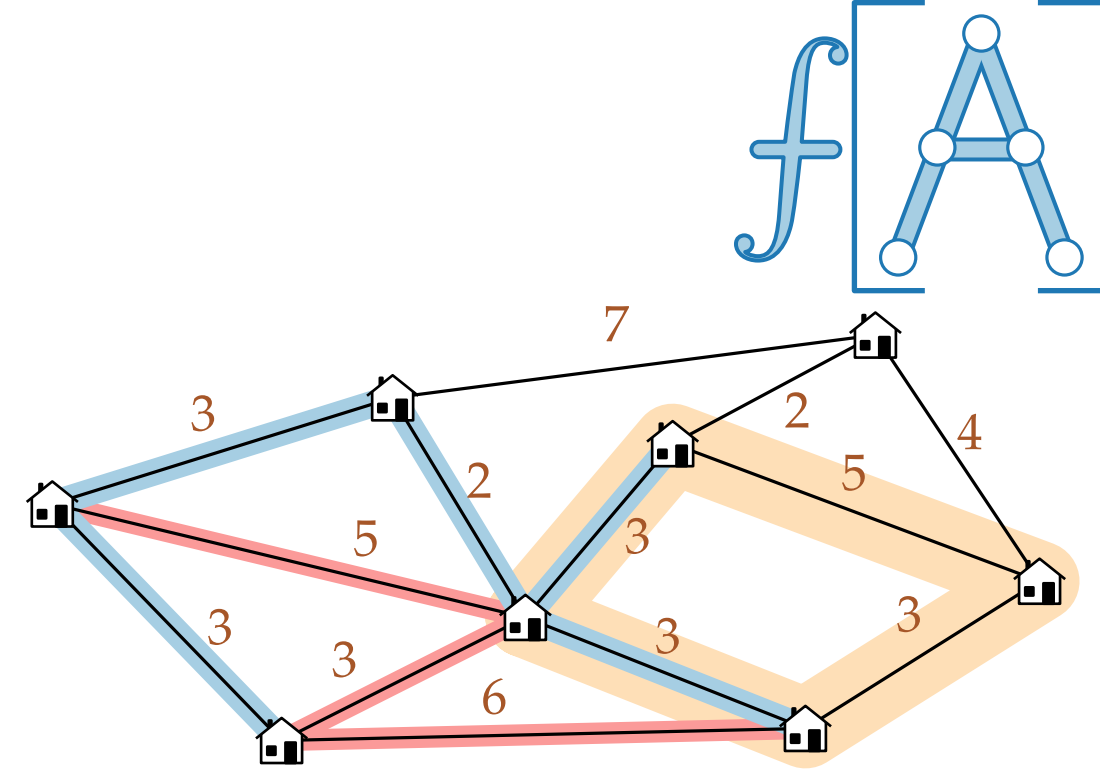
Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungefärbte Kante auf Kreis **rot**



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

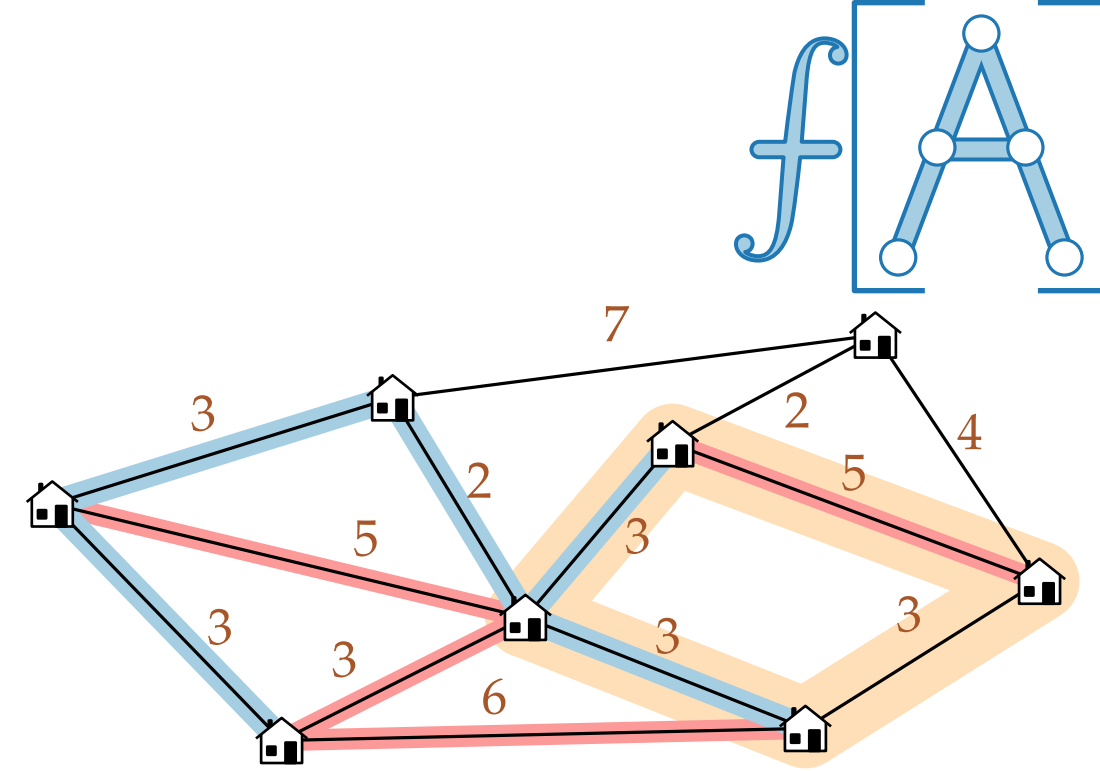
Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungefärbte Kante auf Kreis **rot**



Allgemeiner Greedy Algorithmus

Färbe alle Kanten des Graphen:

- **blau**: Kante aus MSB
- **rot**: Kante nicht aus MSB
- **ungefärbt**: Noch nicht entschieden

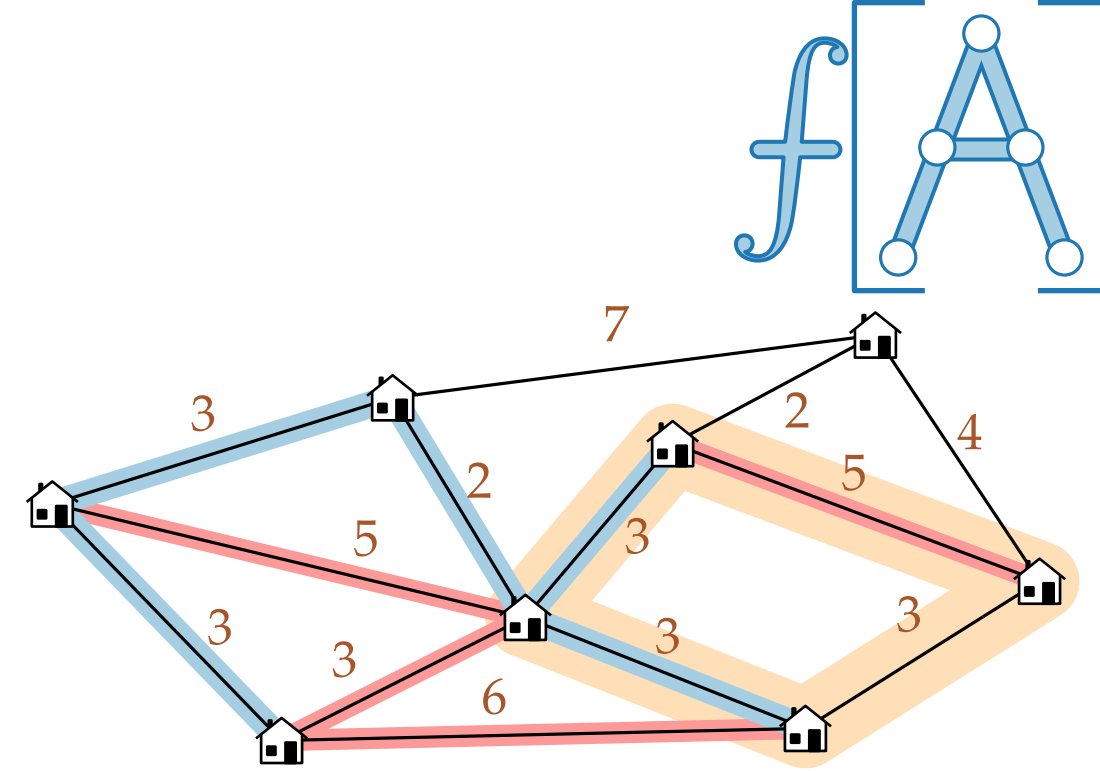
Verwende 2 Regeln:

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungefärbte Kante auf Kreis **rot**



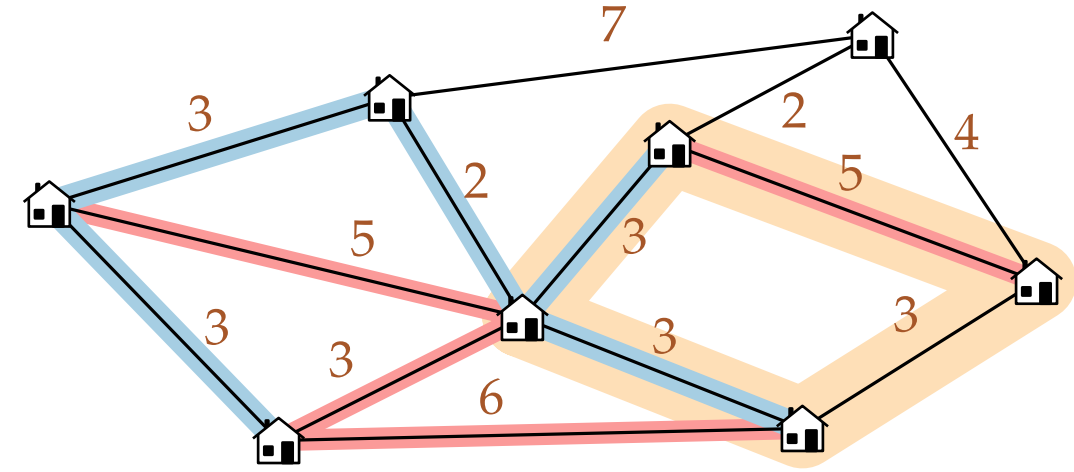
GREEDYSPANNBAUM($G = (V, E; w)$)

- blau: Kante aus MSB
- rot: Kante nicht aus MSB
- ungefärbt: Noch nicht entschieden

Blaue Regel:

Rote Regel:

Wähle Kreis ohne rote Kante
Färbe größte ungefärbte Kante auf Kreis rot



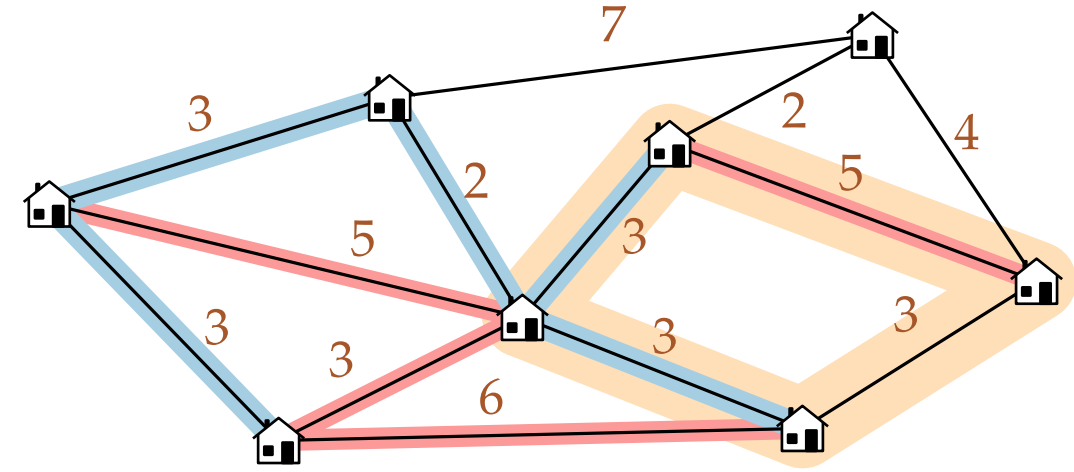
GREEDYSPANNBAUM($G = (V, E; w)$)
 Wende **blaue Regel** oder **rote Regel**
 an, bis alle Kanten gefärbt sind.

- blau: Kante aus MSB
- rot: Kante nicht aus MSB
- ungefärbt: Noch nicht entschieden

Blaue Regel:

Rote Regel:

Wähle Kreis ohne rote Kante
Färbe größte ungefärbte Kante auf Kreis rot



GREEDYSPANNBAUM($G = (V, E; w)$)
 Wende **blaue Regel** oder **rote Regel**
 an, bis alle Kanten gefärbt sind.
 Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel**
an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel**
an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel**
an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel**
an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel**
an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel** an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel** an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Satz. GREEDYSPANNBAUM findet einen minimalen Spannbaum.

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel** an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Satz. GREEDYSPANNBAUM findet einen minimalen Spannbaum.

Beweis. ■ Jede Kante ist entweder **blau** oder **rot**

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel** an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Satz. GREEDYSPANNBAUM findet einen minimalen Spannbaum.

Beweis.

- Jede Kante ist entweder **blau** oder **rot**
- Es gibt einen MSB T , der alle **blauen Kanten** und keine **rote Kante** enthält

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel** an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Beweis Greedy Algorithmus

Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

FI ist am Anfang offensichtlich erfüllt.

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Satz. GREEDYSPANNBAUM findet einen minimalen Spannbaum.

Beweis.

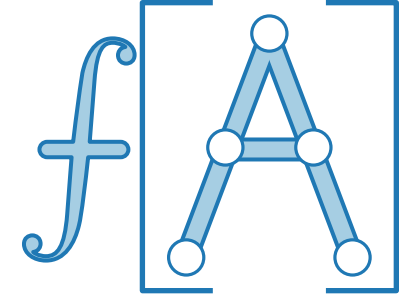
- Jede Kante ist entweder **blau** oder **rot**
- Es gibt einen MSB T , der alle **blauen Kanten** und keine **rote Kante** enthält
 $\Rightarrow$ **Blaue Kanten** bilden MSB

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel** an, bis alle Kanten gefärbt sind.

Gebe $E' = \{\text{blaue Kanten}\}$ zurück

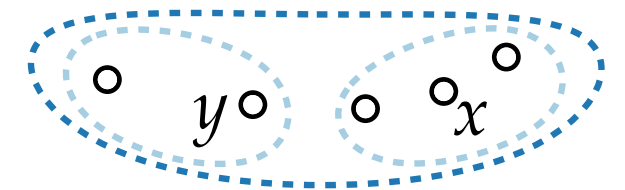
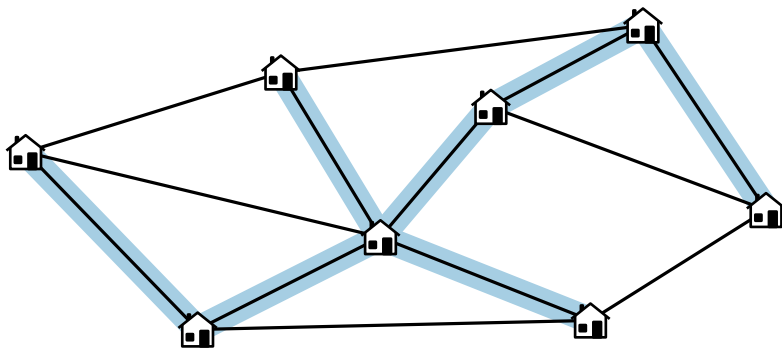




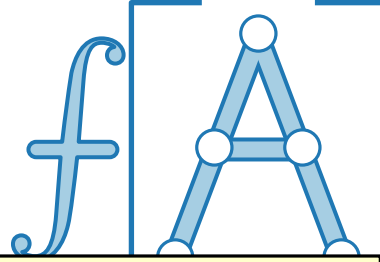
Fortgeschrittene Algorithmen

7. Vorlesung Graphalgorithmen I: Minimale Spannbäume

Teil IV: Beweis der Regeln



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

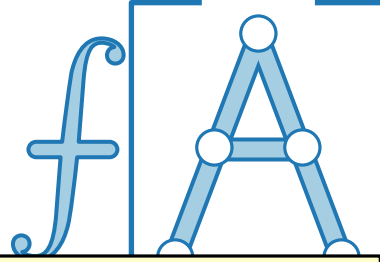
- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

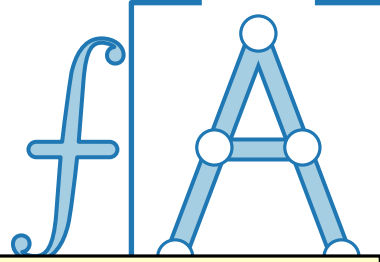
Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis.

Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

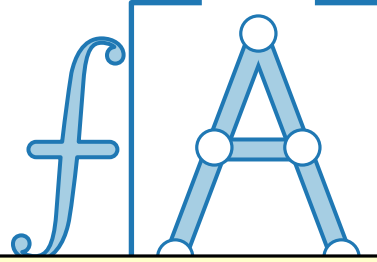
Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

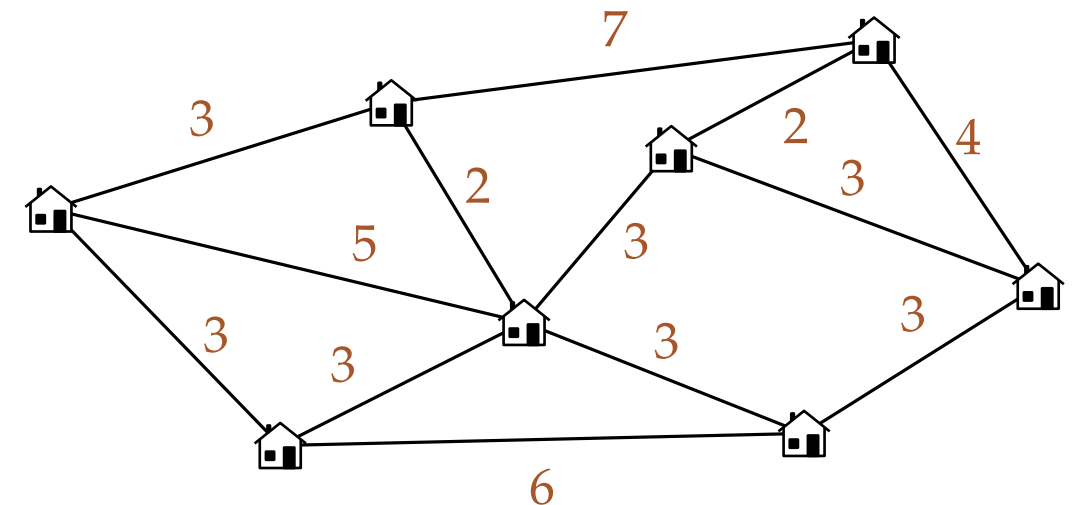
- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

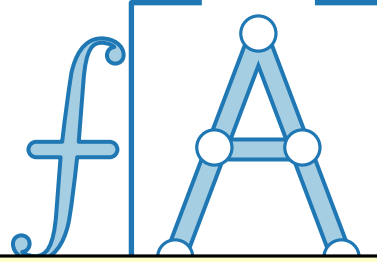
Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

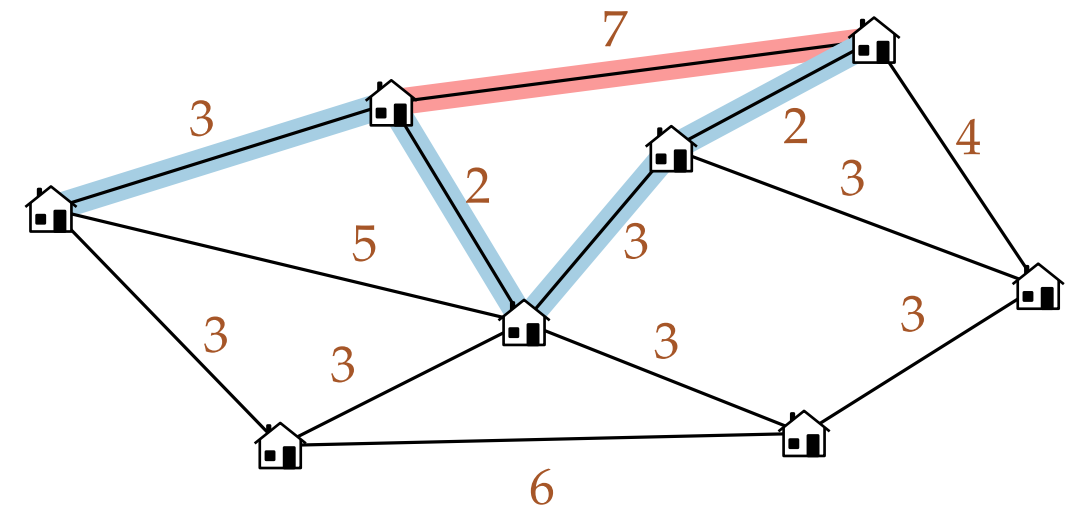
- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

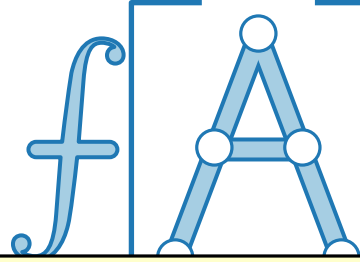
Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

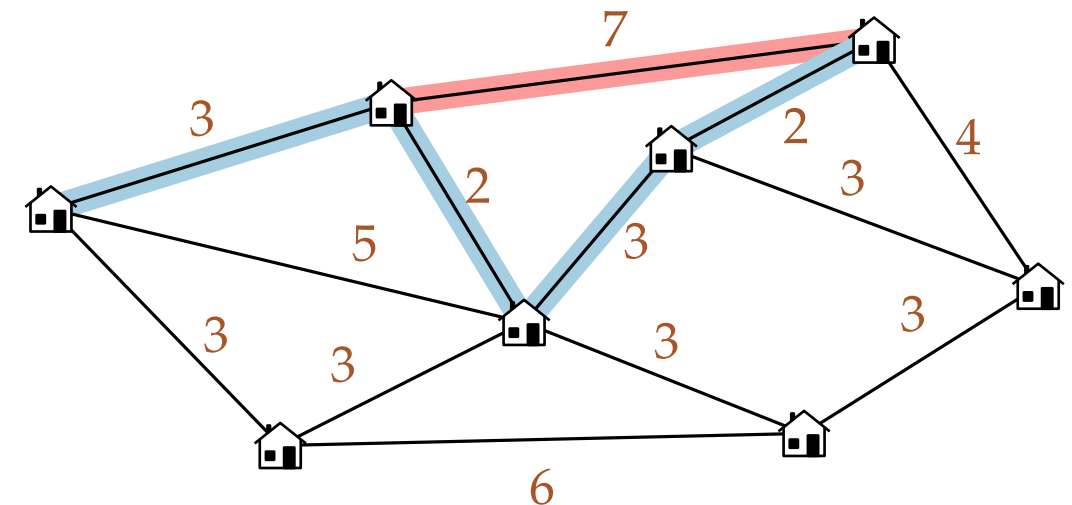
- T enthält alle **blauen** Kanten
- T enthält keine **rote** Kante

Blaue Regel:

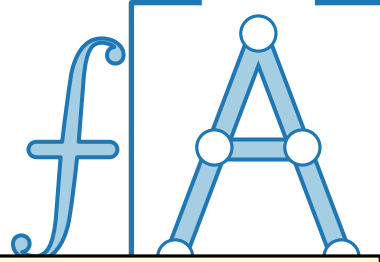
Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI
Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

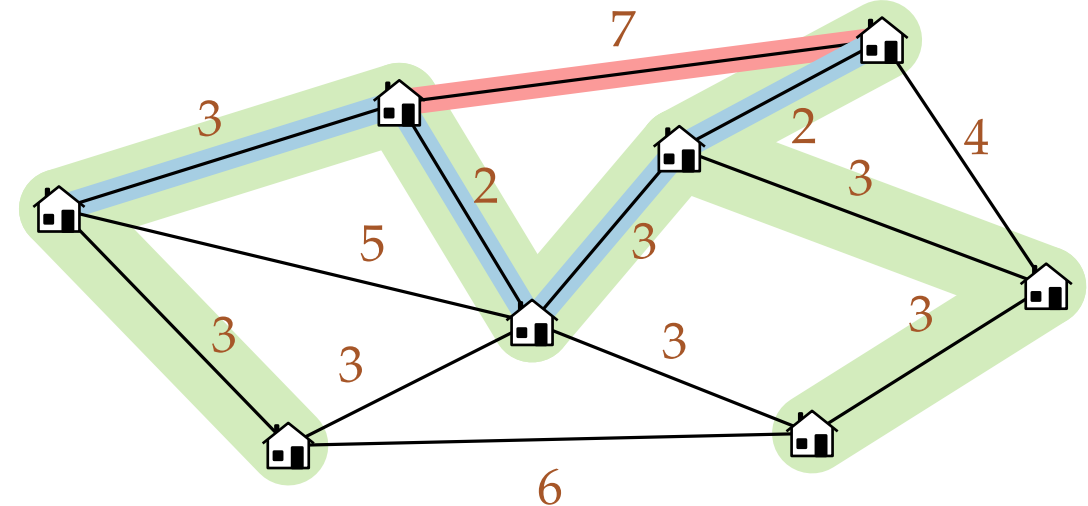
- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

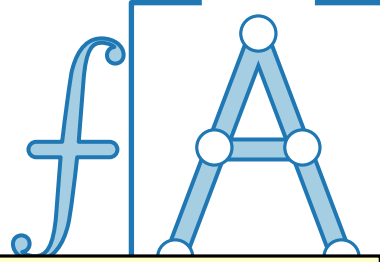
Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI
Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

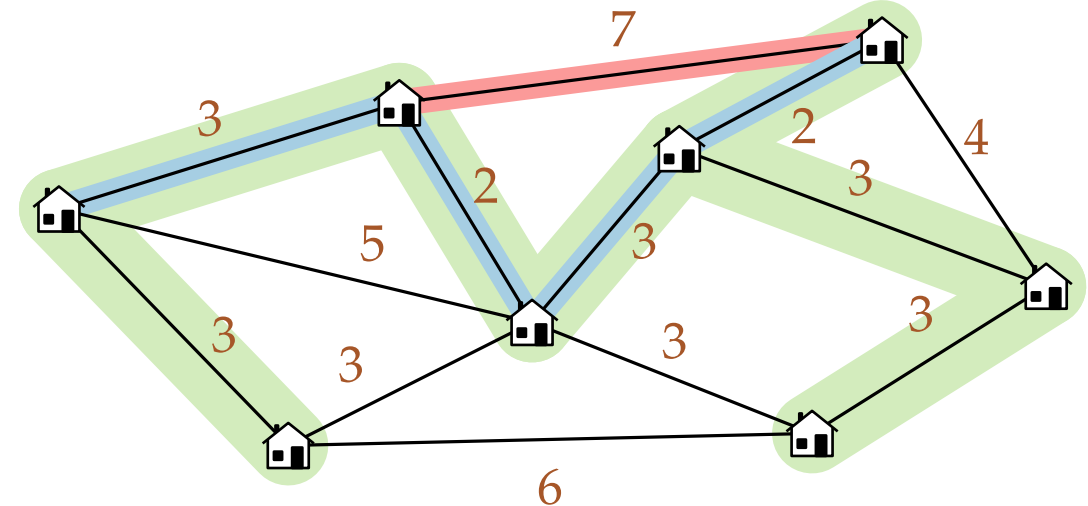
- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

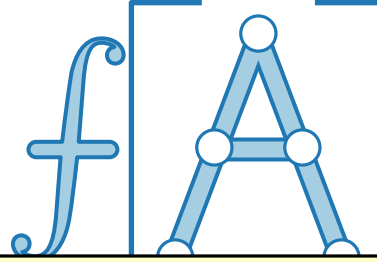
Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI
Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt
Sei $uv \in E$ von **blauer Regel** ausgewählte Kante



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

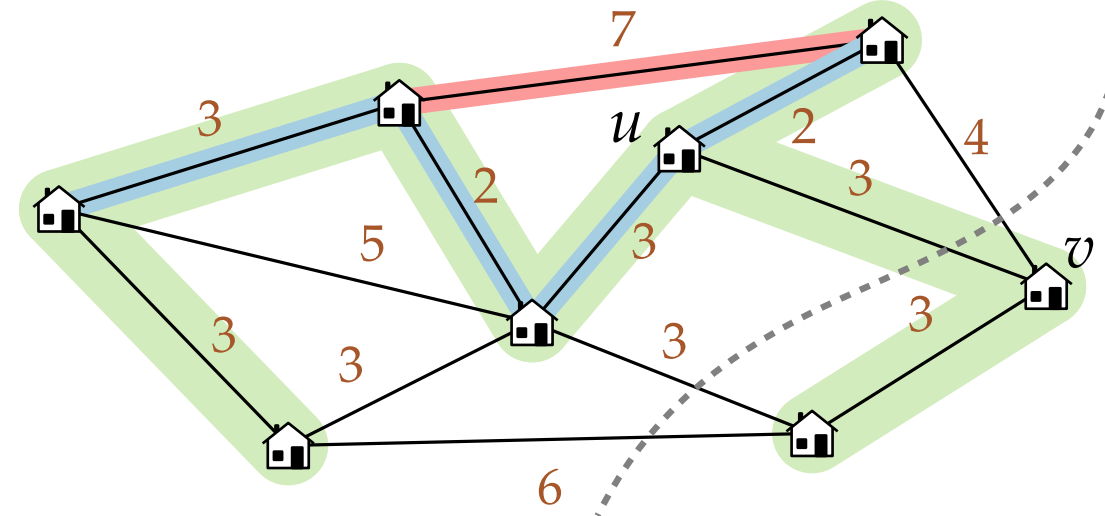
Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

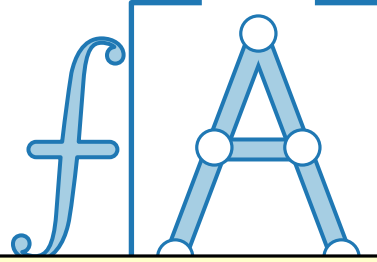
Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E'$



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

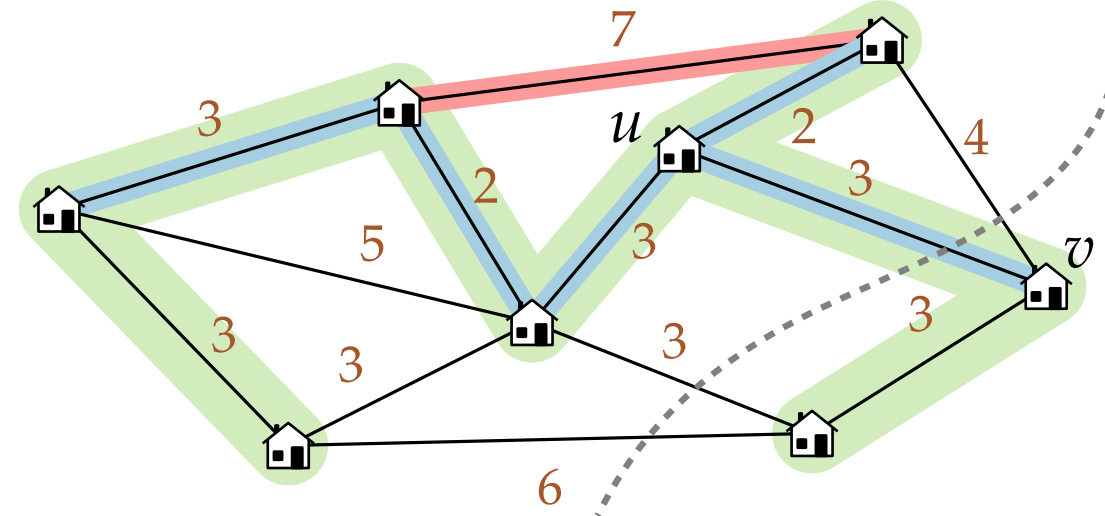
Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

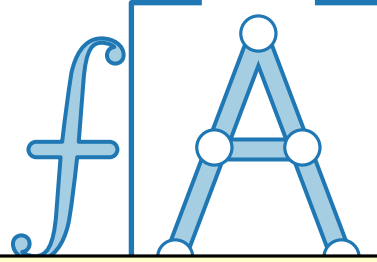
Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E'$



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

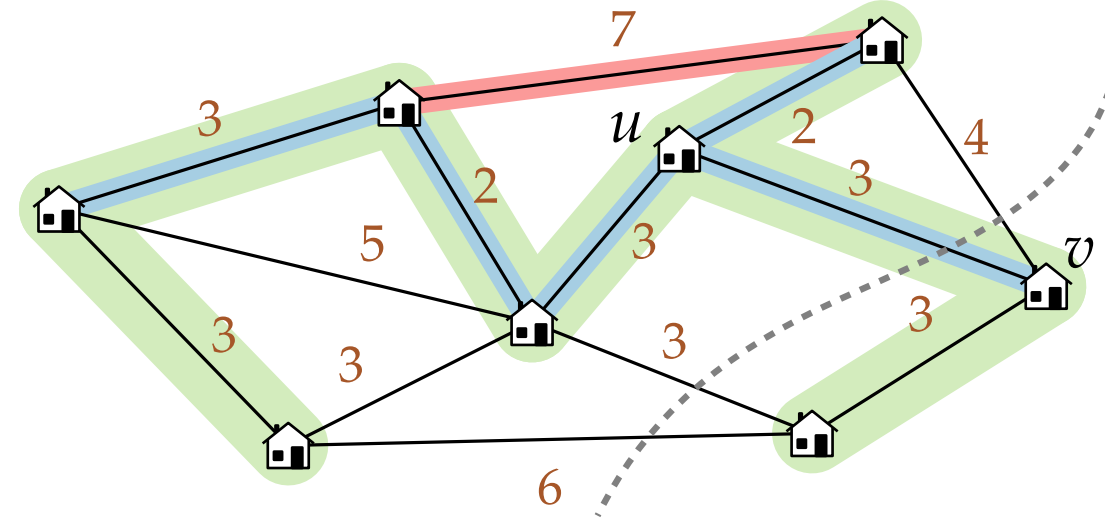
Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

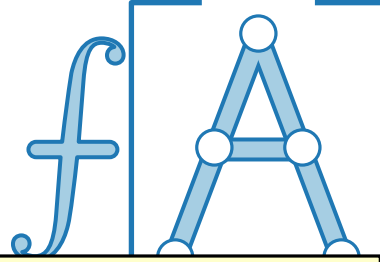
Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten



Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

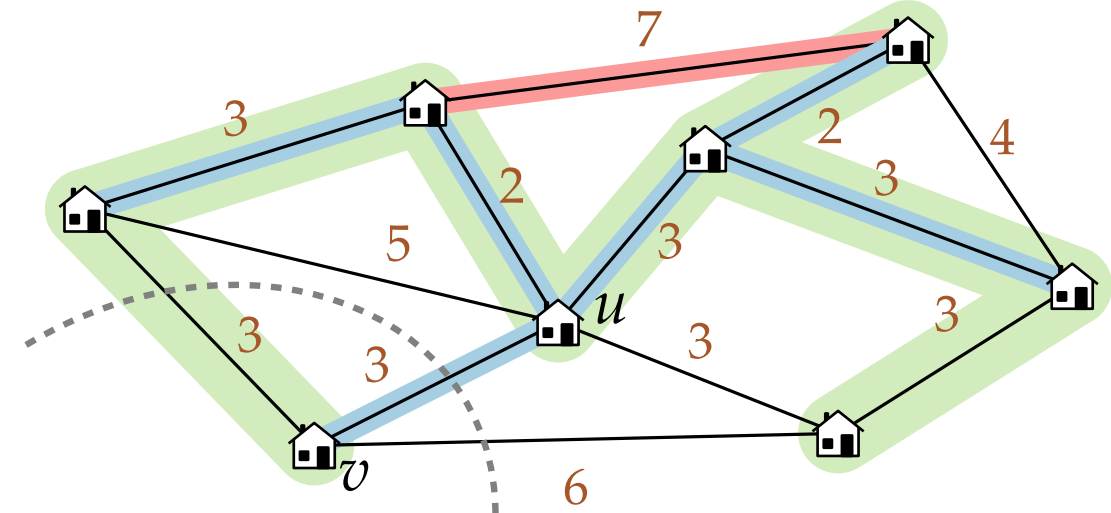
Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

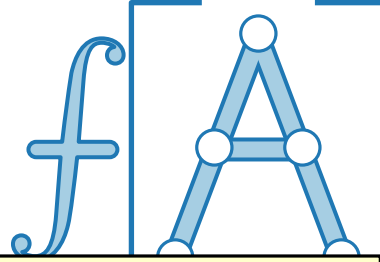
Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten
2. Fall: $uv \notin E'$



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

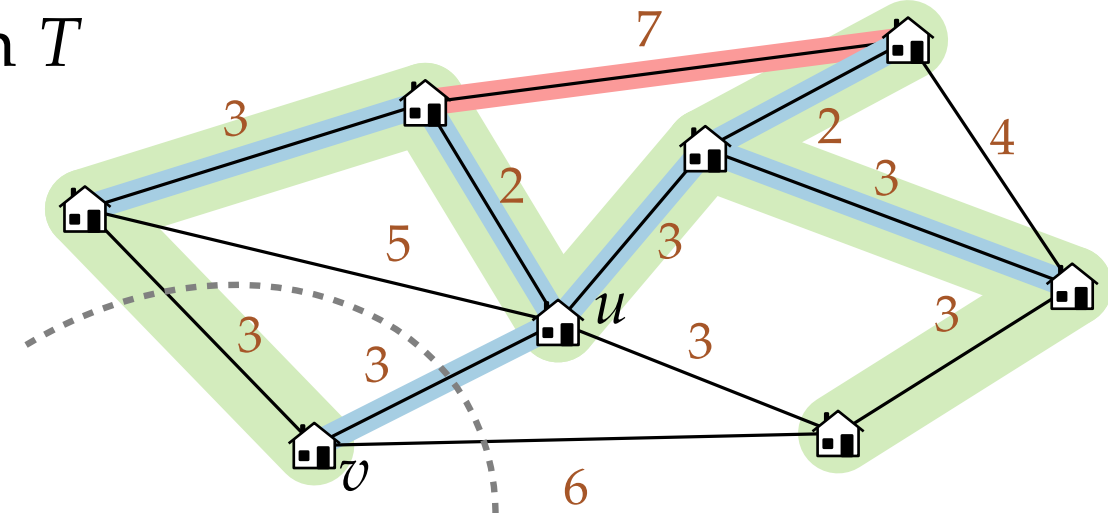
Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

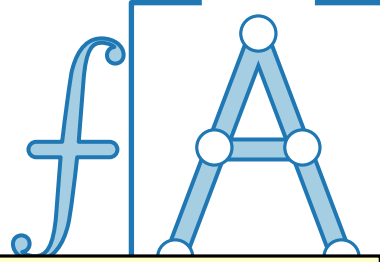
Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten
2. Fall: $uv \notin E' \Rightarrow$ es gibt **Pfad** p von u nach v in T



Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

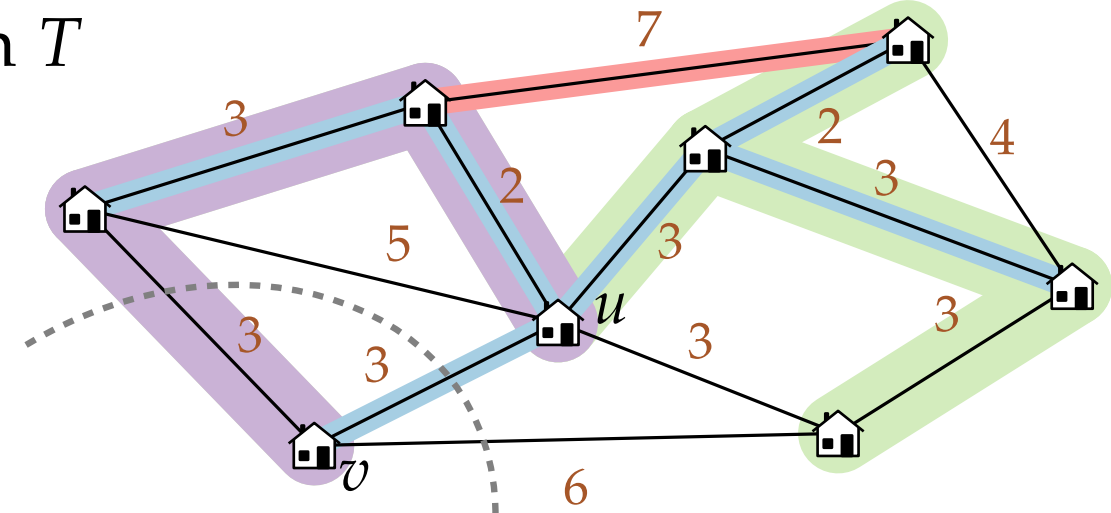
Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

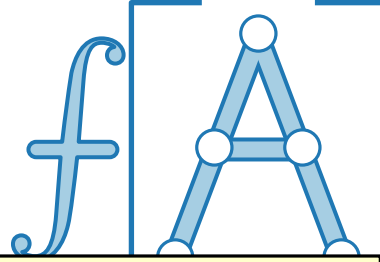
Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten
2. Fall: $uv \notin E' \Rightarrow$ es gibt **Pfad** p von u nach v in T
 $\Rightarrow p$ enthält Kante xy , die Schnitt kreuzt



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

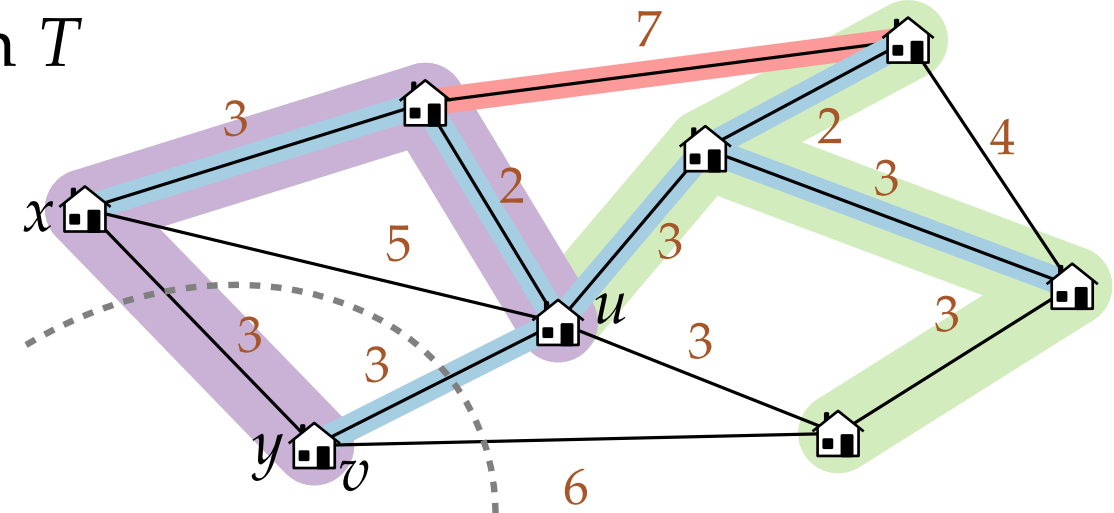
Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten
2. Fall: $uv \notin E' \Rightarrow$ es gibt **Pfad** p von u nach v in T
 $\Rightarrow p$ enthält Kante xy , die Schnitt kreuzt

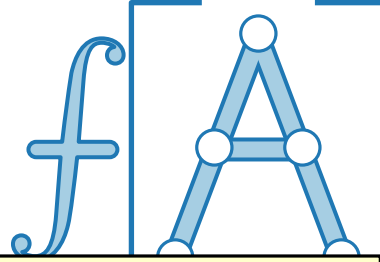


Wähle Schnitt, den keine blaue Kante kreuzt
Färbe leichte Kante blau

Wähle Schnitt, den keine blaue Kante kreuzt
Färbe leichte Kante blau

Wähle Schnitt, den keine blaue Kante kreuzt
Färbe leichte Kante blau

Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den **keine blaue Kante kreuzt**
Färbe **leichte Kante** **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

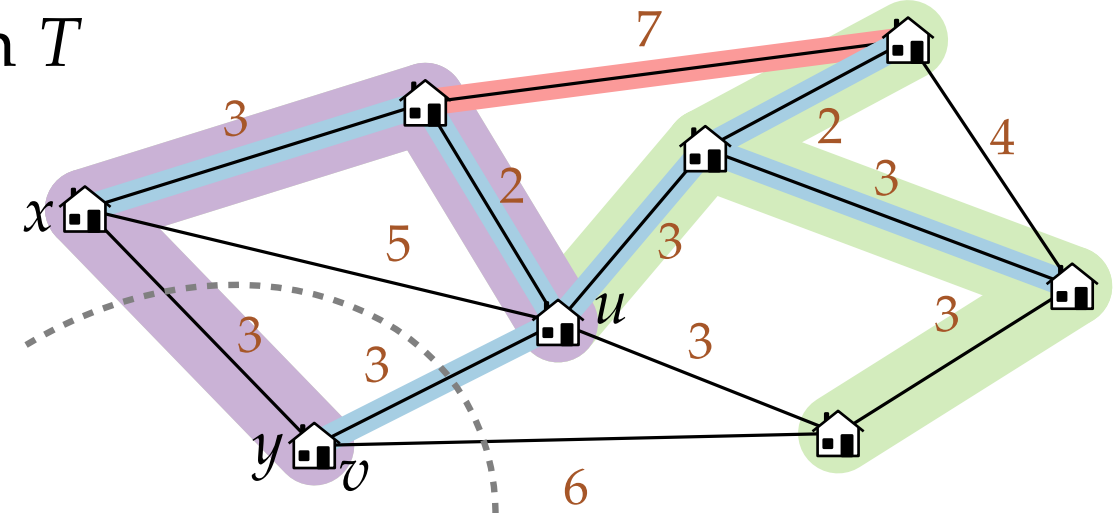
1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \notin E' \Rightarrow$ es gibt **Pfad** p von u nach v in T

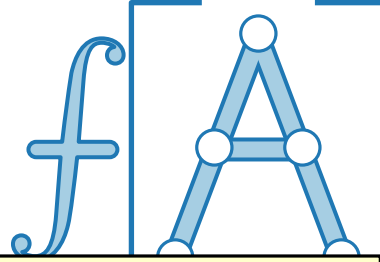
$\Rightarrow p$ enthält Kante xy , die Schnitt kreuzt

keine blaue Kante kreuzt $\Rightarrow xy$ ist ungefärbt

leichte Kante $\Rightarrow w(xy) \geq w(uv)$



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den **keine blaue Kante kreuzt**
Färbe **leichte Kante** **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten

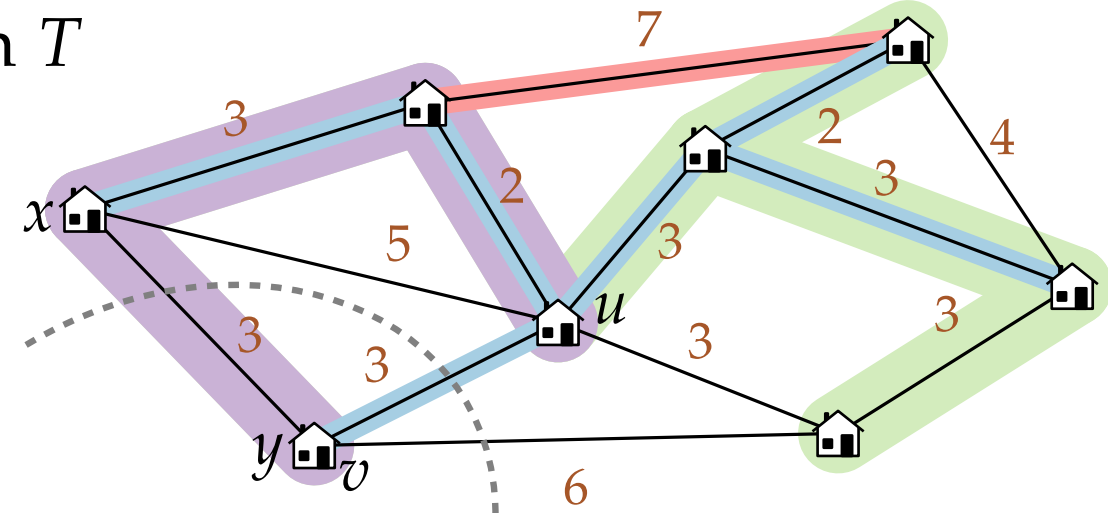
2. Fall: $uv \notin E' \Rightarrow$ es gibt **Pfad** p von u nach v in T

$\Rightarrow p$ enthält Kante xy , die Schnitt kreuzt

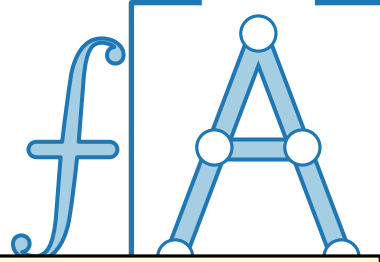
keine blaue Kante kreuzt $\Rightarrow xy$ ist ungefärbt

leichte Kante $\Rightarrow w(xy) \geq w(uv)$

Wähle $E'' = E' \cup \{uv\} \setminus \{xy\}$



Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den **keine blaue Kante kreuzt**
Färbe **leichte Kante** **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten

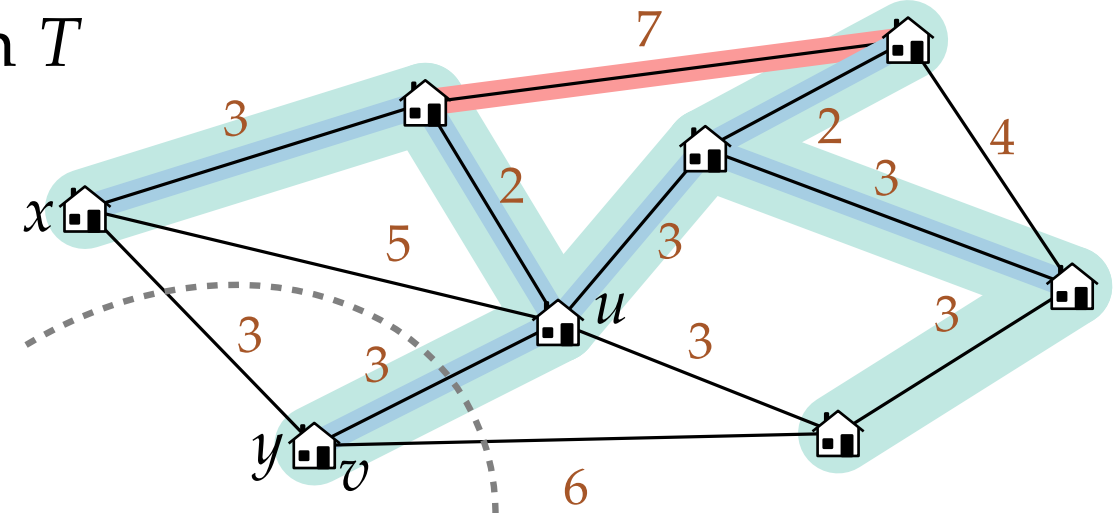
2. Fall: $uv \notin E' \Rightarrow$ es gibt **Pfad** p von u nach v in T

$\Rightarrow p$ enthält Kante xy , die Schnitt kreuzt

keine blaue Kante kreuzt $\Rightarrow xy$ ist ungefärbt

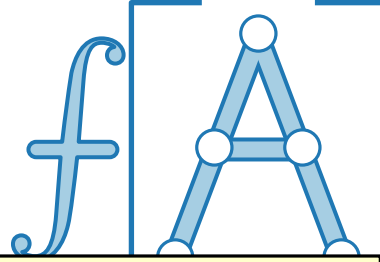
leichte Kante $\Rightarrow w(xy) \geq w(uv)$

Wähle $E'' = E' \cup \{uv\} \setminus \{xy\}$



Wähle Schnitt, den keine blaue Kante kreuzt
Färbe leichte Kante blau

Beweis der blauen Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Blaue Regel:

Wähle Schnitt, den **keine blaue Kante kreuzt**
Färbe **leichte Kante** **blau**

Lemma. Die **blaue Regel** hält die Farbinvariante aufrecht.

Beweis. Alle Kanten ungefärbt $\Rightarrow$ jeder MSB **bezeugt** FI

Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **blauer Regel** ausgewählte Kante

1. Fall: $uv \in E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \notin E' \Rightarrow$ es gibt **Pfad** p von u nach v in T

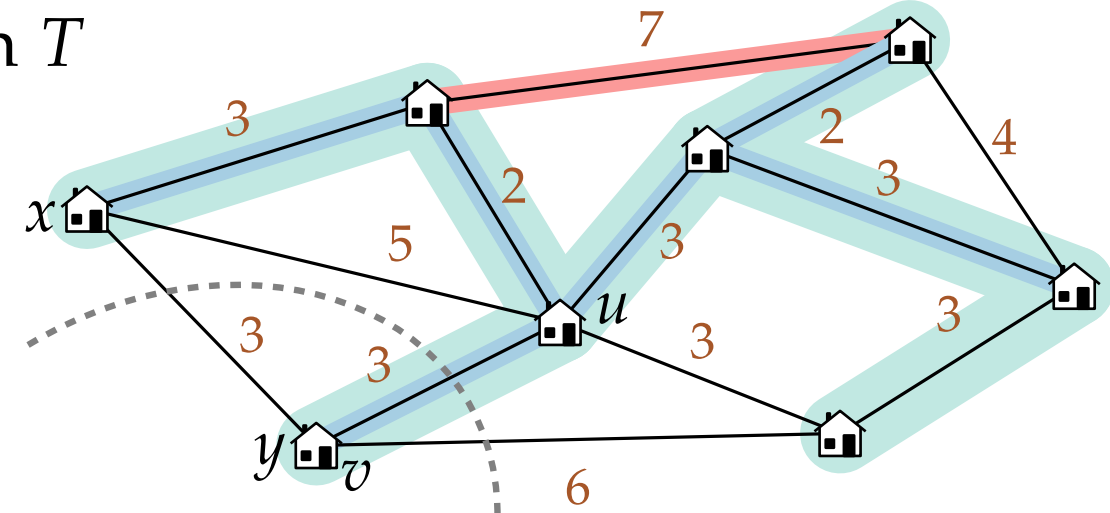
$\Rightarrow p$ enthält Kante xy , die Schnitt kreuzt

keine blaue Kante kreuzt $\Rightarrow xy$ ist ungefärbt

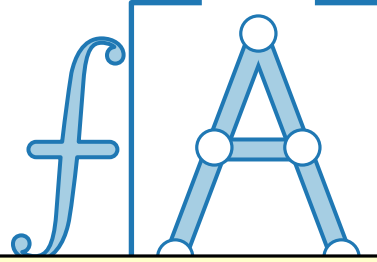
leichte Kante $\Rightarrow w(xy) \geq w(uv)$

Wähle $E'' = E' \cup \{uv\} \setminus \{xy\}$

$\Rightarrow T' = (V, E'')$ ist MSB, der FI bezeugt $\square$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

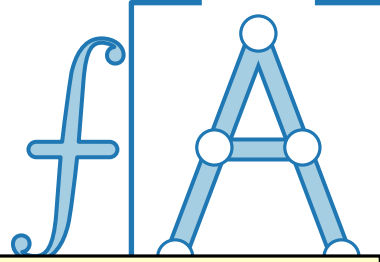
Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

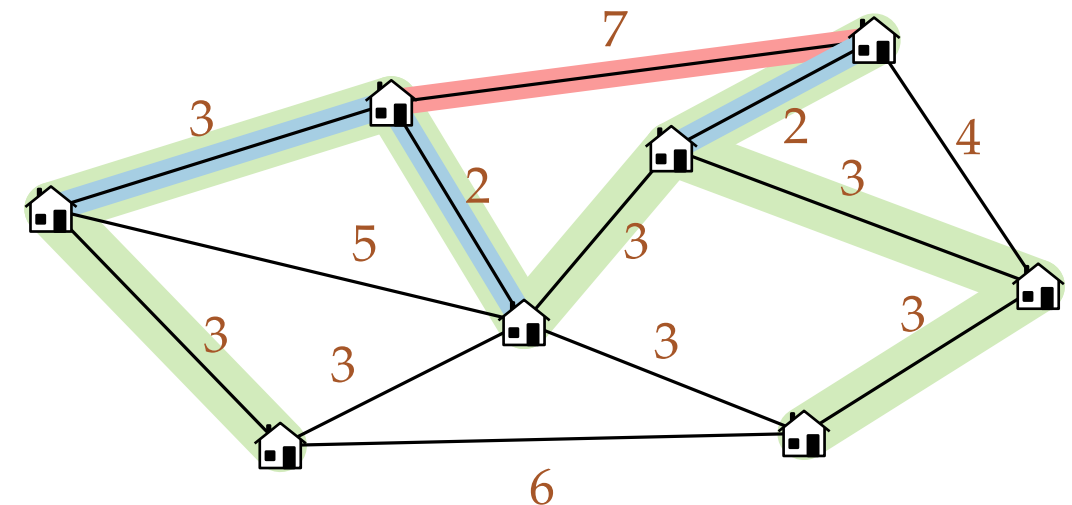
Rote Regel:

Wähle Kreis ohne **rote** Kante

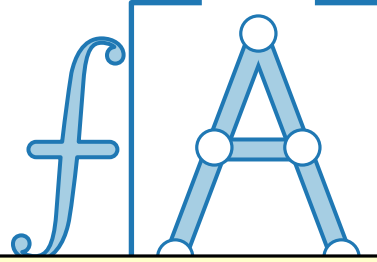
Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

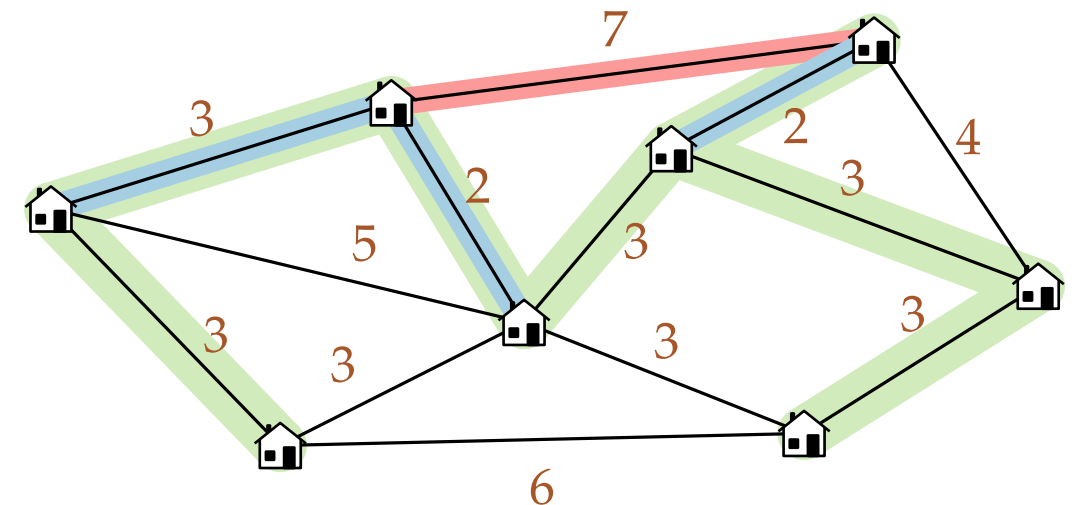
Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

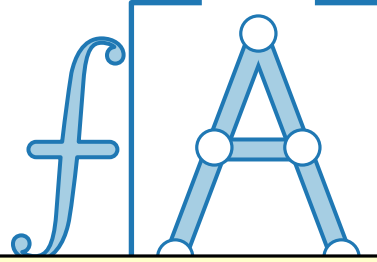
Sei K von **roter Regel** ausgewählter Kreis



Wähle Kreis ohne rote Kante
Färbe größte ungefärbte Kante auf Kreis rot

A weighted undirected graph with 8 nodes (houses) and 12 edges. The edges are labeled with weights: 3, 5, 2, 7, 2, 3, 4, 3, 3, 3, 6, 3. A red path of length 7 is highlighted, starting from the top-left house and ending at the top-right house.

Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

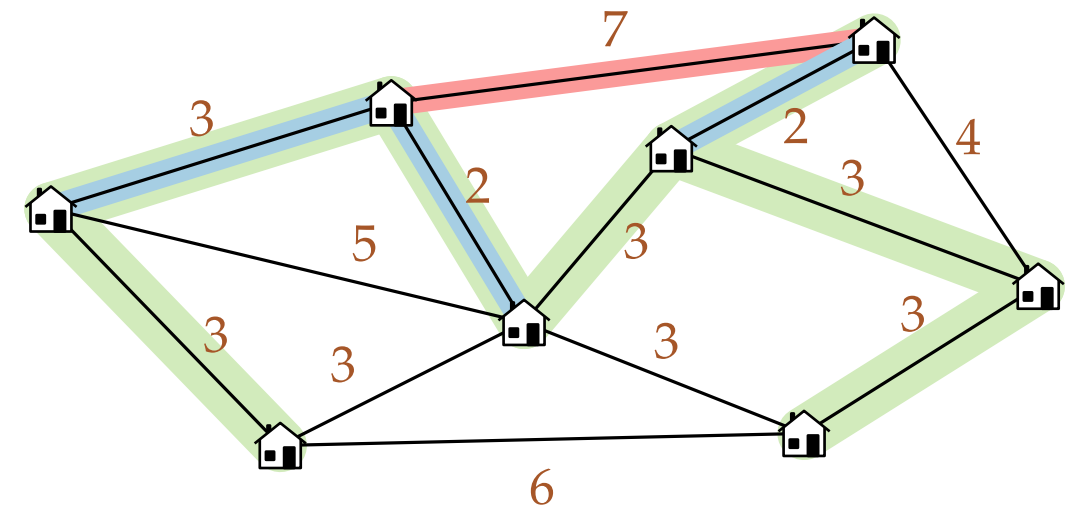
Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

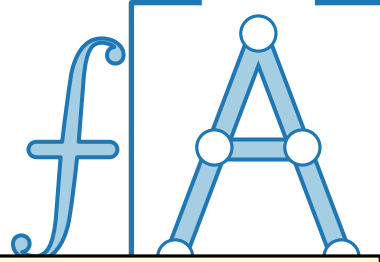
Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E'$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

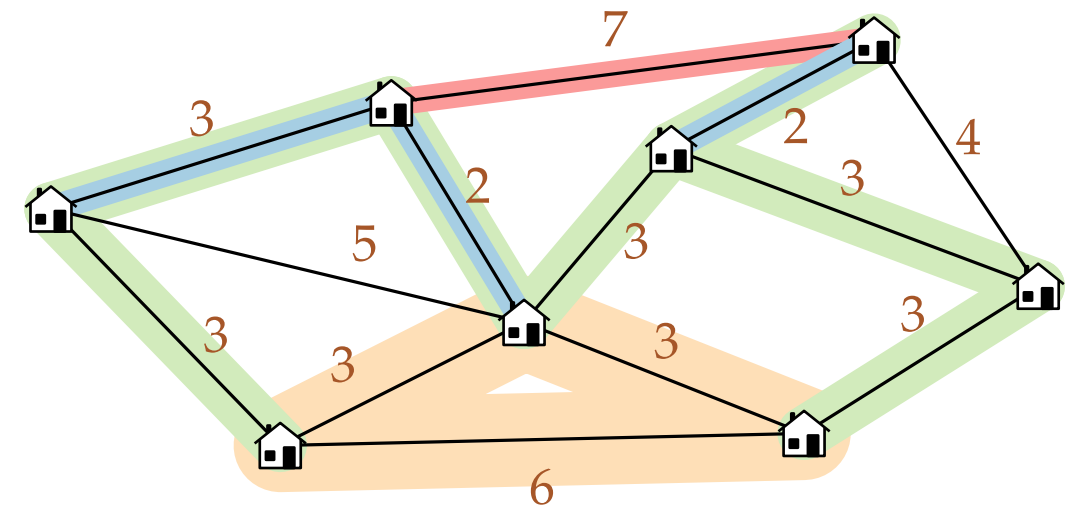
Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

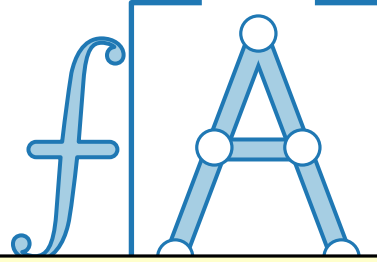
Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E'$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

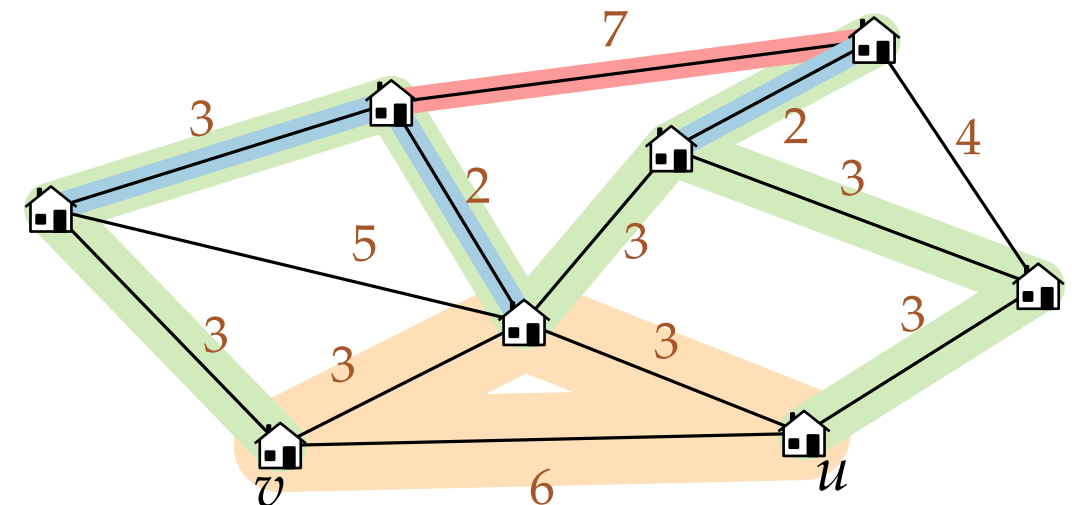
Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

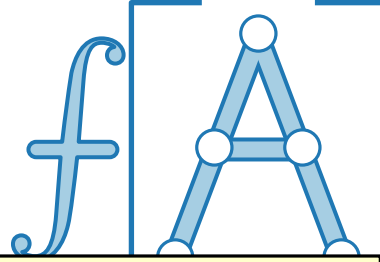
Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E'$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

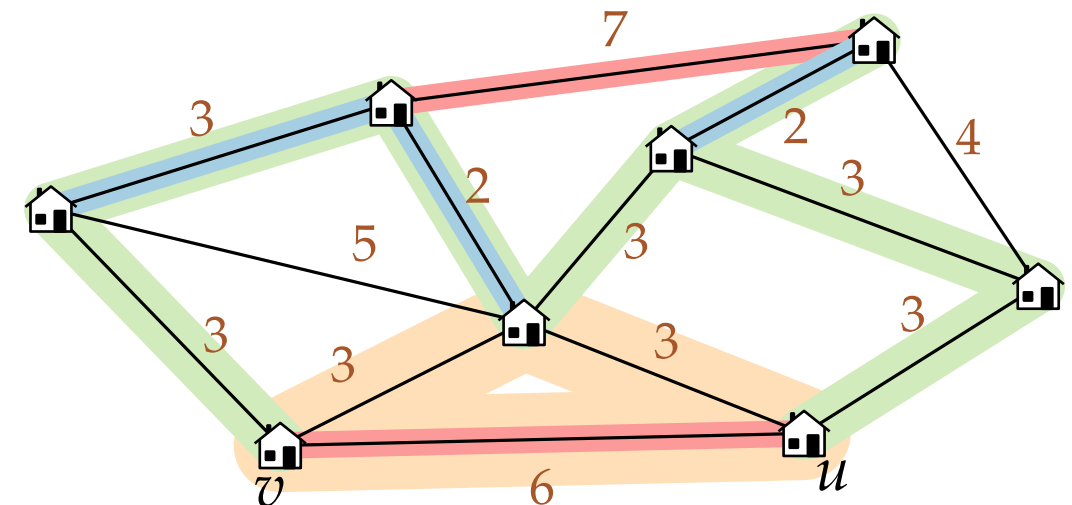
Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

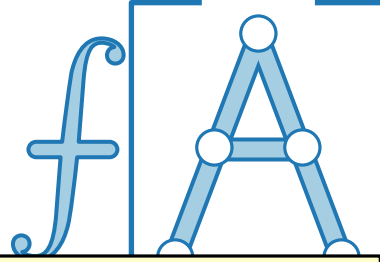
Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E'$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

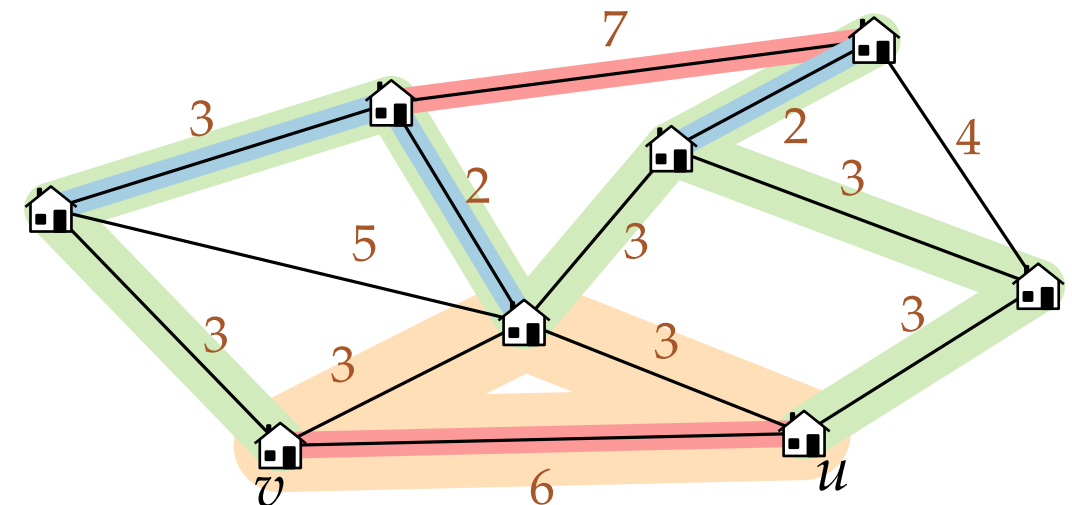
Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

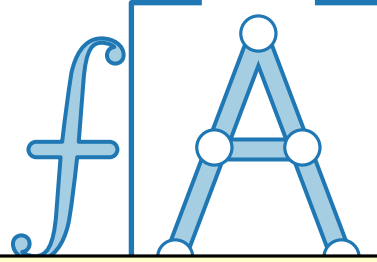
Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

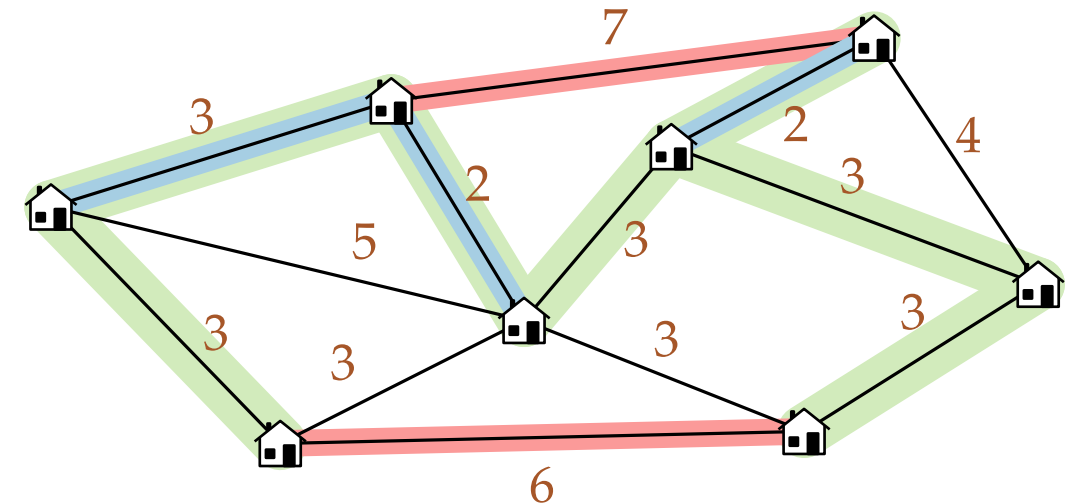
Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

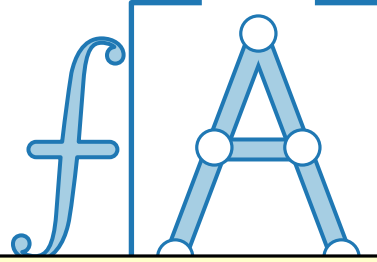
Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E'$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

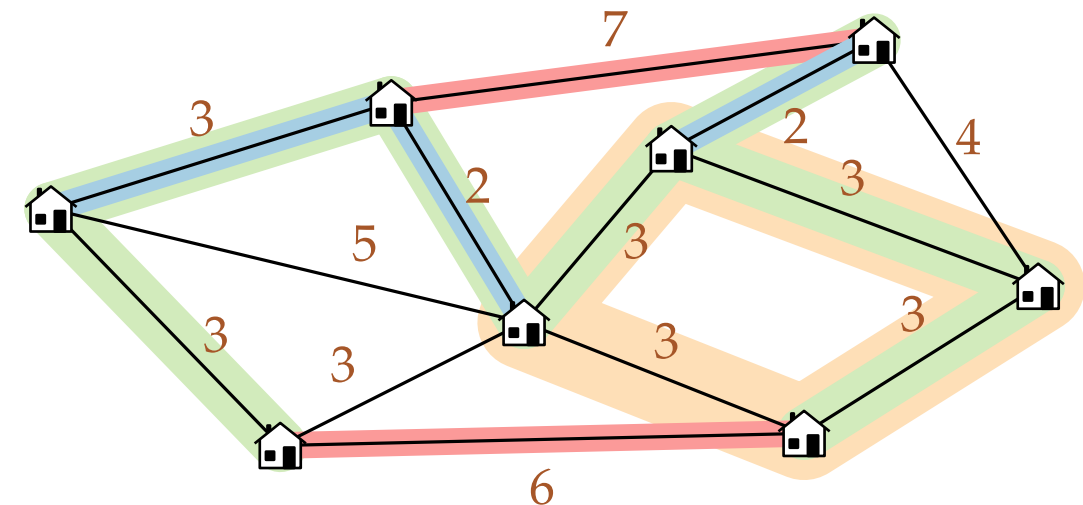
Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E'$



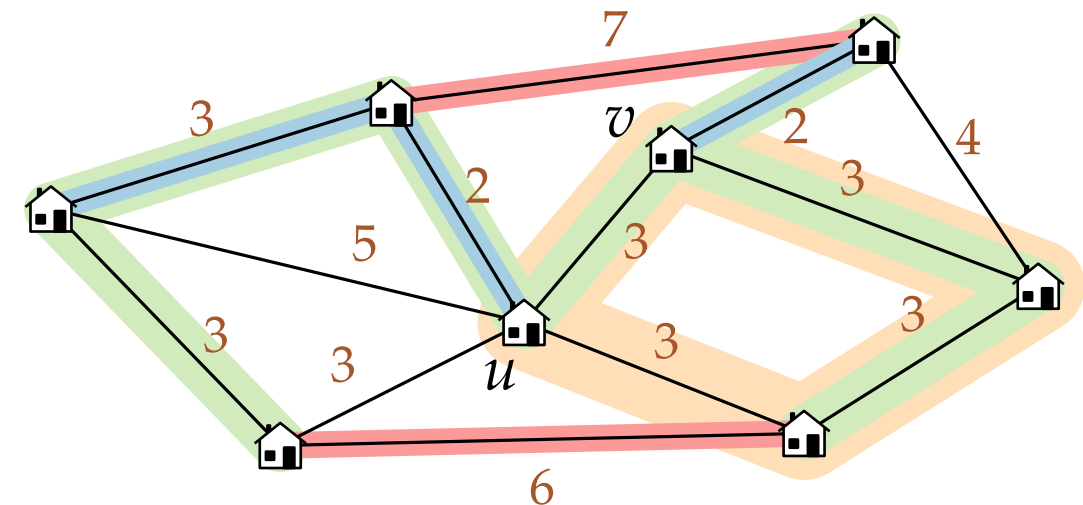
Wähle Kreis ohne rote Kante

- Färbe größte ungefärbte Kante auf Kreis **rot**

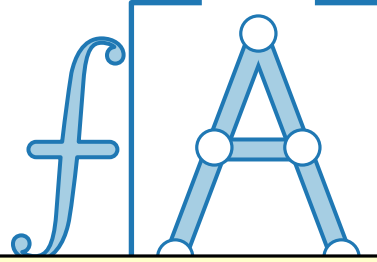
Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E'$ $\Rightarrow$ FI bleibt erhalten
2. Fall: $uv \in E'$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

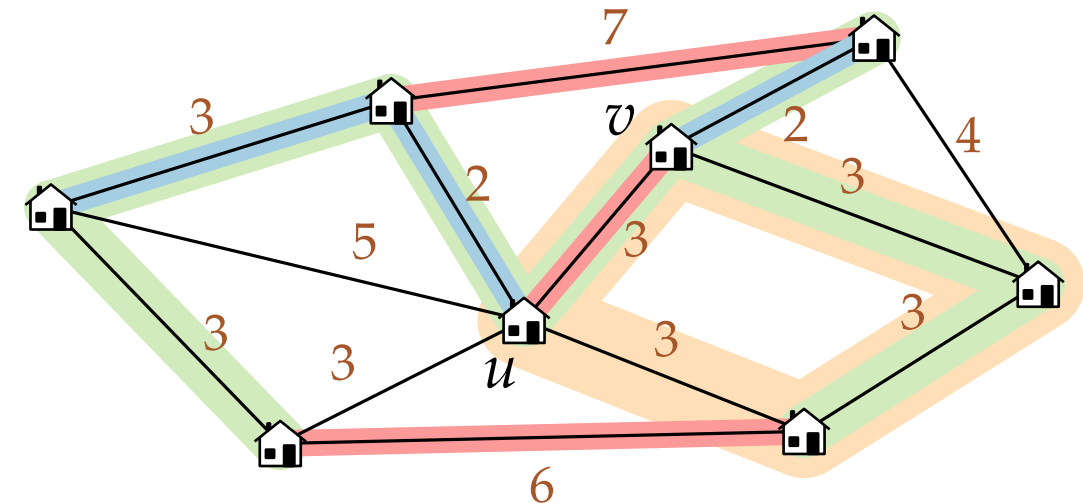
Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

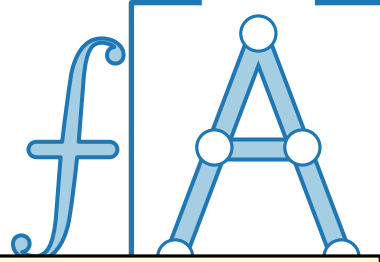
Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E'$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

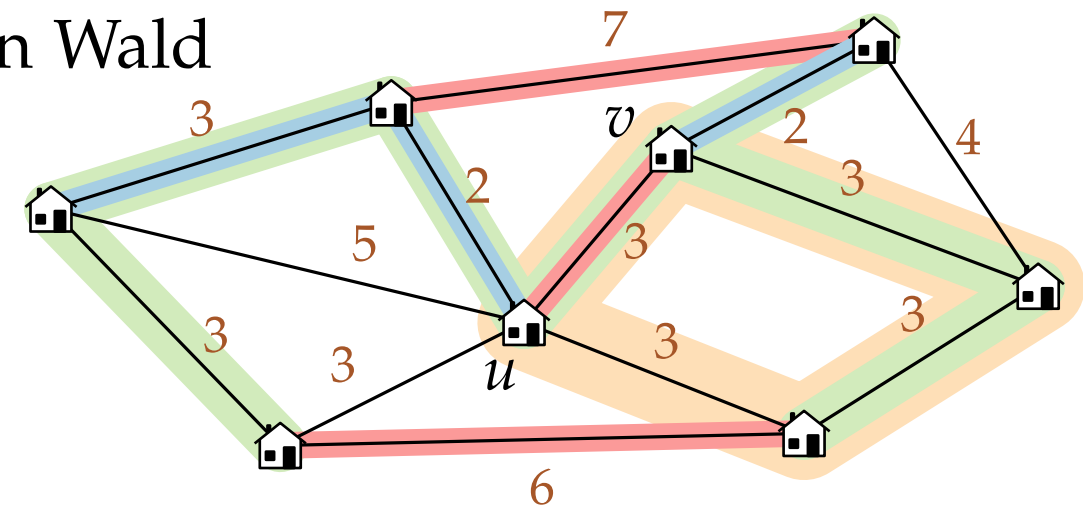
Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

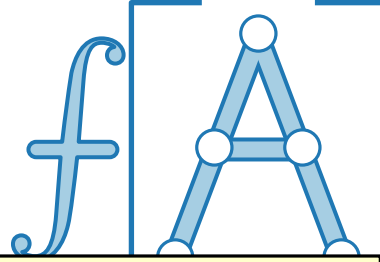
1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald mit zwei Bäumen T_1, T_2



Wähle Kreis ohne rote Kante
Färbe größte ungefärbte Kante auf Kreis rot

Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

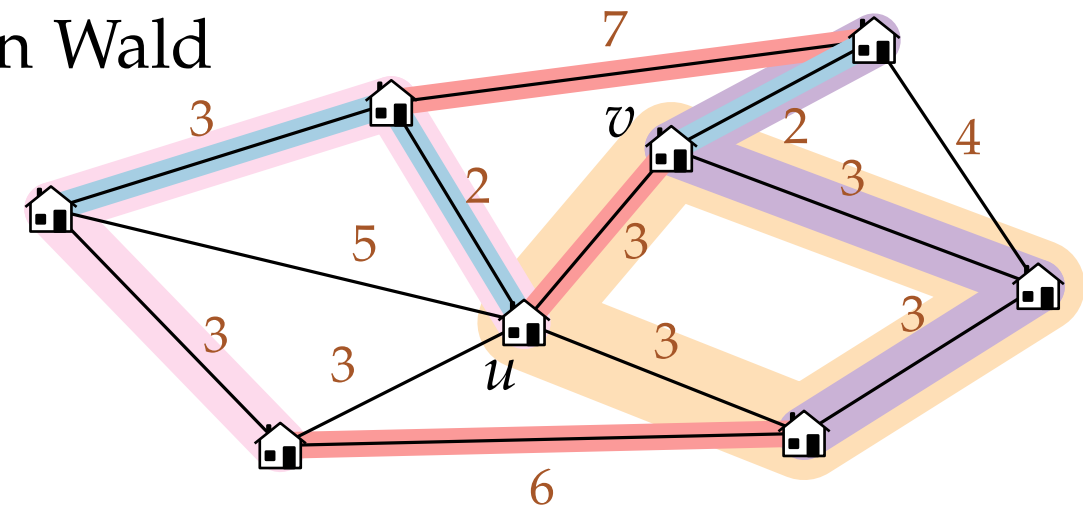
Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

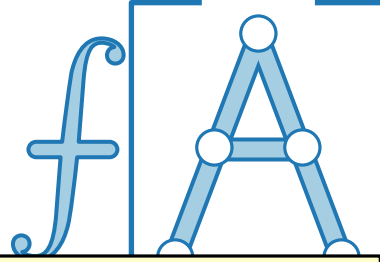
2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald

mit zwei Bäumen T_1, T_2

Sei $u \in T_1, v \in T_2$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

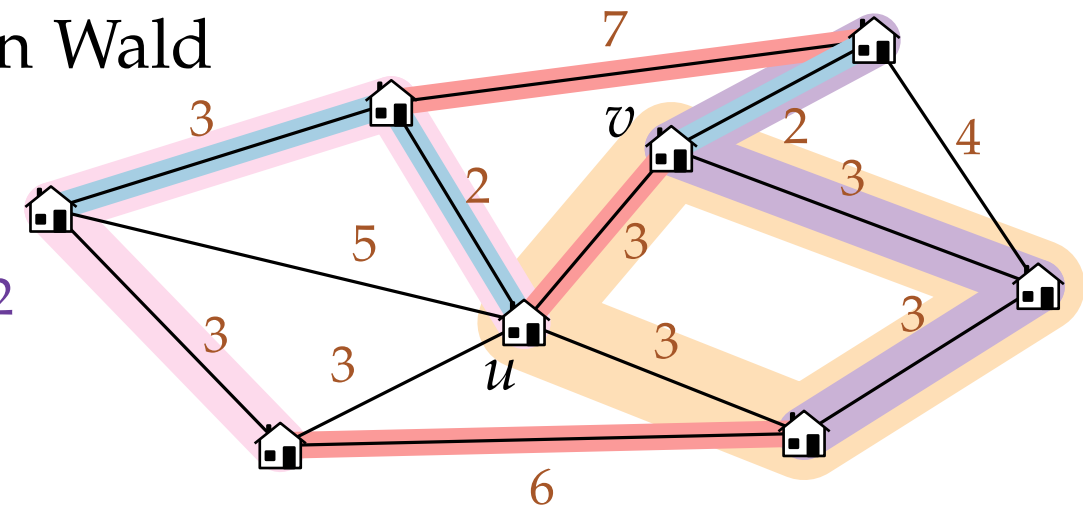
Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

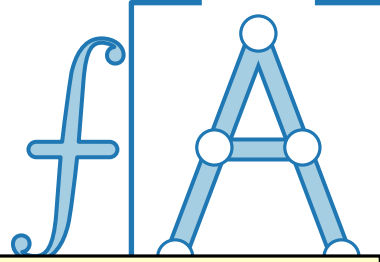
2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

Sei $u \in T_1, v \in T_2$

$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

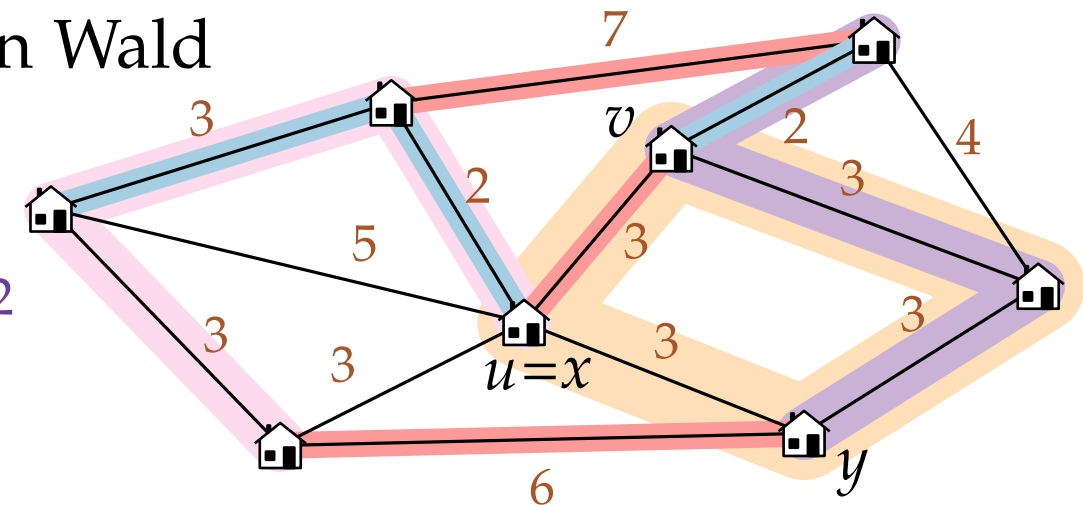
Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

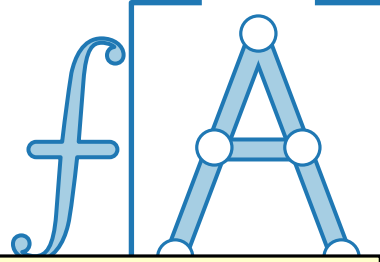
2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

Sei $u \in T_1, v \in T_2$

$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungefärbte Kante auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

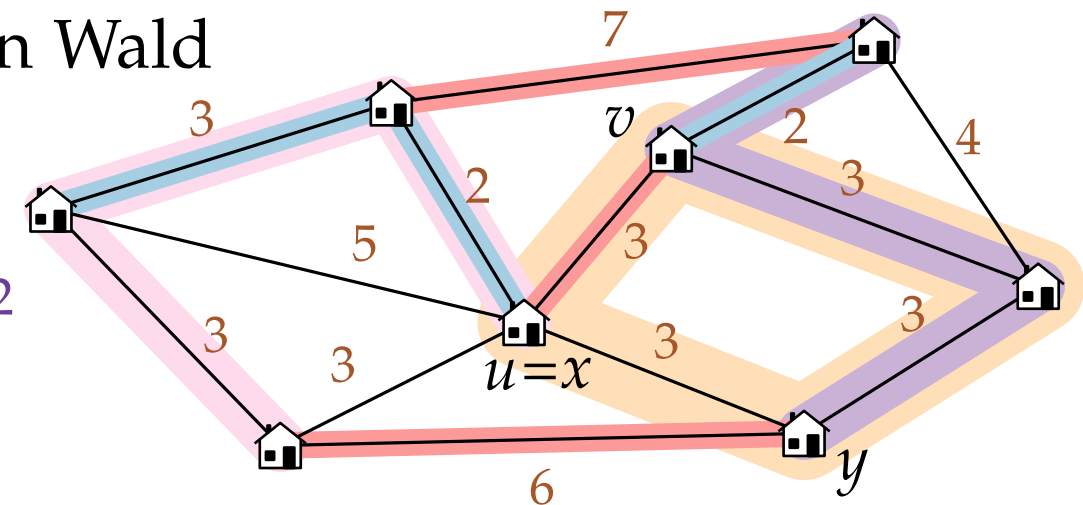
1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

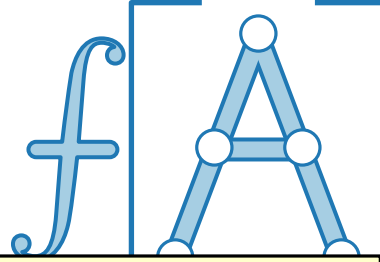
$xy \notin E'$

Sei $u \in T_1, v \in T_2$

$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe **größte ungefärbte Kante** auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

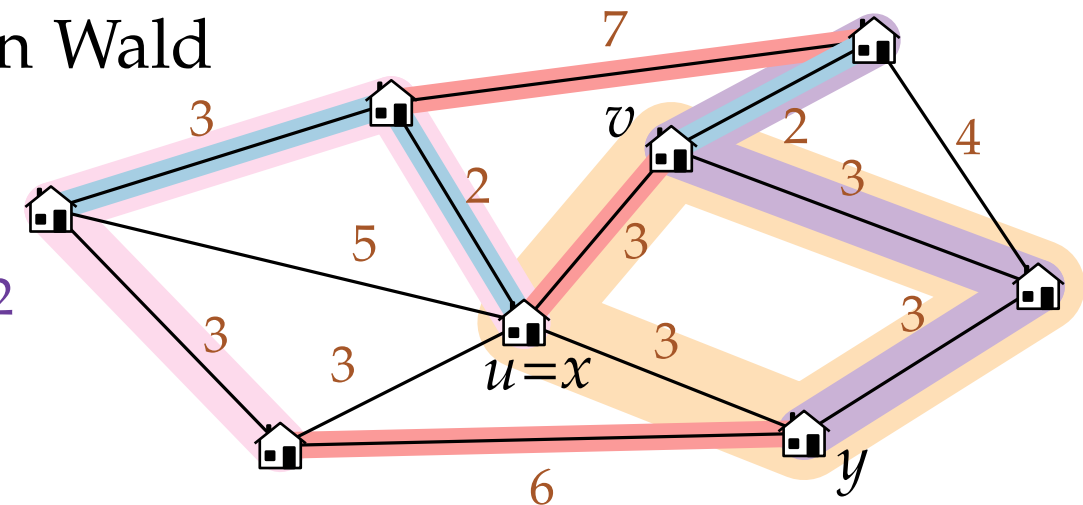
1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

$xy \notin E'$

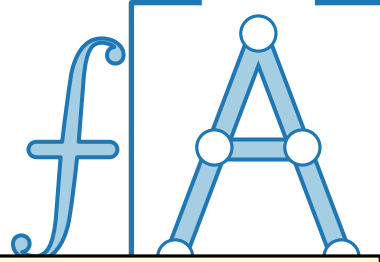
Sei $u \in T_1, v \in T_2$

$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$



Wähle Kreis ohne rote Kante
Färbe größte ungefärbte Kante auf Kreis rot

Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle blauen Kanten
- T enthält keine rote Kante

Rote Regel:

Wähle Kreis ohne rote Kante

Färbe größte ungefärbte Kante auf Kreis rot

Lemma. Die rote Regel hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von roter Regel ausgewählter Kreis

Sei $uv \in E$ von roter Regel gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

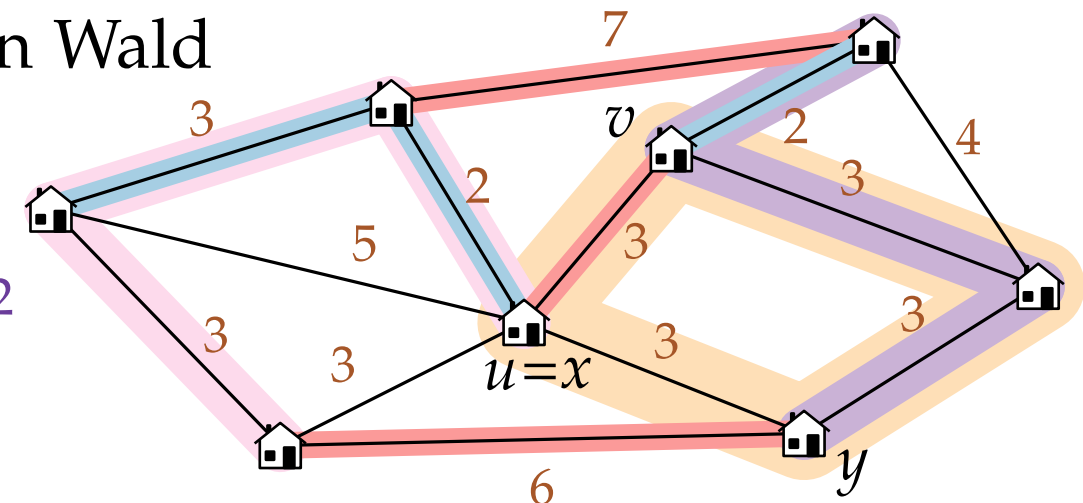
2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

$xy \notin E'$

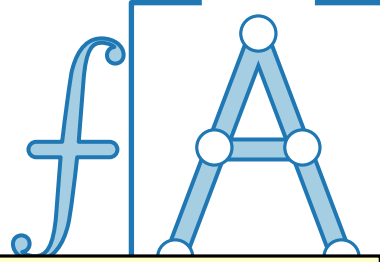
Sei $u \in T_1, v \in T_2$

$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$

größte ungefärbte Kante $\Rightarrow w(xy) \leq w(uv)$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis **ohne rote Kante**

Färbe **größte ungefärbte Kante** auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

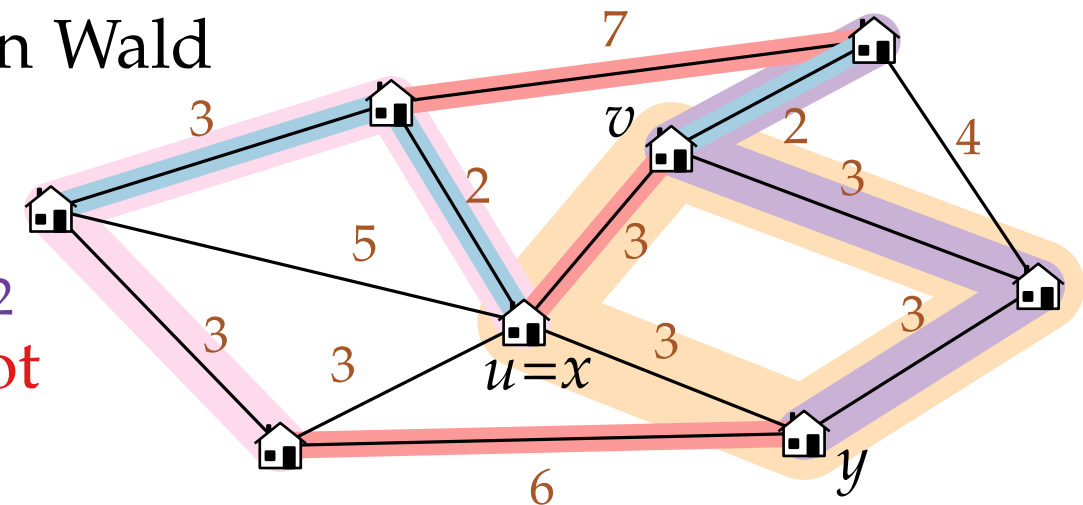
2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

$xy \notin E'$

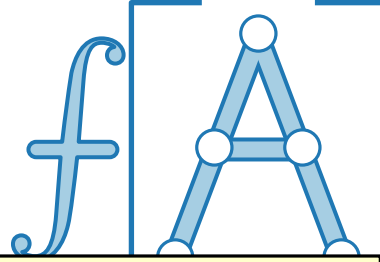
Sei $u \in T_1, v \in T_2$

$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$

größte ungefärbte Kante $\Rightarrow w(xy) \leq w(uv)$ **ohne rote Kante** $\Rightarrow xy$ nicht **rot**



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis **ohne rote Kante**

Färbe **größte ungefärbte Kante** auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

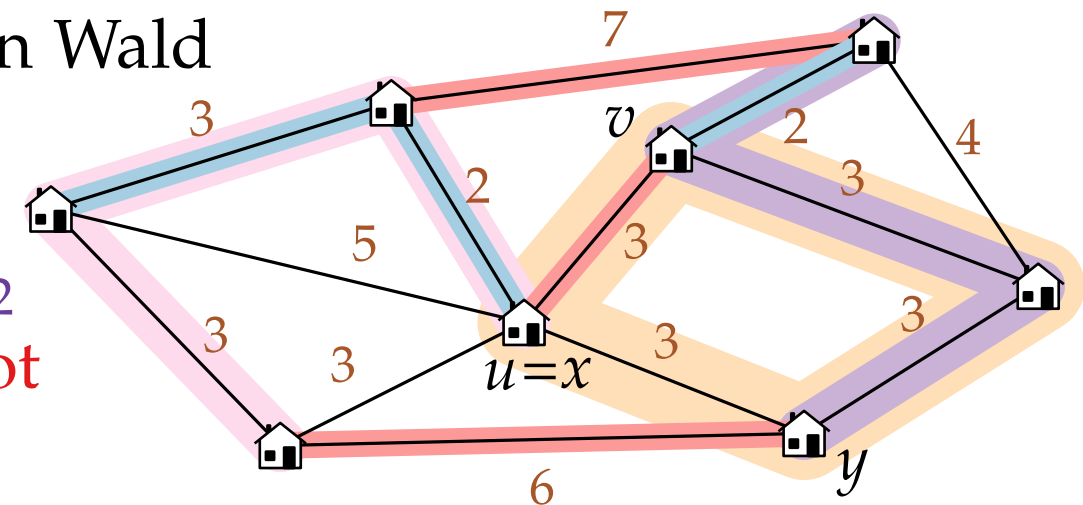
$xy \notin E'$

Sei $u \in T_1, v \in T_2$

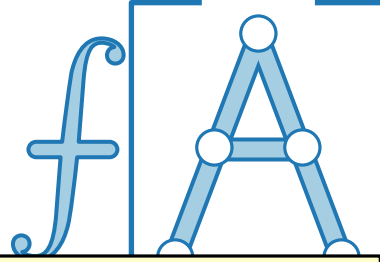
$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$

größte ungefärbte Kante $\Rightarrow w(xy) \leq w(uv)$ **ohne rote Kante** $\Rightarrow xy$ nicht **rot**

Wähle $E'' = E' \cup \{xy\} \setminus \{uv\}$



Beweis der roten Regel



Farbinvariante (FI): Es gibt einen MSB T :

- T enthält alle **blauen Kanten**
- T enthält keine **rote Kante**

Rote Regel:

Wähle Kreis **ohne rote Kante**

Färbe **größte ungefärbte Kante** auf Kreis **rot**

Lemma. Die **rote Regel** hält die Farbinvariante aufrecht.

Beweis. Sei $T = (V, E')$ min. Spannbaum, der FI bezeugt

Sei K von **roter Regel** ausgewählter Kreis

Sei $uv \in E$ von **roter Regel** gefärbte Kante

1. Fall: $uv \notin E' \Rightarrow$ FI bleibt erhalten

2. Fall: $uv \in E' \Rightarrow E' \setminus \{uv\}$ bildet aufspannenden Wald
mit zwei Bäumen T_1, T_2

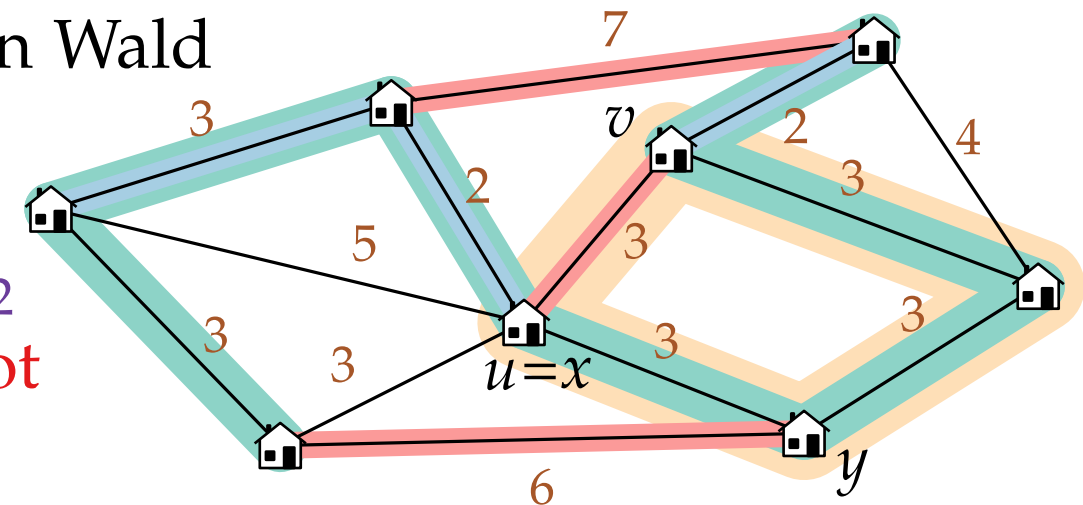
$xy \notin E'$

Sei $u \in T_1, v \in T_2$

$\Rightarrow$ es gibt Kante $xy \neq uv$ in K mit $x \in T_1, y \in T_2$

größte ungefärbte Kante $\Rightarrow w(xy) \leq w(uv)$ **ohne rote Kante** $\Rightarrow xy$ nicht **rot**

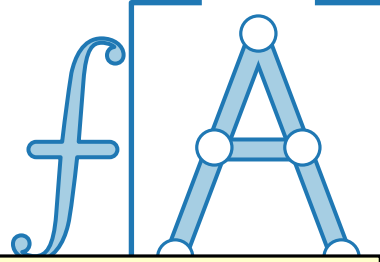
Wähle $E'' = E' \cup \{xy\} \setminus \{uv\}$



Färbe größte ungefärbte Kante auf Kreis rot

-

Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt

Färbe leichte Kante **blau**

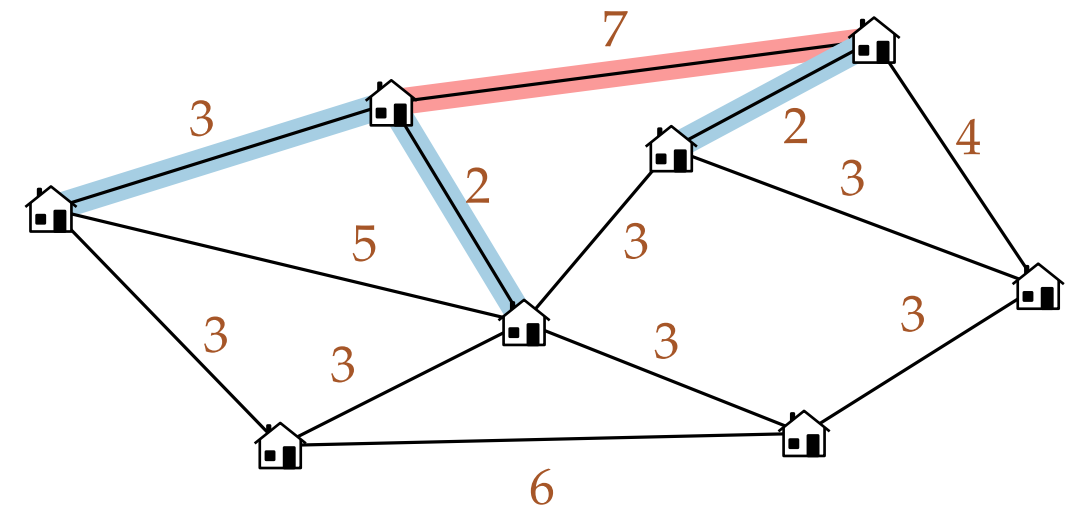
Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Wähle Kreis ohne rote Kante
Färbe größte ungef. Kante auf Kreis rot

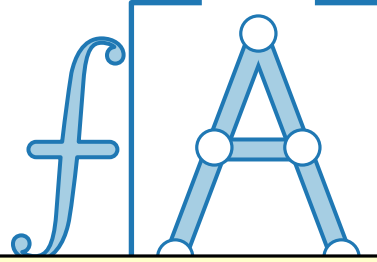
Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis.



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

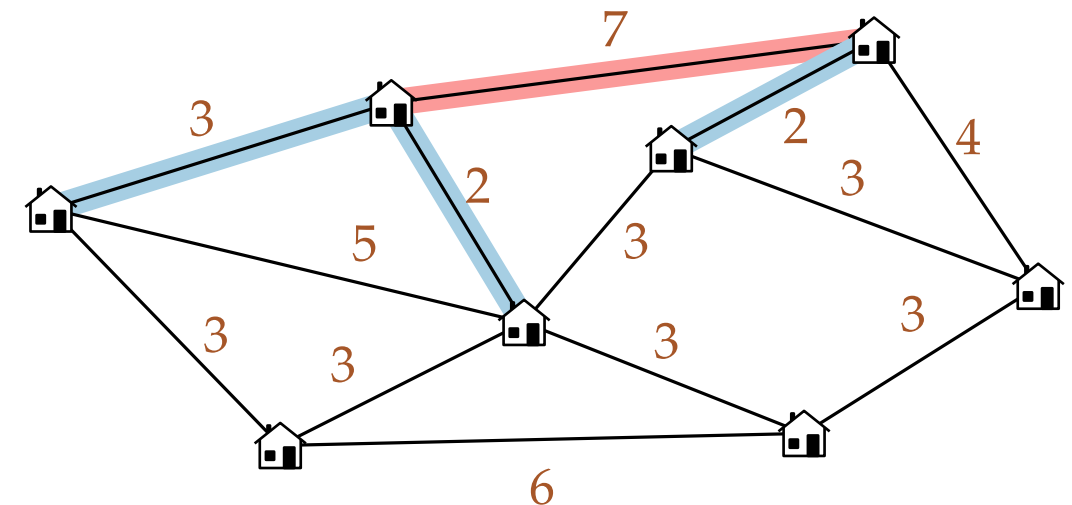
Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt

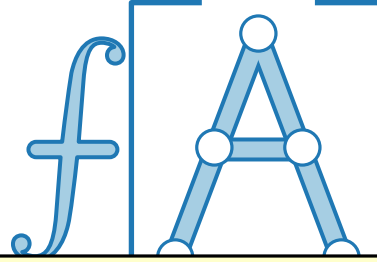
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

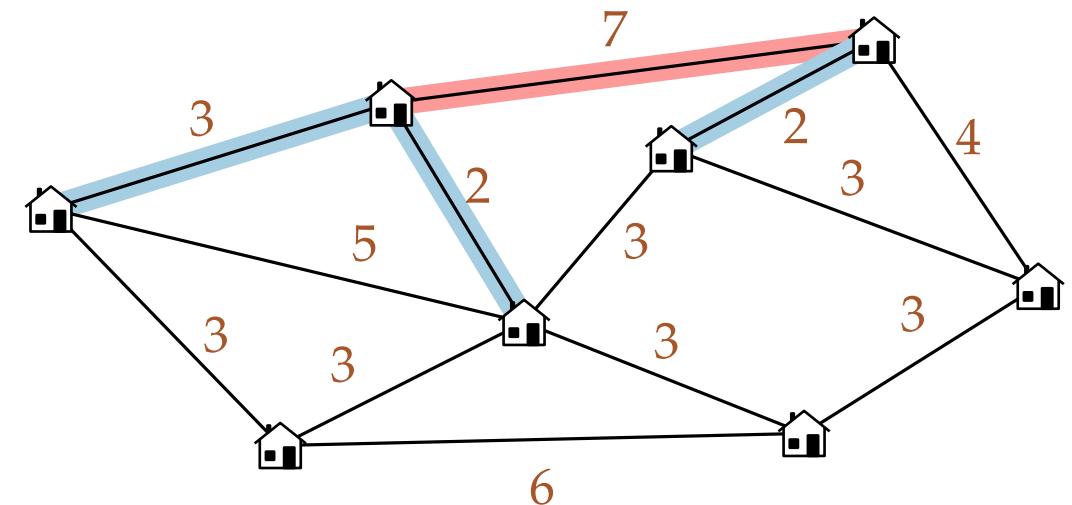
Wähle Schnitt, den keine **blaue** Kante kreuzt

Färbe leichte Kante **blau**

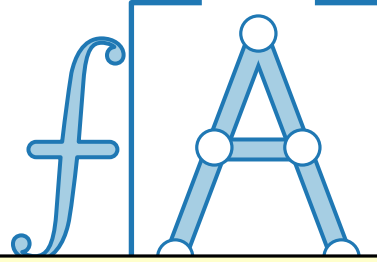
Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt

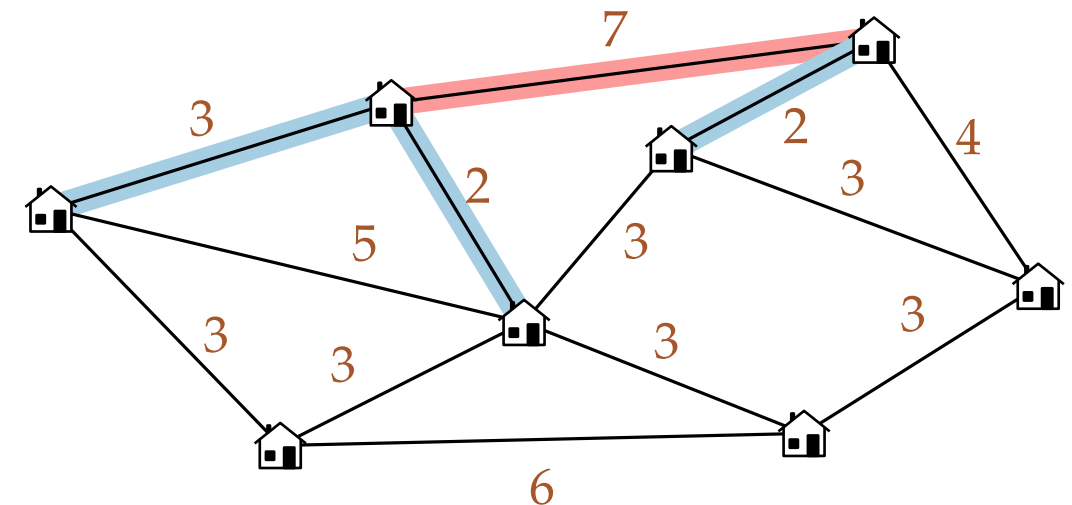
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

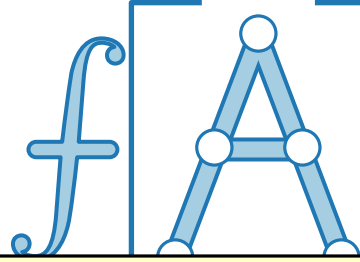
Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus B



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt

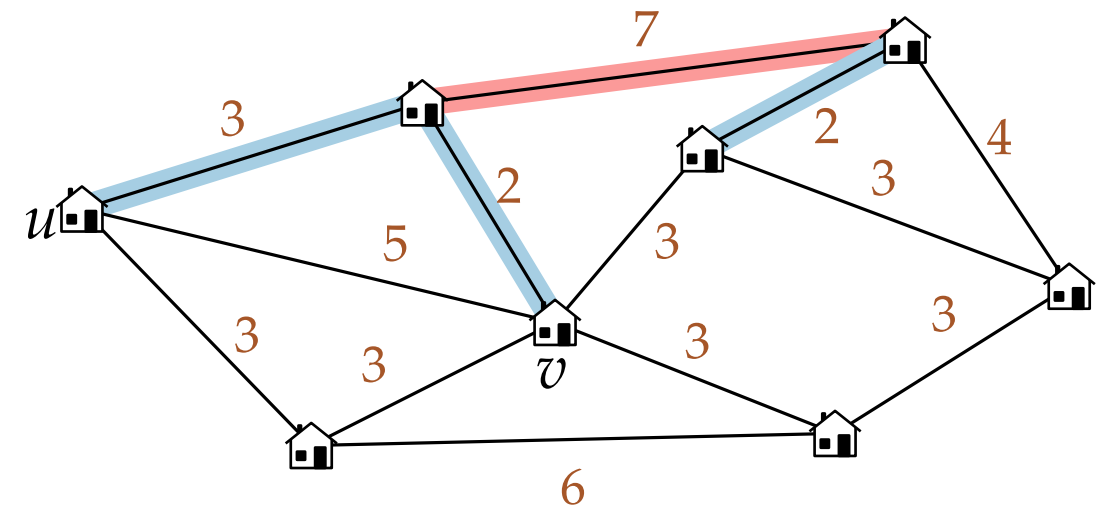
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

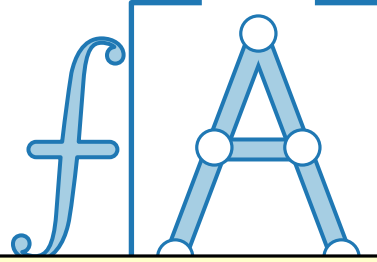
Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus B



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

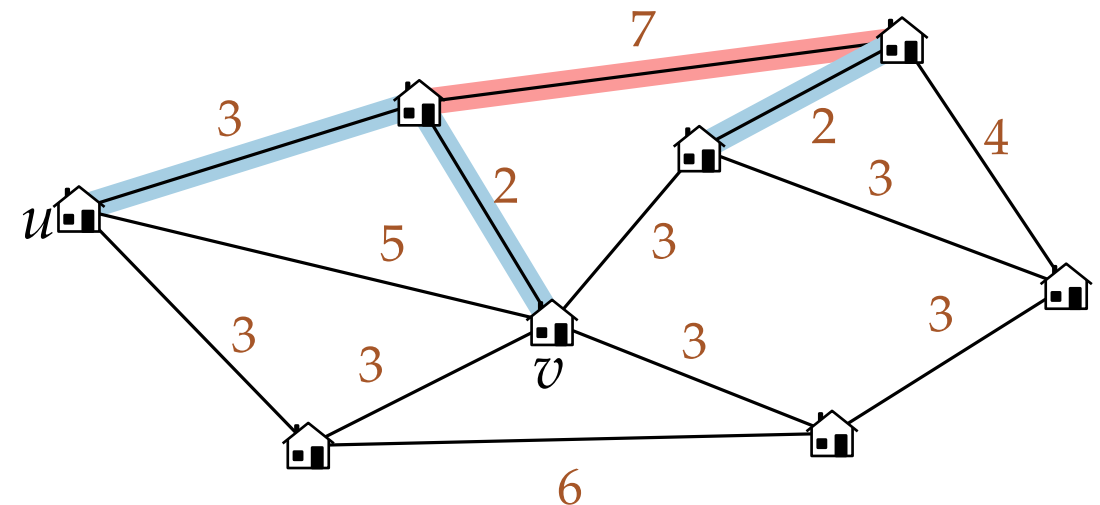
Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

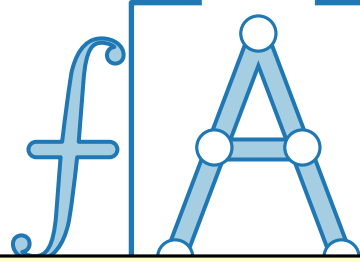
Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus B

Wähle Kreis C : Pfad in B von v zu u + Kante uv



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt

Färbe leichte Kante **blau**

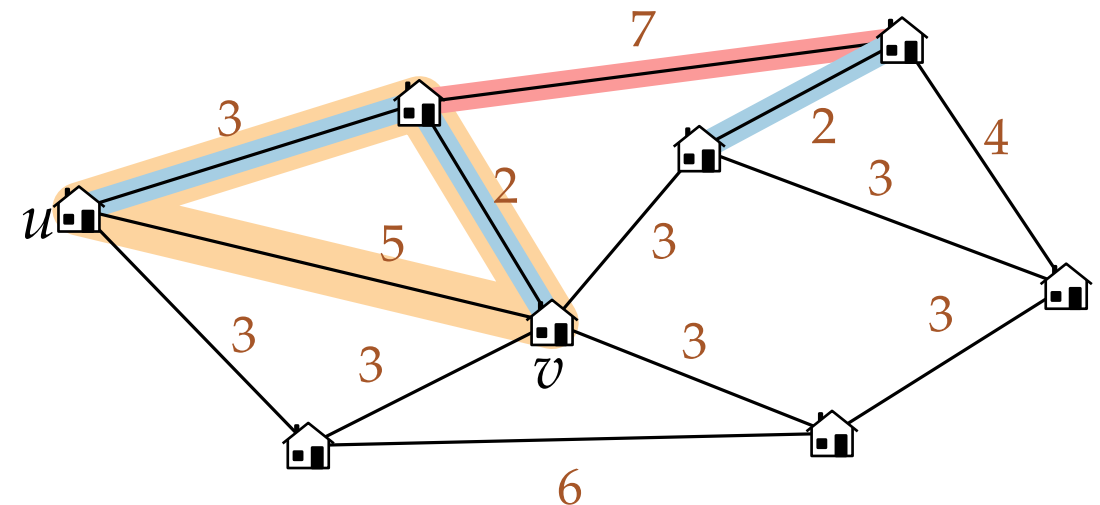
Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

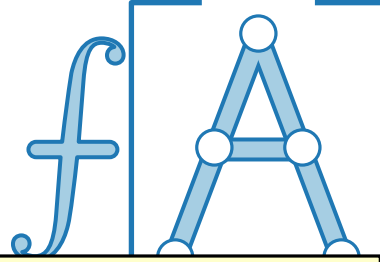
1. Fall: uv verbindet Knoten *eines* Baumes aus B

Wähle Kreis C : Pfad in B von v zu u + Kante uv



Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

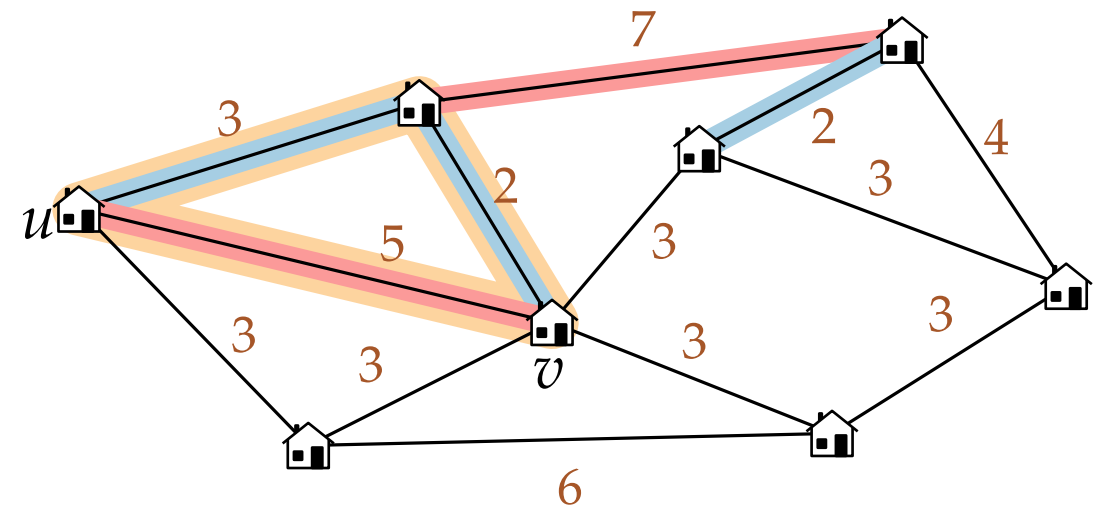
Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus B

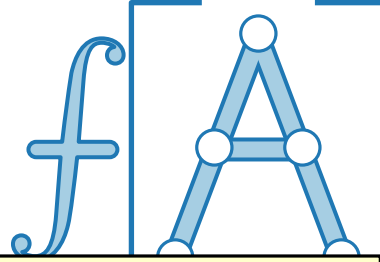
Wähle Kreis C : Pfad in B von v zu u + Kante uv

$\Rightarrow$ Kanten auf C alle **blau** bis auf uv

$\Rightarrow$ **rote Regel** anwendbar



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

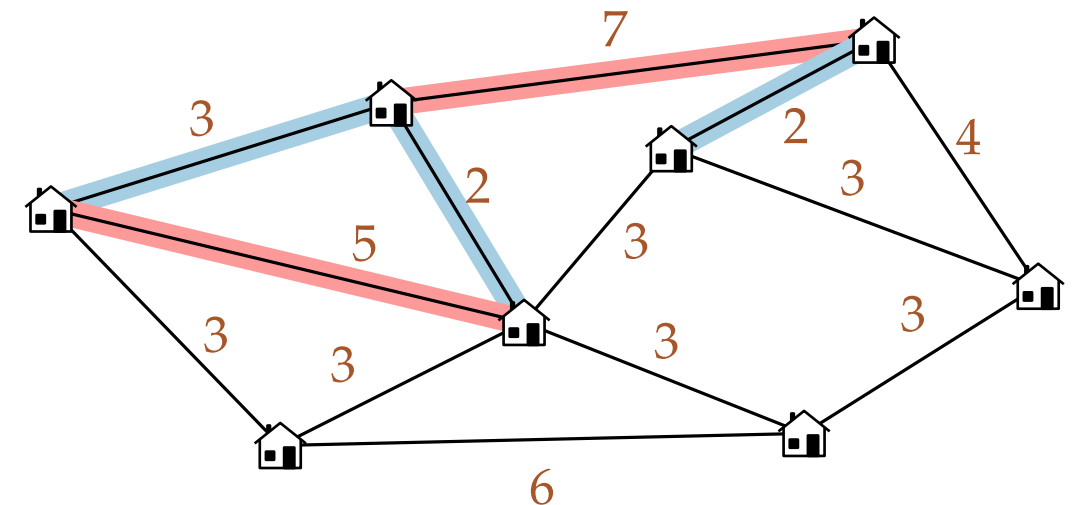
1. Fall: uv verbindet Knoten *eines* Baumes aus B

Wähle Kreis C : Pfad in B von v zu u + Kante uv

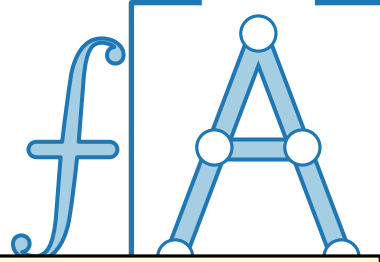
$\Rightarrow$ Kanten auf C alle **blau** bis auf uv

$\Rightarrow$ **rote Regel** anwendbar

2. Fall: uv verbindet *unterschiedliche* Bäume aus B



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt

Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

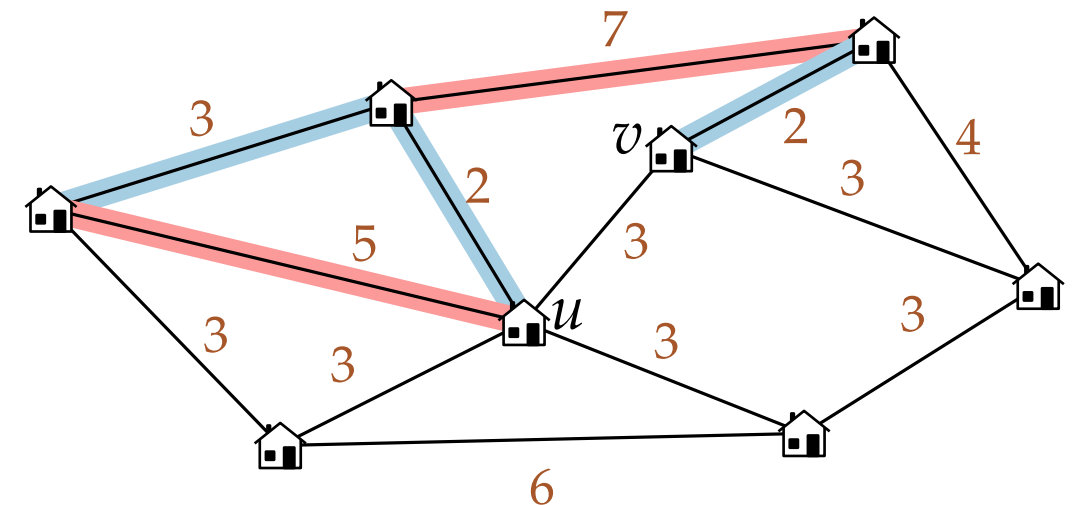
1. Fall: uv verbindet Knoten *eines* Baumes aus B

Wähle Kreis C : Pfad in B von v zu u + Kante uv

$\Rightarrow$ Kanten auf C alle **blau** bis auf uv

$\Rightarrow$ **rote Regel** anwendbar

2. Fall: uv verbindet *unterschiedliche* Bäume aus B



Wähle Kreis ohne rote Kante
Färbe größte ungef. Kante auf Kreis rot

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus $\mathcal{B}$

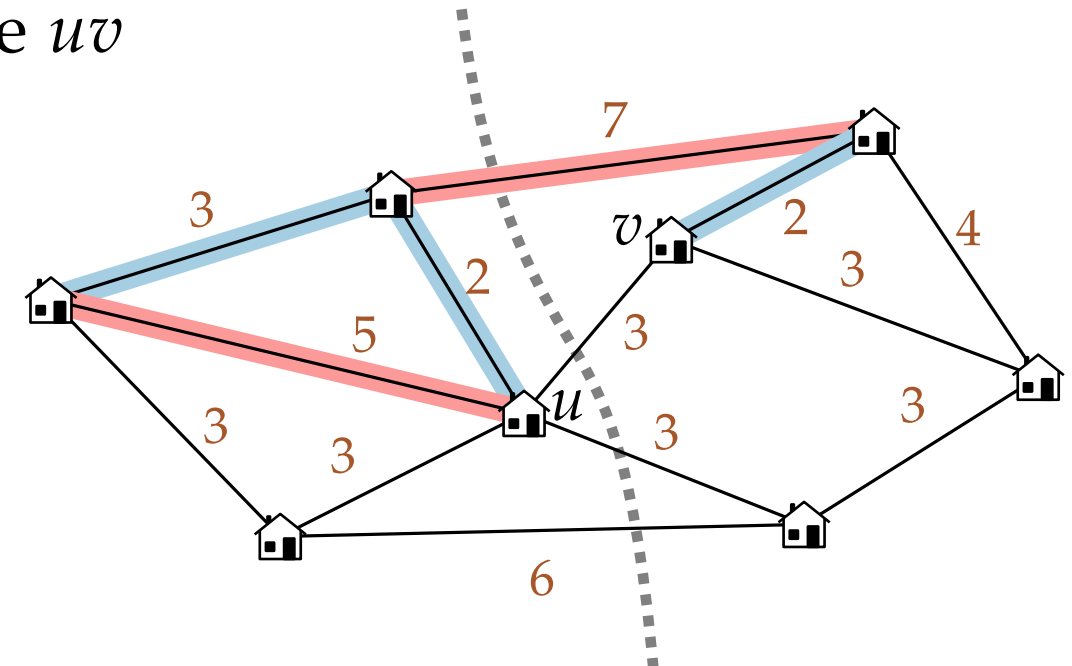
Wähle Kreis **C**: Pfad in **B** von v zu u + Kante uv

$\Rightarrow$ Kanten auf **C** alle **blau** bis auf uv

⇒ rote Regel anwendbar

2. Fall: uv verbindet *unterschiedliche* Bäume aus B

$\Rightarrow$ es gibt Schnitt ohne blaue Kanten



Wähle Kreis ohne rote Kante
Färbe größte ungef. Kante auf Kreis rot

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus $\mathcal{B}$

Wähle Kreis **C**: Pfad in **B** von v zu u + Kante uv

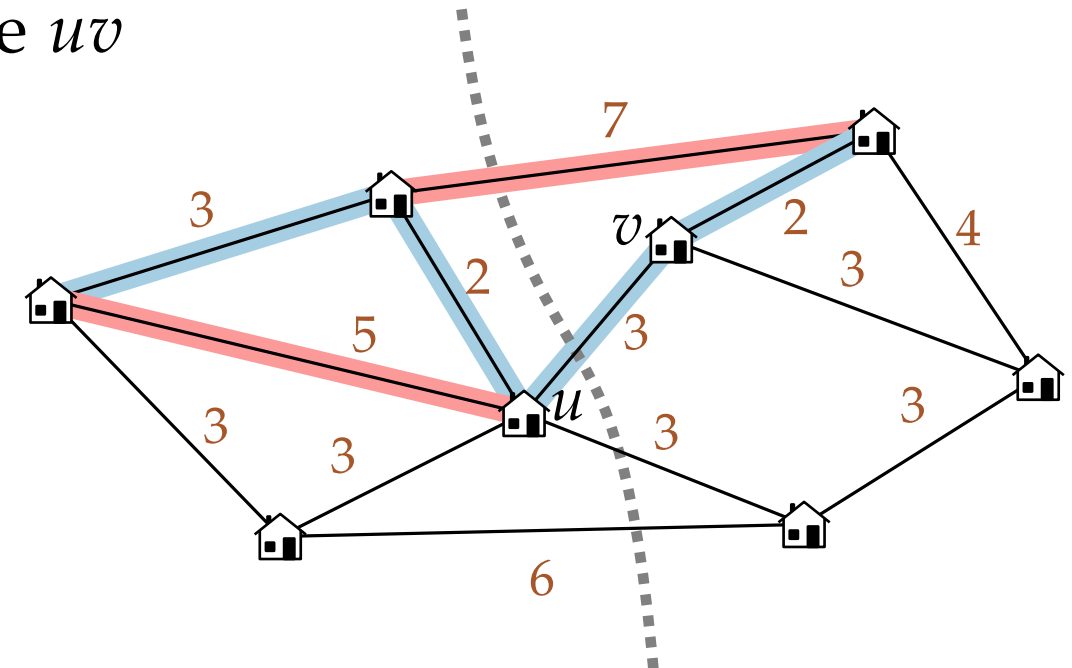
$\Rightarrow$ Kanten auf C alle blau bis auf uv

⇒ rote Regel anwendbar

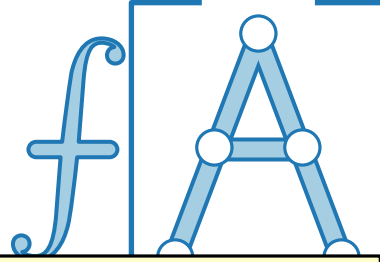
2. Fall: uv verbindet *unterschiedliche* Bäume aus B

$\Rightarrow$ es gibt Schnitt ohne blaue Kanten

⇒ blaue Regel anwendbar



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus B

Wähle Kreis C : Pfad in B von v zu u + Kante uv

$\Rightarrow$ Kanten auf C alle **blau** bis auf uv

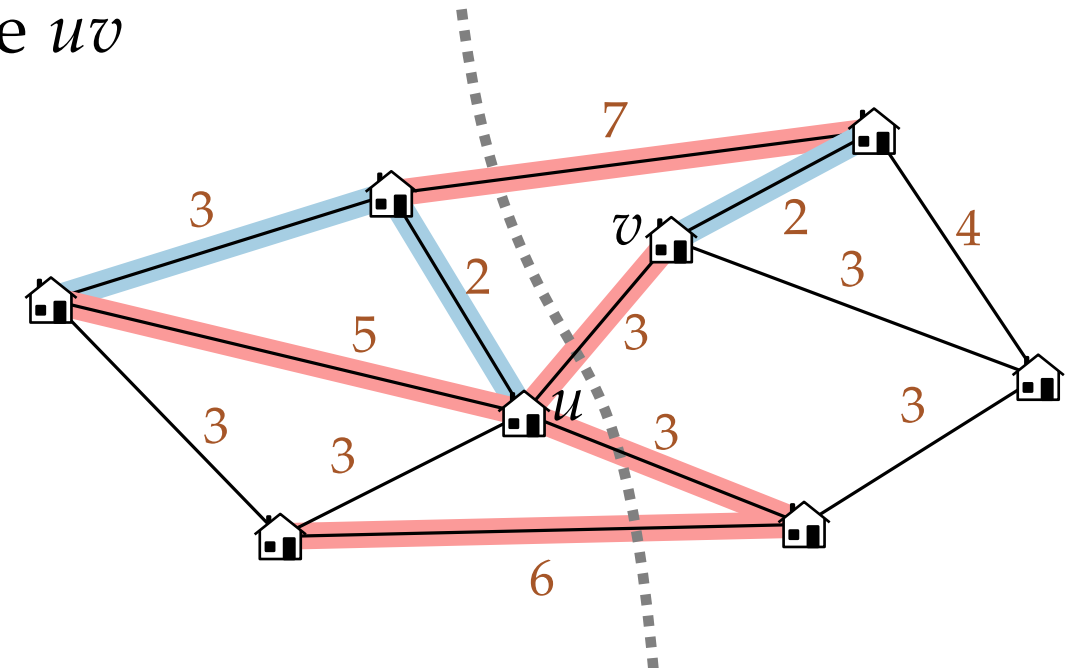
$\Rightarrow$ **rote Regel** anwendbar

2. Fall: uv verbindet *unterschiedliche* Bäume aus B

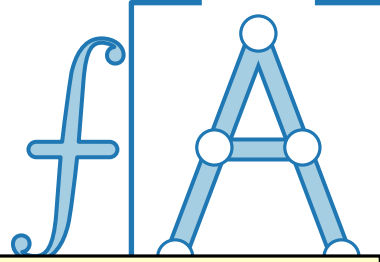
$\Rightarrow$ es gibt Schnitt ohne **blaue Kanten**

$\Rightarrow$ **blaue Regel** anwendbar

Was wenn alle Kanten auf Schnitt **rot**?



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus B

Wähle Kreis C : Pfad in B von v zu u + Kante uv

$\Rightarrow$ Kanten auf C alle **blau** bis auf uv

$\Rightarrow$ **rote Regel** anwendbar

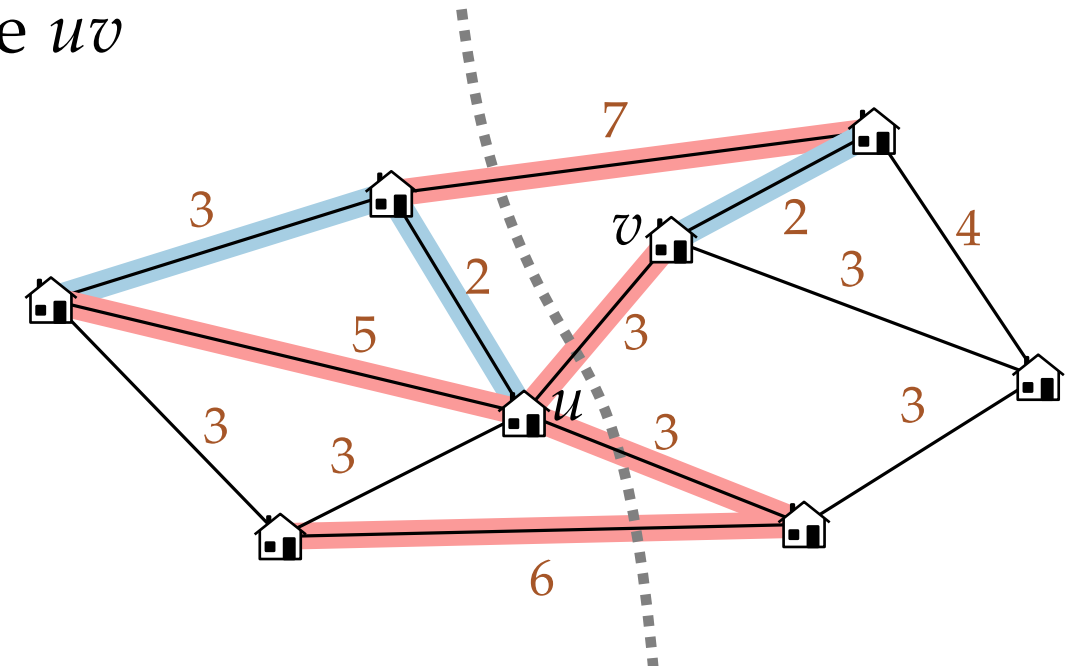
2. Fall: uv verbindet *unterschiedliche* Bäume aus B

$\Rightarrow$ es gibt Schnitt ohne **blaue Kanten**

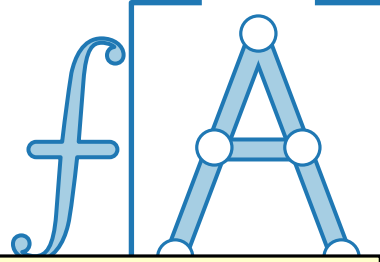
$\Rightarrow$ **blaue Regel** anwendbar

Was wenn alle Kanten auf Schnitt **rot**?

Widerspruch zu uv ungefärbt ⚡



Alle Kanten werden gefärbt



Rote Regel:

Wähle Kreis ohne **rote** Kante

Färbe größte ungef. Kante auf Kreis **rot**

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt

Färbe leichte Kante **blau**

Lemma. GREEDYSPANNBAUM färbt alle Kanten.

Beweis. Blaue Kanten bilden Wald B (ggfs. isolierte Knoten)

Sei uv ungefärbte Kante

1. Fall: uv verbindet Knoten *eines* Baumes aus B

Wähle Kreis C : Pfad in B von v zu u + Kante uv

$\Rightarrow$ Kanten auf C alle **blau** bis auf uv

$\Rightarrow$ **rote Regel** anwendbar

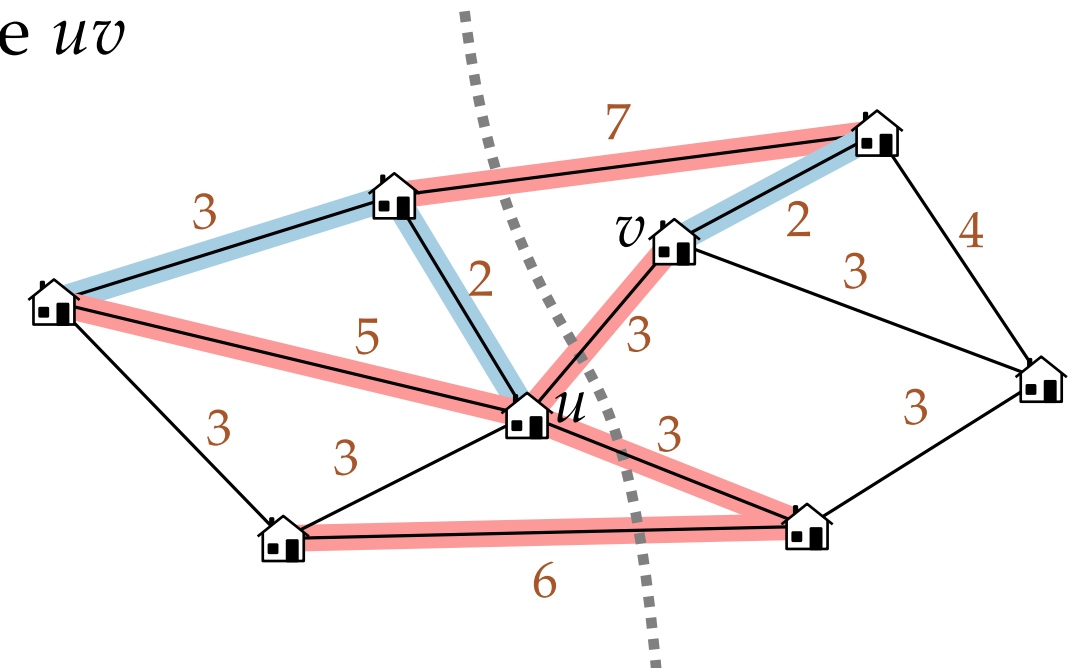
2. Fall: uv verbindet *unterschiedliche* Bäume aus B

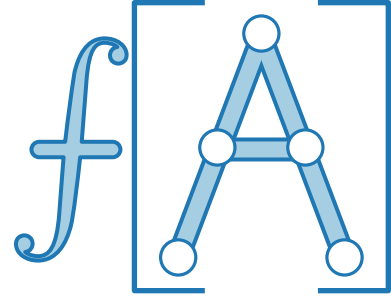
$\Rightarrow$ es gibt Schnitt ohne **blaue Kanten**

$\Rightarrow$ **blaue Regel** anwendbar

Was wenn alle Kanten auf Schnitt **rot**?

Widerspruch zu uv ungefärbt ⚡

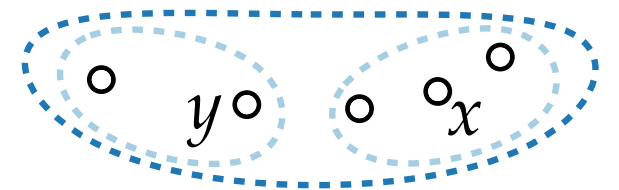
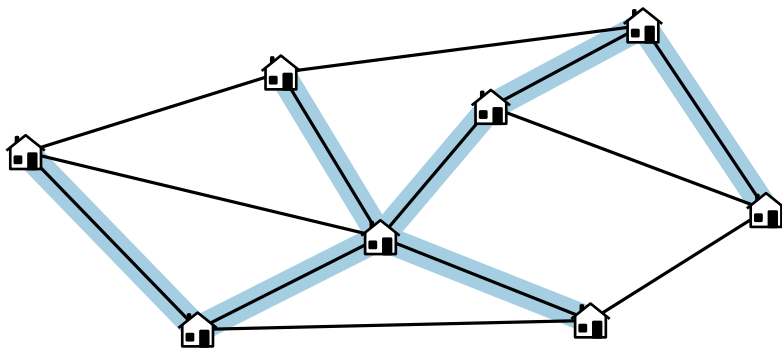




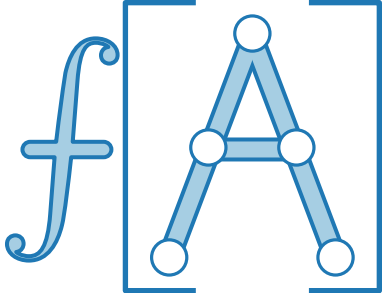
Fortgeschrittene Algorithmen

7. Vorlesung Graphalgorithmen I: Minimale Spannbäume

Teil IV: Algorithmen

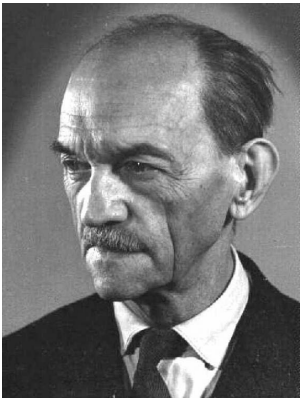


Der Algorithmus von Jarník-Prim (1930/1957)

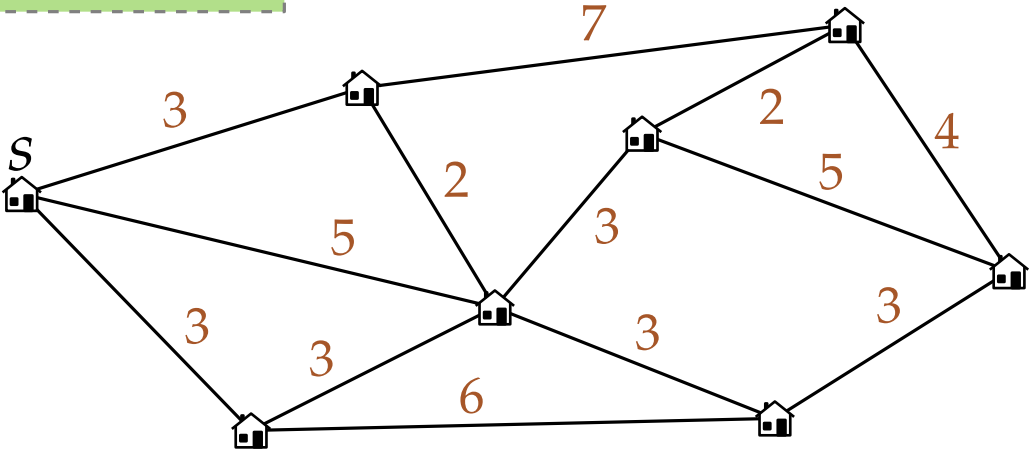


JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

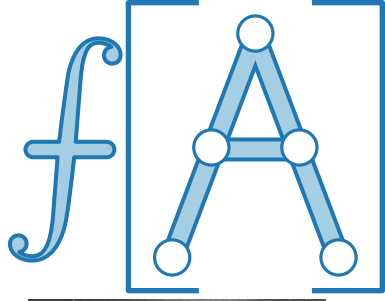
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)

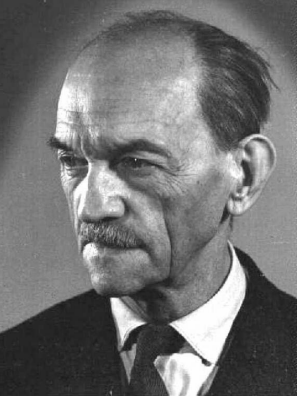


JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

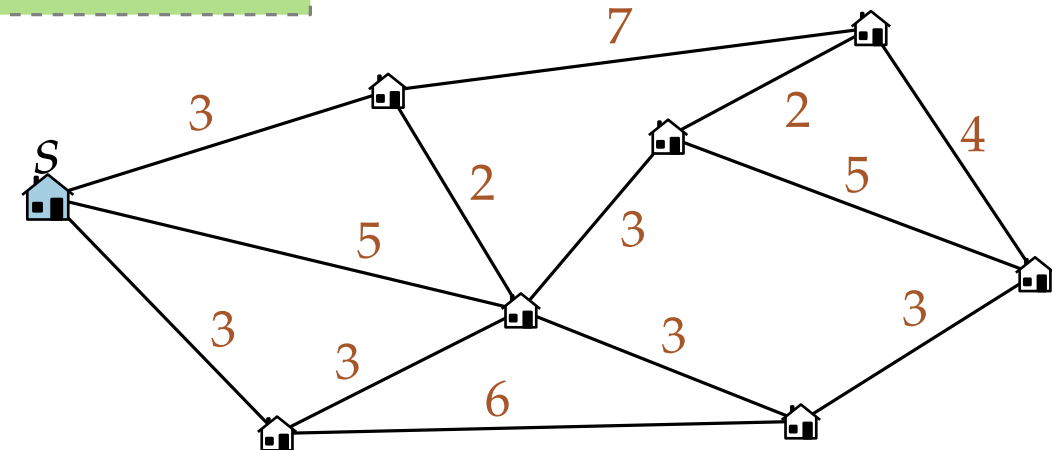
$$S = \{s\}$$

$$E' = \emptyset$$

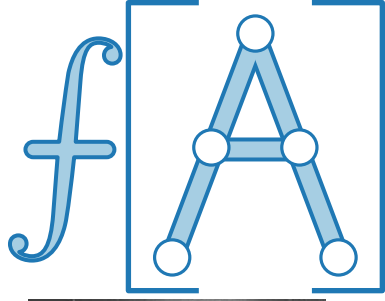
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



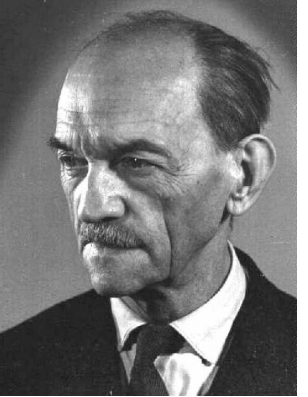
JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

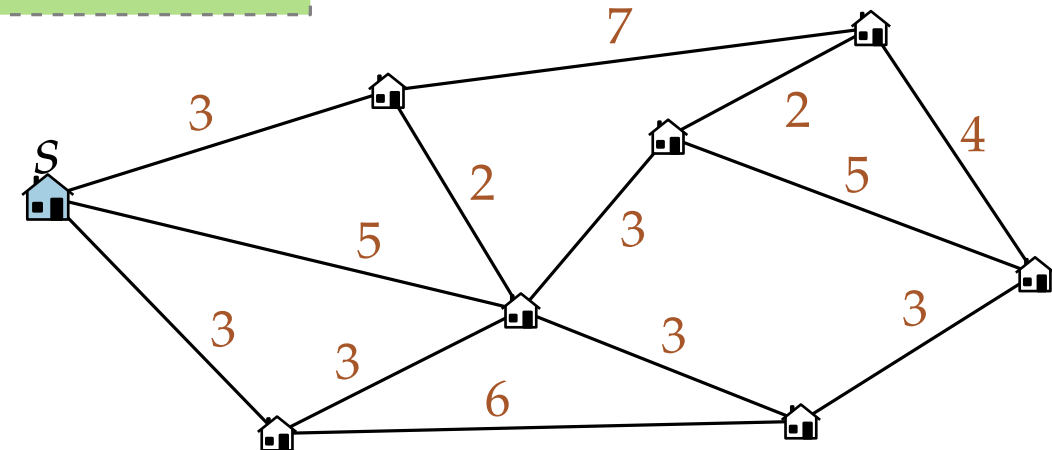
$E' = \emptyset$

while not $S == V$ do

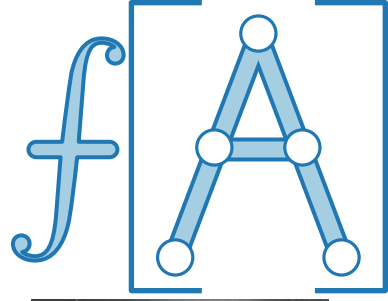
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

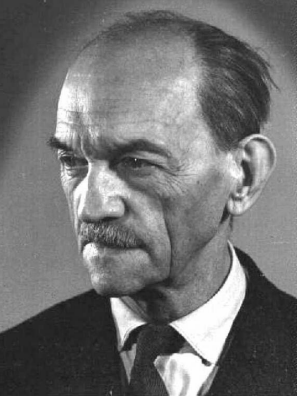
$E' = \emptyset$

while not $S == V$ do

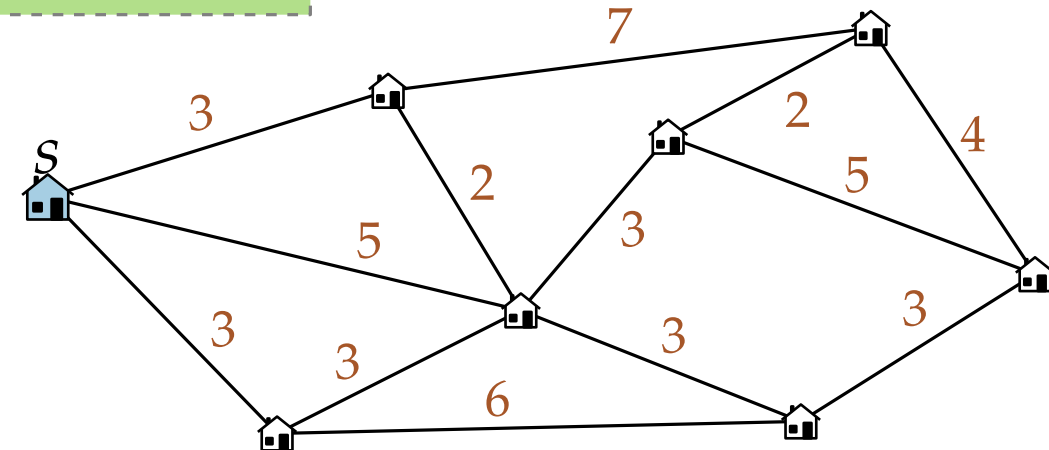
 Wähle Schnitt $(S, V \setminus S)$

Blaue Regel

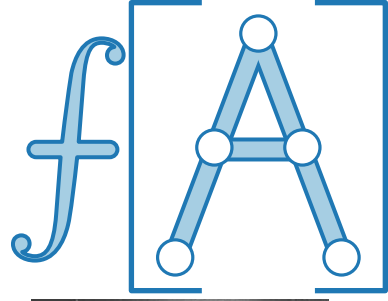
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

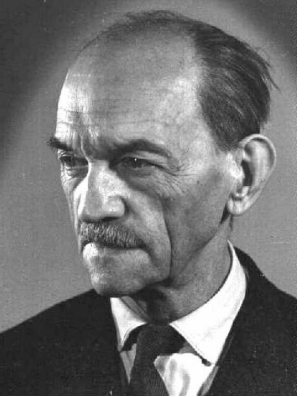
$E' = \emptyset$

while not $S == V$ do

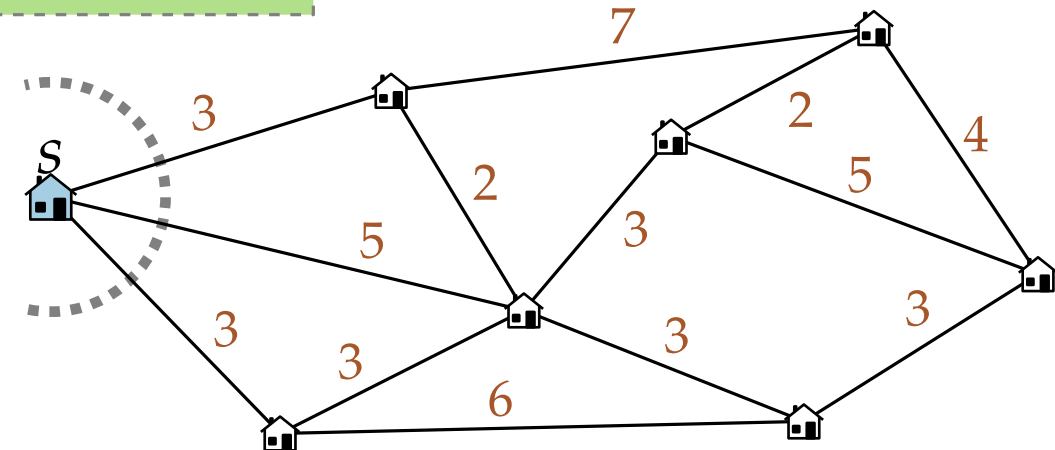
 Wähle Schnitt $(S, V \setminus S)$

Blaue Regel

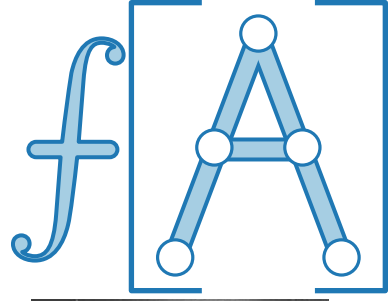
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

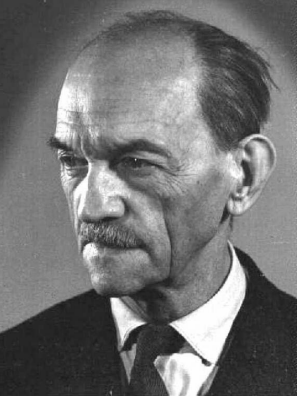
while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

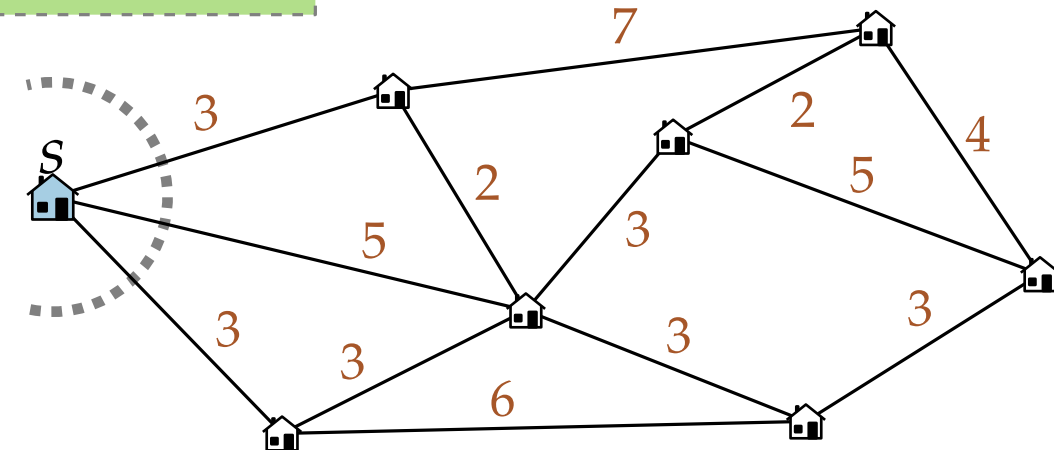
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

Blaue Regel

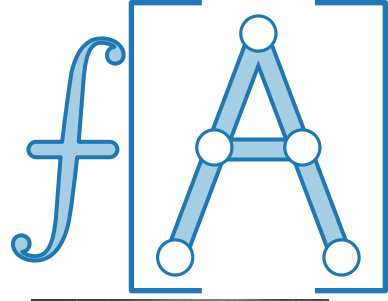
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

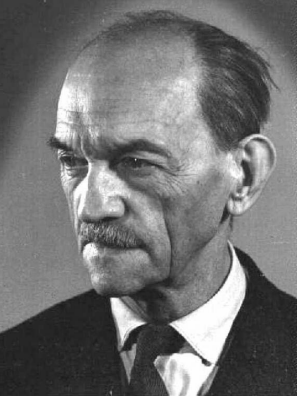
while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

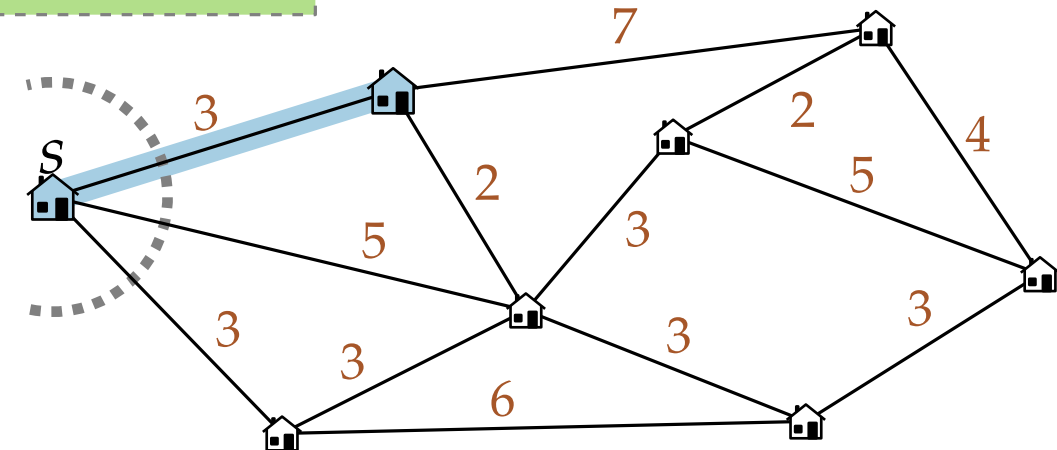
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

Blaue Regel

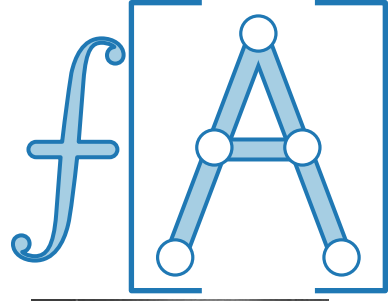
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

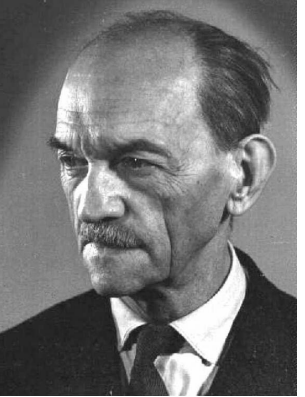
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

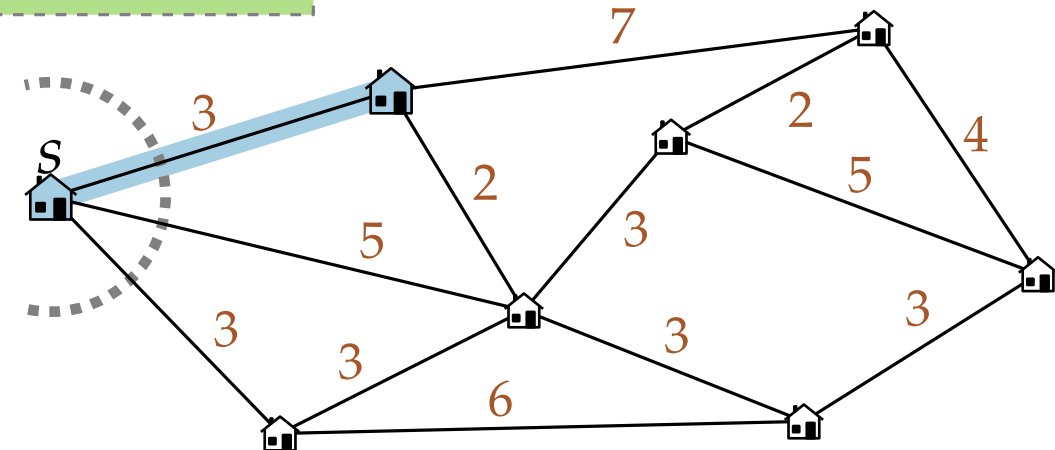
$E' = E' \cup \{uv\}$

Blaue Regel

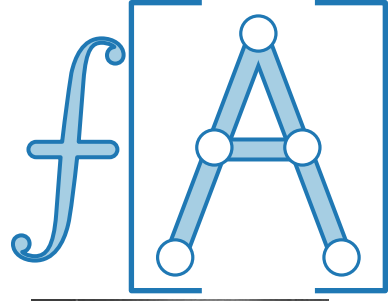
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

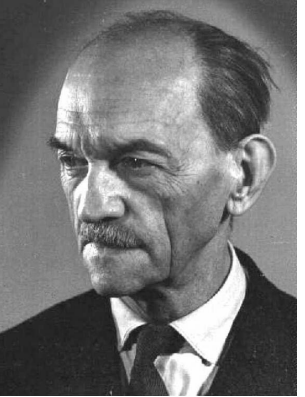
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

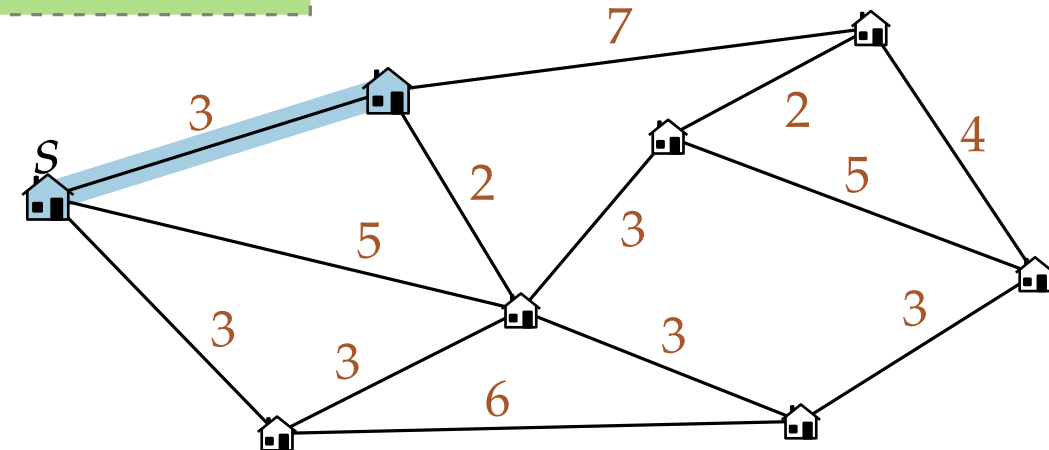
$E' = E' \cup \{uv\}$

Blaue Regel

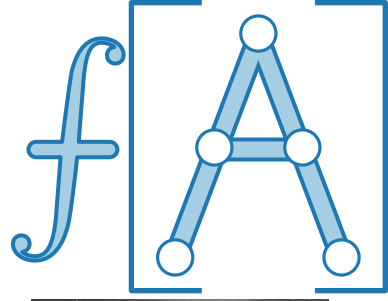
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

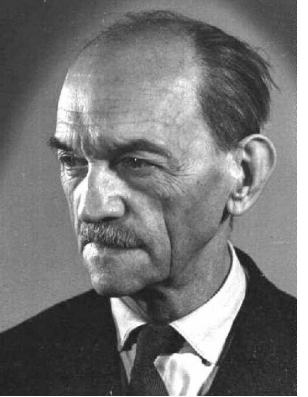
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

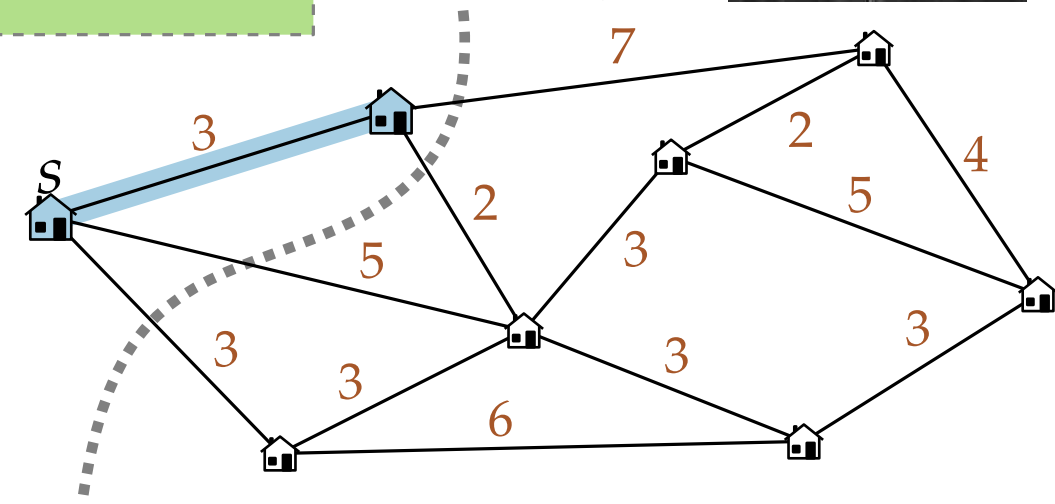
$E' = E' \cup \{uv\}$

Blaue Regel

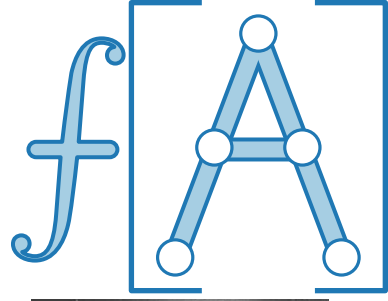
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

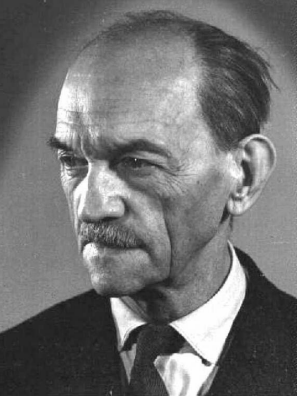
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

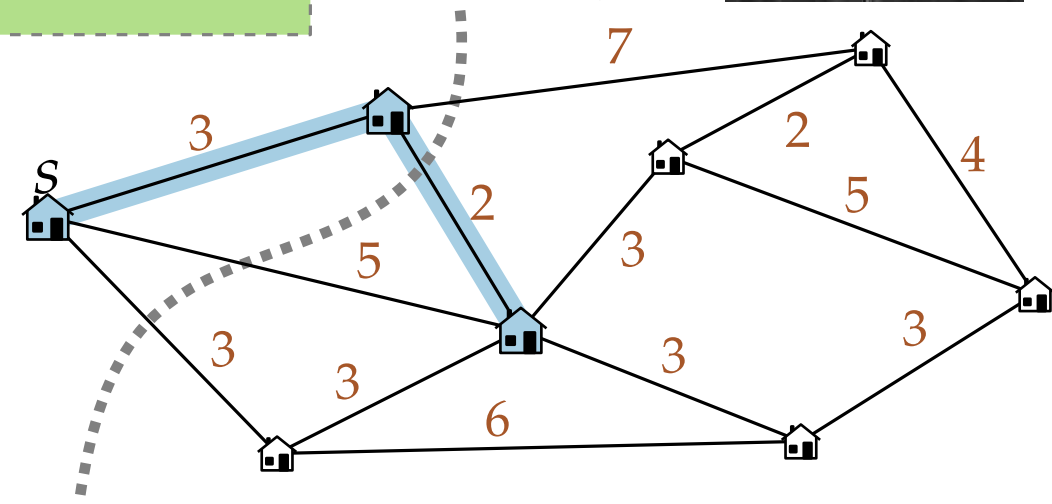
$E' = E' \cup \{uv\}$

Blaue Regel

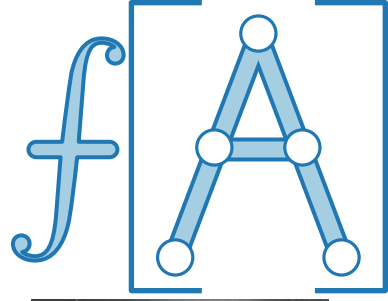
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

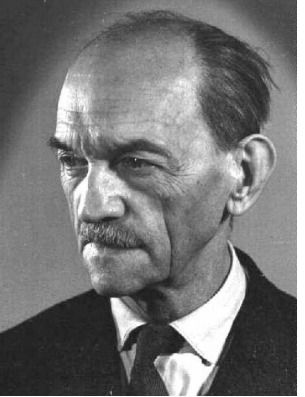
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

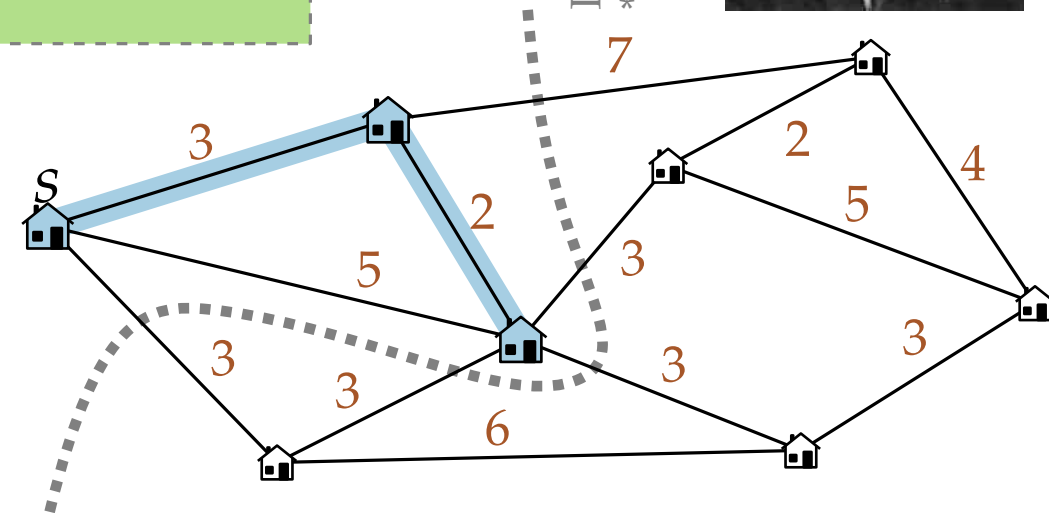
$E' = E' \cup \{uv\}$

Blaue Regel

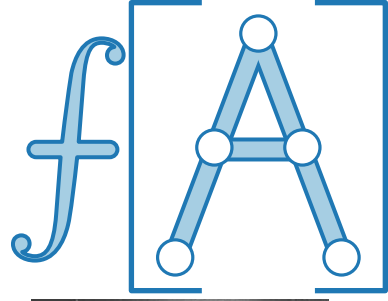
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

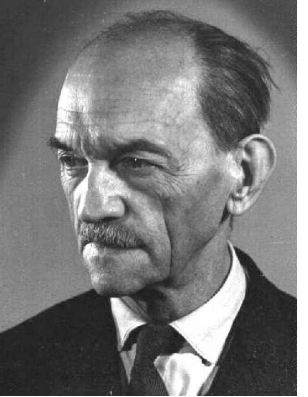
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

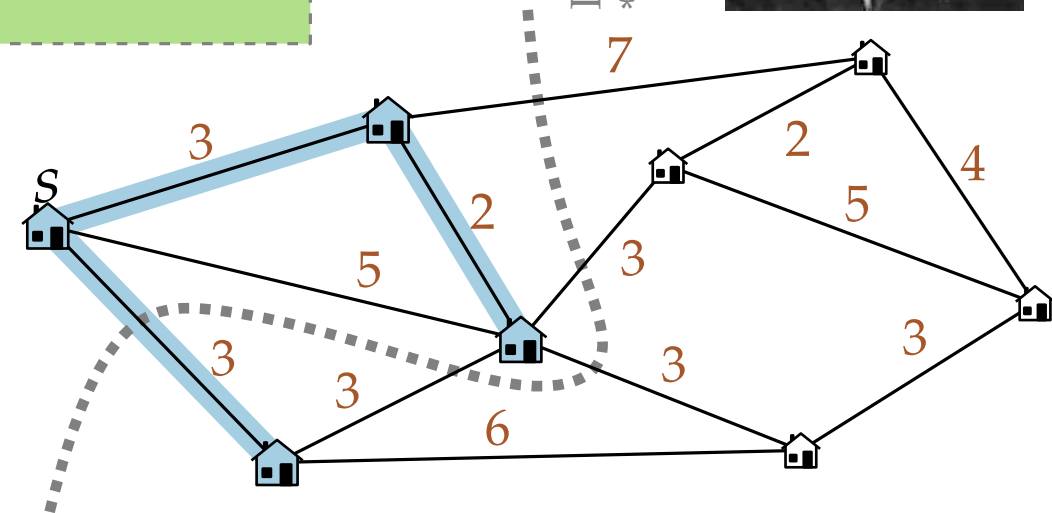
$E' = E' \cup \{uv\}$

Blaue Regel

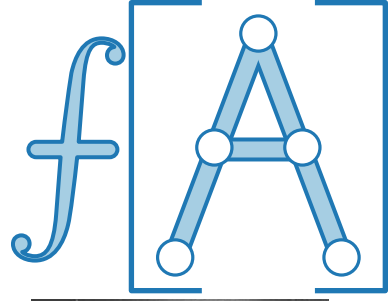
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

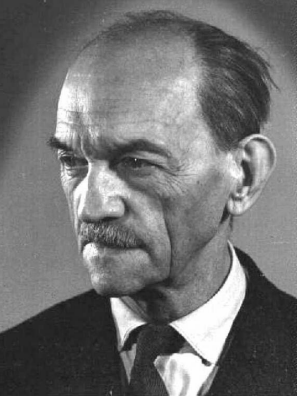
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

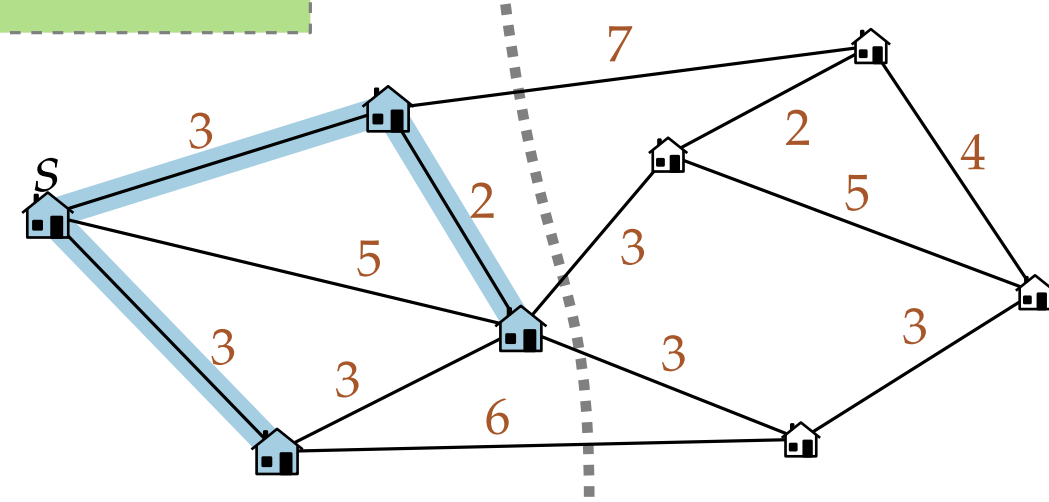
$E' = E' \cup \{uv\}$

Blaue Regel

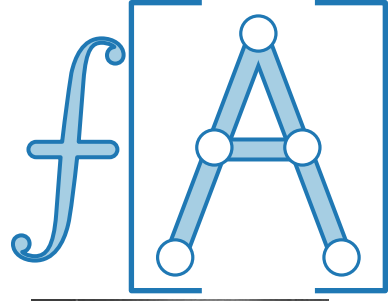
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

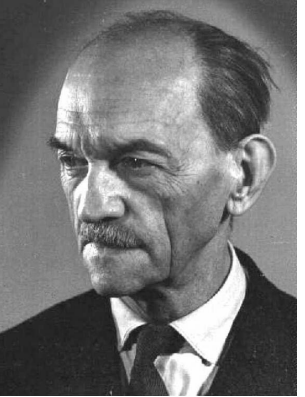
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

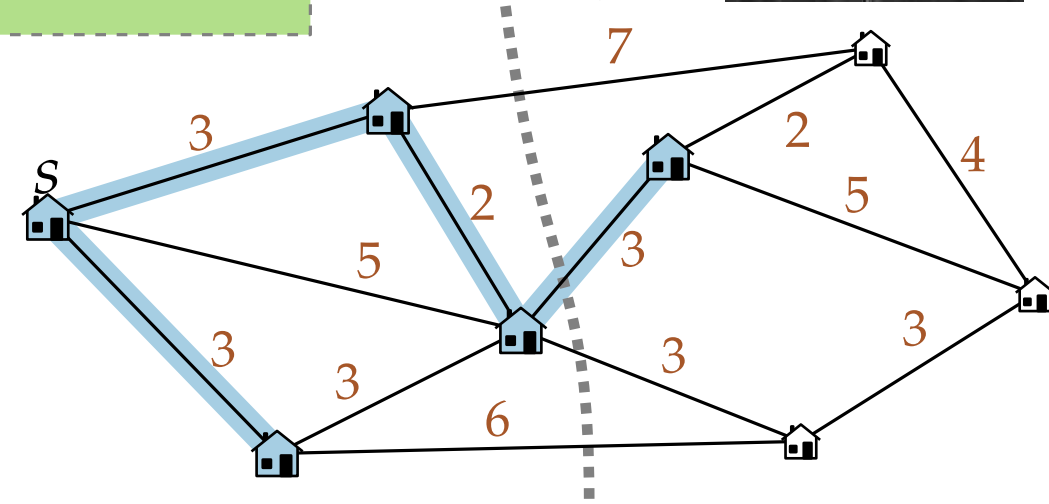
$E' = E' \cup \{uv\}$

Blaue Regel

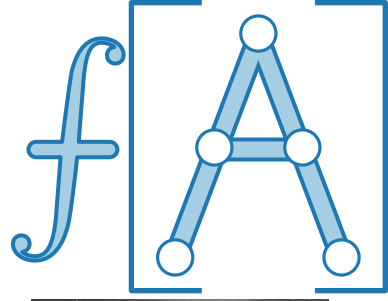
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

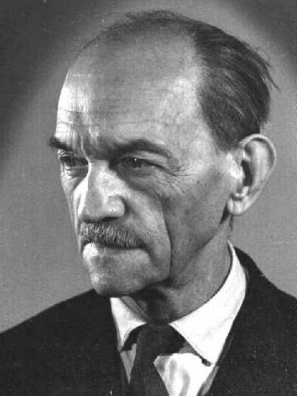
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

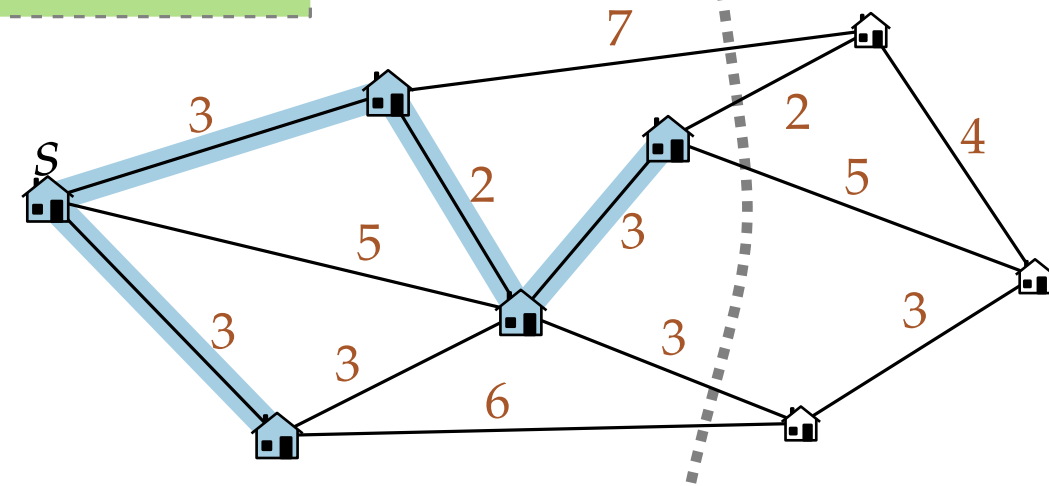
$E' = E' \cup \{uv\}$

Blaue Regel

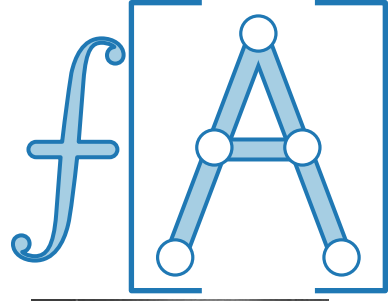
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

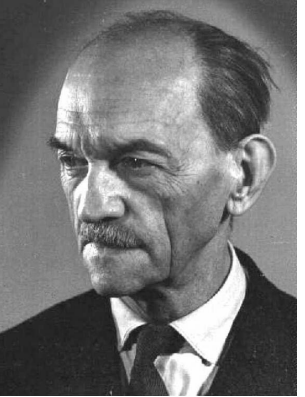
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

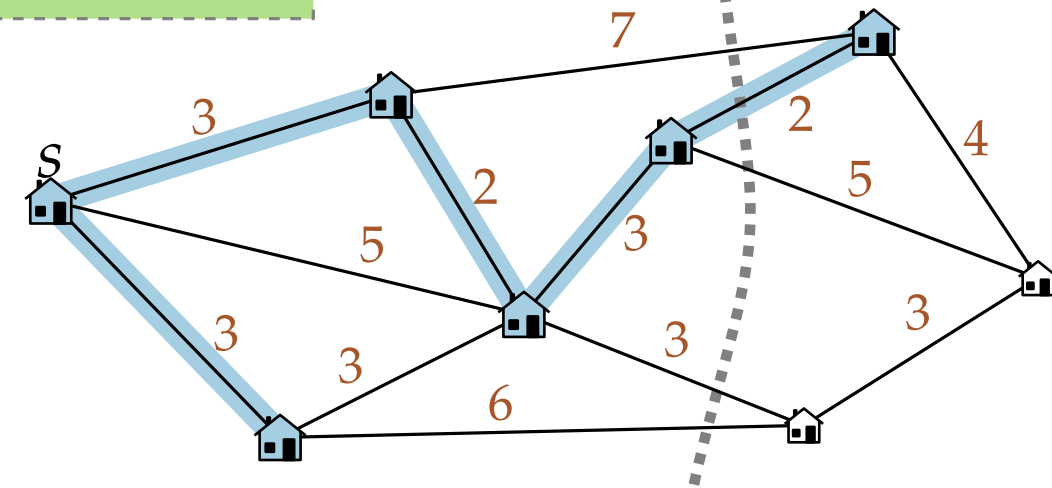
$E' = E' \cup \{uv\}$

Blaue Regel

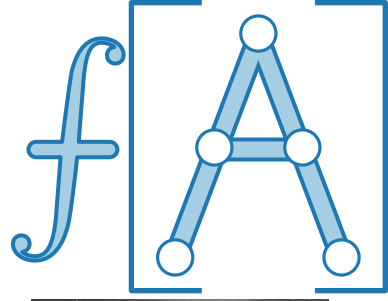
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

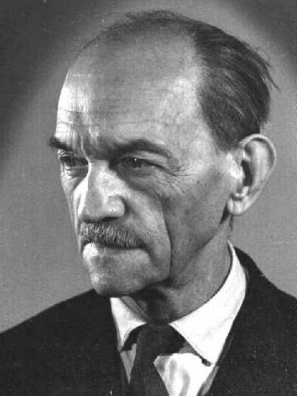
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

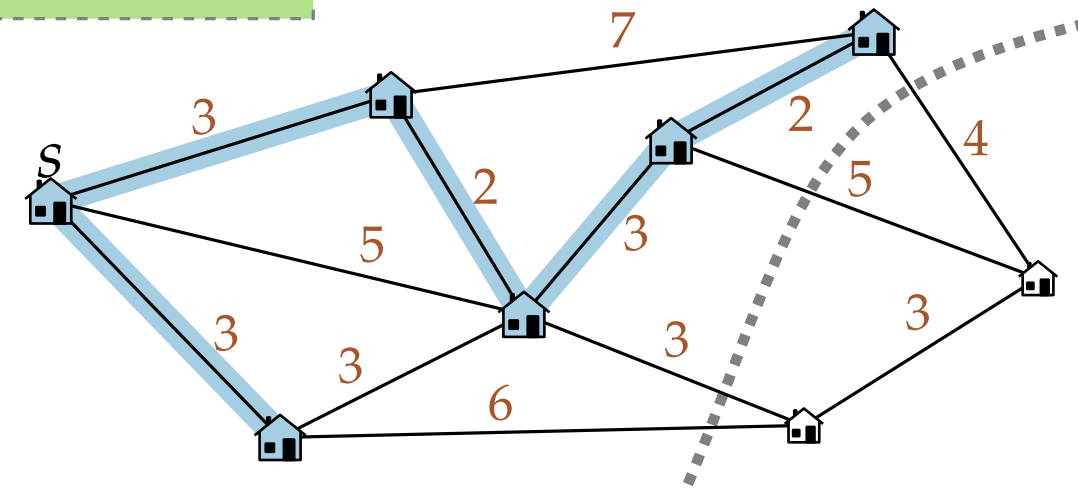
$E' = E' \cup \{uv\}$

Blaue Regel

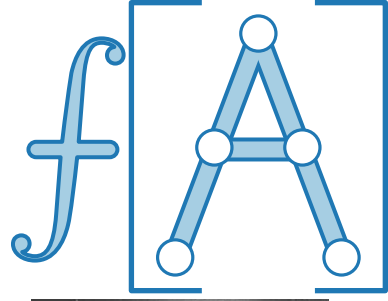
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

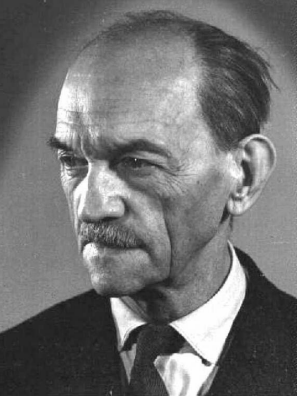
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

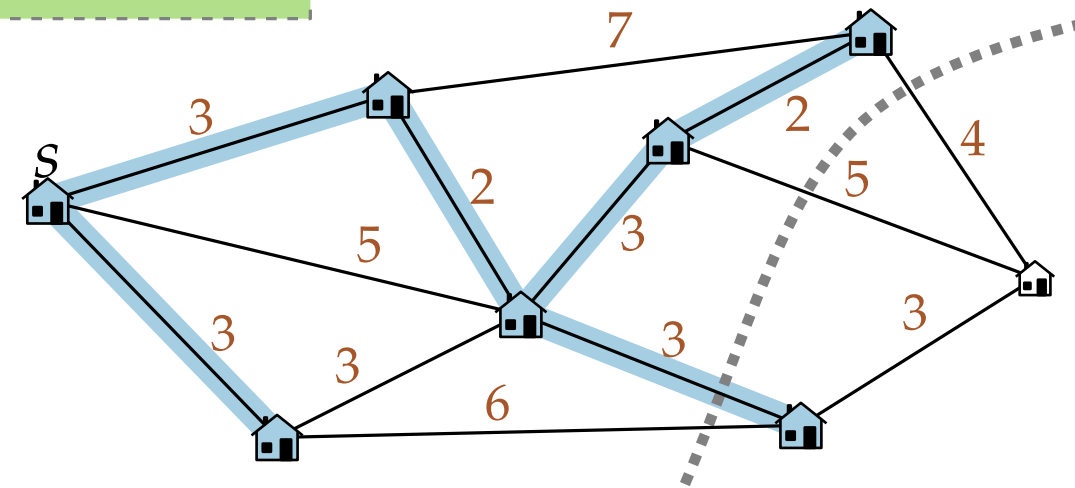
$E' = E' \cup \{uv\}$

Blaue Regel

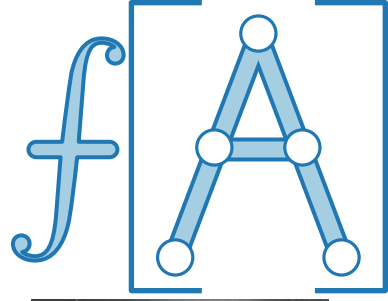
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

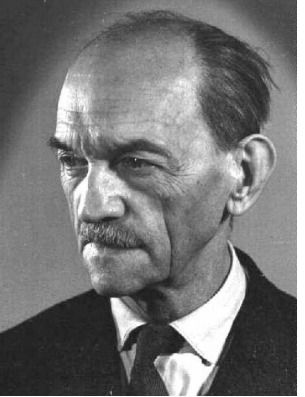
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

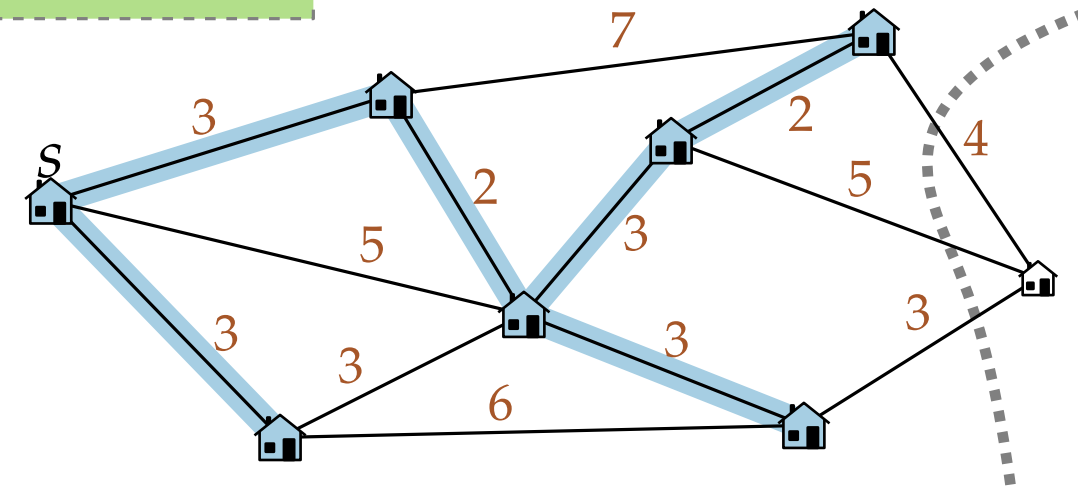
$E' = E' \cup \{uv\}$

Blaue Regel

Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



$$S = \{s\}$$

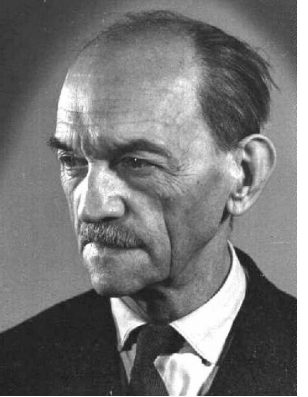
while not $S == V$ do

Färbe leichte Kante uv **blau** ($u \in S, v \in V \setminus S$)

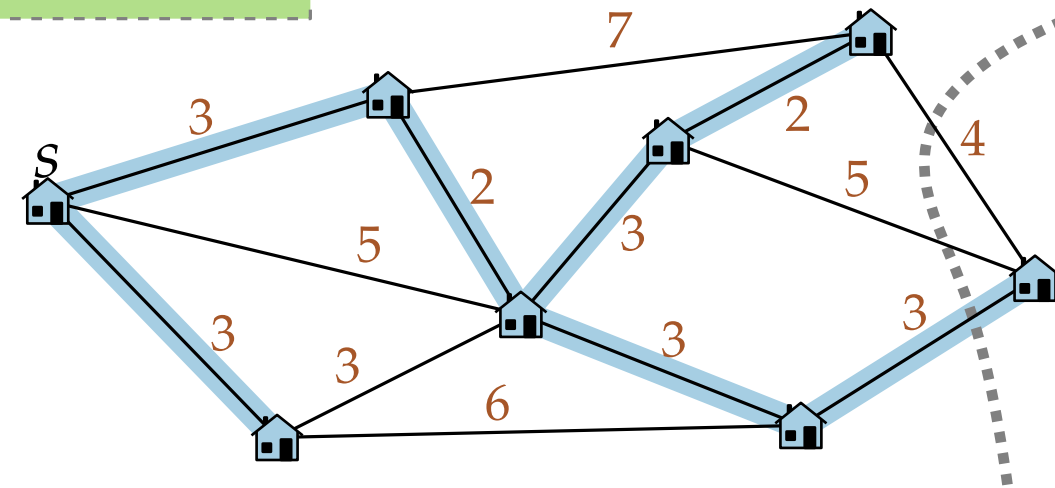
$$S = S \cup \{v\}$$
$$E' = E' \cup \{uv\}$$

Blaue Regel

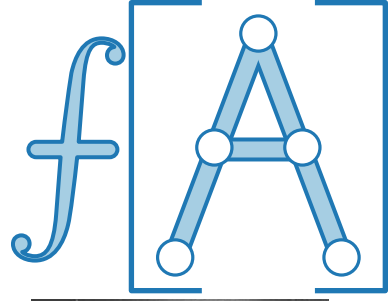
Vojtěch Jarník
*1897 Prag
+ 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ do

 Wähle Schnitt $(S, V \setminus S)$

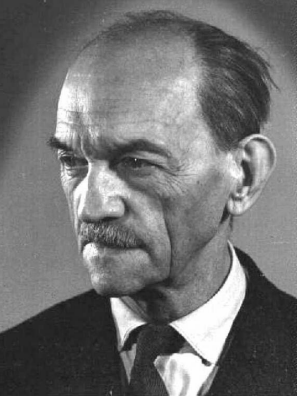
 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

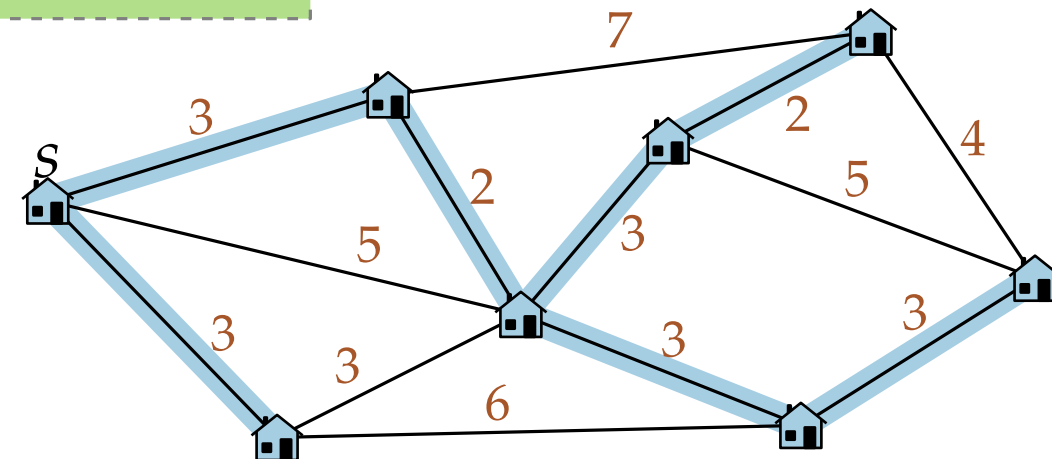
$E' = E' \cup \{uv\}$

Blaue Regel

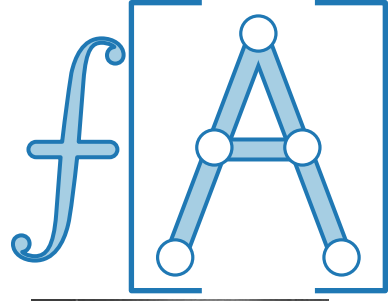
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ **do**

 Wähle Schnitt $(S, V \setminus S)$

 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

$E' = E' \cup \{uv\}$

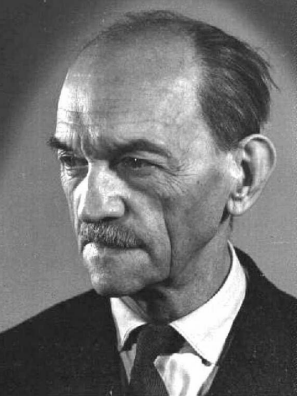
Blaue Regel

Färbe alle anderen Kanten rot

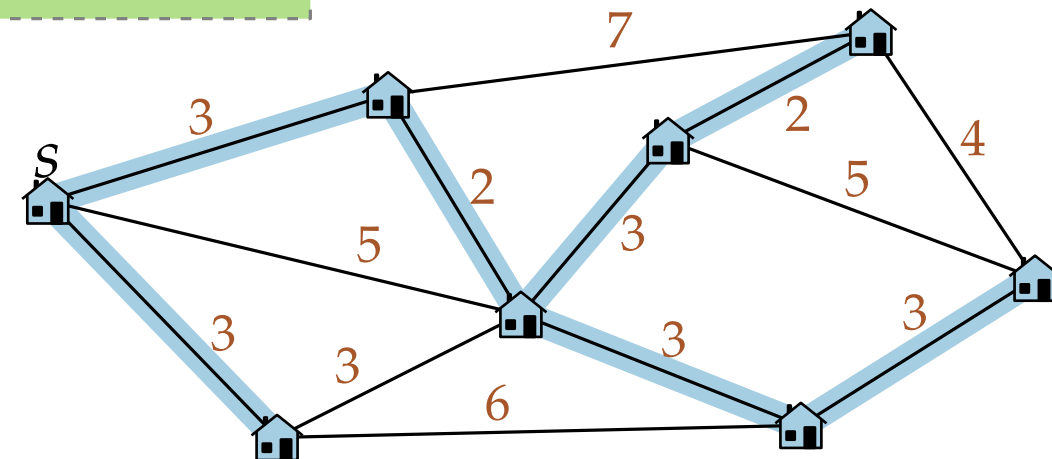
Rote Regel

return E'

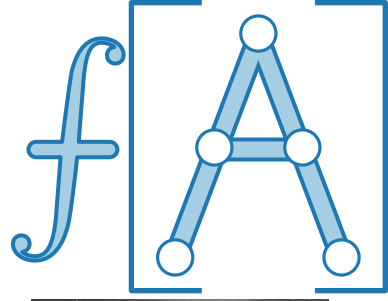
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ **do**

 Wähle Schnitt $(S, V \setminus S)$

 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

$E' = E' \cup \{uv\}$

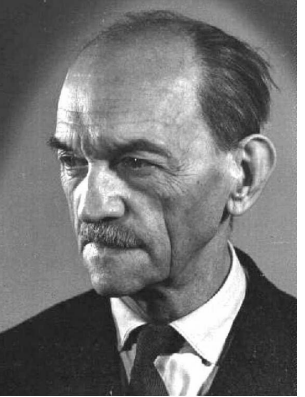
Blaue Regel

Färbe alle anderen Kanten rot

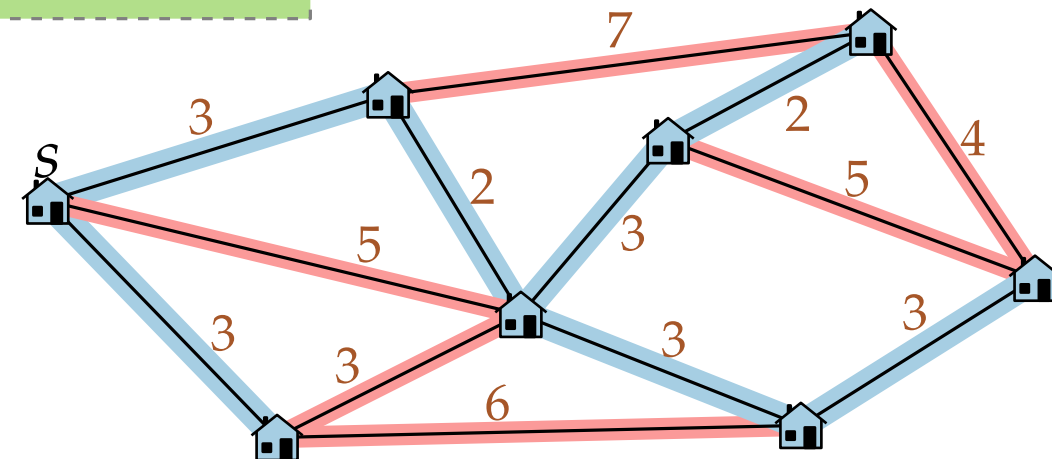
Rote Regel

return E'

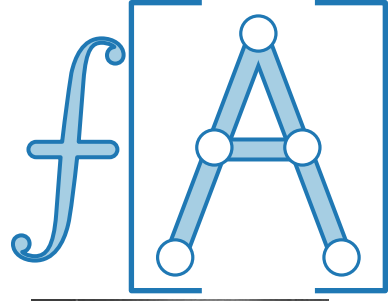
Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ **do**

 Wähle Schnitt $(S, V \setminus S)$

 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

$E' = E' \cup \{uv\}$

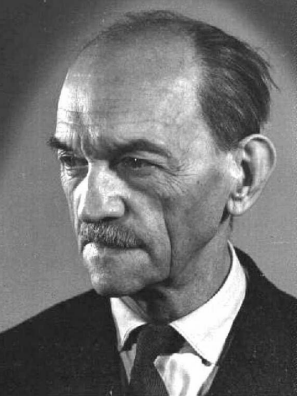
Blaue Regel

Färbe alle anderen Kanten rot

Rote Regel

return E'

Vojtěch Jarník
*1897 Prag
† 1970 Prag

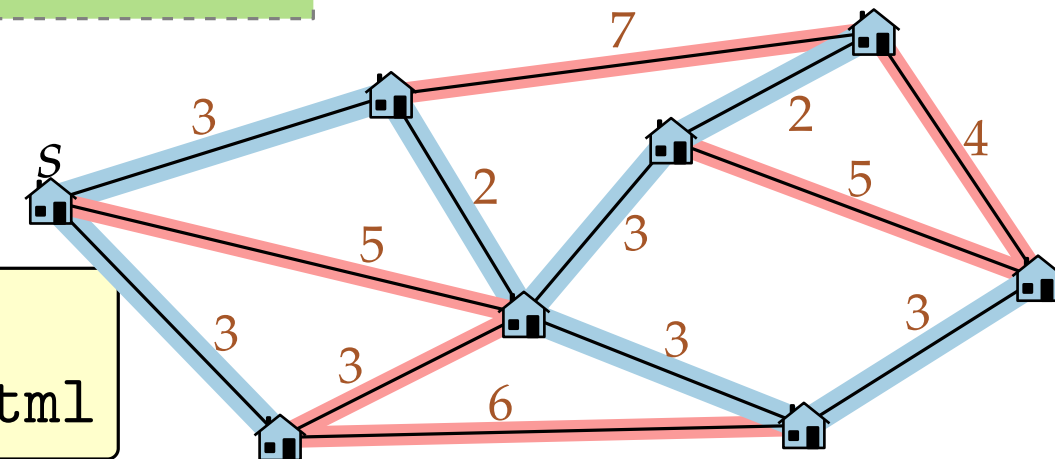


Robert C. Prim
*1921 Sweetwater, TX

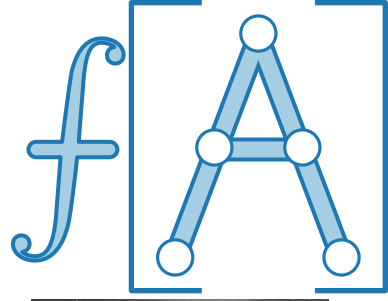


Demo.

<https://algo.uni-trier.de/demos/spanningtree.html>



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ **do**

 Wähle Schnitt $(S, V \setminus S)$

 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

$E' = E' \cup \{uv\}$

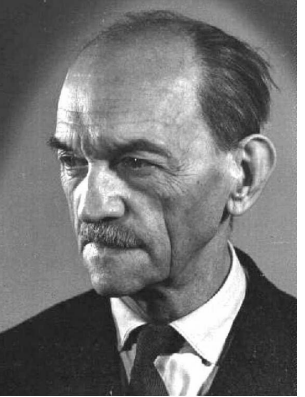
Blaue Regel

Färbe alle anderen Kanten rot

Rote Regel

return E'

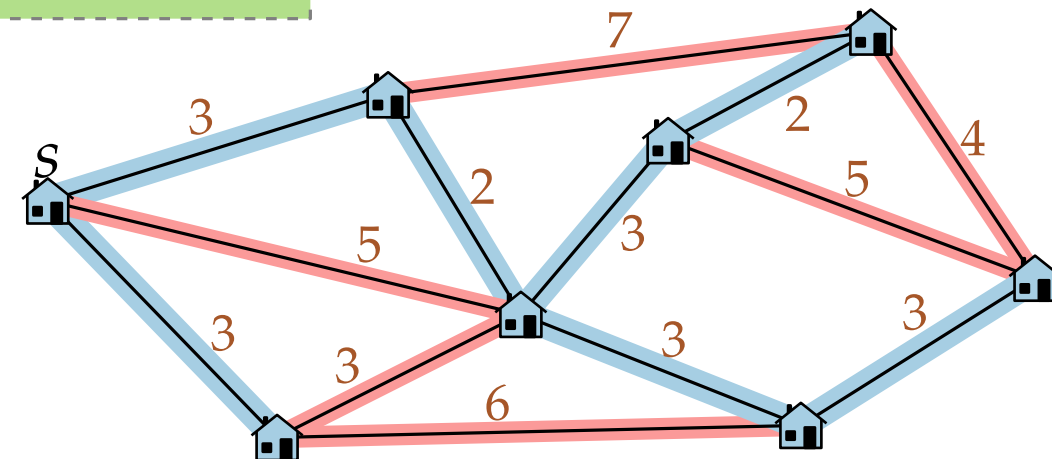
Vojtěch Jarník
*1897 Prag
† 1970 Prag



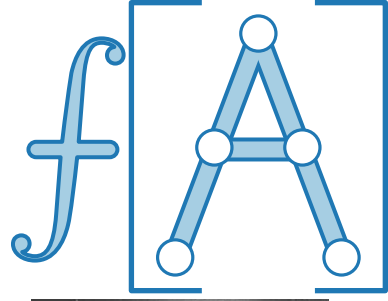
Robert C. Prim
*1921 Sweetwater, TX



Laufzeit?



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ **do**

 Wähle Schnitt $(S, V \setminus S)$

 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

$E' = E' \cup \{uv\}$

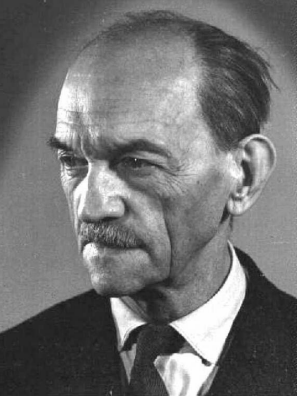
Blaue Regel

Färbe alle anderen Kanten rot

Rote Regel

return E'

Vojtěch Jarník
*1897 Prag
† 1970 Prag

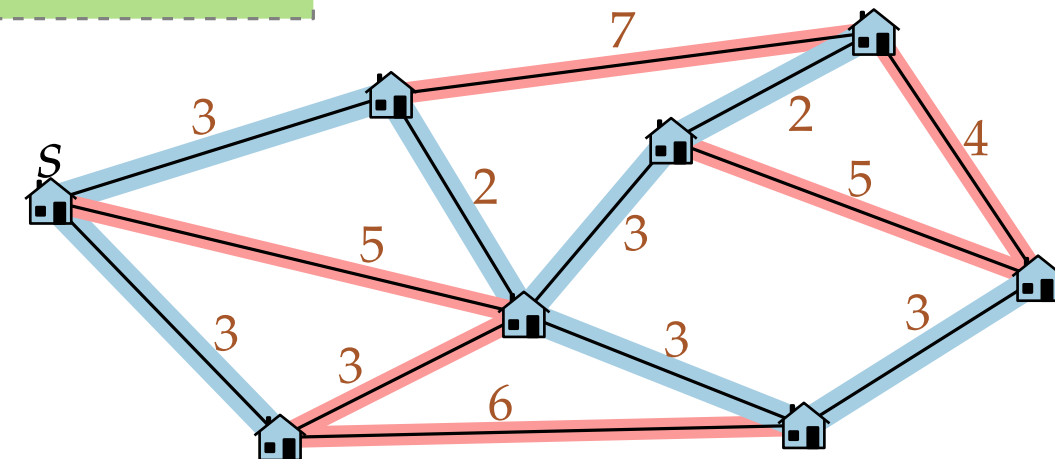


Robert C. Prim
*1921 Sweetwater, TX

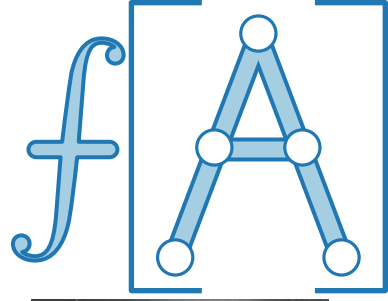


Laufzeit?

Wie DIJKSTRA!



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ **do**

 Wähle Schnitt $(S, V \setminus S)$

 Färbe leichte Kante uv **blau** ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

$E' = E' \cup \{uv\}$

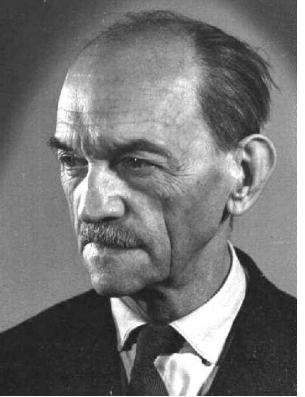
Blaue Regel

Färbe alle anderen Kanten **rot**

Rote Regel

return E'

Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX

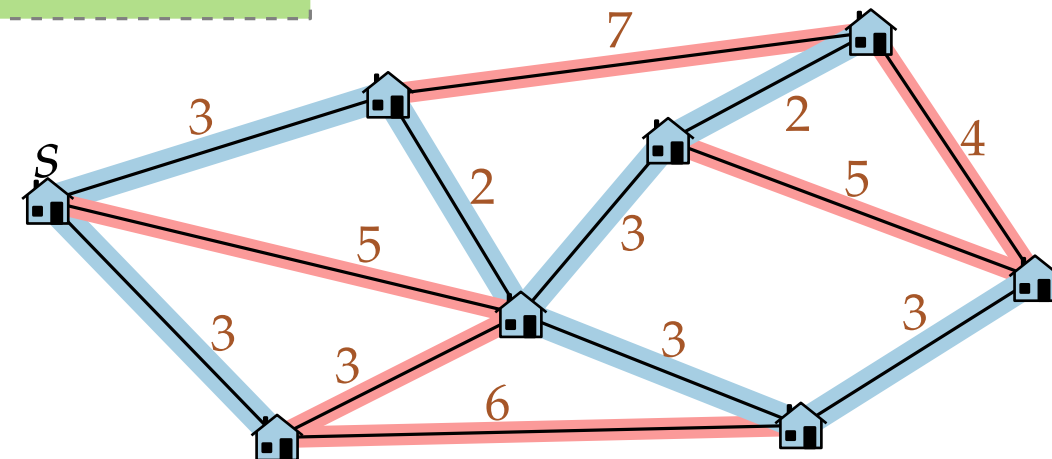


Laufzeit?

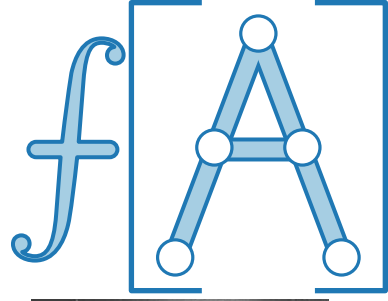
Wie DIJKSTRA!

$\Rightarrow \mathcal{O}((n + m) \log n)$

Heap/RS-Baum



Der Algorithmus von Jarník-Prim (1930/1957)



JARNÍK-PRIM(WeightedGraph $G = (V, E; w)$, Vertex s)

$S = \{s\}$

$E' = \emptyset$

while not $S == V$ **do**

 Wähle Schnitt $(S, V \setminus S)$

 Färbe leichte Kante uv blau ($u \in S, v \in V \setminus S$)

$S = S \cup \{v\}$

$E' = E' \cup \{uv\}$

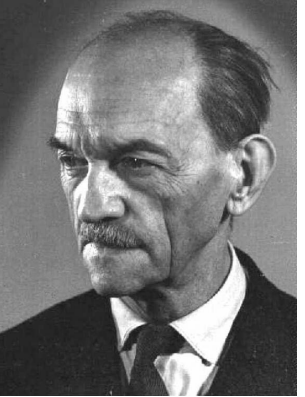
Blaue Regel

Färbe alle anderen Kanten rot

Rote Regel

return E'

Vojtěch Jarník
*1897 Prag
† 1970 Prag



Robert C. Prim
*1921 Sweetwater, TX



Laufzeit?

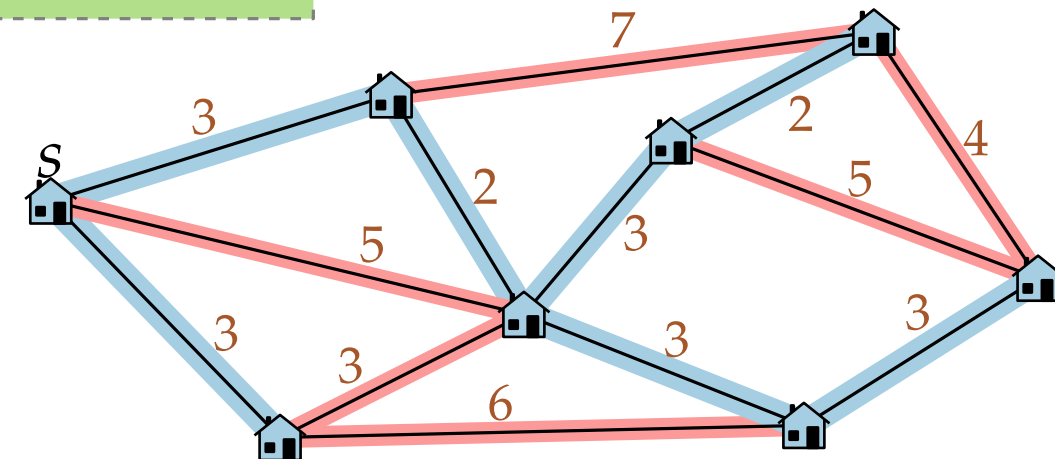
Wie DIJKSTRA!

$\Rightarrow \mathcal{O}((n + m) \log n)$

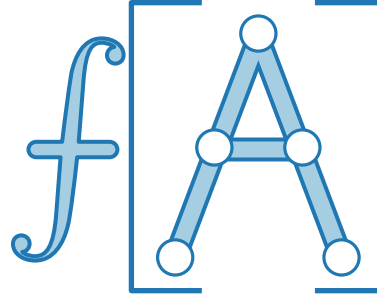
$\Rightarrow \mathcal{O}(m + n \log n)$

Heap/RS-Baum

Fibonacci-Heap

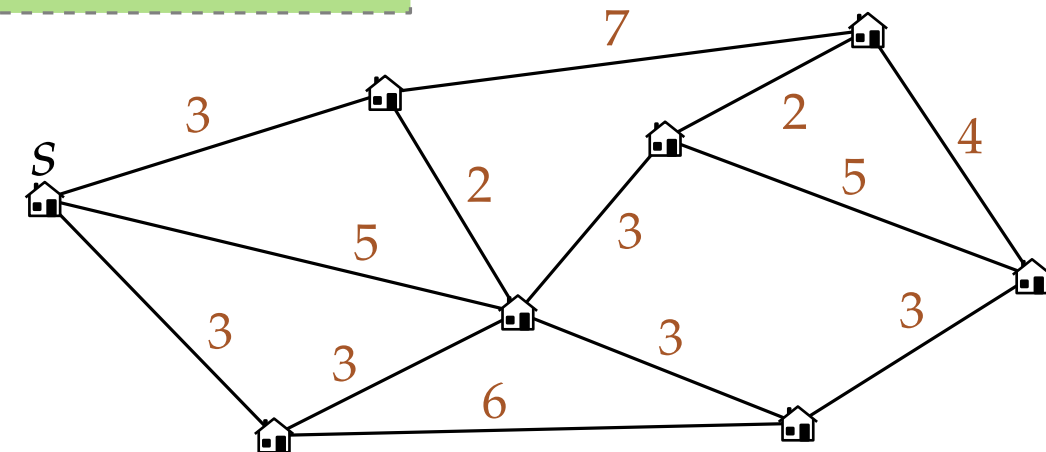
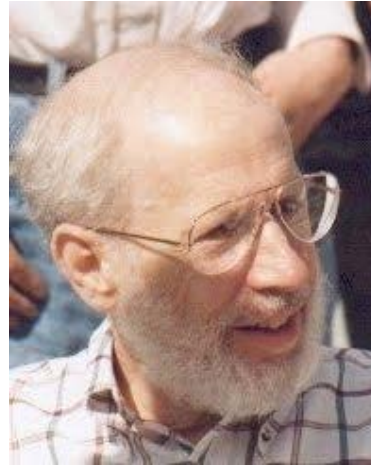


Der Algorithmus von Kruskal (1956)

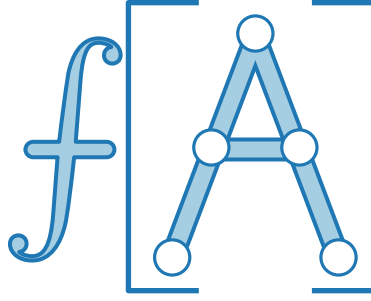


KRUSKAL(WeightedGraph $G = (V, E; w)$)

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



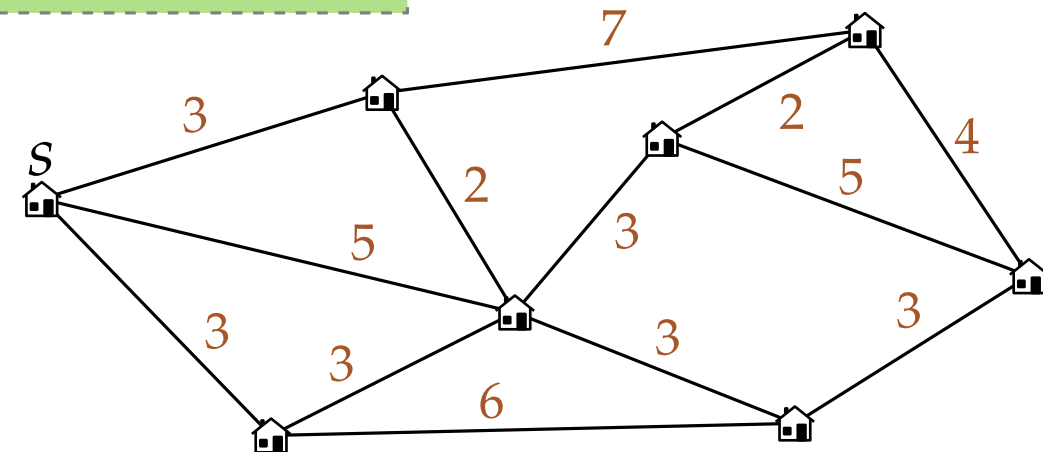
Der Algorithmus von Kruskal (1956)



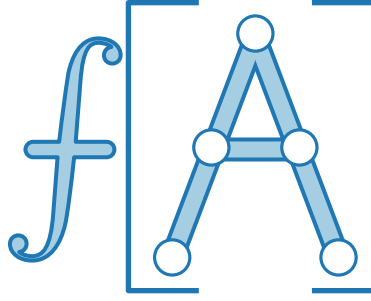
KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)

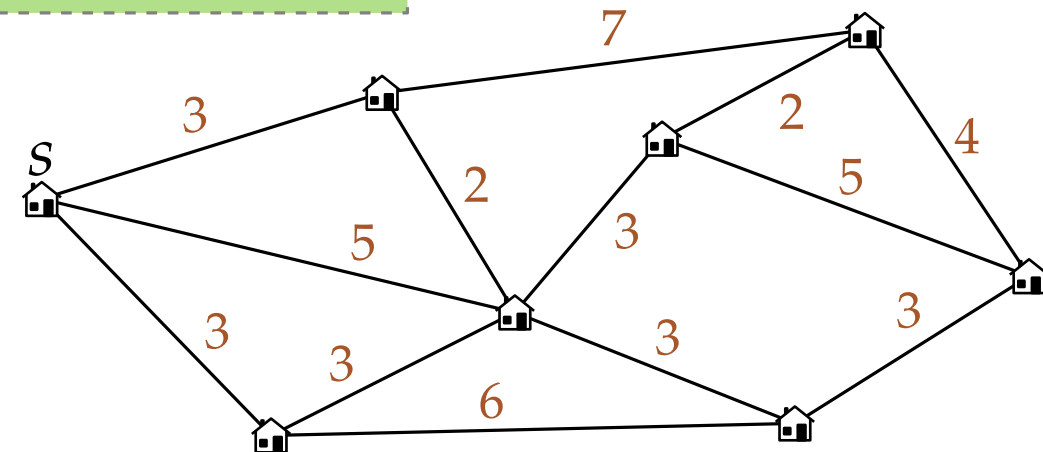


KRUSKAL(WeightedGraph $G = (V, E; w)$)

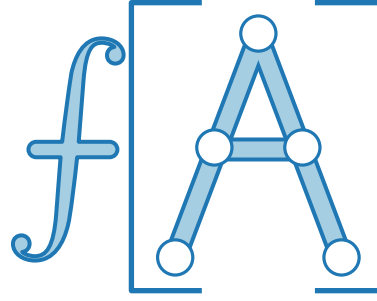
$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

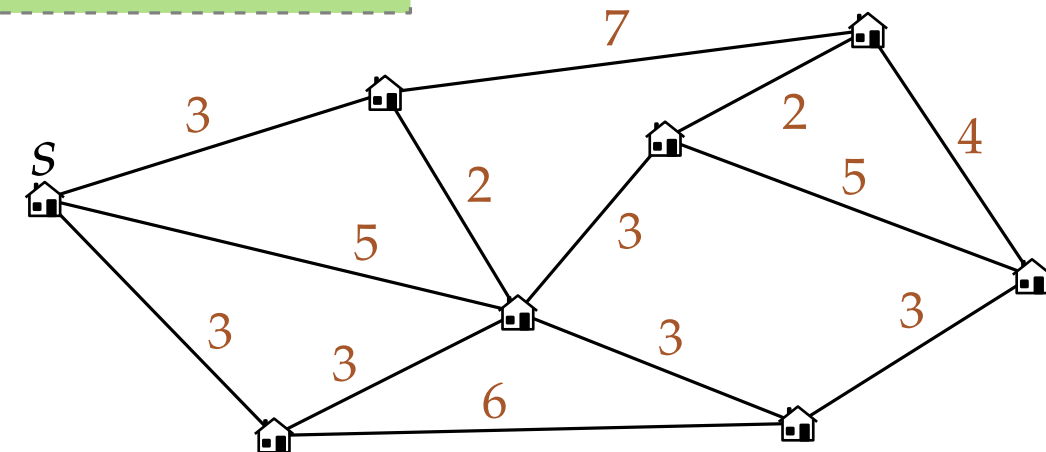
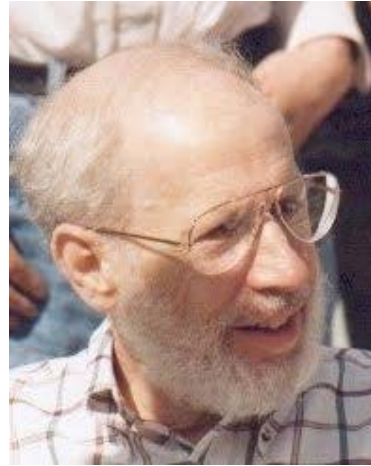
$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

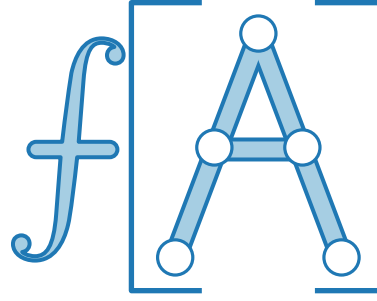
foreach $uv \in E$ **do**

|

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

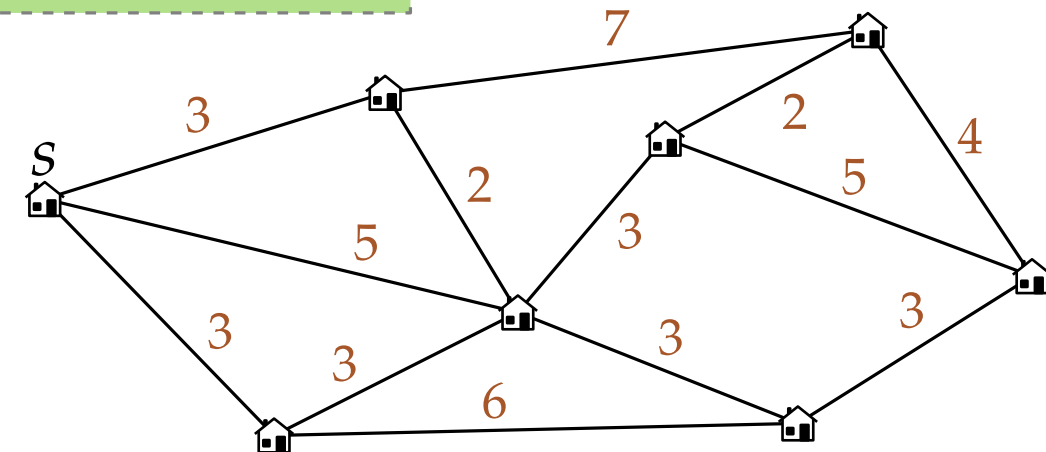
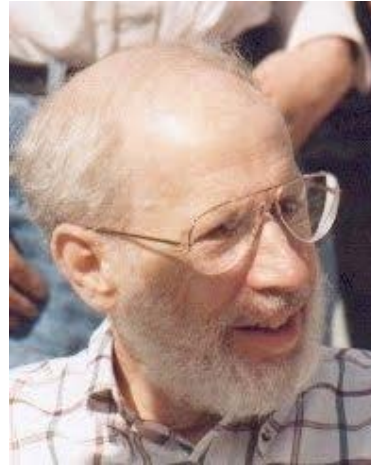
$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

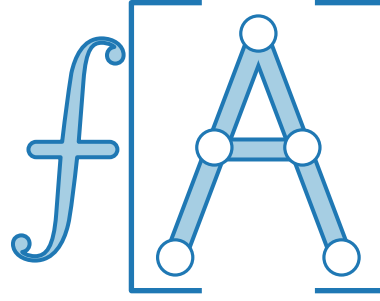
foreach $uv \in E$ **do** (in sortierter Reihenfolge)

|

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

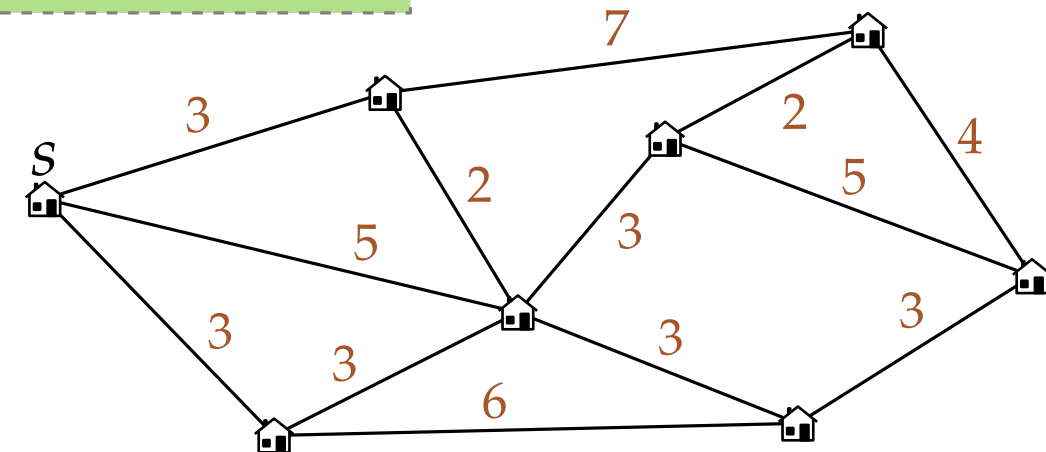
$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

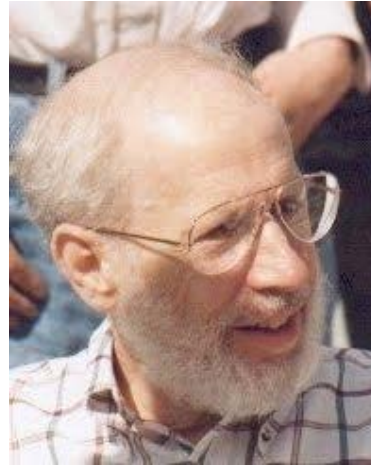
foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

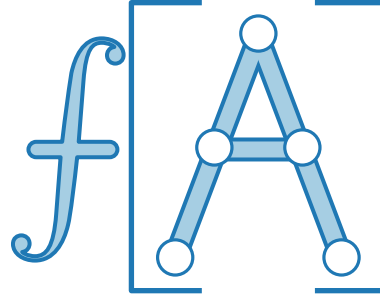
else



Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

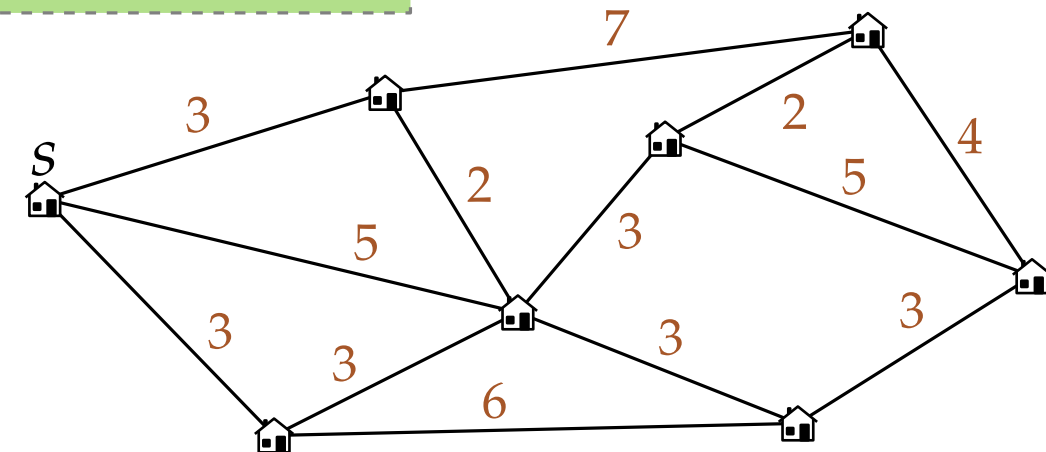
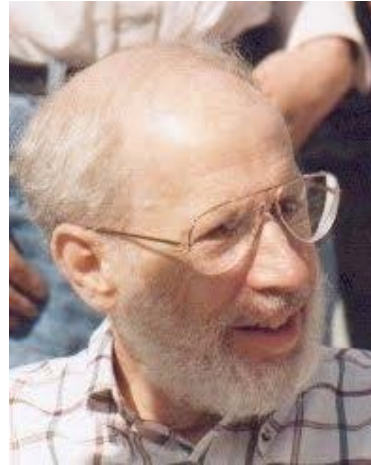
 Färbe uv blau

$E' = E' \cup \{uv\}$

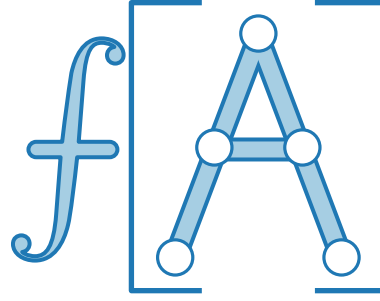
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

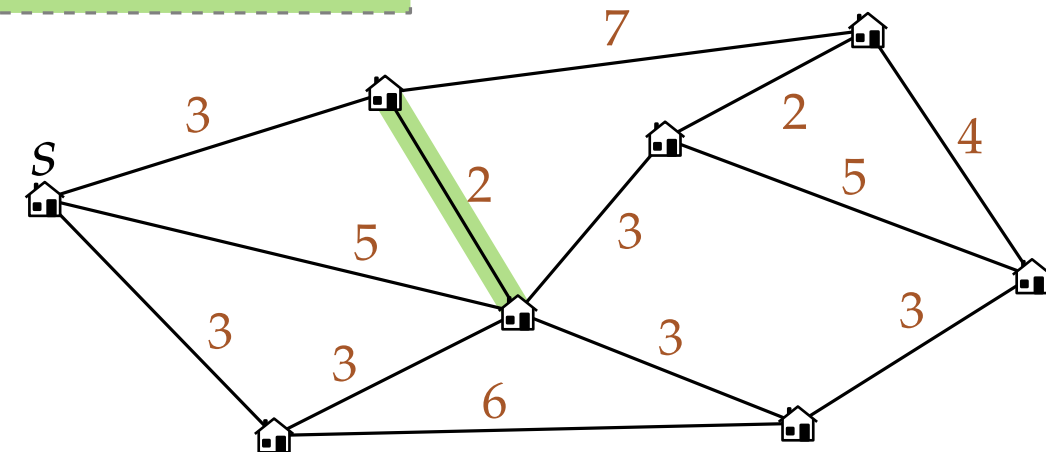
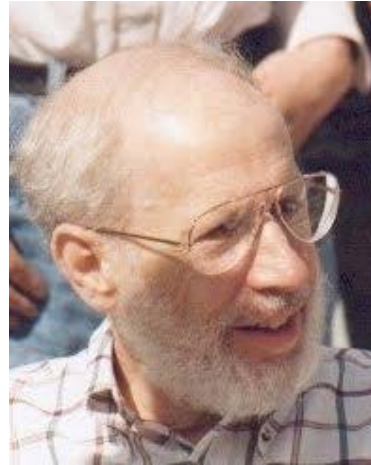
 Färbe uv blau

$E' = E' \cup \{uv\}$

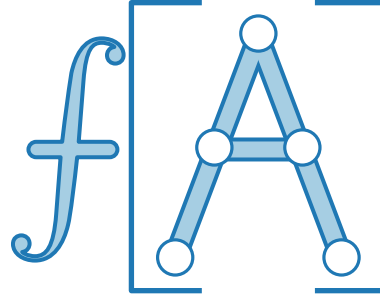
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

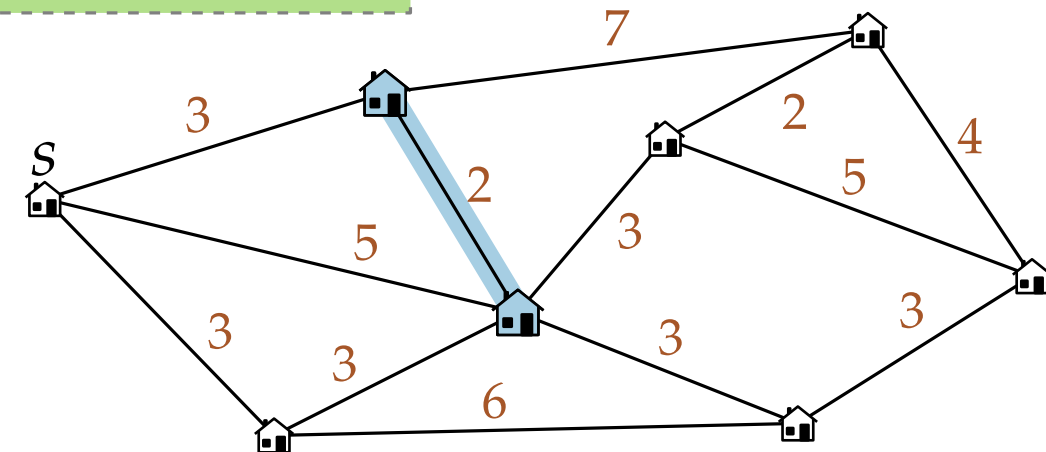
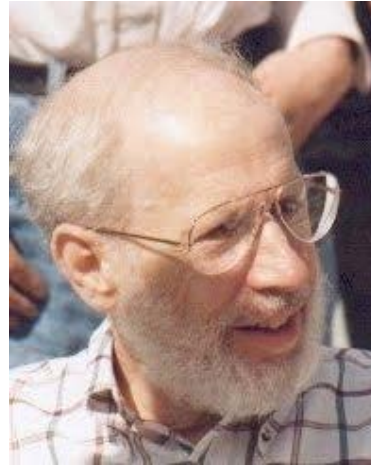
 Färbe uv **blau**

$E' = E' \cup \{uv\}$

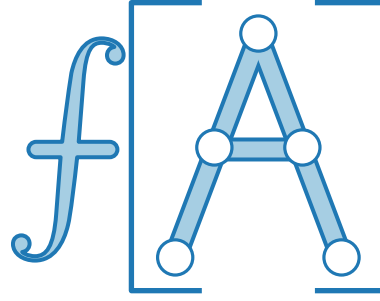
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

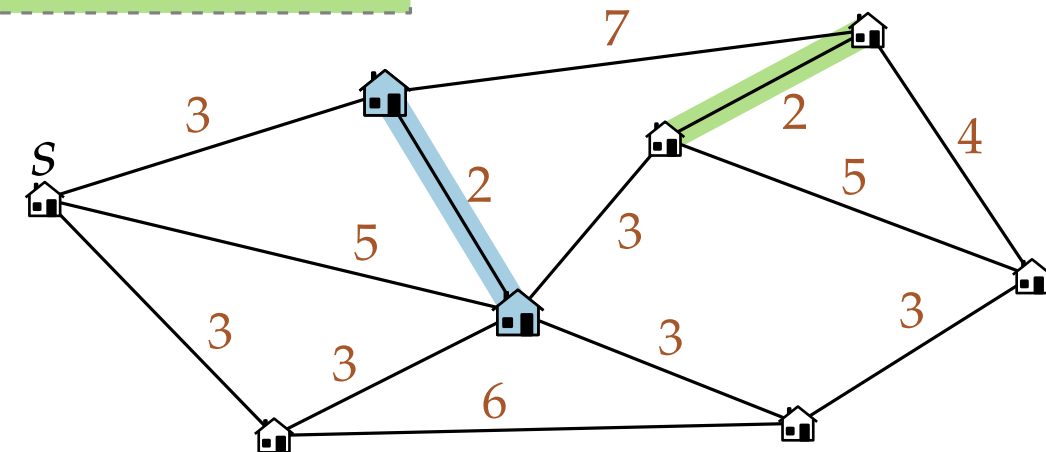
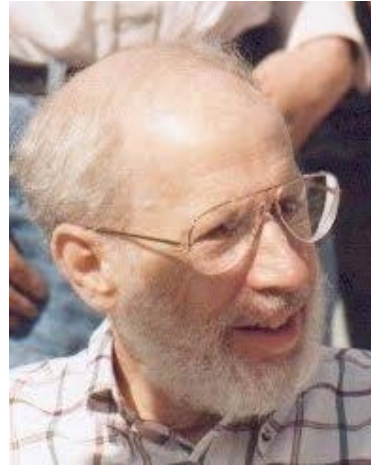
 Färbe uv blau

$E' = E' \cup \{uv\}$

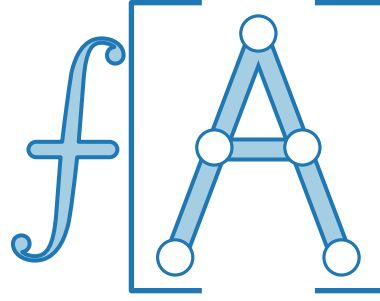
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

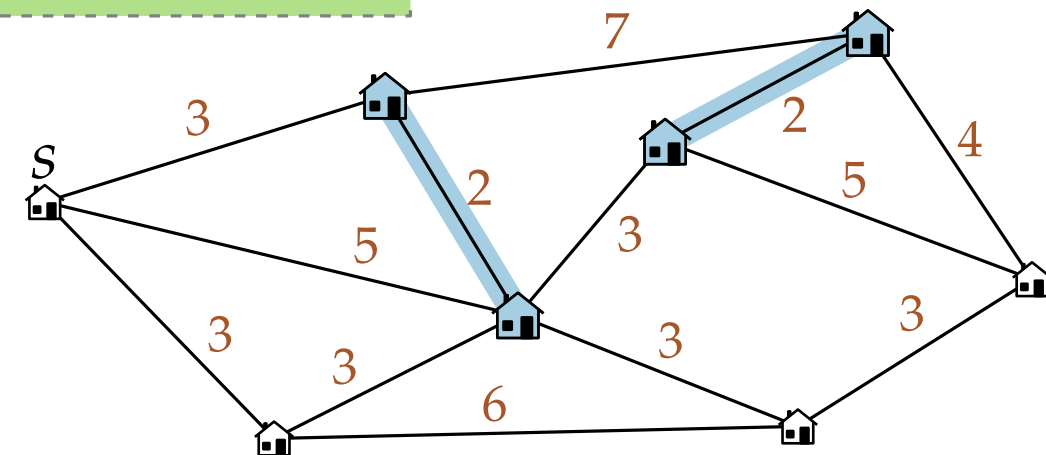
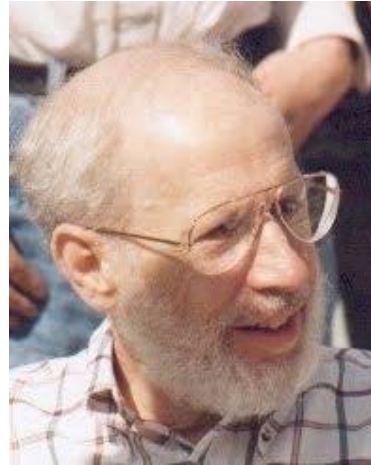
 Färbe uv blau

$E' = E' \cup \{uv\}$

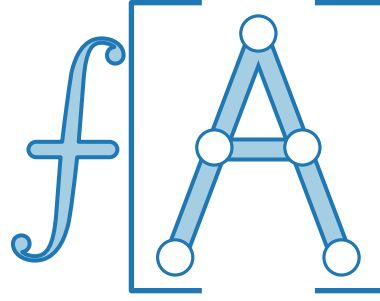
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

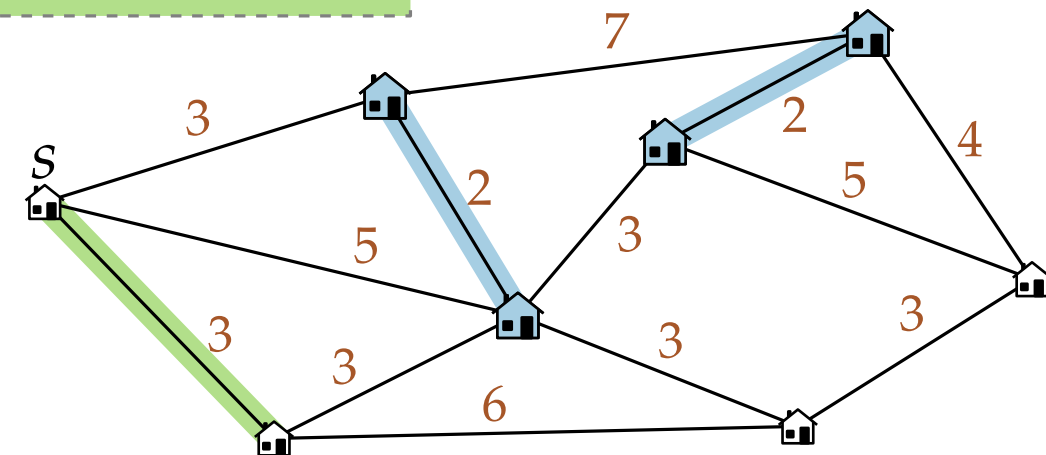
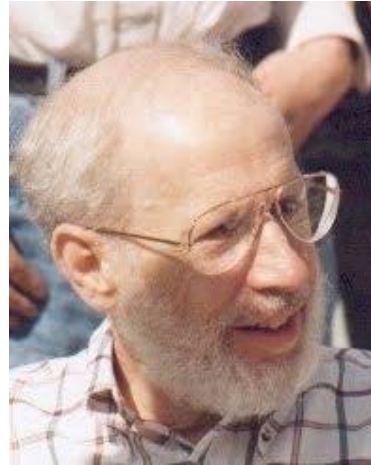
 Färbe uv blau

$E' = E' \cup \{uv\}$

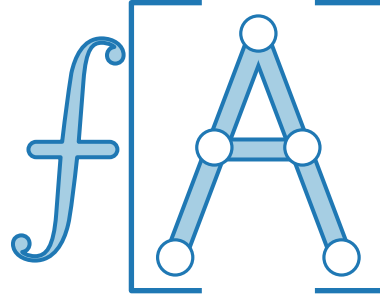
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

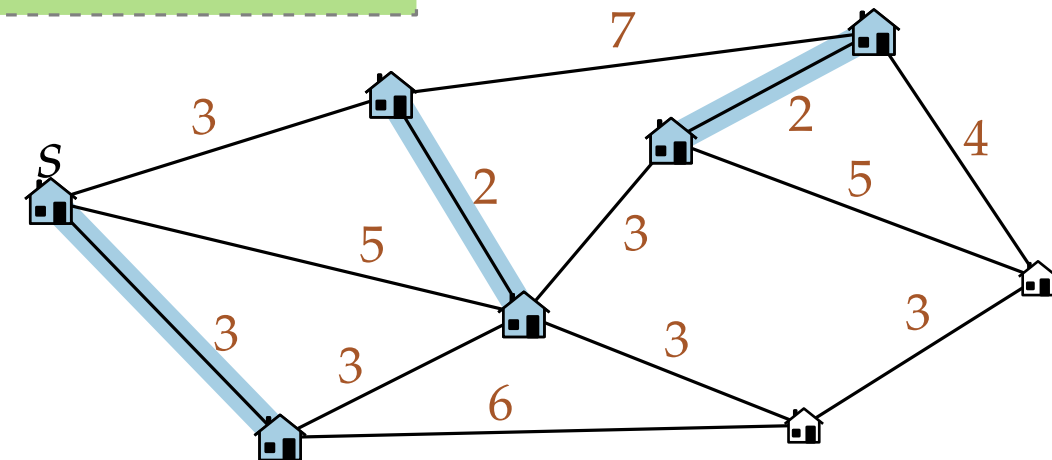
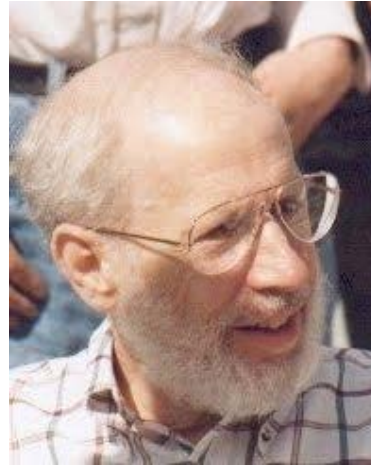
 Färbe uv blau

$E' = E' \cup \{uv\}$

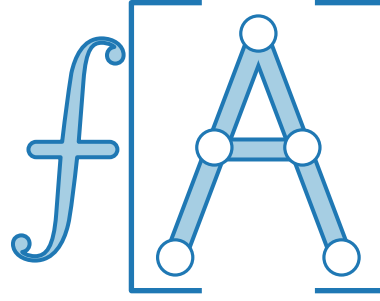
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

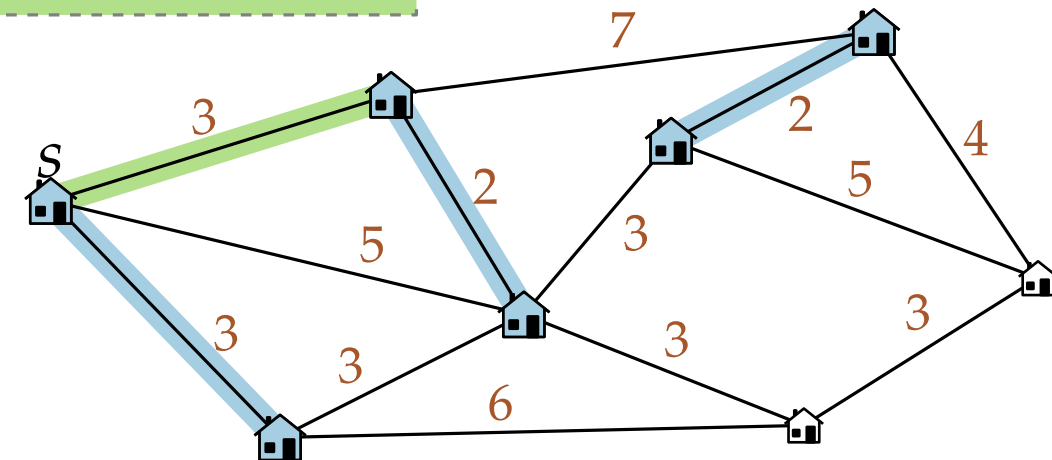
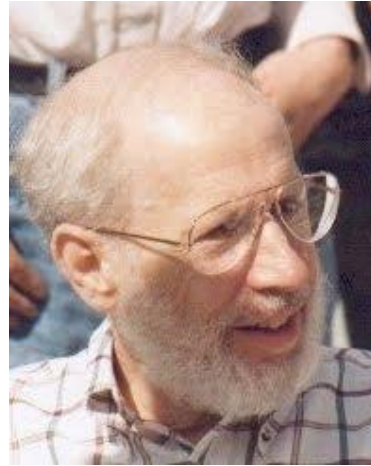
 Färbe uv blau

$E' = E' \cup \{uv\}$

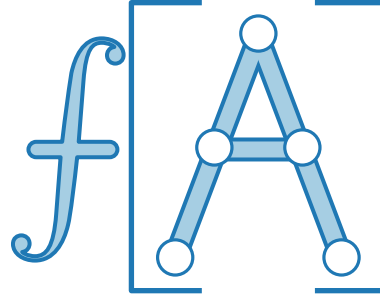
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

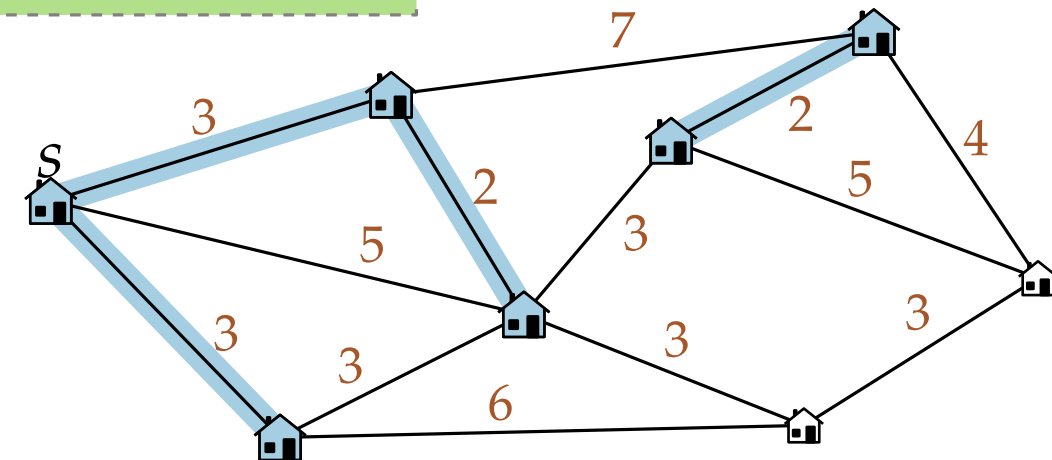
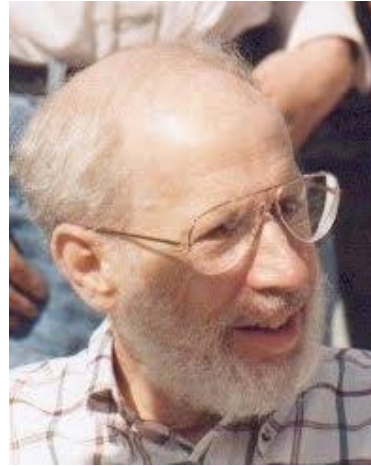
 Färbe uv blau

$E' = E' \cup \{uv\}$

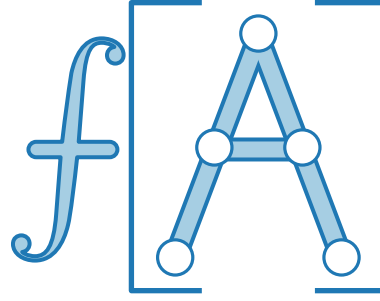
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

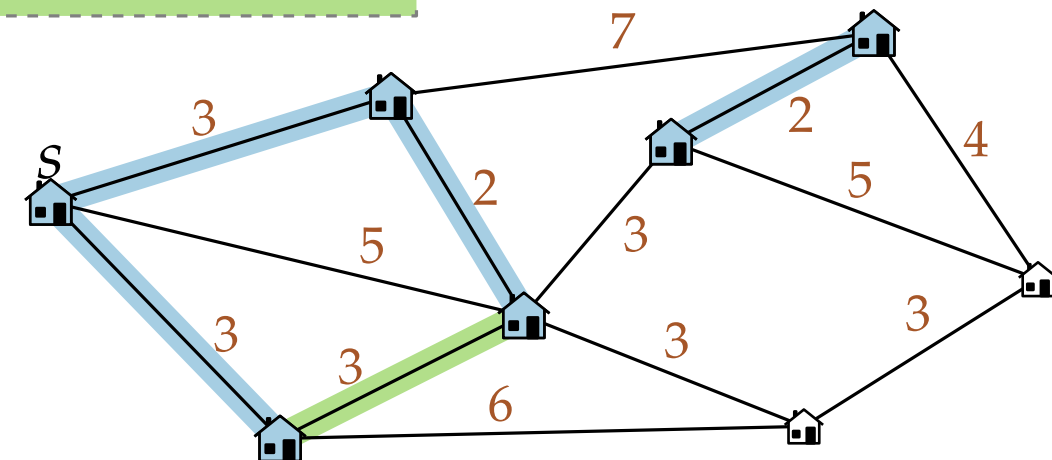
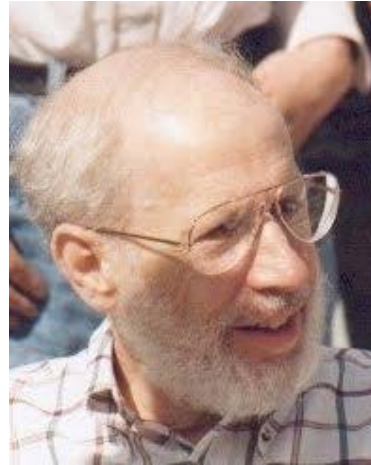
 Färbe uv blau

$E' = E' \cup \{uv\}$

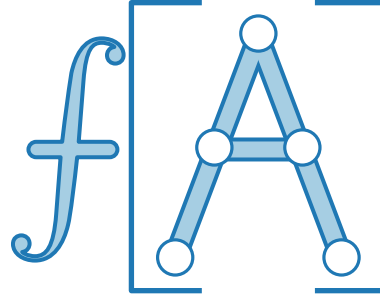
else

Blaue Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

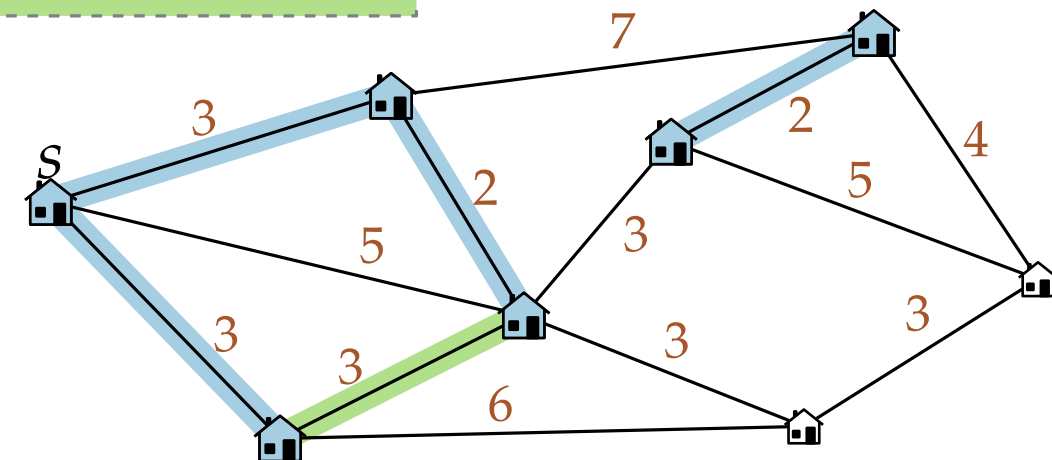
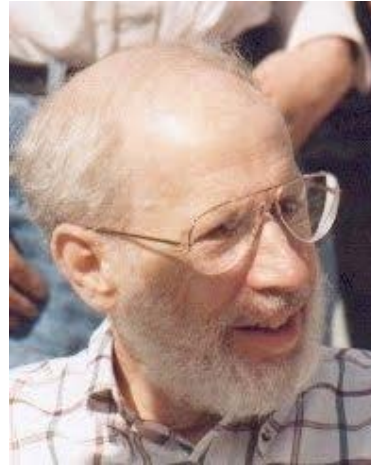
else

 Färbe uv **rot**

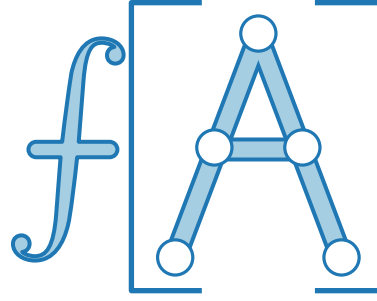
Blaue Regel

Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

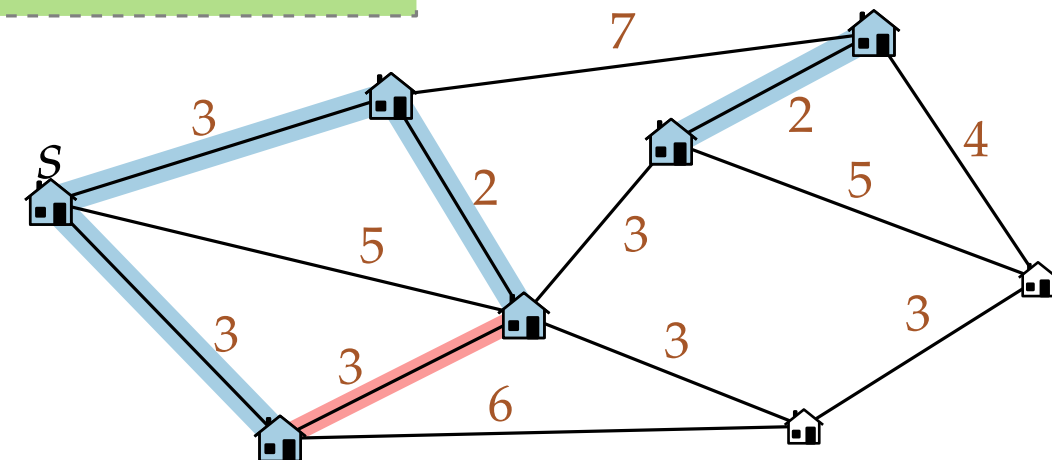
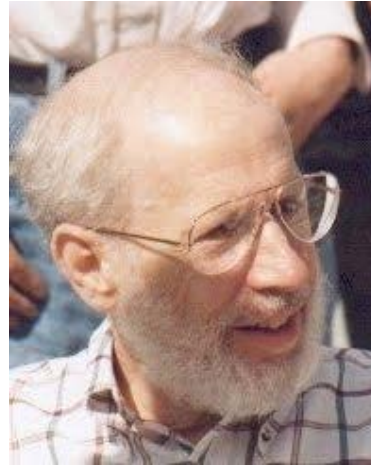
else

 Färbe uv **rot**

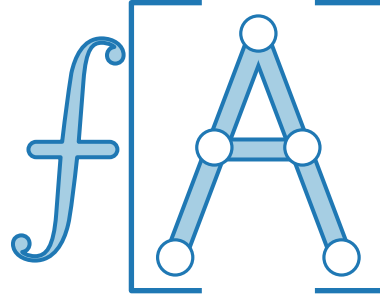
Blaue Regel

Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

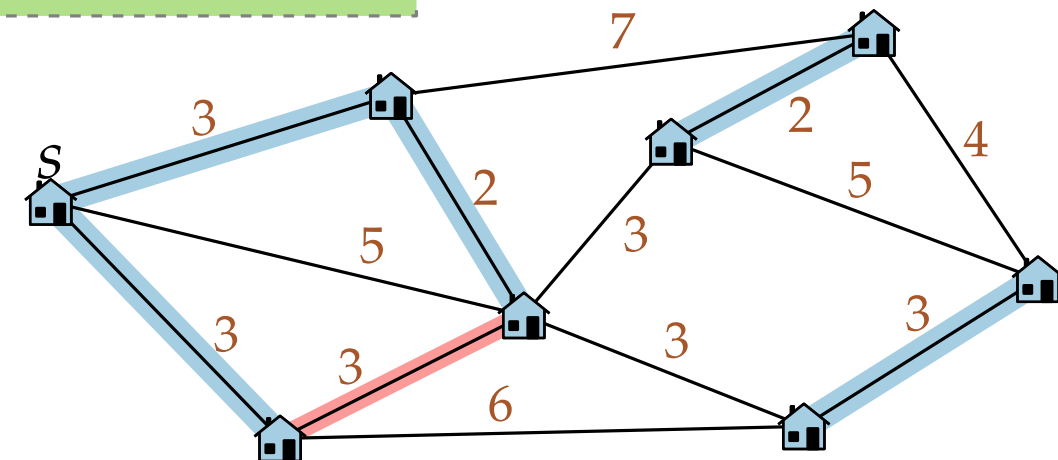
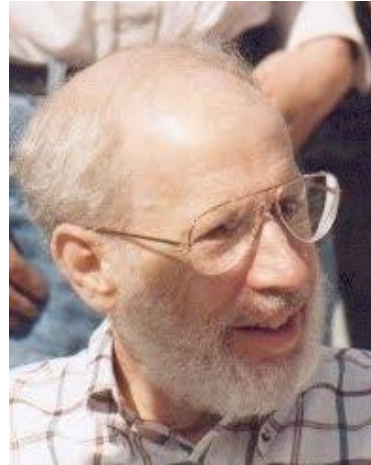
else

 Färbe uv **rot**

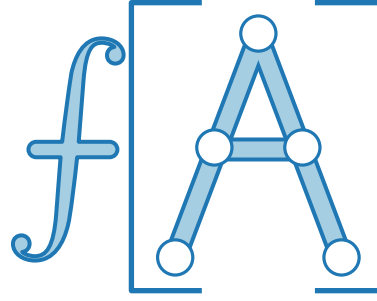
Blaue Regel

Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

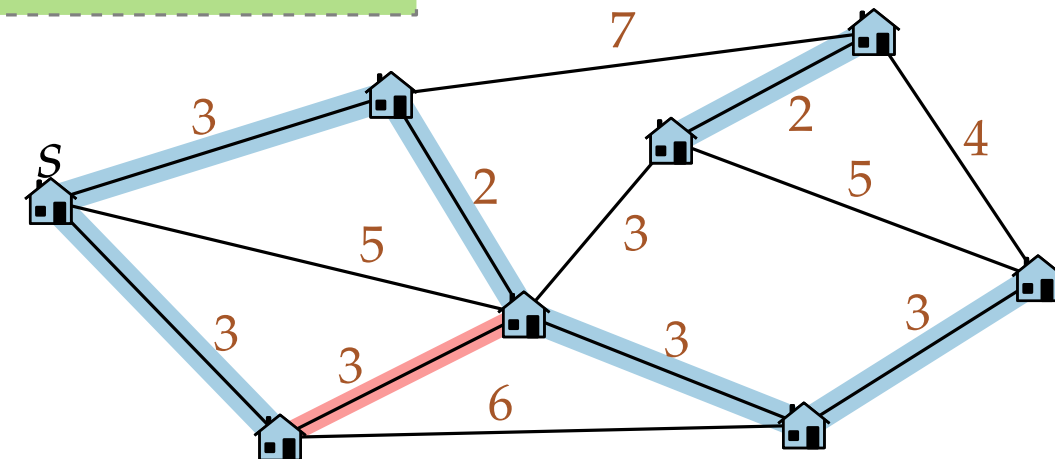
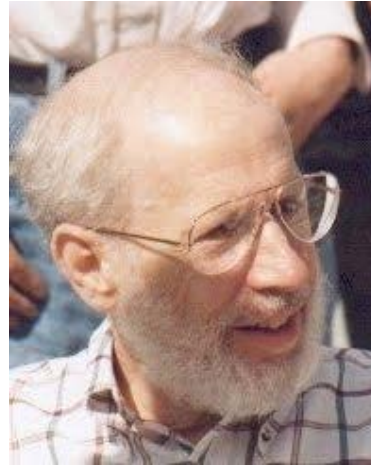
else

 Färbe uv **rot**

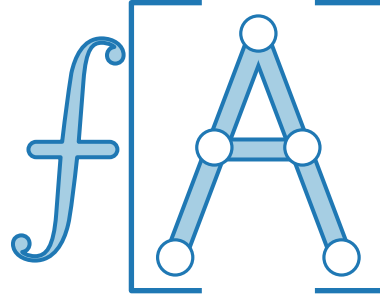
Blaue Regel

Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

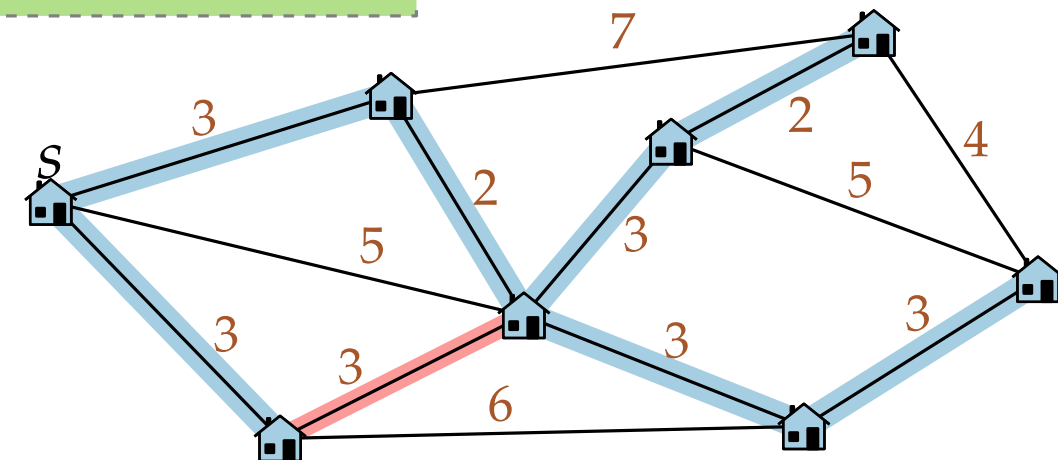
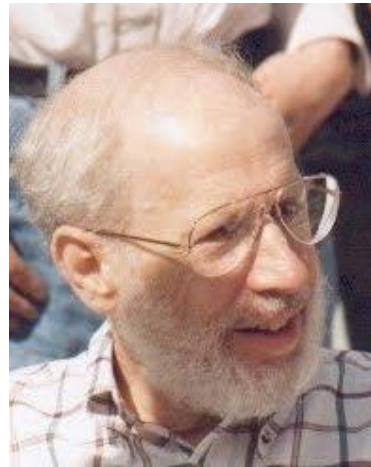
else

 Färbe uv **rot**

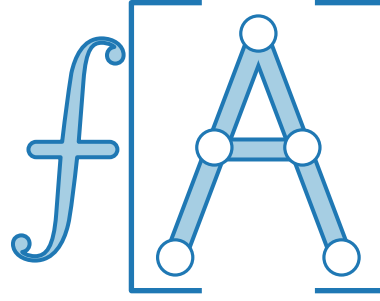
Blaue Regel

Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv blau

$E' = E' \cup \{uv\}$

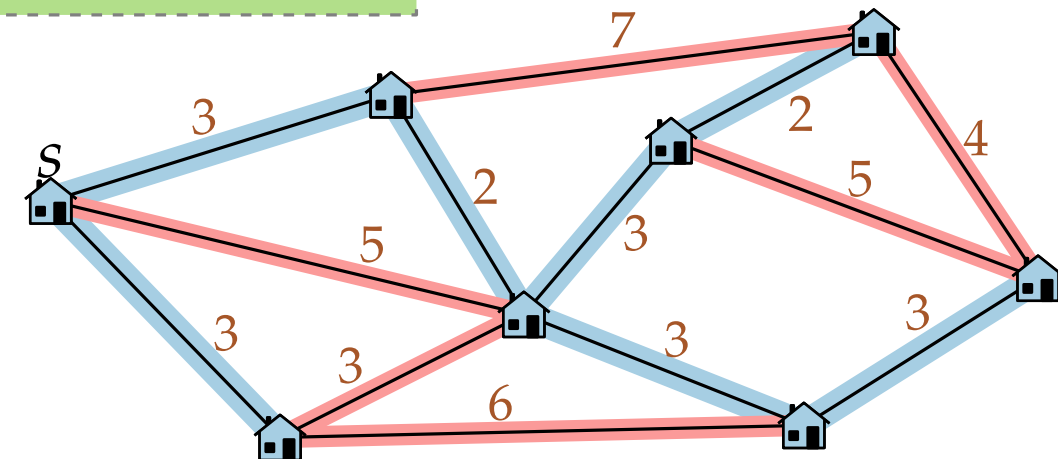
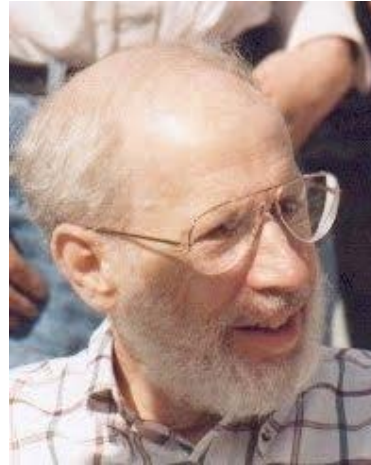
else

 Färbe uv rot

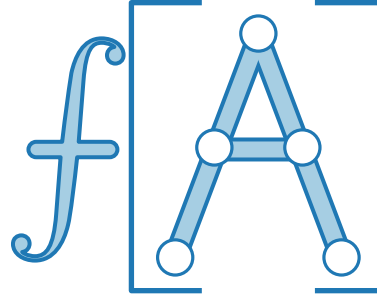
Blaue Regel

Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

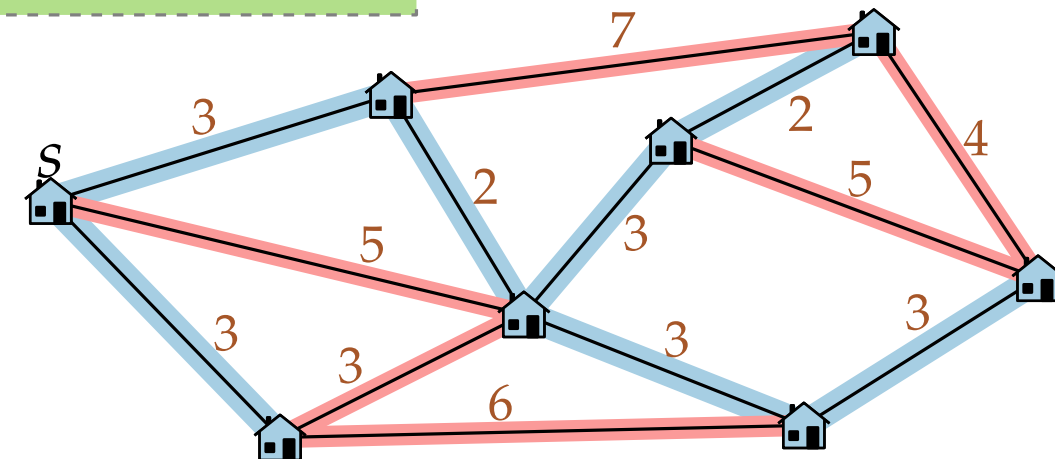
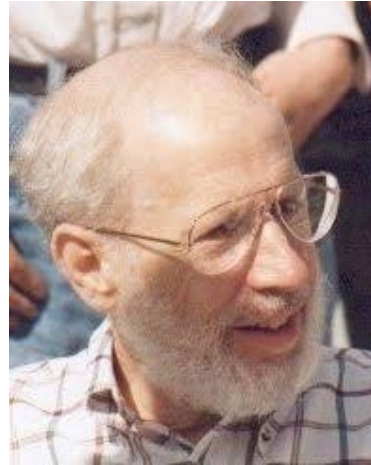
 Färbe uv **rot**

return E'

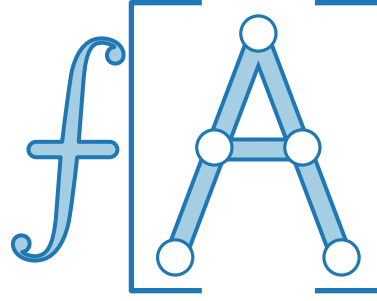
Blaue Regel

Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

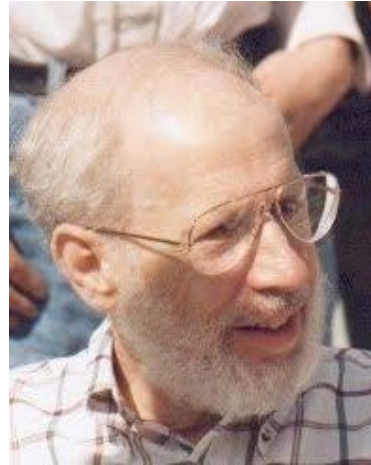
 Färbe uv **rot**

return E'

Blaue Regel

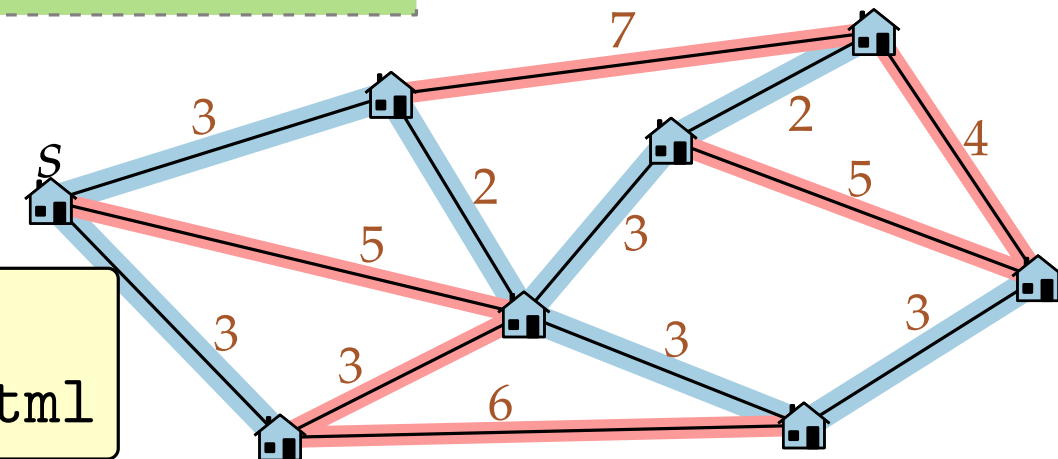
Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010

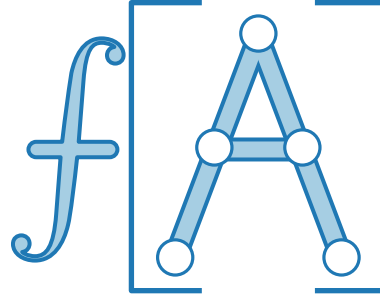


Demo.

<https://algo.uni-trier.de/demos/spanningtree.html>



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

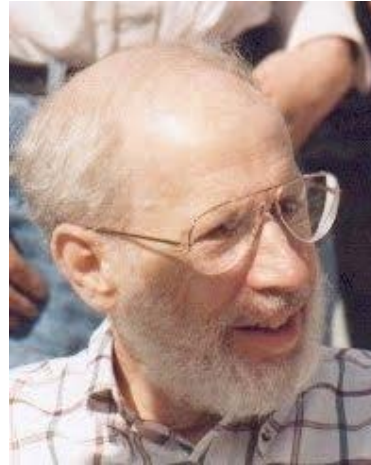
 Färbe uv **rot**

return E'

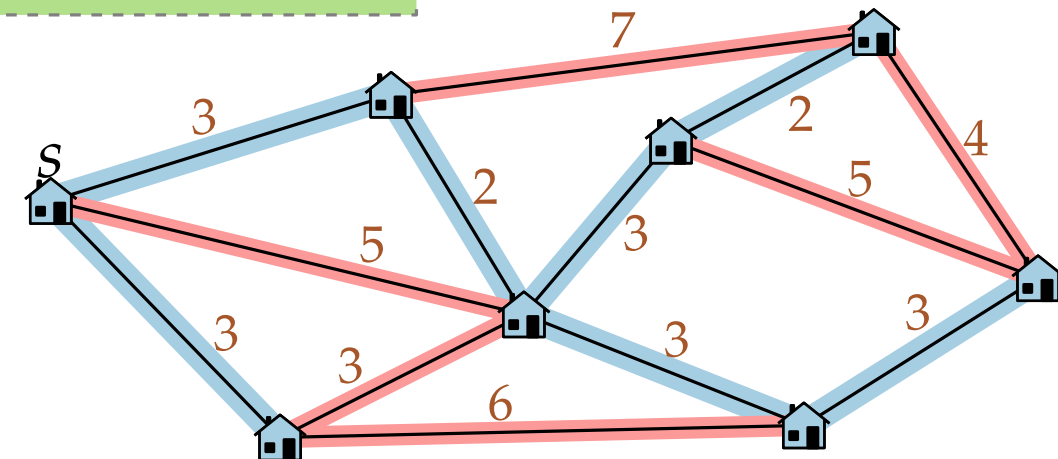
Blaue Regel

Rote Regel

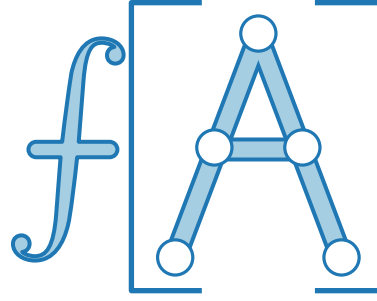
Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Laufzeit?



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

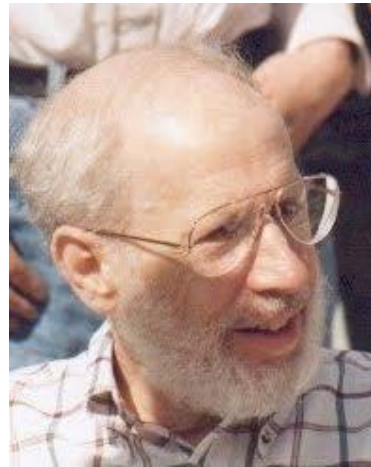
 Färbe uv **rot**

return E'

Blaue Regel

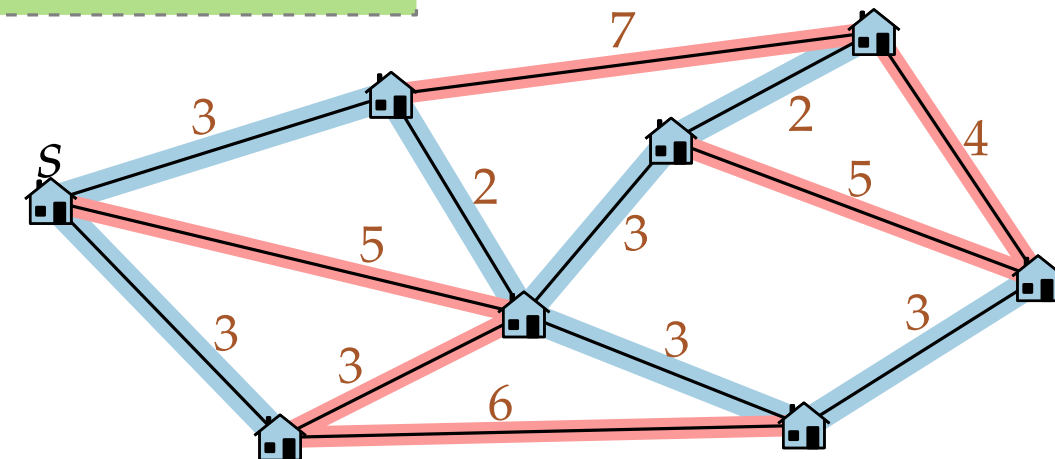
Rote Regel

Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010

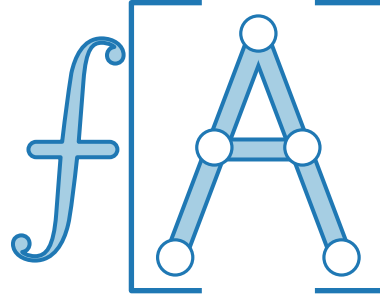


Laufzeit?

$\mathcal{O}(m \log n)$



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

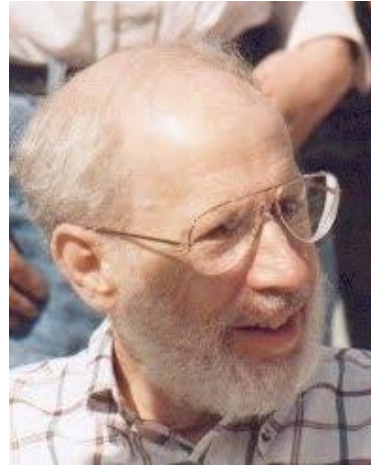
 Färbe uv **rot**

return E'

Blaue Regel

Rote Regel

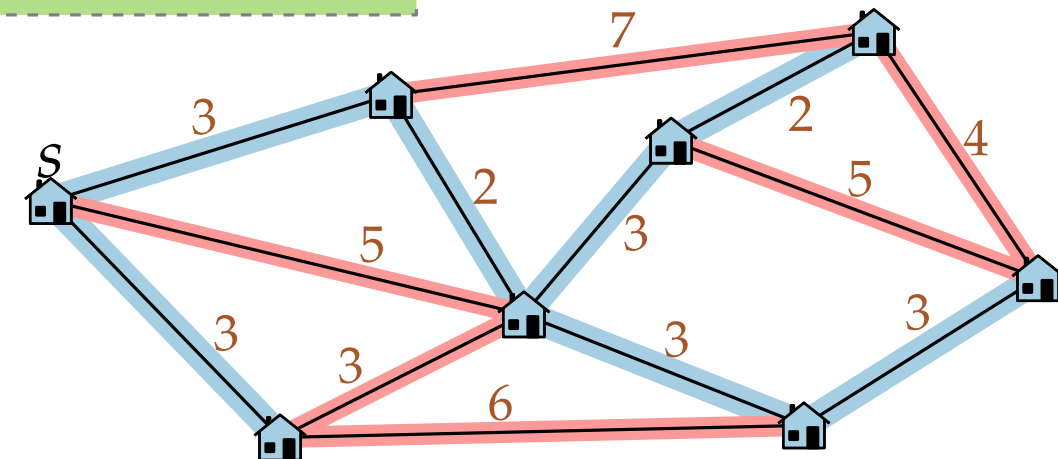
Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



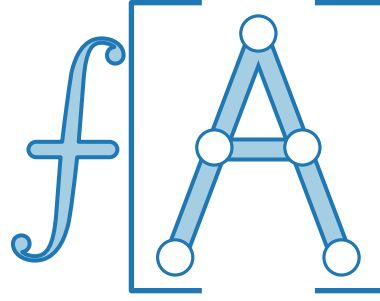
Laufzeit?

$\mathcal{O}(m \log n)$

$\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert



Der Algorithmus von Kruskal (1956)



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

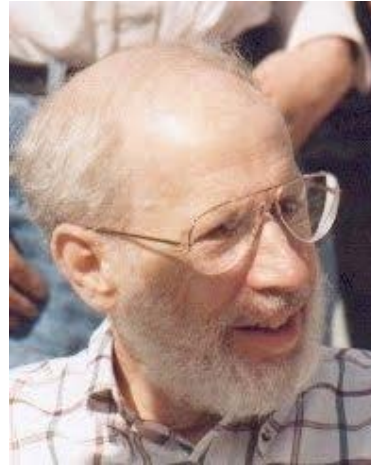
 Färbe uv **rot**

return E'

Blaue Regel

Rote Regel

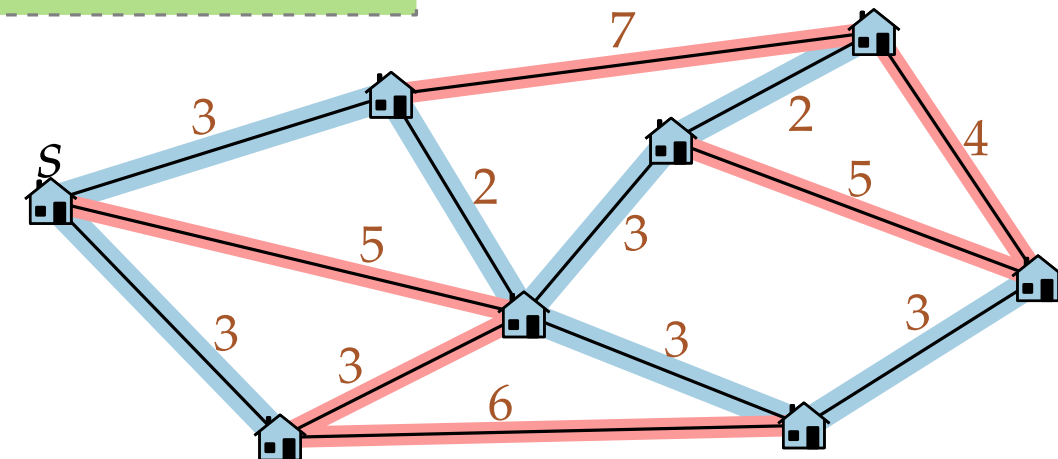
Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010

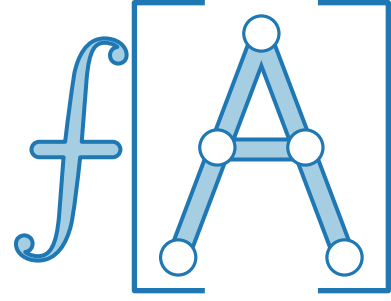


Laufzeit?

$\mathcal{O}(m \log n)$

$\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert

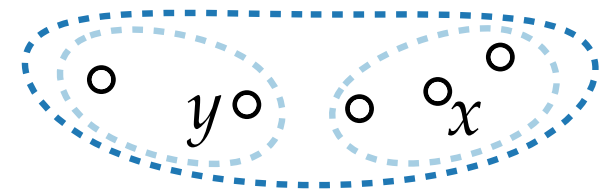
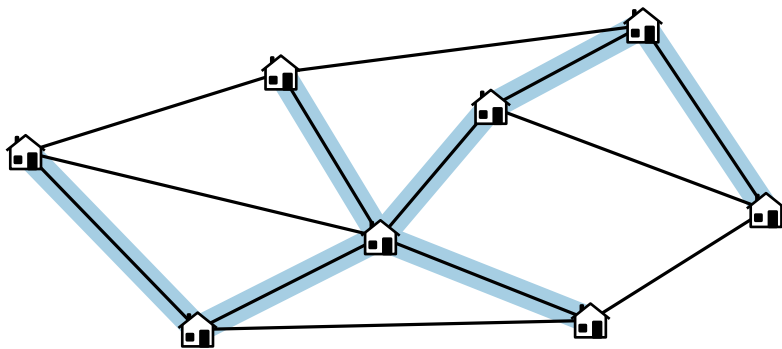




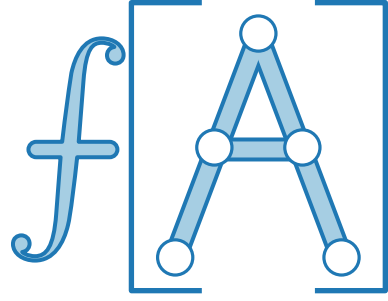
Fortgeschrittene Algorithmen

7. Vorlesung Graphalgorithmen I: Minimale Spannbäume

Teil V: UNIONFIND Datenstruktur

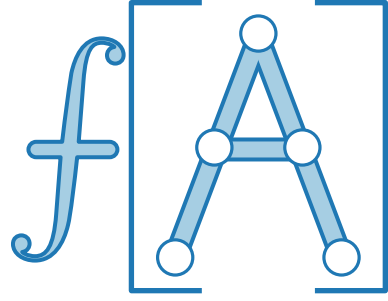


UNIONFIND Datenstruktur



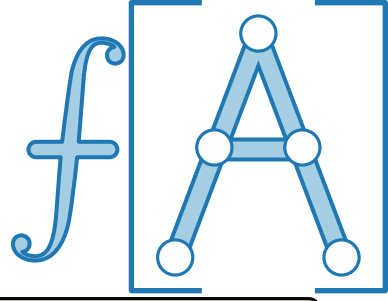
UNIONFIND Datenstruktur

Datenstruktur für **halbdynamische Mengen**



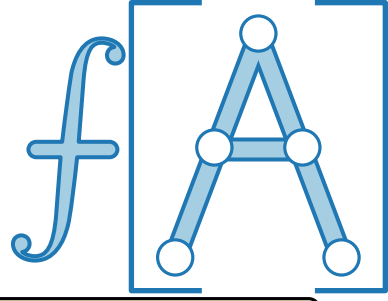
UNIONFIND Datenstruktur

Datenstruktur für **halbdynamische Mengen**



(wachsen nur, schrumpfen nicht)

UNIONFIND Datenstruktur

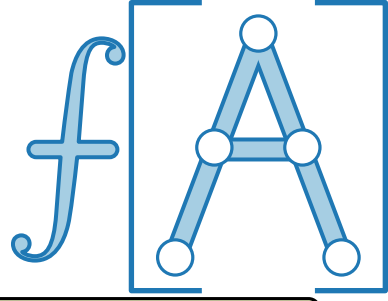


Datenstruktur für **halbdynamische Mengen**

(wachsen nur, schrumpfen nicht)

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

UNIONFIND Datenstruktur



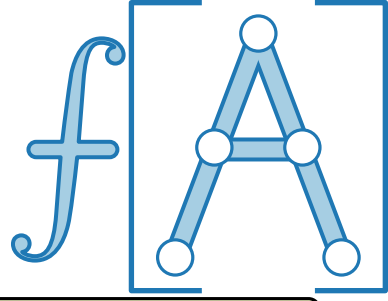
Datenstruktur für **halbdynamische Mengen**

(wachsen nur, schrumpfen nicht)

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(bei Kruskal: $X = V$)

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

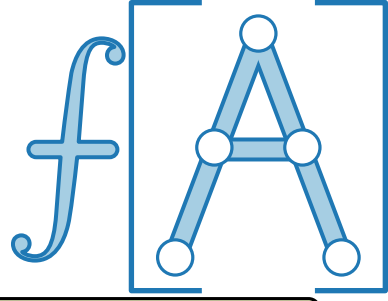
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

(bei Kruskal: $X = V$)

Drei Operationen:

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

(bei Kruskal: $X = V$)

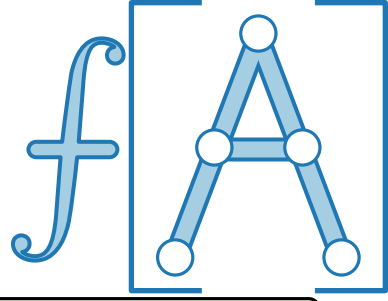
Drei Operationen:

MAKE(Element x)

FIND(Element x)

UNION(Elem. x , Elem. y)

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

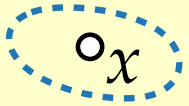
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

(bei Kruskal: $X = V$)

Drei Operationen:

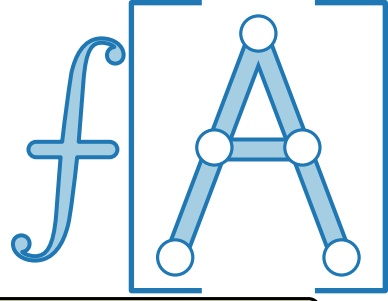
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x)

UNION(Elem. x , Elem. y)

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

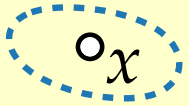
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

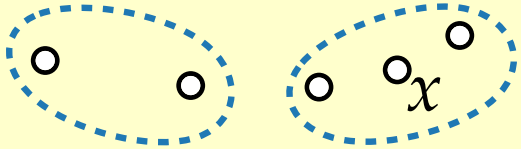
(bei Kruskal: $X = V$)

Drei Operationen:

MAKE(Element x) legt die Menge $\{x\}$ an.

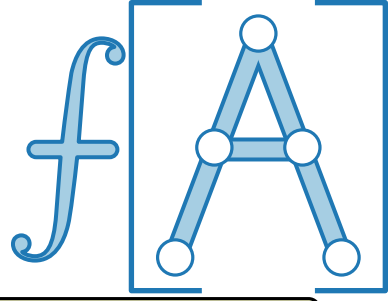


FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y)

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

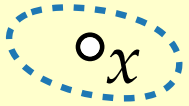
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

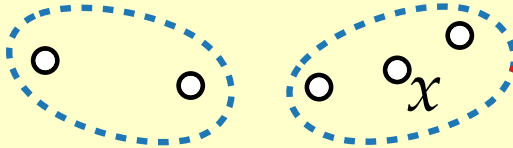
(bei Kruskal: $X = V$)

Drei Operationen:

MAKE(Element x) legt die Menge $\{x\}$ an.

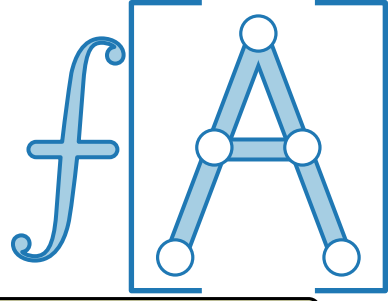


FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y)

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

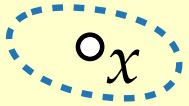
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

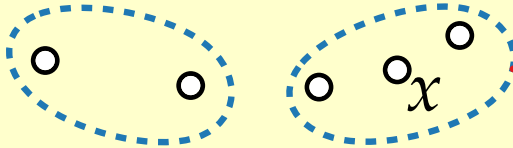
(bei Kruskal: $X = V$)

Drei Operationen:

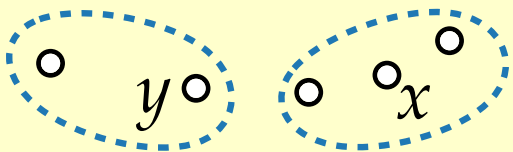
MAKE(Element x) legt die Menge $\{x\}$ an.



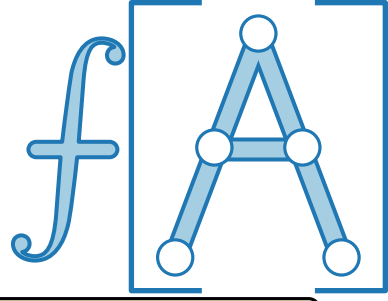
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

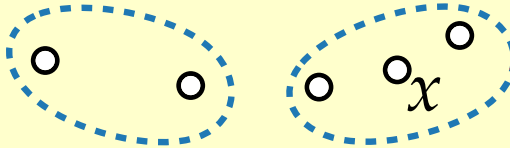
(bei Kruskal: $X = V$)

Drei Operationen:

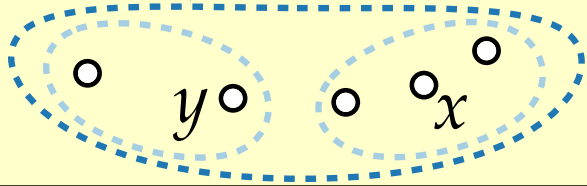
MAKE(Element x) legt die Menge $\{x\}$ an.



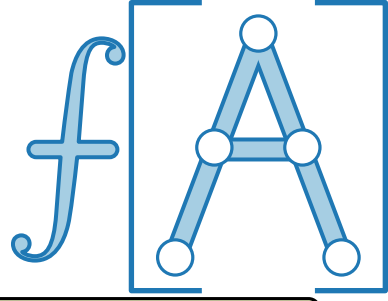
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

(bei Kruskal: $X = V$)

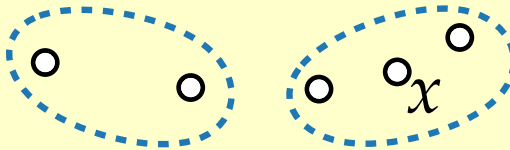
Drei Operationen:

MAKE(Element x)



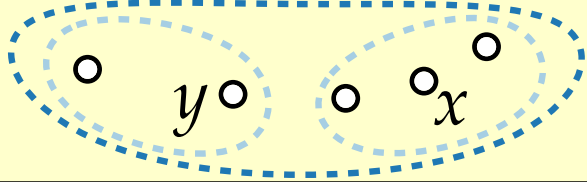
legt die Menge $\{x\}$ an.

FIND(Element x)



liefert (Zeiger auf) die Menge zurück, die momentan x enthält.

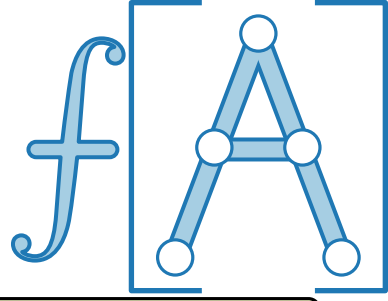
UNION(Elem. x , Elem. y)



vereinigt die Mengen, die momentan x und y enthalten.

Beispiel.

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

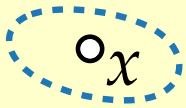
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

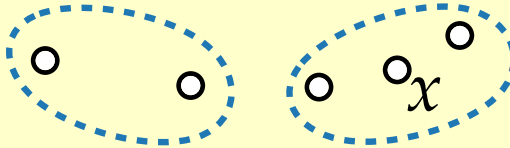
(bei Kruskal: $X = V$)

Drei Operationen:

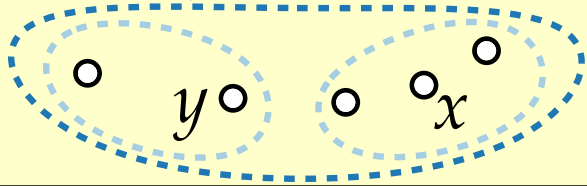
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



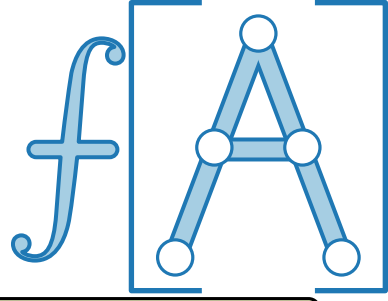
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.



UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

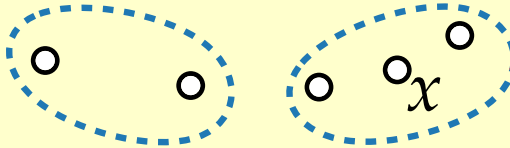
(bei Kruskal: $X = V$)

Drei Operationen:

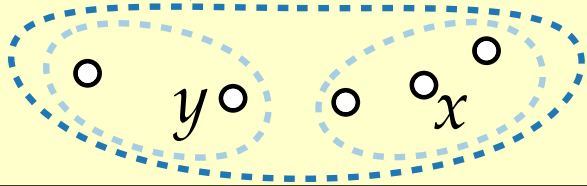
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.

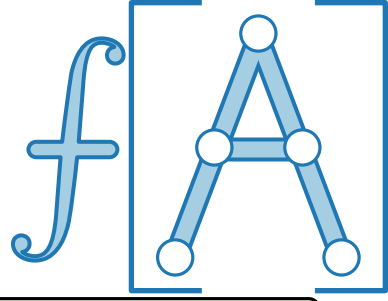


Beispiel.



■ UNION(1,2)

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

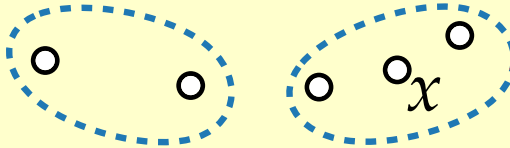
(bei Kruskal: $X = V$)

Drei Operationen:

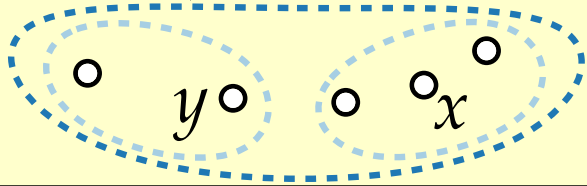
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



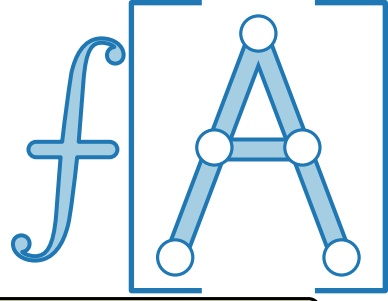
Beispiel.



■ UNION(1, 2)



UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

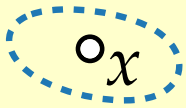
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

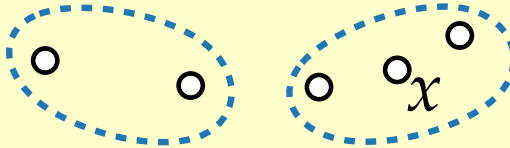
(bei Kruskal: $X = V$)

Drei Operationen:

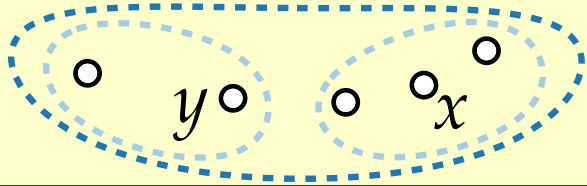
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.

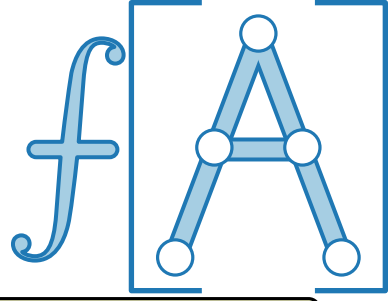


■ UNION(1,2)



■ UNION(2,3)

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

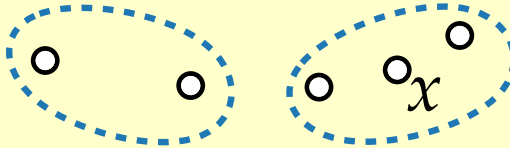
(bei Kruskal: $X = V$)

Drei Operationen:

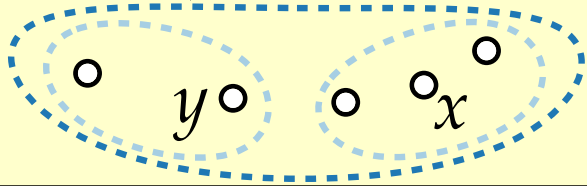
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.



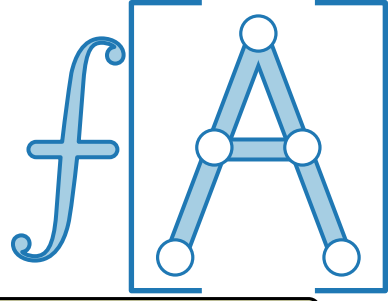
■ UNION(1,2)



■ UNION(2,3)



UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

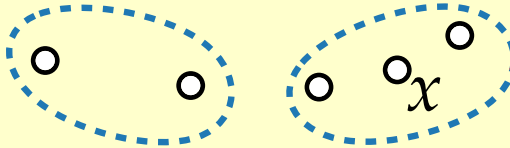
(bei Kruskal: $X = V$)

Drei Operationen:

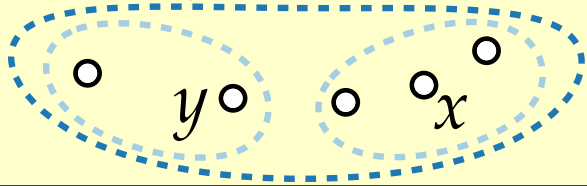
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.



■ UNION(1,2)

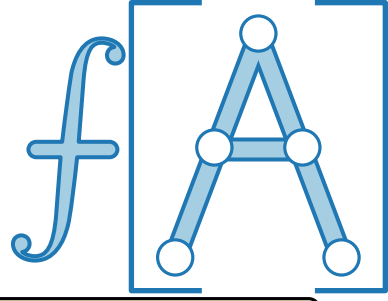


■ UNION(2,3)



■ FIND(1) = FIND(3)?

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

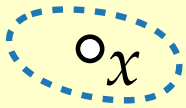
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

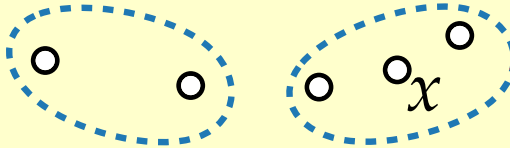
(bei Kruskal: $X = V$)

Drei Operationen:

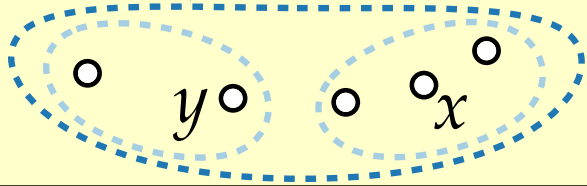
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.



■ UNION(1,2)

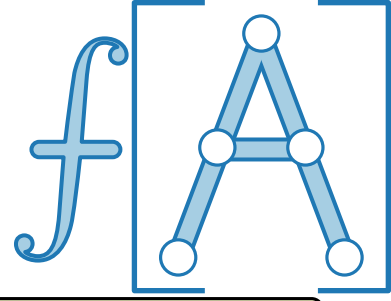


■ UNION(2,3)



■ FIND(1) = FIND(3)?
➔ true

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

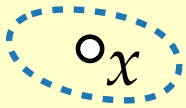
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

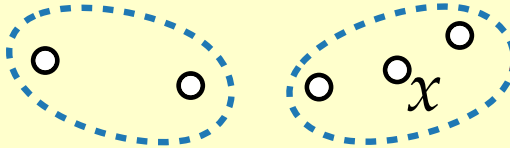
(bei Kruskal: $X = V$)

Drei Operationen:

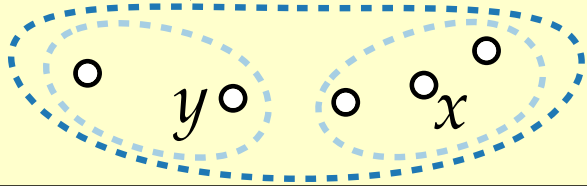
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.



■ UNION(1,2)



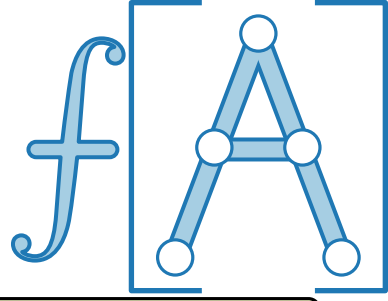
■ UNION(2,3)



■ FIND(1) = FIND(3)?
➔ true

■ FIND(2) = FIND(4)?

UNIONFIND Datenstruktur



Datenstruktur für **halbdynamische Mengen**

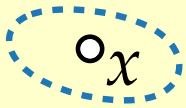
Die halbdynamischen Mengen zerlegen immer eine Grundmenge X

(wachsen nur, schrumpfen nicht)

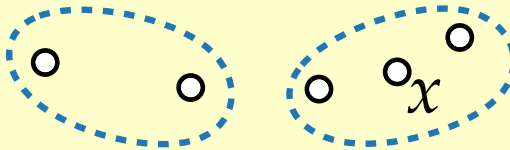
(bei Kruskal: $X = V$)

Drei Operationen:

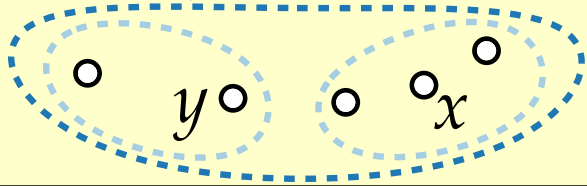
MAKE(Element x) legt die Menge $\{x\}$ an.



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.



■ UNION(1,2)



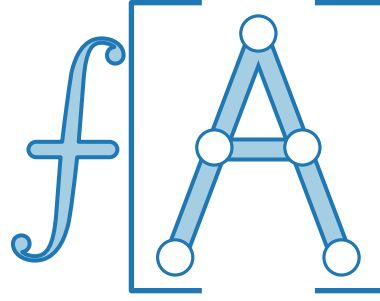
■ UNION(2,3)



■ FIND(1) = FIND(3)?
➔ true

■ FIND(2) = FIND(4)?
➔ false

Anpassung Kruskal



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv blau

$E' = E' \cup \{uv\}$

else

 Färbe uv rot

return E'

Blaue Regel

Rote Regel

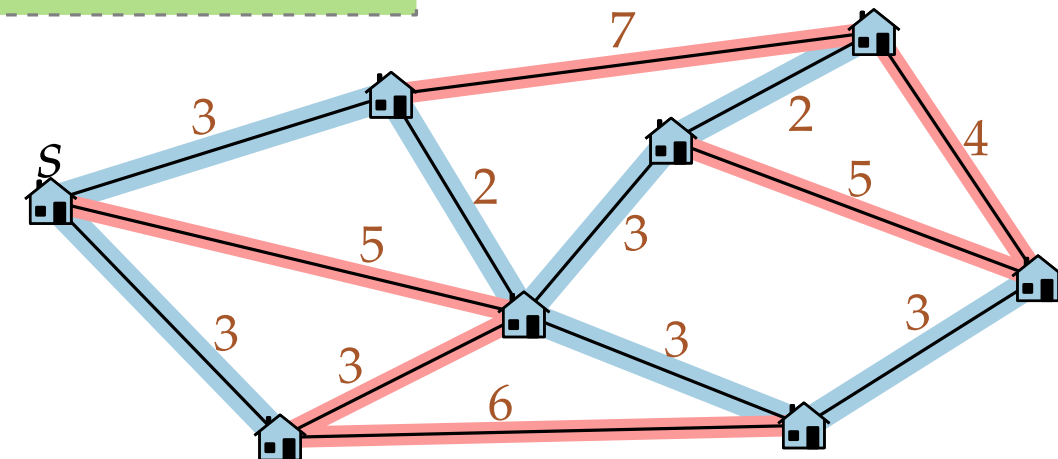
Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



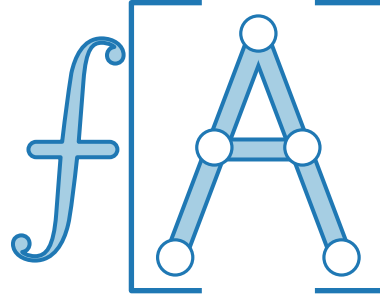
Laufzeit?

$\mathcal{O}(m \log n)$

$\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert



Anpassung Kruskal



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

 Färbe uv **rot**

return E'

$\forall v \in V : \text{MAKE}(v)$

Blaue Regel

Rote Regel

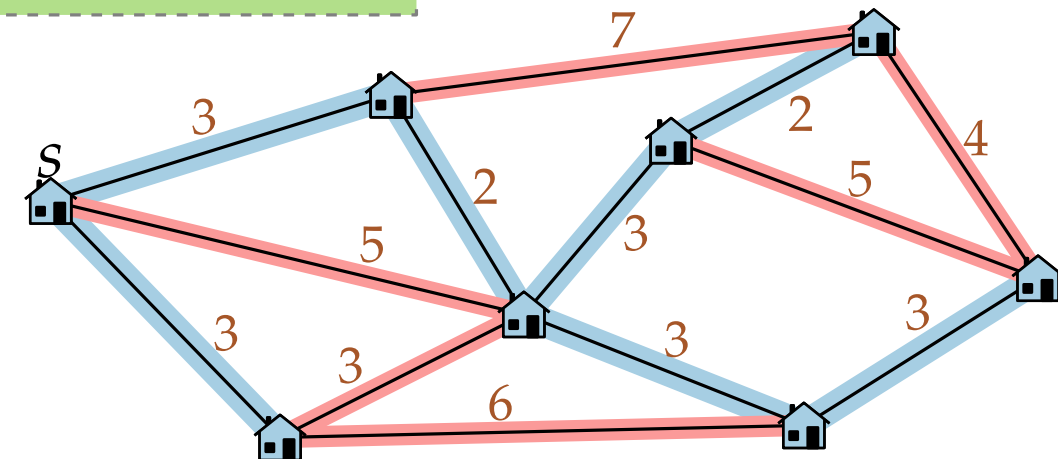
Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



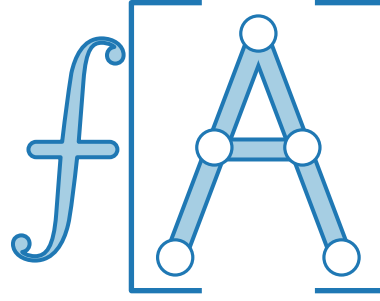
Laufzeit?

$\mathcal{O}(m \log n)$

$\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert



Anpassung Kruskal



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

$\forall v \in V : \text{MAKE}(v)$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

 Färbe uv **rot**

return E'

if $\text{FIND}(u) \neq \text{FIND}(v)$

Blaue Regel

Rote Regel

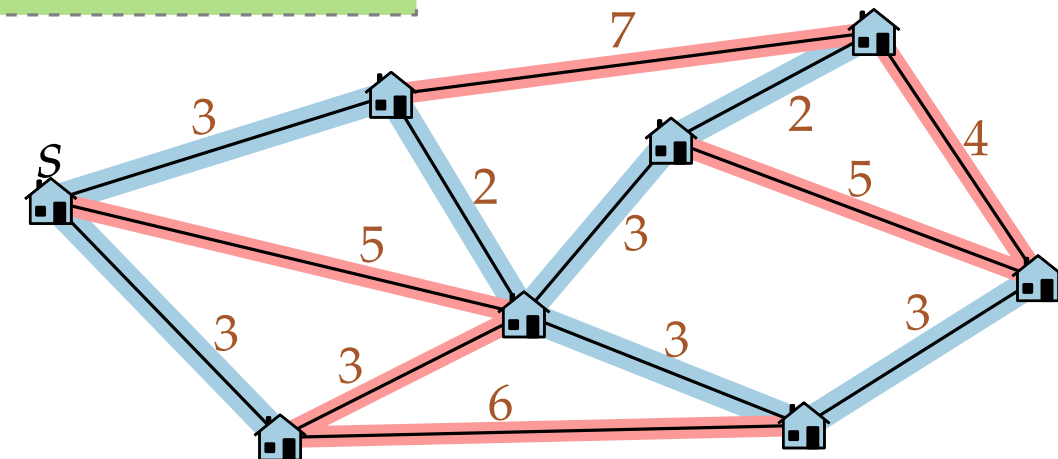
Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



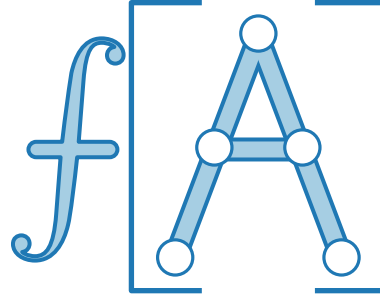
Laufzeit?

$\mathcal{O}(m \log n)$

$\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert



Anpassung Kruskal



KRUSKAL(WeightedGraph $G = (V, E; w)$)

$E' = \emptyset$

$\forall v \in V : \text{MAKE}(v)$

Sortiere E nicht-absteigend nach Gewicht w

foreach $uv \in E$ **do** (in sortierter Reihenfolge)

if $E' \cup \{uv\}$ enthält keinen Kreis **then**

 Färbe uv **blau**

$E' = E' \cup \{uv\}$

else

 Färbe uv **rot**

return E'

if $\text{FIND}(u) \neq \text{FIND}(v)$

Blaue Regel

$\text{UNION}(u, v)$

Rote Regel

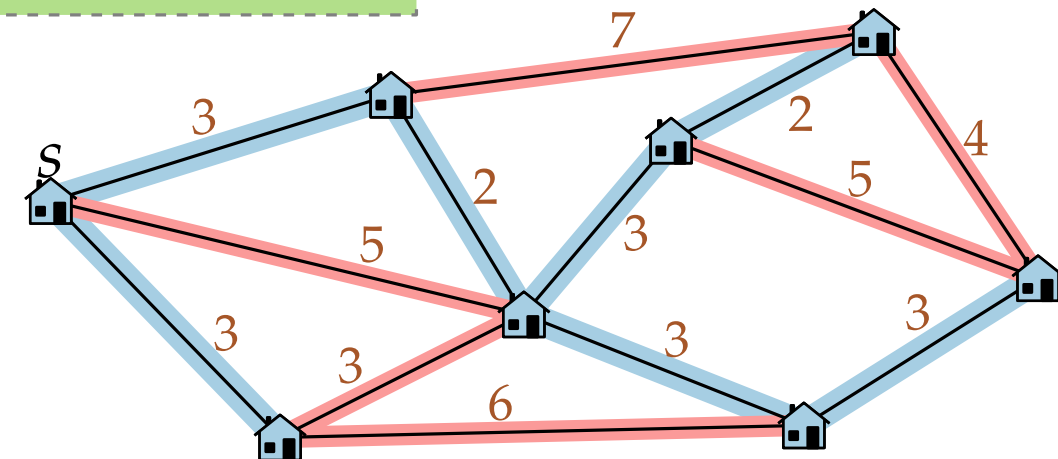
Joseph Bernard Kruskal, Jr.
*1928 New York City
+ 2010



Laufzeit?

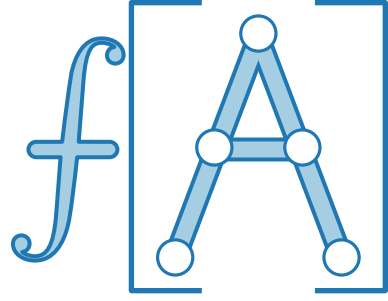
$\mathcal{O}(m \log n)$

$\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert



Realisierung UNIONFIND

Baumstruktur für jede Menge



Realisierung UNIONFIND

Baumstruktur für jede Menge

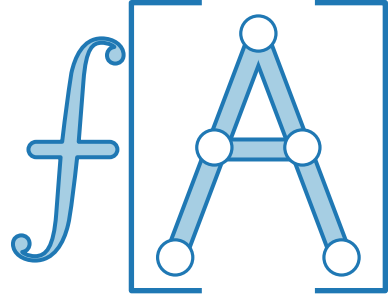
Beispiel.

○
1

○
2

○
3

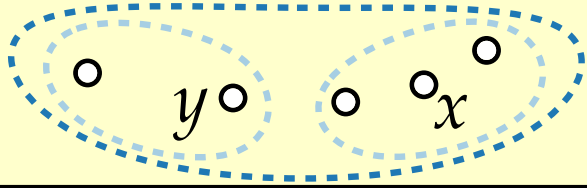
○
4



Realisierung UNIONFIND

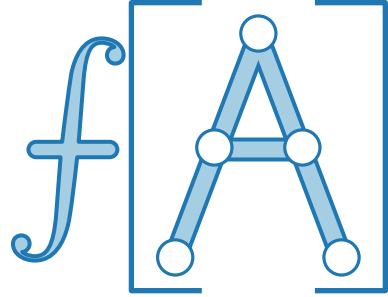
Baumstruktur für jede Menge

UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.

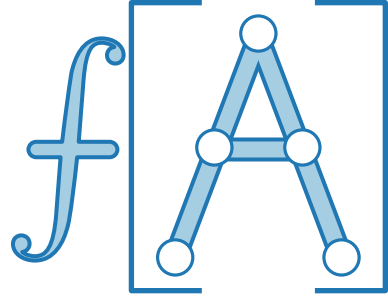
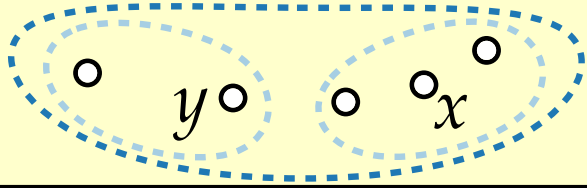
○
1 ○
2 ○
3 ○
4



Realisierung UNIONFIND

Baumstruktur für jede Menge

UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



Beispiel.

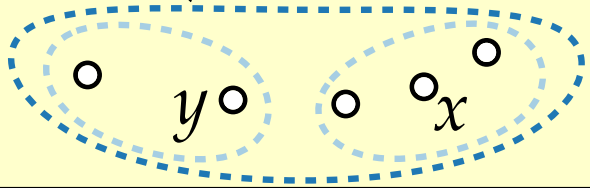
1 2 3 4

■ UNION(1,2)

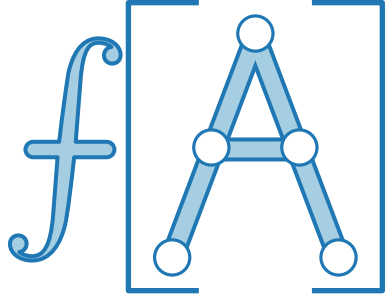
Realisierung UNIONFIND

Baumstruktur für jede Menge

$\text{UNION}(\text{Elem. } x, \text{Elem. } y)$ vereinigt die Mengen, die momentan x und y enthalten.



➡ Hänge einen Baum an den anderen



Beispiel.

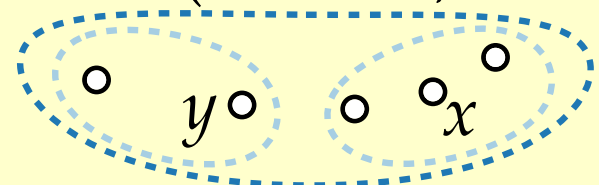
○ 1 ○ 2 ○ 3 ○ 4

■ $\text{UNION}(1, 2)$

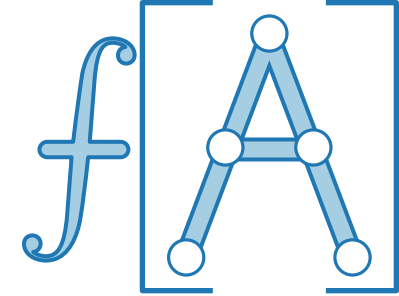
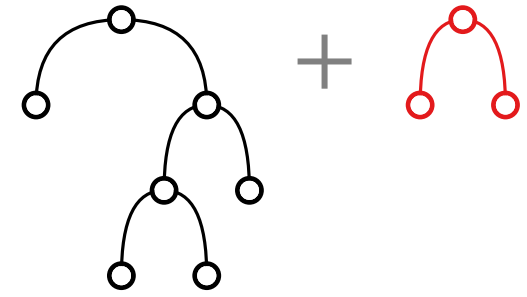
Realisierung UNIONFIND

Baumstruktur für jede Menge

UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



➡ Hänge einen Baum an den anderen



Beispiel.

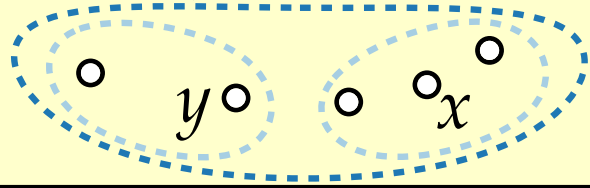


■ UNION(1,2)

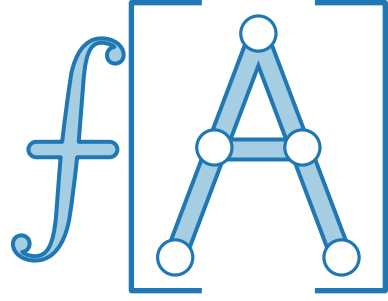
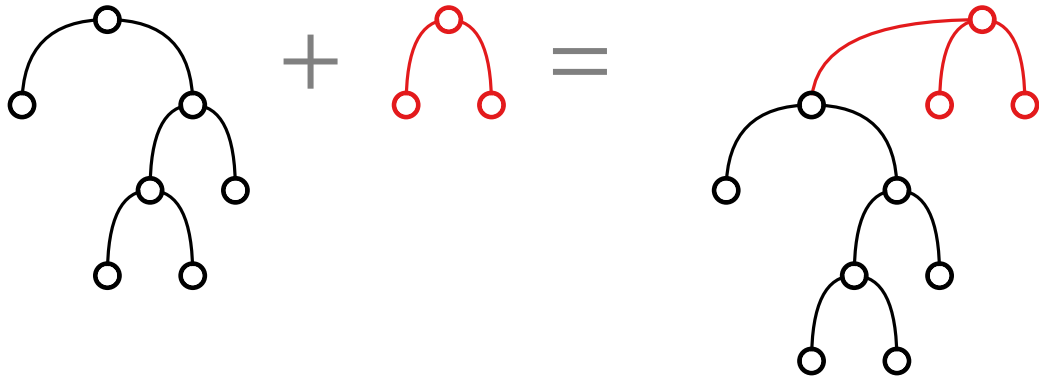
Realisierung UNIONFIND

Baumstruktur für jede Menge

UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



→ Hänge einen Baum an den anderen



Beispiel.

1 2 3 4

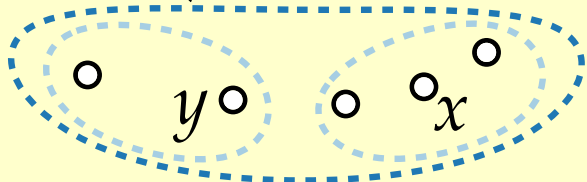
■ UNION(1,2)



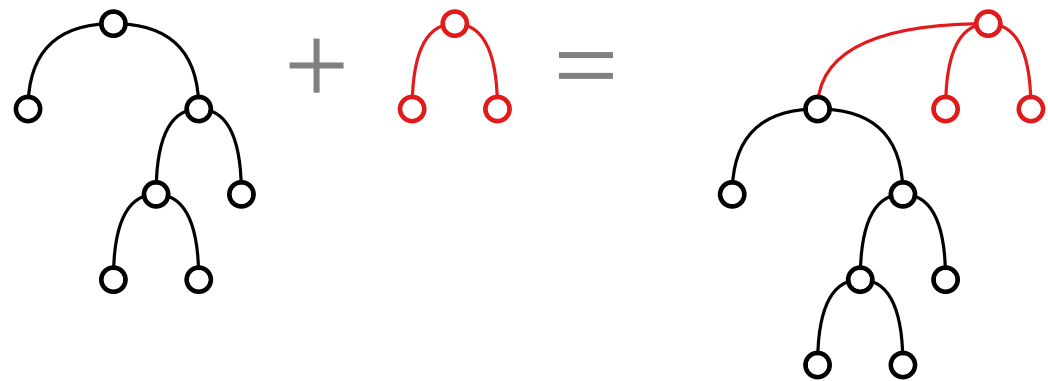
Realisierung UNIONFIND

Baumstruktur für jede Menge

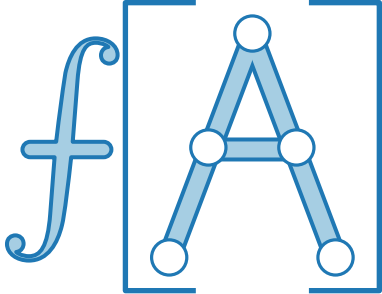
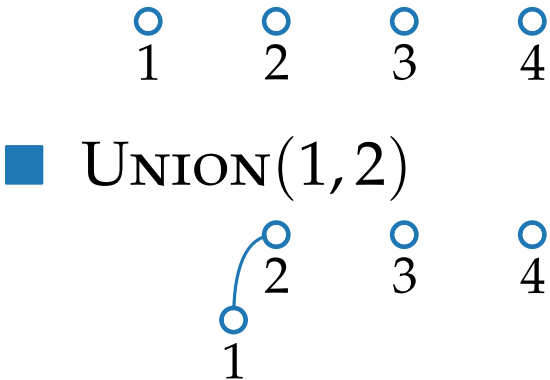
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



➡ Hänge einen Baum an den anderen



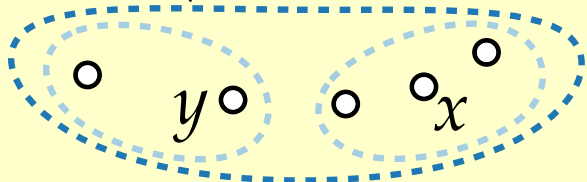
Beispiel.



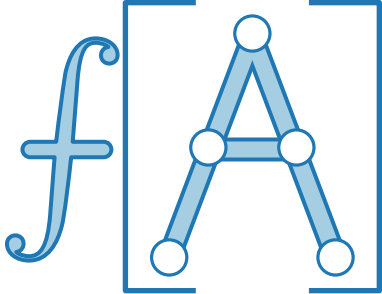
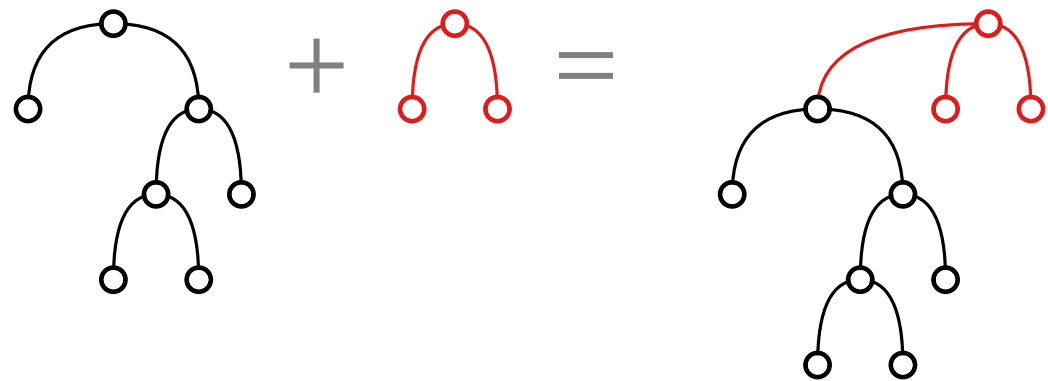
Realisierung UNIONFIND

Baumstruktur für jede Menge

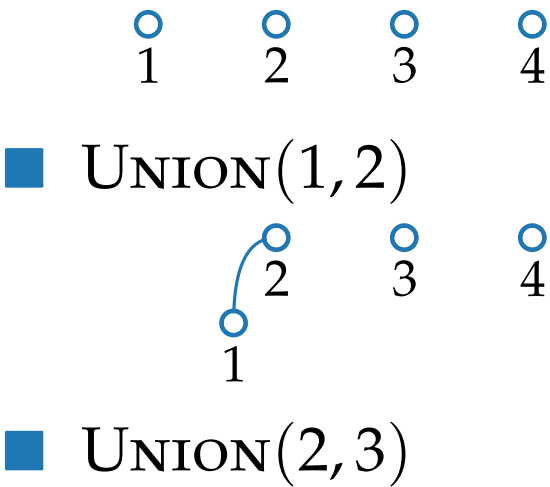
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



➡ Hänge einen Baum an den anderen



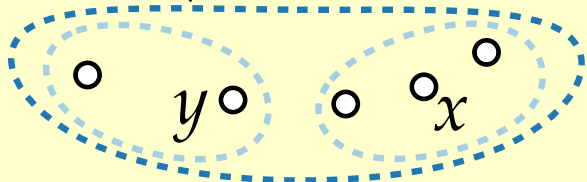
Beispiel.



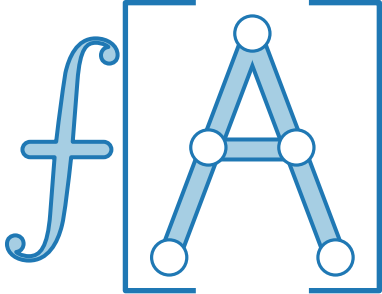
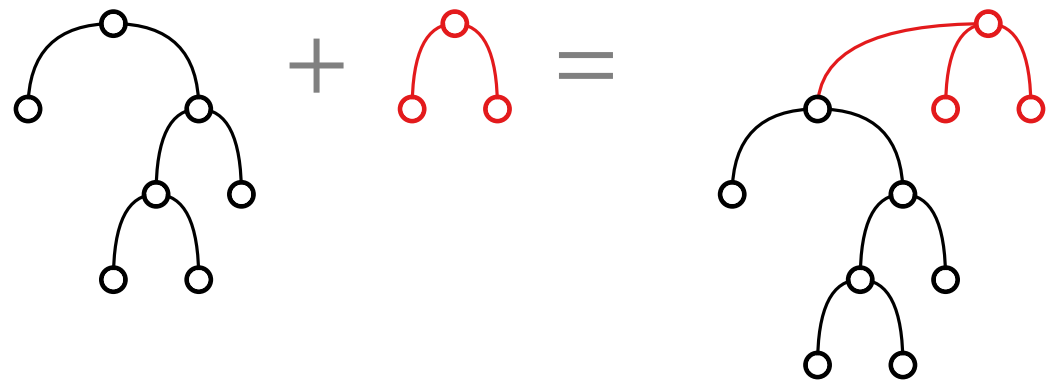
Realisierung UNIONFIND

Baumstruktur für jede Menge

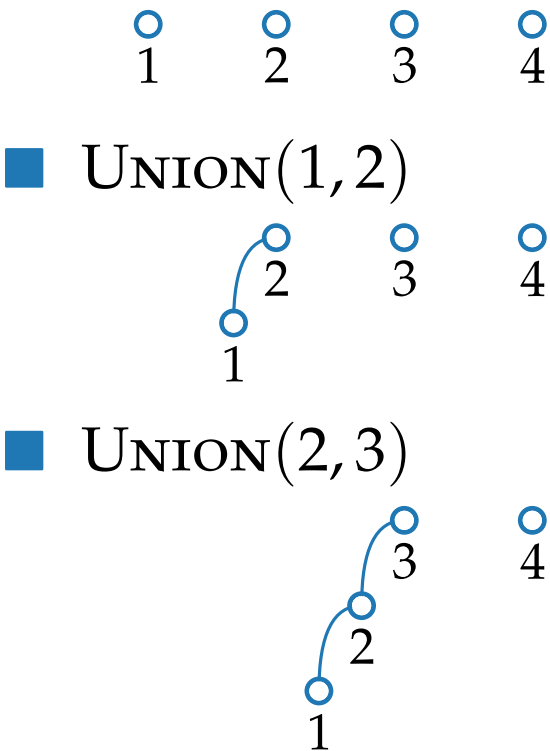
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



➡ Hänge einen Baum an den anderen



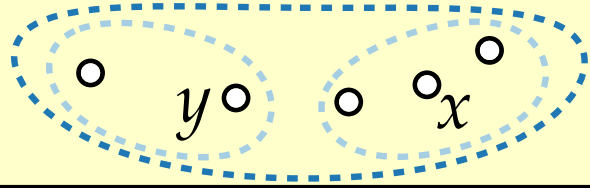
Beispiel.



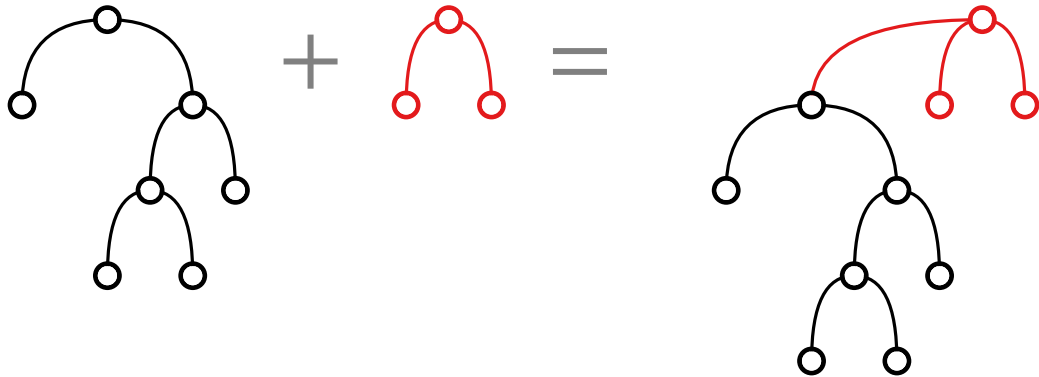
Realisierung UNIONFIND

Baumstruktur für jede Menge

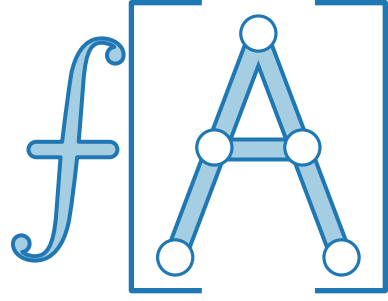
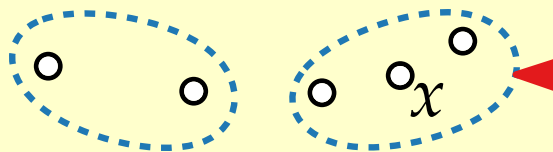
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



→ Hänge einen Baum an den anderen



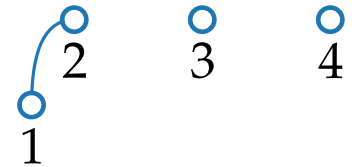
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



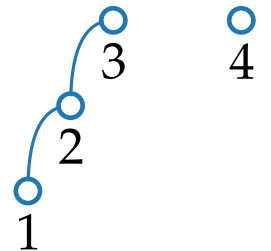
Beispiel.

1 2 3 4

■ UNION(1,2)



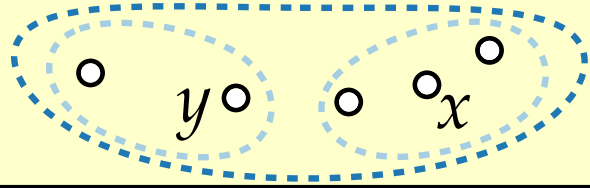
■ UNION(2,3)



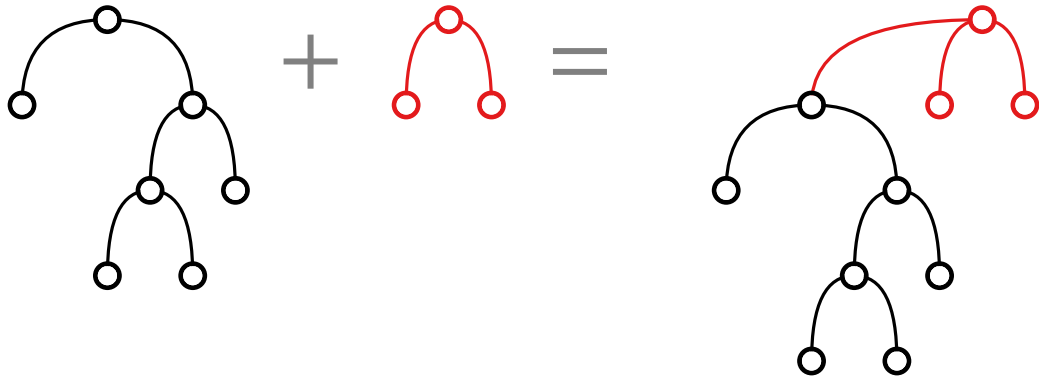
Realisierung UNIONFIND

Baumstruktur für jede Menge

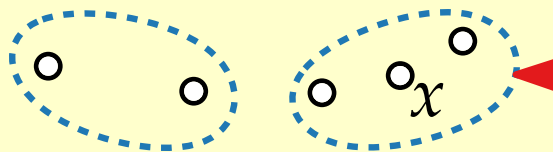
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



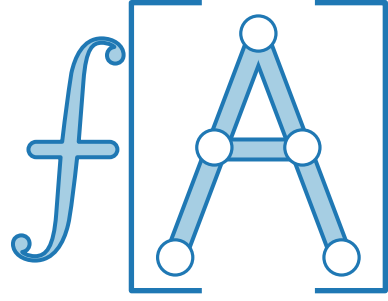
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



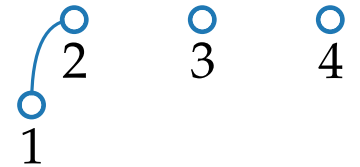
→ Laufe zur Wurzel, gebe Wurzel zurück



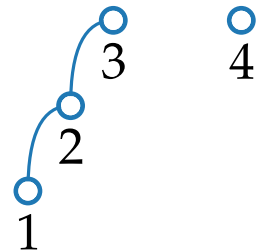
Beispiel.

1 2 3 4

■ UNION(1,2)



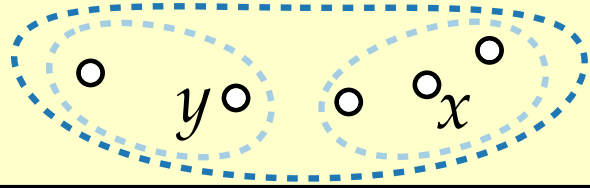
■ UNION(2,3)



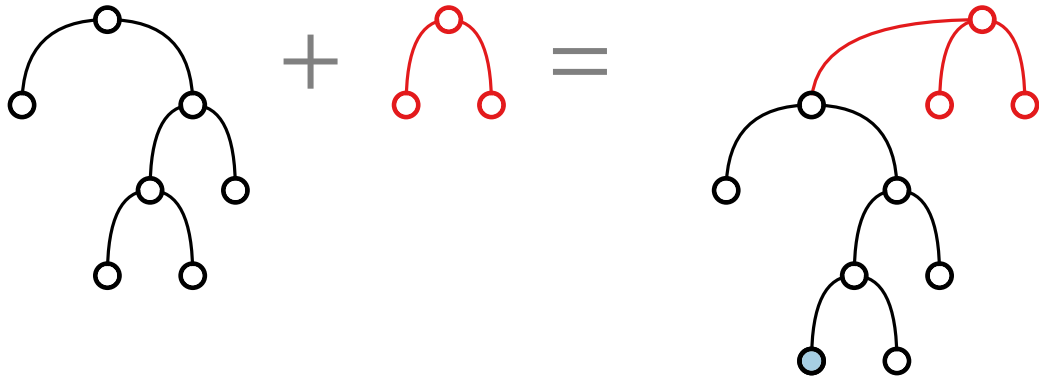
Realisierung UNIONFIND

Baumstruktur für jede Menge

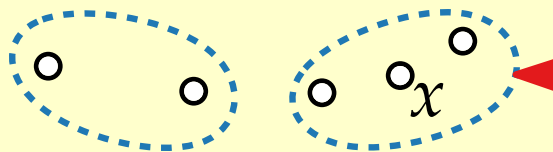
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



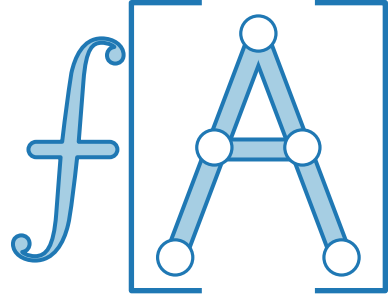
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



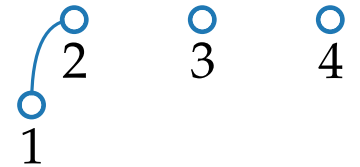
→ Laufe zur Wurzel, gebe Wurzel zurück



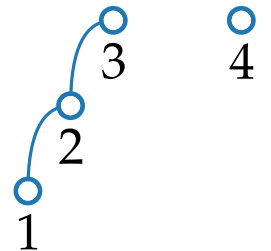
Beispiel.

1 2 3 4

■ UNION(1,2)



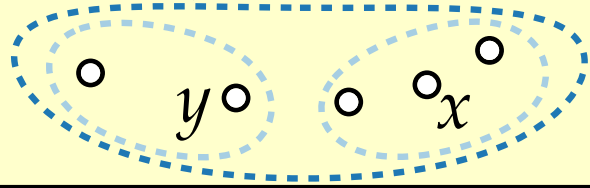
■ UNION(2,3)



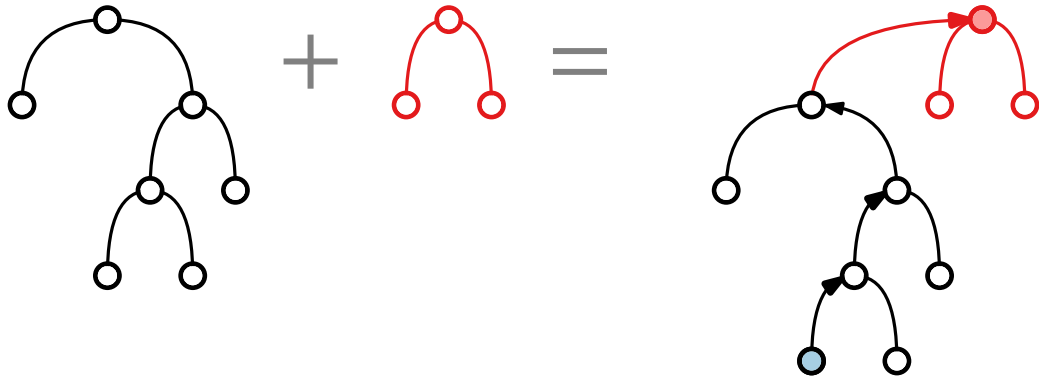
Realisierung UNIONFIND

Baumstruktur für jede Menge

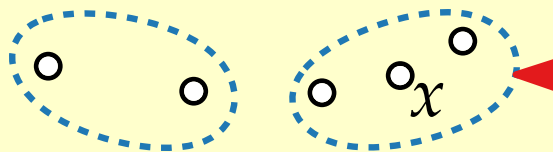
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



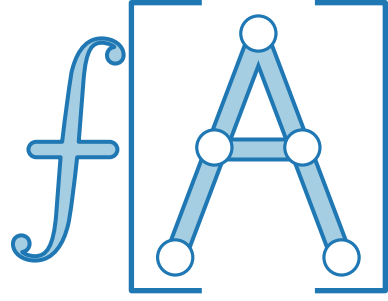
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



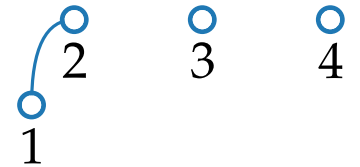
→ Laufe zur Wurzel, gebe Wurzel zurück



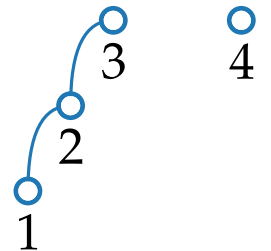
Beispiel.

1 2 3 4

■ UNION(1,2)



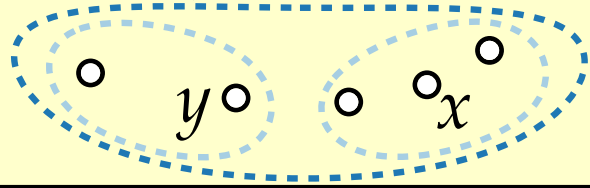
■ UNION(2,3)



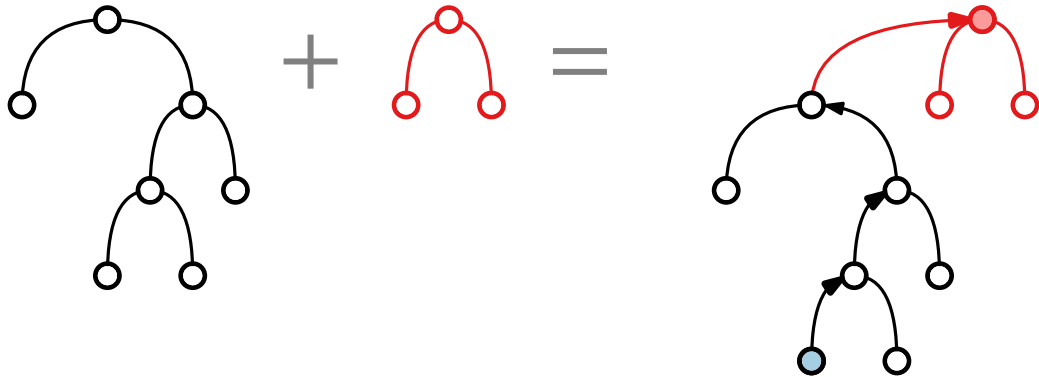
Realisierung UNIONFIND

Baumstruktur für jede Menge

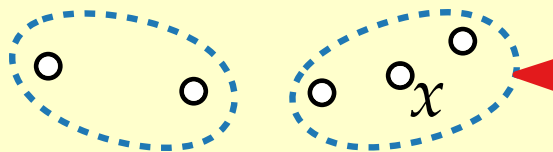
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



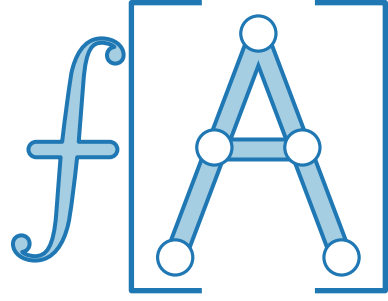
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



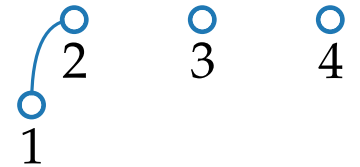
→ Laufe zur Wurzel, gebe Wurzel zurück



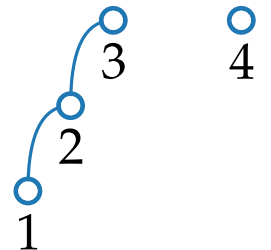
Beispiel.

1 2 3 4

■ UNION(1,2)



■ UNION(2,3)

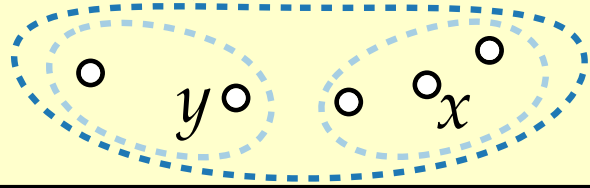


■ FIND(1)?

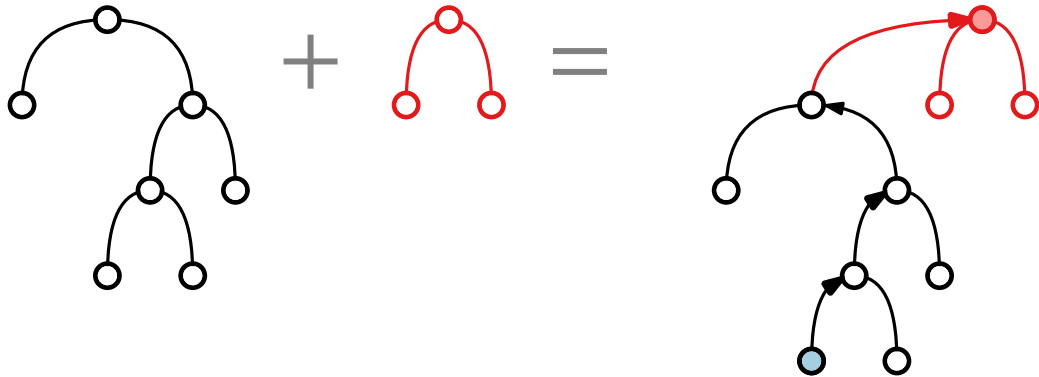
Realisierung UNIONFIND

Baumstruktur für jede Menge

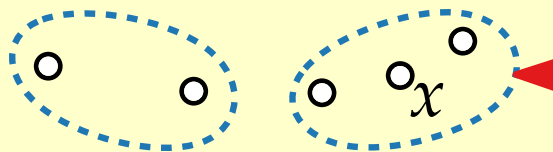
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



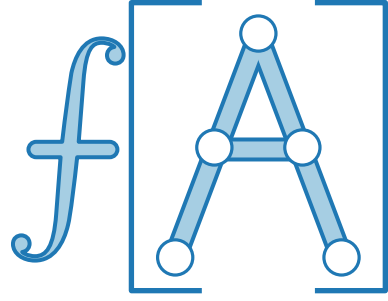
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



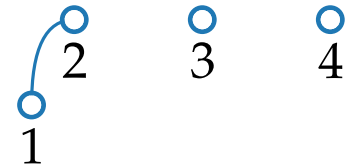
→ Laufe zur Wurzel, gebe Wurzel zurück



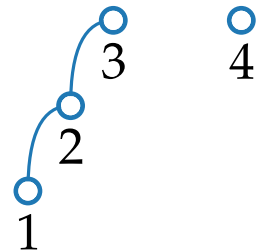
Beispiel.

1 2 3 4

■ UNION(1,2)



■ UNION(2,3)

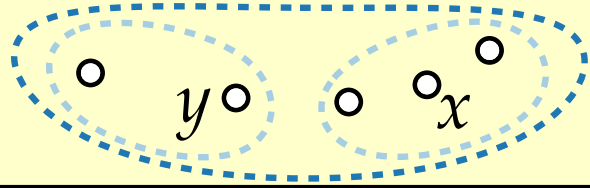


■ FIND(1)? → 3

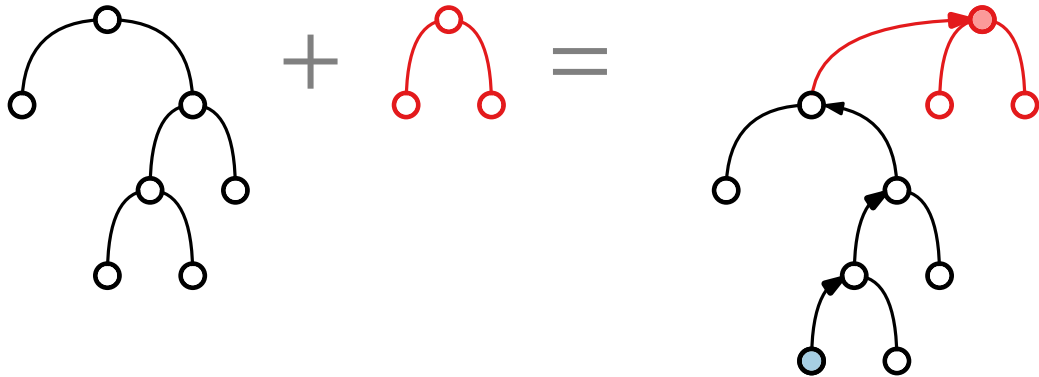
Realisierung UNIONFIND

Baumstruktur für jede Menge

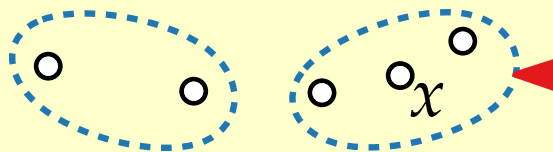
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



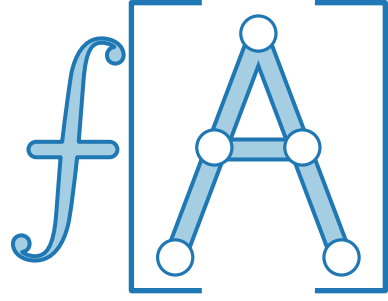
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



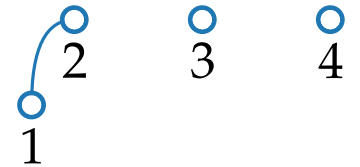
→ Laufe zur Wurzel, gebe Wurzel zurück



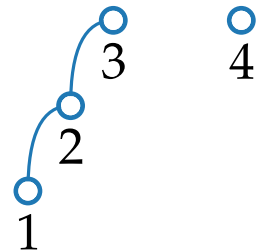
Beispiel.

1 2 3 4

■ UNION(1,2)



■ UNION(2,3)



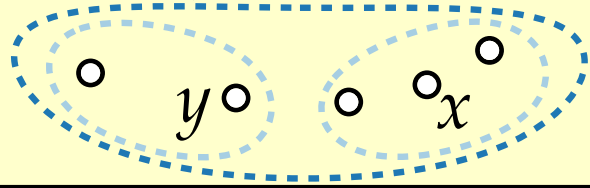
■ FIND(1)? → 3

■ FIND(3)?

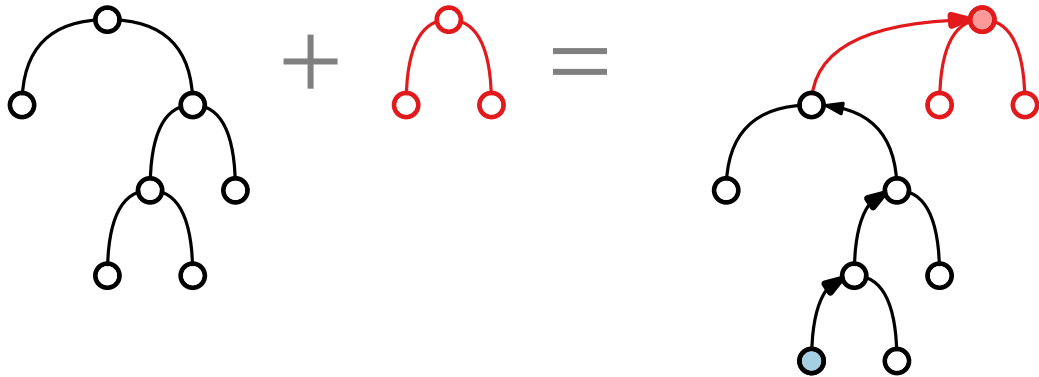
Realisierung UNIONFIND

Baumstruktur für jede Menge

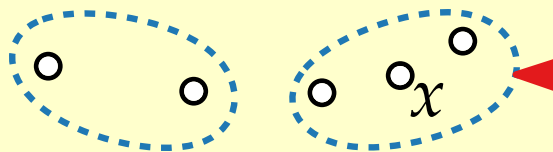
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



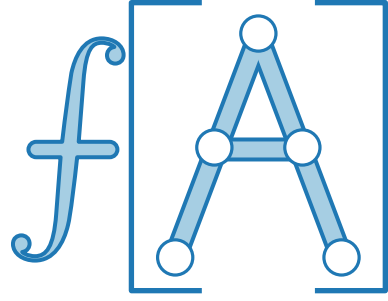
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



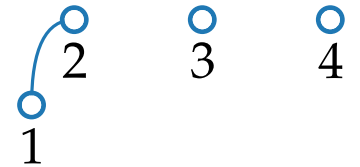
→ Laufe zur Wurzel, gebe Wurzel zurück



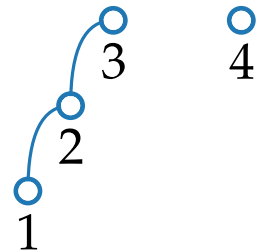
Beispiel.

1 2 3 4

■ UNION(1,2)



■ UNION(2,3)



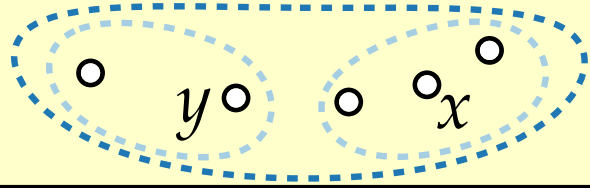
■ FIND(1)? → 3

■ FIND(3)? → 3

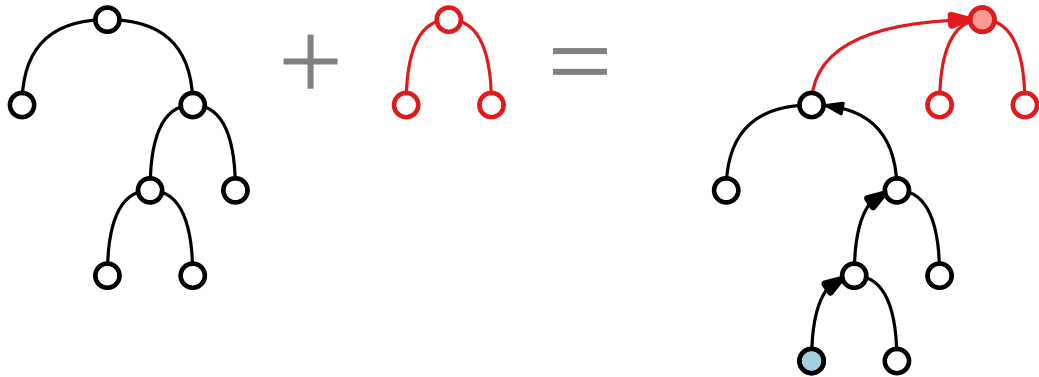
Realisierung UNIONFIND

Baumstruktur für jede Menge

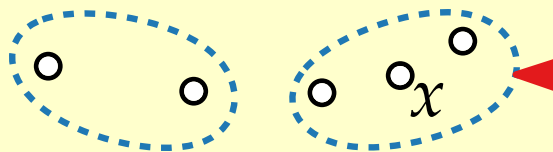
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



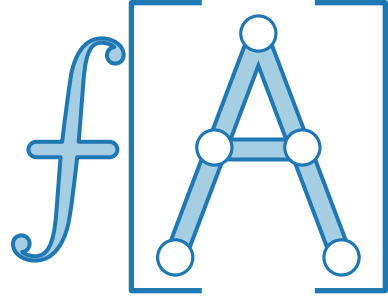
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



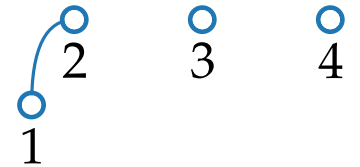
→ Laufe zur Wurzel, gebe Wurzel zurück



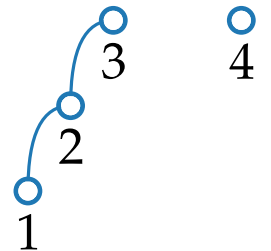
Beispiel.

1 2 3 4

■ UNION(1,2)



■ UNION(2,3)



■ FIND(1)? → 3

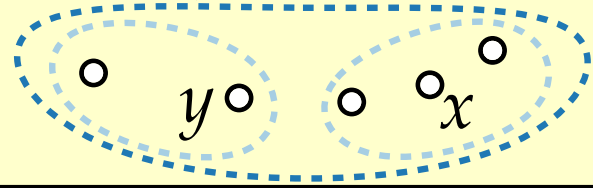
■ FIND(3)? → 3

■ FIND(1) = FIND(3)?

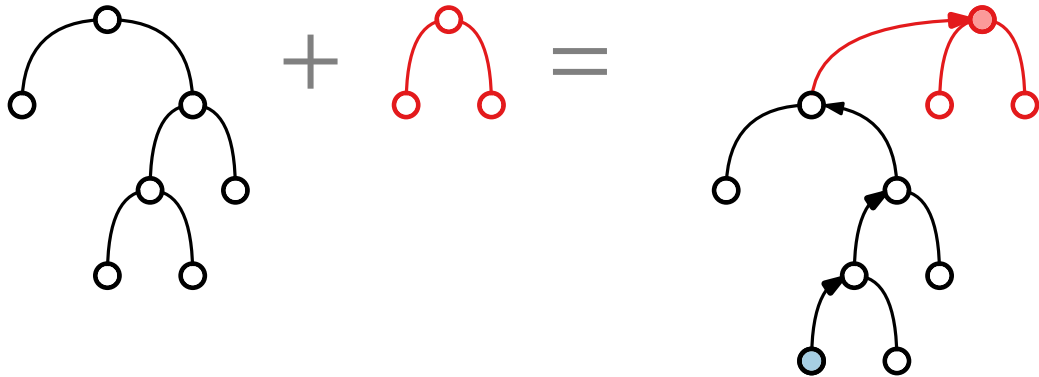
Realisierung UNIONFIND

Baumstruktur für jede Menge

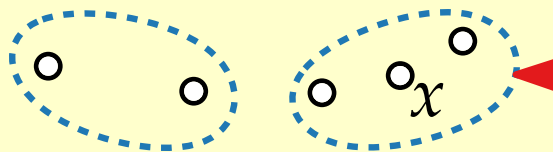
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



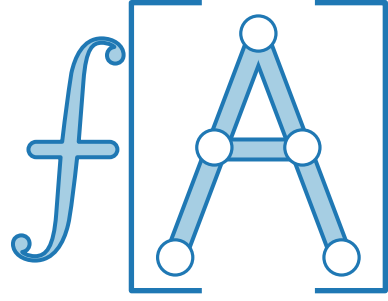
→ Hänge einen Baum an den anderen



FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



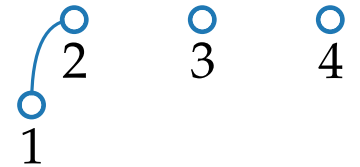
→ Laufe zur Wurzel, gebe Wurzel zurück



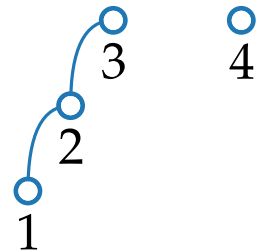
Beispiel.

1 2 3 4

■ UNION(1,2)



■ UNION(2,3)



■ FIND(1)? → 3

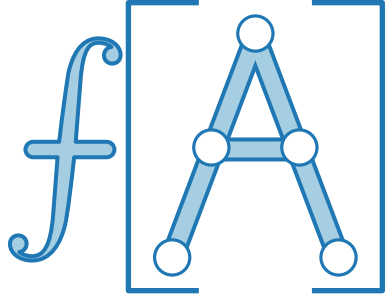
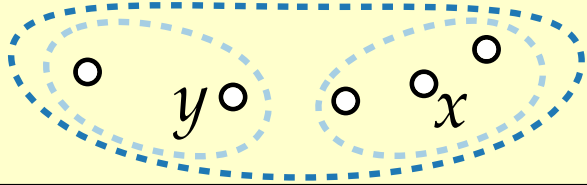
■ FIND(3)? → 3

■ FIND(1) = FIND(3)?
→ true

Zwei Verbesserungen

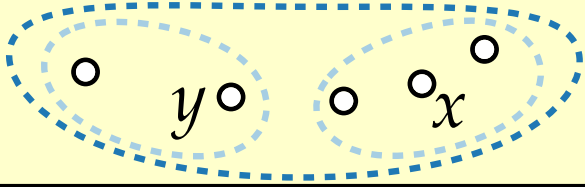
UNION(Elem. x , Elem. y)

vereinigt die Mengen, die momentan x und y enthalten.

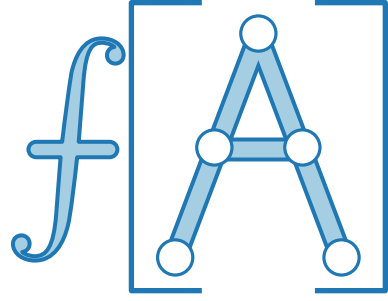


Zwei Verbesserungen

UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.

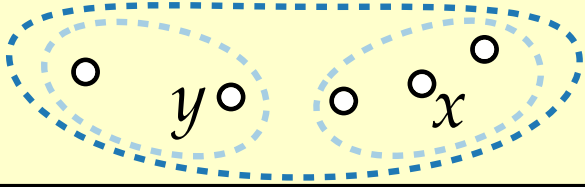


Union-by-Rank: Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat



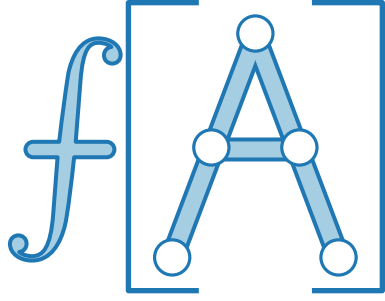
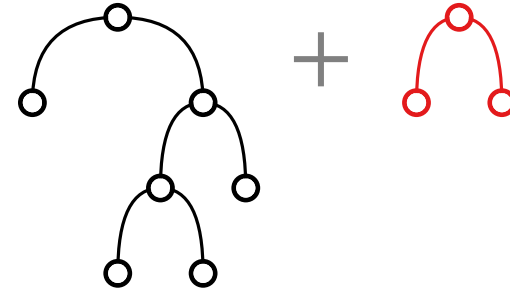
Zwei Verbesserungen

UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



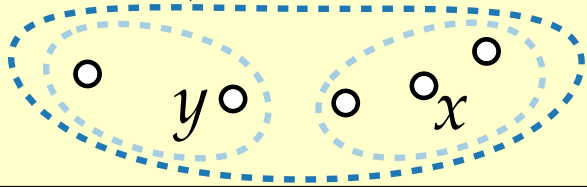
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat



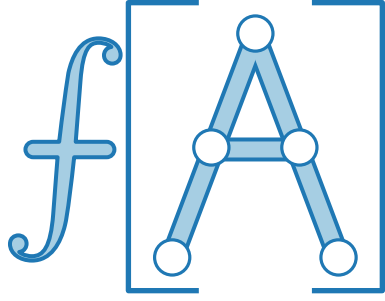
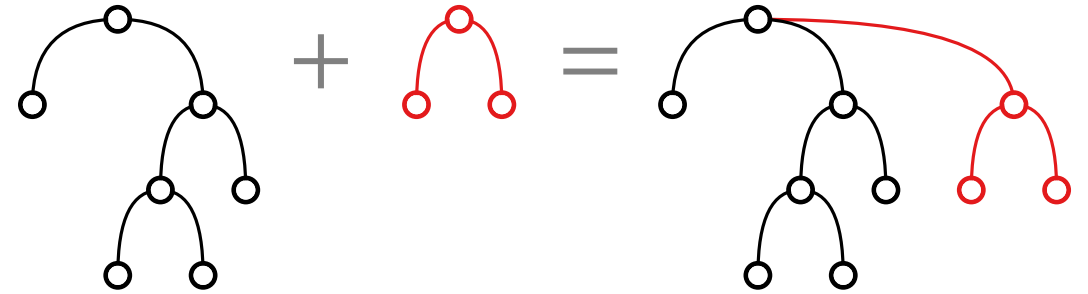
Zwei Verbesserungen

UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



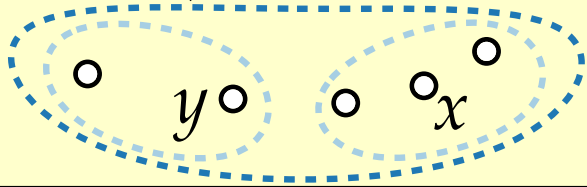
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat



Zwei Verbesserungen

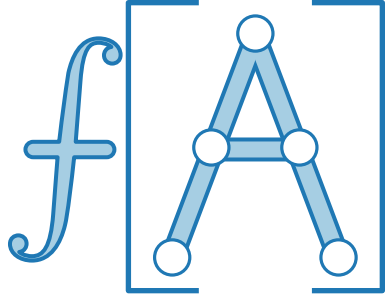
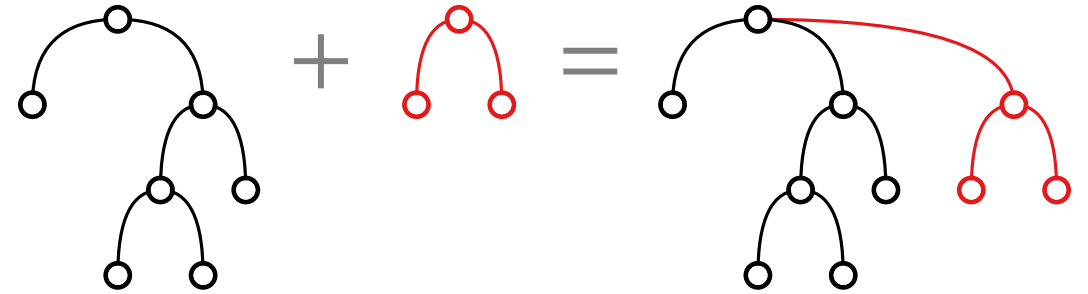
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



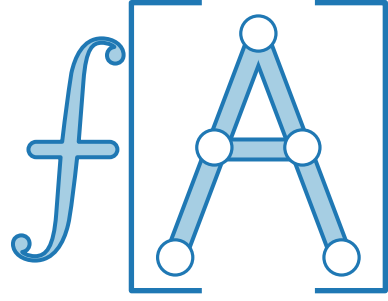
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

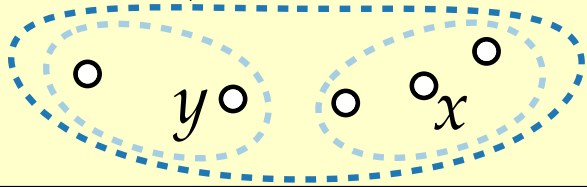
➡ Hänge Baum mit kleinerer Tiefe an den mit größerer



Zwei Verbesserungen



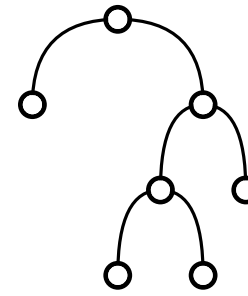
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



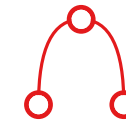
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

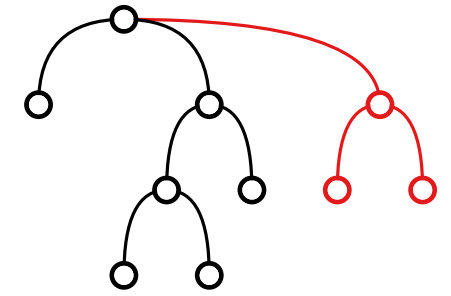
→ Hänge Baum mit kleinerer Tiefe an den mit größerer



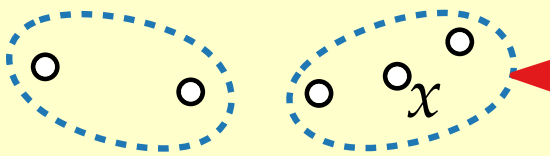
+



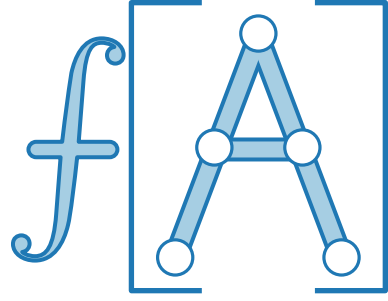
=



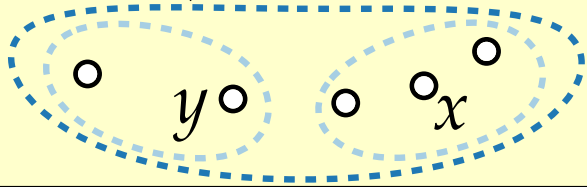
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



Zwei Verbesserungen



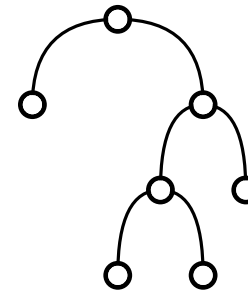
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



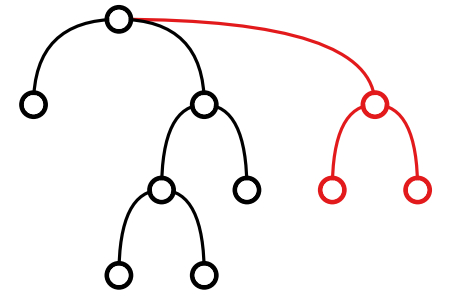
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

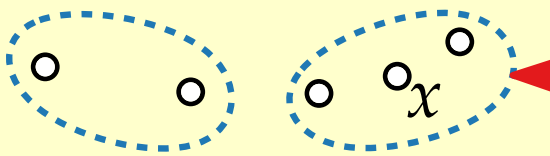
→ Hänge Baum mit kleinerer Tiefe an den mit größerer



=

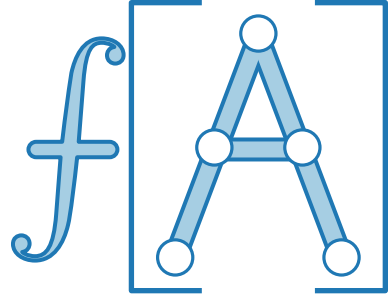


FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.

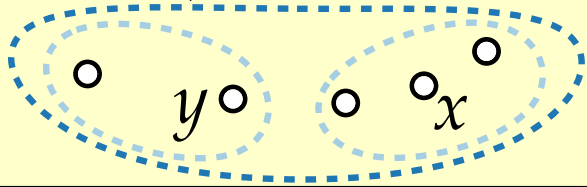


Pfadkompression: Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r

Zwei Verbesserungen



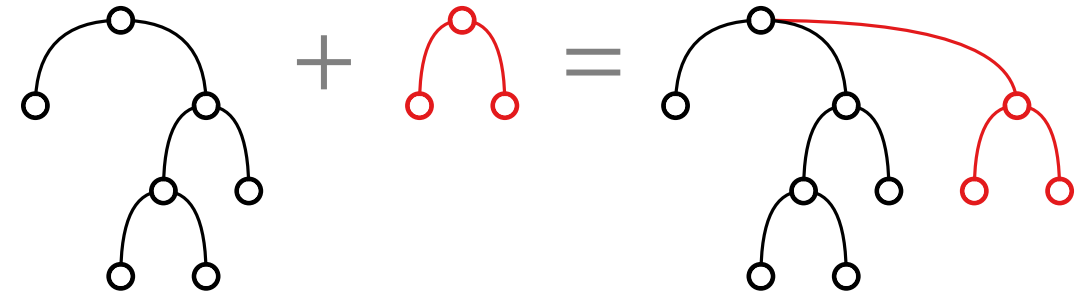
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



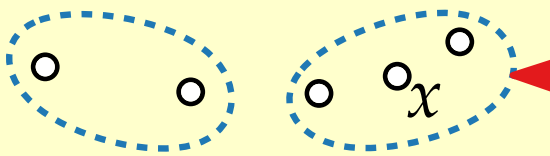
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

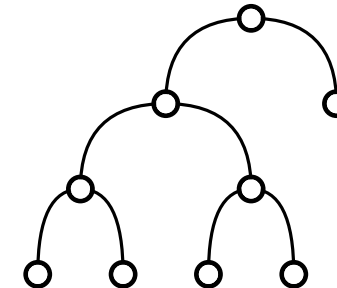
→ Hänge Baum mit kleinerer Tiefe an den mit größerer



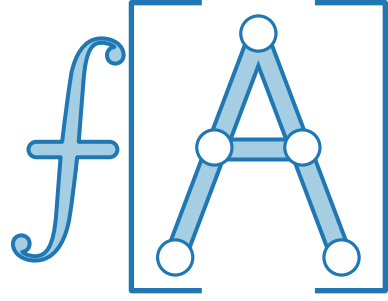
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



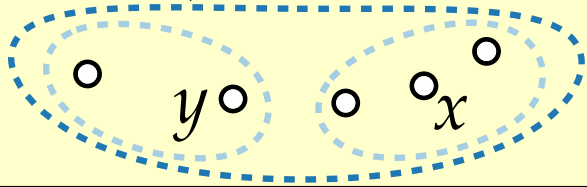
Pfadkompression: Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r



Zwei Verbesserungen



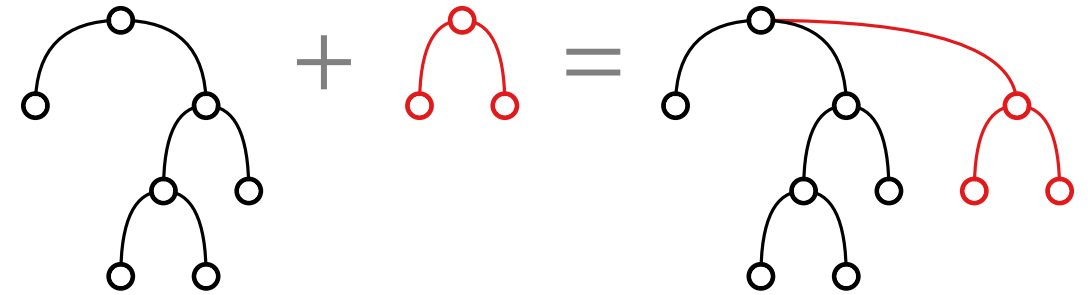
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



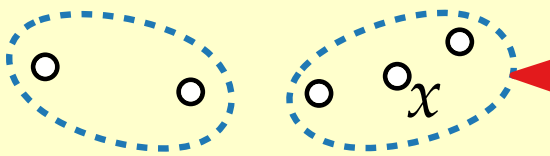
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

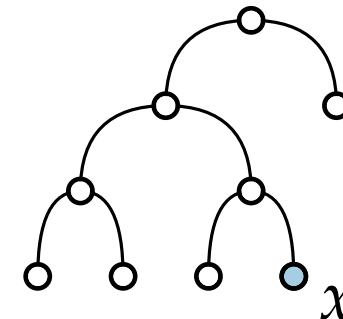
→ Hänge Baum mit kleinerer Tiefe an den mit größerer



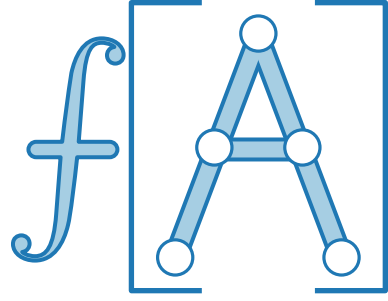
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



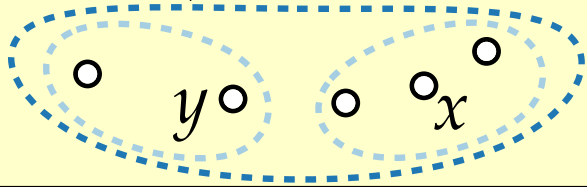
Pfadkompression: Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r



Zwei Verbesserungen



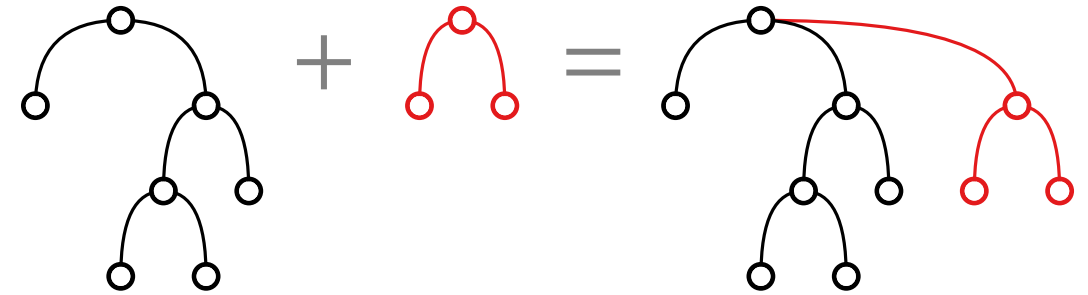
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



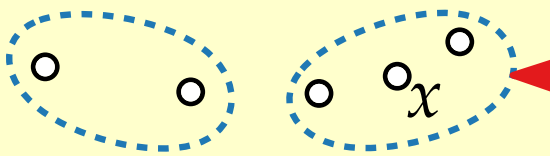
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

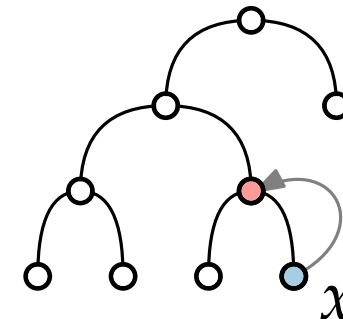
→ Hänge Baum mit kleinerer Tiefe an den mit größerer



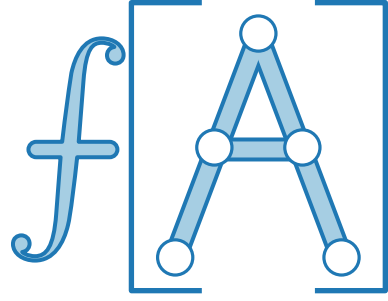
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



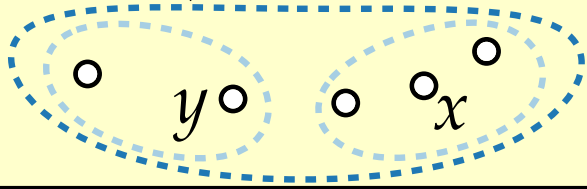
Pfadkompression: Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r



Zwei Verbesserungen



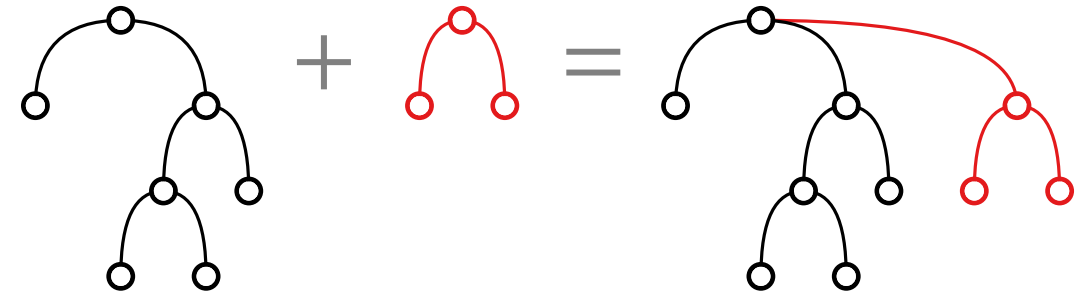
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



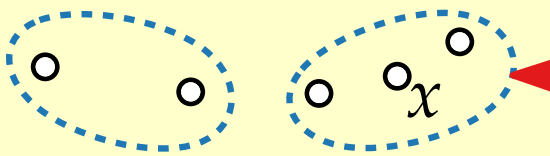
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

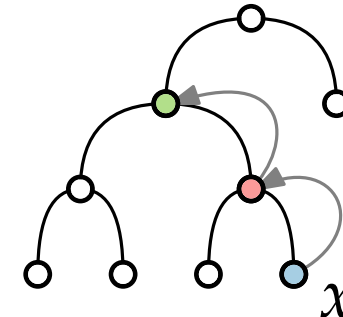
→ Hänge Baum mit kleinerer Tiefe an den mit größerer



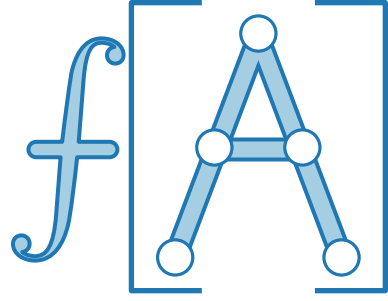
FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.



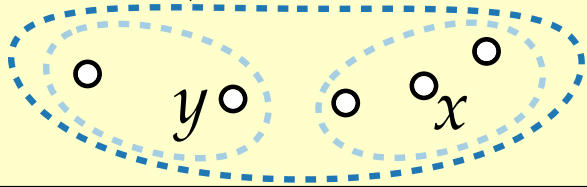
Pfadkompression: Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r



Zwei Verbesserungen



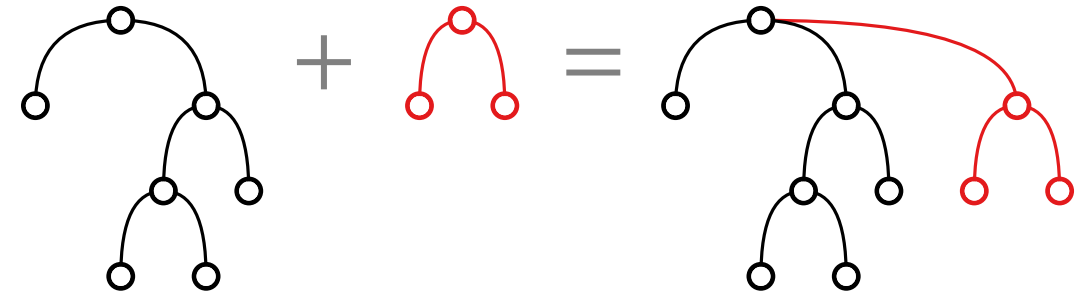
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



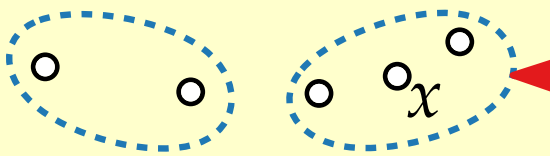
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

→ Hänge Baum mit kleinerer Tiefe an den mit größerer

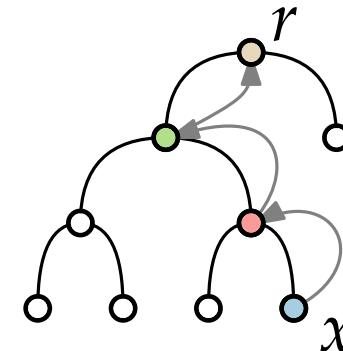


FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.

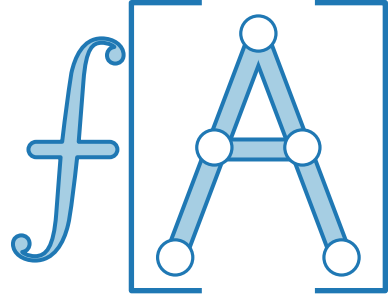


Pfadkompression:

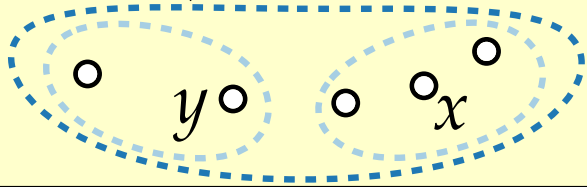
Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r



Zwei Verbesserungen



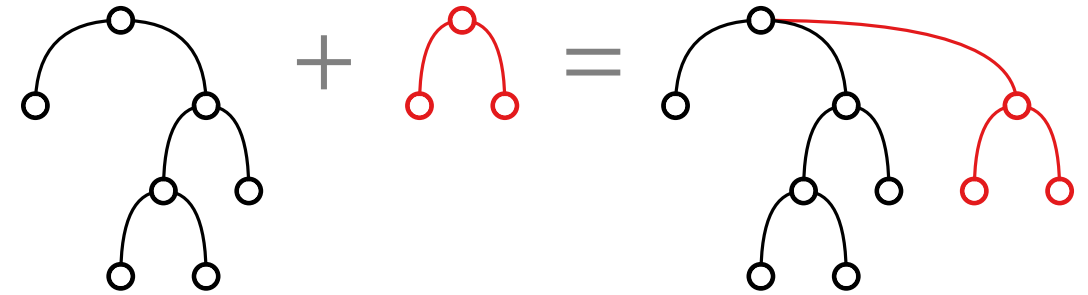
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



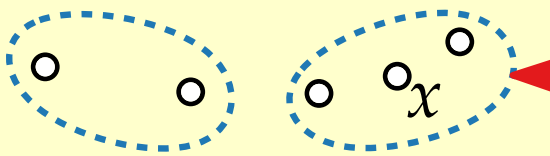
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

→ Hänge Baum mit kleinerer Tiefe an den mit größerer

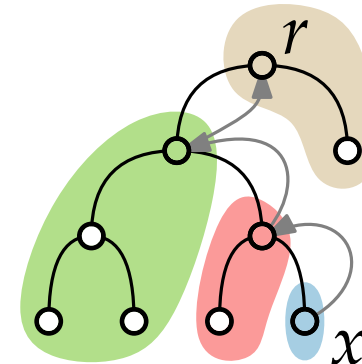


FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.

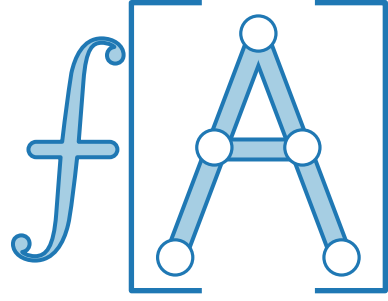


Pfadkompression:

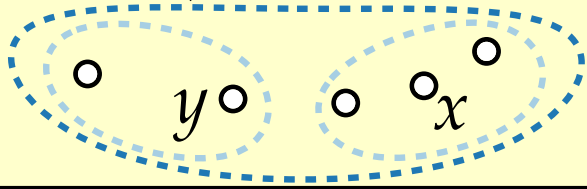
Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r



Zwei Verbesserungen



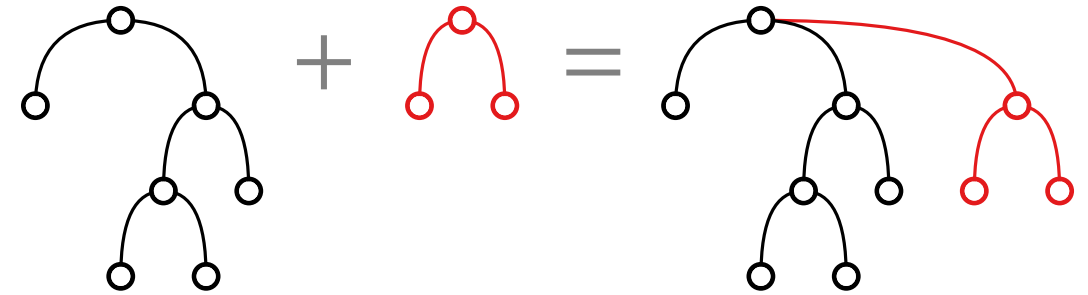
UNION(Elem. x , Elem. y) vereinigt die Mengen, die momentan x und y enthalten.



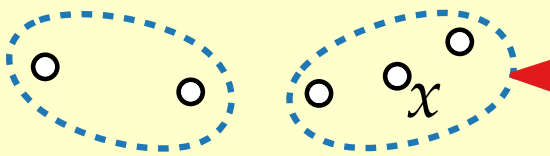
Union-by-Rank:

Führe die Op. so aus, dass der neue Baum möglichst geringe Tiefe hat

→ Hänge Baum mit kleinerer Tiefe an den mit größerer

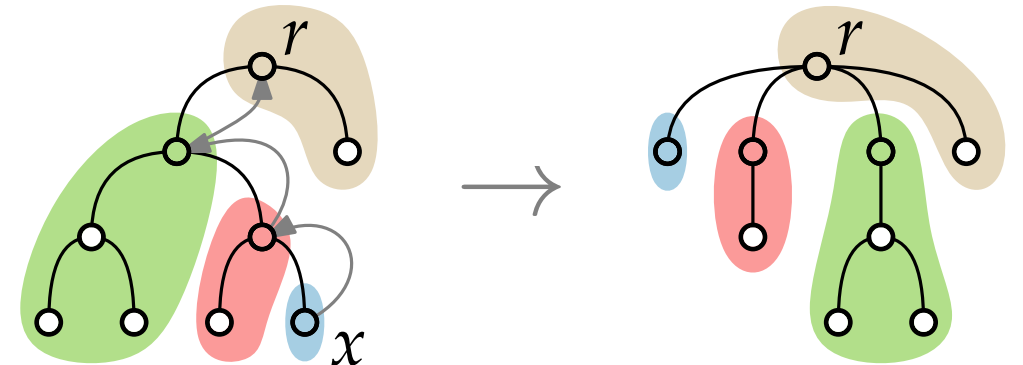


FIND(Element x) liefert (Zeiger auf) die Menge zurück, die momentan x enthält.

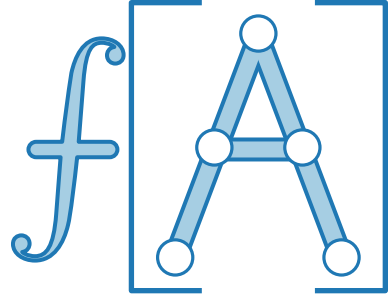


Pfadkompression:

Laufe zur Wurzel r , merke alle besuchten Knoten und mache sie zu Kindern von r



Kosten für Union-Find

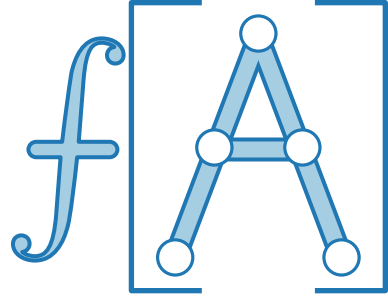


Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

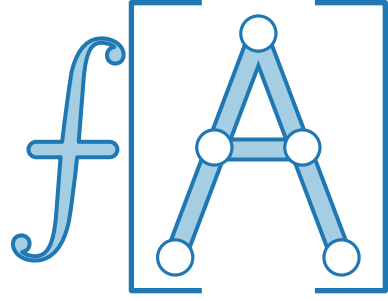
Kosten für Union-Find

Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank



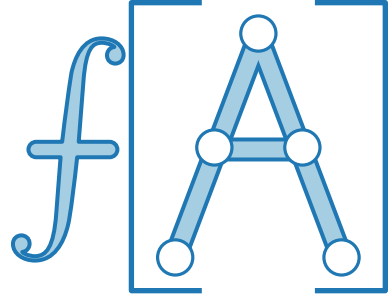
Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

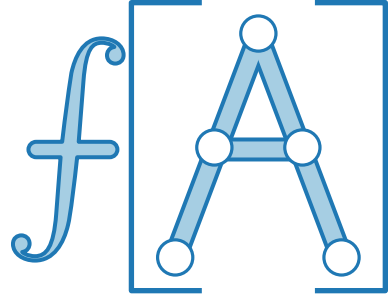
Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

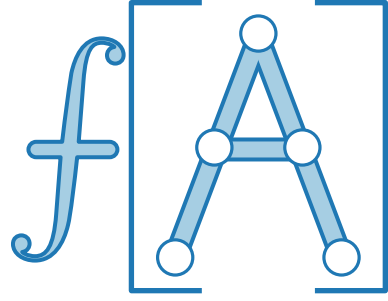
Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression
- $\alpha(n)$ ist die inverse Ackermannfunktion

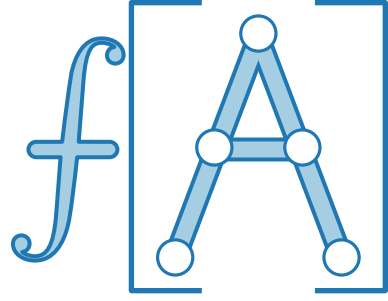
Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression
- $\alpha(n)$ ist die inverse Ackermannfunktion
- $\alpha_1(n) = \lceil n/2 \rceil$

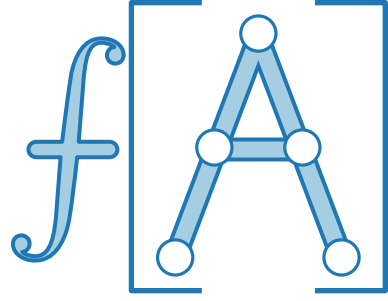
Kosten für Union-Find



Satz. Kosten für $m \times$ FIND und $n \times$ UNION:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression
- $\alpha(n)$ ist die inverse Ackermannfunktion
- $\alpha_1(n) = \lceil n/2 \rceil$
- $\alpha_k(n) =$ „wie oft muss ich $\alpha_{k-1}(n)$ auf n anwenden, um auf 1 zu kommen?“

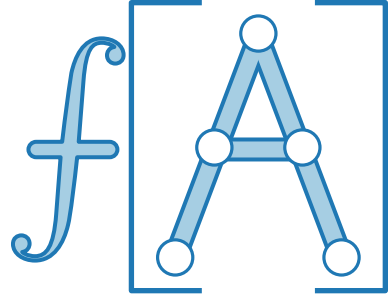
Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression
- $\alpha(n)$ ist die inverse Ackermannfunktion
- $\alpha_1(n) = \lceil n/2 \rceil$
- $\alpha_k(n) =$ „wie oft muss ich $\alpha_{k-1}(n)$ auf n anwenden, um auf 1 zu kommen?“
- $\alpha_2(n) = \lceil \log n \rceil$

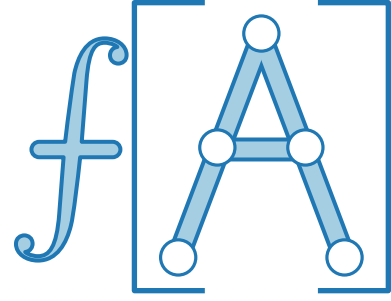
Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression
- $\alpha(n)$ ist die inverse Ackermannfunktion
- $\alpha_1(n) = \lceil n/2 \rceil$
- $\alpha_k(n) = \text{„wie oft muss ich } \alpha_{k-1}(n) \text{ auf } n \text{ anwenden, um auf 1 zu kommen?“}$
- $\alpha_2(n) = \lceil \log n \rceil$
- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

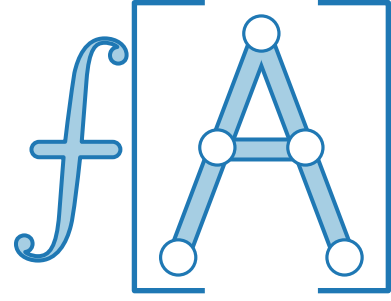
Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression
- $\alpha(n)$ ist die inverse Ackermannfunktion
- $\alpha_1(n) = \lceil n/2 \rceil$
- $\alpha_k(n) = \text{„wie oft muss ich } \alpha_{k-1}(n) \text{ auf } n \text{ anwenden, um auf 1 zu kommen?“}$
- $\alpha_2(n) = \lceil \log n \rceil$
- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$ z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$

Kosten für Union-Find

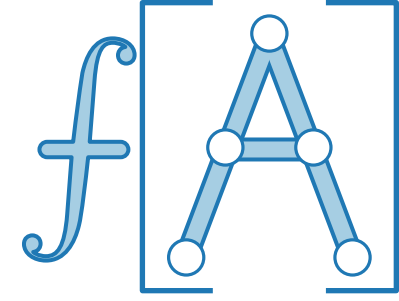


Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression
- $\alpha(n)$ ist die inverse Ackermannfunktion
- $\alpha_1(n) = \lceil n/2 \rceil$
- $\alpha_k(n) = \text{„wie oft muss ich } \alpha_{k-1}(n) \text{ auf } n \text{ anwenden, um auf 1 zu kommen?“}$
- $\alpha_2(n) = \lceil \log n \rceil$
- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$
 $\log^*(2^{2^{2^{2^2}}}) = \log^*(2^{65536}) = 5$

Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

- $\alpha(n)$ ist die inverse Ackermannfunktion

- $\alpha_1(n) = \lceil n/2 \rceil$

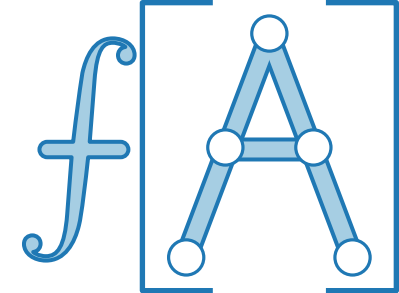
- $\alpha_k(n) = \text{„wie oft muss ich } \alpha_{k-1}(n) \text{ auf } n \text{ anwenden, um auf 1 zu kommen?“}$

- $\alpha_2(n) = \lceil \log n \rceil$

- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$

$\log^*(2^{2^{2^{2^2}}}) = \log^*(2^{65536}) = 5$
 $\approx 2 \cdot 10^{19729}$



Kosten für Union-Find

Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

- $\alpha(n)$ ist die inverse Ackermannfunktion

- $\alpha_1(n) = \lceil n/2 \rceil$

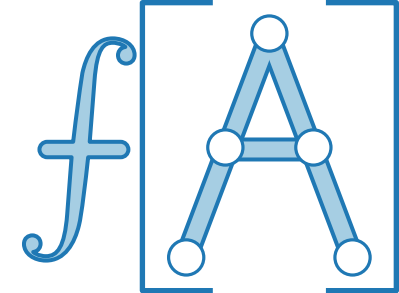
- $\alpha_k(n) = \text{„wie oft muss ich } \alpha_{k-1}(n) \text{ auf } n \text{ anwenden, um auf 1 zu kommen?“}$

- $\alpha_2(n) = \lceil \log n \rceil$

- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$
 $\log^*(2^{2^{2^{2^2}}}) = \log^*(\underbrace{2^{65536}}_{\approx 2 \cdot 10^{19729}}) = 5$

- $\alpha_4(n) = \log^{**}(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^{**}(\log^* n) & \text{sonst} \end{cases}$



Kosten für Union-Find

Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

- $\alpha(n)$ ist die inverse Ackermannfunktion

- $\alpha_1(n) = \lceil n/2 \rceil$

- $\alpha_k(n) = \text{„wie oft muss ich } \alpha_{k-1}(n) \text{ auf } n \text{ anwenden, um auf 1 zu kommen?“}$

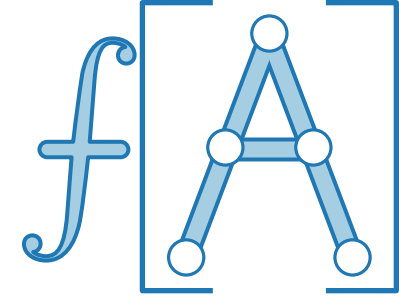
- $\alpha_2(n) = \lceil \log n \rceil$

- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$
 $\log^*(2^{2^{2^{2^2}}}) = \log^*(2^{65536}) = 5$
 $\approx 2 \cdot 10^{19729}$

- $\alpha_4(n) = \log^{**}(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^{**}(\log^* n) & \text{sonst} \end{cases}$

...



Kosten für Union-Find

Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

- $\alpha(n)$ ist die inverse Ackermannfunktion

- $\alpha_1(n) = \lceil n/2 \rceil$

- $\alpha_k(n) =$ „wie oft muss ich $\alpha_{k-1}(n)$ auf n anwenden, um auf 1 zu kommen?“

- $\alpha_2(n) = \lceil \log n \rceil$

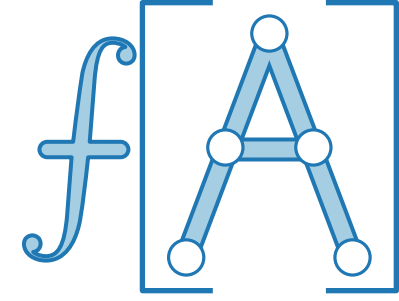
- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$
 $\log^*(2^{2^{2^{2^2}}}) = \log^*(2^{65536}) = 5$
 $\approx 2 \cdot 10^{19729}$

- $\alpha_4(n) = \log^{**}(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^{**}(\log^* n) & \text{sonst} \end{cases}$
- ...

- $\alpha(n)$ ist das kleinste k , so dass $\alpha_k(n) \leq 3$

Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

- $\alpha(n)$ ist die inverse Ackermannfunktion

- $\alpha_1(n) = \lceil n/2 \rceil$

- $\alpha_k(n) =$ „wie oft muss ich $\alpha_{k-1}(n)$ auf n anwenden, um auf 1 zu kommen?“

- $\alpha_2(n) = \lceil \log n \rceil$

- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

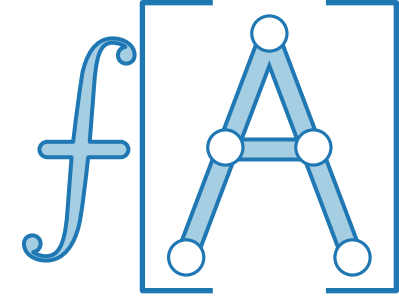
z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$
 $\log^*(2^{2^{2^{2^2}}}) = \log^*(\underbrace{2^{65536}}_{\approx 2 \cdot 10^{19729}}) = 5$

- $\alpha_4(n) = \log^{**}(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^{**}(\log^* n) & \text{sonst} \end{cases}$
- ...

- $\alpha(n)$ ist das kleinste k , so dass $\alpha_k(n) \leq 3$

$$\alpha(n) \leq 4 \text{ für } n \leq 2^{2^{2^{2^{2^2}}}} \approx 10^{10^{10^{19729}}}$$

Kosten für Union-Find



Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

- $\alpha(n)$ ist die inverse Ackermannfunktion

- $\alpha_1(n) = \lceil n/2 \rceil$

- $\alpha_k(n) =$ „wie oft muss ich $\alpha_{k-1}(n)$ auf n anwenden, um auf 1 zu kommen?“

- $\alpha_2(n) = \lceil \log n \rceil$

- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

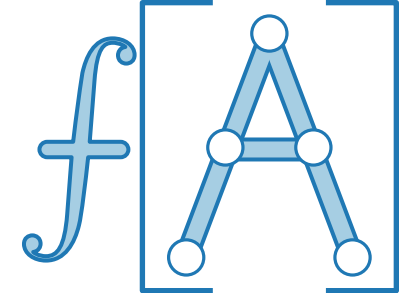
z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$
 $\log^*(2^{2^{2^{2^2}}}) = \log^*(2^{65536}) = 5$
 $\approx 2 \cdot 10^{19729}$

- $\alpha_4(n) = \log^{**}(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^{**}(\log^* n) & \text{sonst} \end{cases}$
- ...

- $\alpha(n)$ ist das kleinste k , so dass $\alpha_k(n) \leq 3$

$$\alpha(n) \leq 4 \text{ für } n \leq 2^{2^{2^{2^{2^2}}}} \approx 10^{10^{10^{19729}}}$$

$$\alpha(n) \leq 5 \text{ für } n \leq \underbrace{2^{2^{\dots^2}}}_{\text{mal}} \underbrace{2^{2^{\dots^2}}}_{\text{mal}} \dots \underbrace{2^{2^2}}_{\text{mal}}$$



Kosten für Union-Find

Satz. Kosten für $m \times \text{FIND}$ und $n \times \text{UNION}$:

- $\mathcal{O}(n + m \log n)$ mit Union-by-Rank
- $\mathcal{O}(n + m \cdot \alpha(n))$ mit Union-by-Rank und Pfadkompression

$\Omega(n + m \cdot \alpha(n))$ ist untere Schranke für Union-Find [Tarjan '79]

- $\alpha(n)$ ist die inverse Ackermannfunktion
- $\alpha_1(n) = \lceil n/2 \rceil$
- $\alpha_k(n) =$ „wie oft muss ich $\alpha_{k-1}(n)$ auf n anwenden, um auf 1 zu kommen?“

- $\alpha_2(n) = \lceil \log n \rceil$

- $\alpha_3(n) = \log^*(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^*(\log n) & \text{sonst} \end{cases}$

z.B. $\log^*(2^{2^{2^2}}) = \log^*(65536) = 4$

$\log^*(2^{2^{2^{2^2}}}) = \log^*(2^{65536}) = 5$
 $\approx 2 \cdot 10^{19729}$

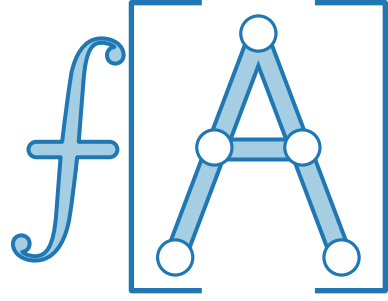
- $\alpha_4(n) = \log^{**}(n) = \begin{cases} 0 & \text{wenn } n \leq 1 \\ 1 + \log^{**}(\log^* n) & \text{sonst} \end{cases}$
- ...

$\alpha(n) \leq 4$ für $n \leq 2^{2^{2^{2^{2^2}}}} \approx 10^{10^{10^{19729}}}$

- $\alpha(n)$ ist das kleinste k , so dass $\alpha_k(n) \leq 3$

$\alpha(n) \leq 5$ für $n \leq 2^{2^{\dots^{2^2}}}$ mal $2^{2^{\dots^{2^2}}}$ mal $2^{2^{2^2}}$ mal

Übersicht: Algorithmen für min. Spannbäume



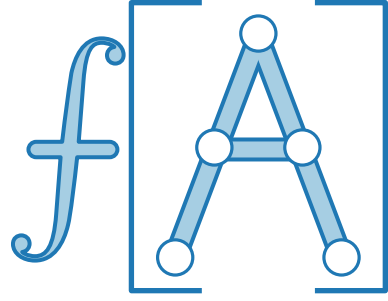
JARNÍK-PRIM

- geht (wie DIJKSTRA / BFS) wellenförmig von einem Startknoten aus

KRUSKAL

- bearbeitet Kanten nach aufsteigendem (genauer: nicht-absteig.) Gewicht

Übersicht: Algorithmen für min. Spannbäume



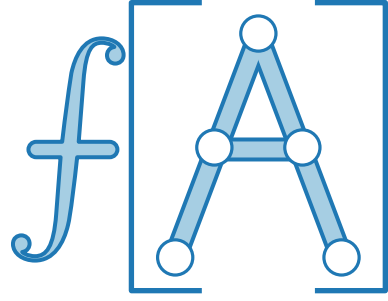
JARNÍK-PRIM

- geht (wie DIJKSTRA / BFS) wellenförmig von einem Startknoten aus
- aktuelle Kantenmenge zusammenhängend

KRUSKAL

- bearbeitet Kanten nach aufsteigendem (genauer: nicht-absteig.) Gewicht
- nach Einfügen der i . Kante gibt es $n - i$ Zusammenhangskomponenten

Übersicht: Algorithmen für min. Spannbäume



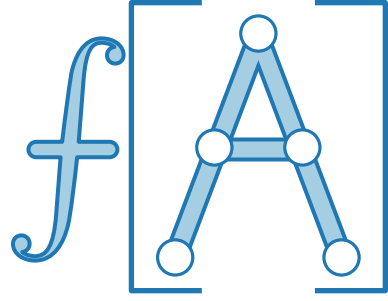
JARNÍK-PRIM

- geht (wie DIJKSTRA / BFS) wellenförmig von einem Startknoten aus
- aktuelle Kantenmenge zusammenhängend
- Laufzeit $\mathcal{O}(m + n \log n)$

KRUSKAL

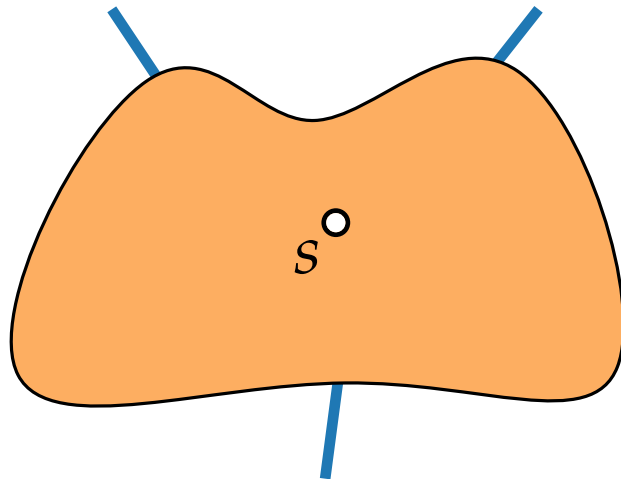
- bearbeitet Kanten nach aufsteigendem (genauer: nicht-absteig.) Gewicht
- nach Einfügen der i . Kante gibt es $n - i$ Zusammenhangskomponenten
- Laufzeit $\mathcal{O}(m \log n)$ oder $\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert

Übersicht: Algorithmen für min. Spannbäume



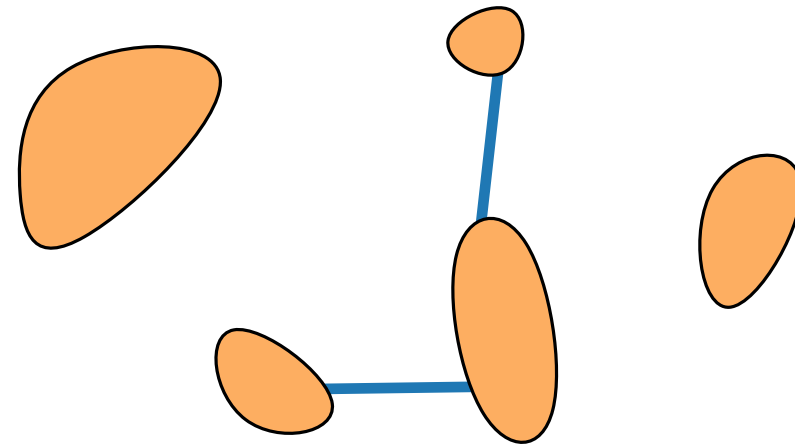
JARNÍK-PRIM

- geht (wie DIJKSTRA / BFS) wellenförmig von einem Startknoten aus
- aktuelle Kantenmenge zusammenhängend
- Laufzeit $\mathcal{O}(m + n \log n)$

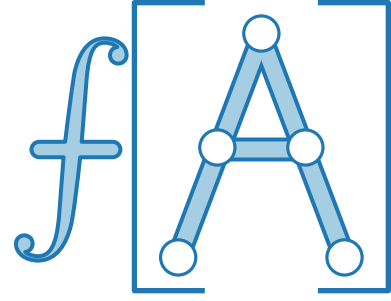


KRUSKAL

- bearbeitet Kanten nach aufsteigendem (genauer: nicht-absteig.) Gewicht
- nach Einfügen der i . Kante gibt es $n - i$ Zusammenhangskomponenten
- Laufzeit $\mathcal{O}(m \log n)$ oder $\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert



Übersicht: Algorithmen für min. Spannbäume



JARNÍK-PRIM

- geht (wie DIJKSTRA / BFS) wellenförmig von einem Startknoten aus
- aktuelle Kantenmenge zusammenhängend
- Laufzeit $\mathcal{O}(m + n \log n)$

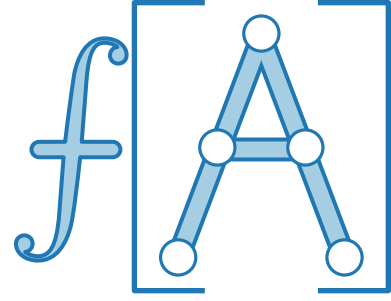
KRUSKAL

- bearbeitet Kanten nach aufsteigendem (genauer: nicht-absteig.) Gewicht
- nach Einfügen der i . Kante gibt es $n - i$ Zusammenhangskomponenten
- Laufzeit $\mathcal{O}(m \log n)$ oder $\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert

GREEDYSPANNBAUM($G = (V, E; w)$)

Wende **blaue Regel** oder **rote Regel**
an, bis alle Kanten gefärbt sind.
Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Übersicht: Algorithmen für min. Spannbäume



JARNÍK-PRIM

- geht (wie DIJKSTRA / BFS) wellenförmig von einem Startknoten aus
- aktuelle Kantenmenge zusammenhängend
- Laufzeit $\mathcal{O}(m + n \log n)$

KRUSKAL

- bearbeitet Kanten nach aufsteigendem (genauer: nicht-absteig.) Gewicht
- nach Einfügen der i . Kante gibt es $n - i$ Zusammenhangskomponenten
- Laufzeit $\mathcal{O}(m \log n)$ oder $\mathcal{O}(m \cdot \alpha(n))$ falls vorsortiert

GREEDYSPANNBAUM($G = (V, E; w)$)
Wende **blaue Regel** oder **rote Regel**
an, bis alle Kanten gefärbt sind.
Gebe $E' = \{\text{blaue Kanten}\}$ zurück

Blaue Regel:

Wähle Schnitt, den keine **blaue** Kante kreuzt
Färbe leichte Kante **blau**

Rote Regel:

Wähle Kreis ohne **rote** Kante
Färbe größte ungefärbte Kante auf Kreis **rot**